



OpenShift Container Platform 4.22

Storage

Configuring and managing storage in OpenShift Container Platform

OpenShift Container Platform 4.22 Storage

Configuring and managing storage in OpenShift Container Platform

Legal Notice

Copyright © Red Hat.

Except as otherwise noted below, the text of and illustrations in this documentation are licensed by Red Hat under the Creative Commons Attribution–Share Alike 3.0 Unported license . If you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, the Red Hat logo, JBoss, Hibernate, and RHCE are trademarks or registered trademarks of Red Hat, LLC. or its subsidiaries in the United States and other countries.

Linux[®] is the registered trademark of Linus Torvalds in the United States and other countries.

XFS is a trademark or registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and other countries.

The OpenStack[®] Word Mark and OpenStack logo are trademarks or registered trademarks of the Linux Foundation, used under license.

All other trademarks are the property of their respective owners.

Abstract

This document provides instructions for configuring persistent volumes from various storage back ends and managing dynamic allocation from Pods.

Table of Contents

CHAPTER 1. OPENSIFT CONTAINER PLATFORM STORAGE OVERVIEW	11
1.1. GLOSSARY OF COMMON TERMS FOR OPENSIFT CONTAINER PLATFORM STORAGE	11
1.2. STORAGE TYPES	14
1.2.1. Ephemeral storage	14
1.2.2. Persistent storage	14
1.3. CONTAINER STORAGE INTERFACE (CSI)	14
1.4. DYNAMIC PROVISIONING	14
CHAPTER 2. UNDERSTANDING EPHEMERAL STORAGE	15
2.1. OVERVIEW OF EPHEMERAL STORAGE	15
2.2. TYPES OF EPHEMERAL STORAGE	15
2.3. EPHEMERAL STORAGE MANAGEMENT OVERVIEW	16
2.3.1. Ephemeral storage management limits and requests	16
2.3.2. Ephemeral storage management configuration affects pod scheduling and eviction	17
2.4. MONITORING EPHEMERAL STORAGE	18
CHAPTER 3. UNDERSTANDING PERSISTENT STORAGE	19
3.1. PERSISTENT STORAGE OVERVIEW	19
3.2. LIFECYCLE OF A VOLUME AND CLAIM	19
3.2.1. Provision storage	20
3.2.2. Bind claims	20
3.2.3. Use pods and claimed PVs	20
3.2.4. Storage Object in Use Protection	20
3.2.5. Release a persistent volume	21
3.2.6. Reclaim policy for persistent volumes	21
3.2.7. Reclaiming a persistent volume manually	21
3.2.8. Changing the reclaim policy of a persistent volume	22
3.3. PERSISTENT VOLUMES	23
3.3.1. Types of PVs	23
3.3.2. Capacity	24
3.3.3. Access modes	24
3.3.4. Phase	27
3.4. PERSISTENT VOLUME CLAIMS	29
3.4.1. Volume Attributes Classes	29
3.4.1.1. Volume Attributes Classes Limitations	29
3.4.1.2. Defining Volume Attributes Classes	30
3.4.1.3. Applying a Volume Attributes Class to a PVC	31
3.4.1.4. Deleting Volume Attributes Classes	32
3.4.2. Block volume support	33
3.4.2.1. Block volume examples	34
3.4.3. Reduce pod timeouts by using fsGroup	36
3.4.3.1. Changing fsGroup at the namespace level	37
3.4.3.2. Changing fsGroup at the pod level	38
3.4.4. Reducing pod timeouts using seLinuxChangePolicy	39
3.4.4.1. RWOP and SELinux mount option	39
3.4.4.2. RWO and RWX and SELinux mount option	40
3.4.4.3. Testing the RWO and RWX and SELinux mount option feature	40
3.4.4.4. Opting out of the SELinux mount option default	41
3.4.4.4.1. Changing seLinuxChangePolicy at the namespace level	41
3.4.4.4.2. Changing seLinuxChangePolicy at the pod level	42
3.4.5. Additional resources	42

CHAPTER 4. CONFIGURING PERSISTENT STORAGE	43
4.1. PERSISTENT STORAGE USING AWS ELASTIC BLOCK STORE	43
4.1.1. About the EBS storage class	43
4.1.2. Creating the persistent volume claim	43
4.1.3. Volume format	44
4.1.4. Maximum number of EBS volumes on a node	44
4.1.5. Encrypting container persistent volumes on AWS with a KMS key	44
4.1.6. Additional resources	46
4.2. PERSISTENT STORAGE USING AZURE	46
4.2.1. Creating the Azure storage class	46
4.2.2. Creating the persistent volume claim	47
4.2.3. Volume format	48
4.2.4. Machine sets that deploy machines with ultra disks using PVCs	48
4.2.4.1. Creating machines with ultra disks by using machine sets	48
4.2.4.2. Troubleshooting resources for machine sets that enable ultra disks	52
4.2.4.2.1. Unable to mount a persistent volume claim backed by an ultra disk	52
4.3. PERSISTENT STORAGE USING AZURE FILE	52
4.3.1. Create the Azure File share persistent volume claim	53
4.3.2. Mount the Azure File share in a pod	54
4.4. PERSISTENT STORAGE USING CINDER	55
4.4.1. Manual provisioning with Cinder	56
4.4.1.1. Creating the persistent volume	56
4.4.1.2. Persistent volume formatting	57
4.4.1.3. Configuring Cinder volume security	57
4.5. PERSISTENT STORAGE USING FIBRE CHANNEL	58
4.5.1. About provisioning storage using Fibre Channel	59
4.5.1.1. Enforcing disk quotas	60
4.5.1.2. Fibre Channel volume security	60
4.6. PERSISTENT STORAGE USING FLEXVOLUME	60
4.6.1. About FlexVolume drivers	61
4.6.2. FlexVolume driver example	61
4.6.3. Installing FlexVolume drivers	62
4.6.4. Consuming storage using FlexVolume drivers	63
4.7. PERSISTENT STORAGE USING GCE PERSISTENT DISK	64
4.7.1. About the GCE storage class	65
4.7.2. Creating the persistent volume claim	65
4.7.3. Volume format	66
4.8. PERSISTENT STORAGE USING ISCSI	66
4.8.1. Provisioning	66
4.8.2. Enforce disk quotas	67
4.8.3. iSCSI volume security	67
4.8.3.1. Challenge Handshake Authentication Protocol (CHAP) configuration	67
4.8.4. iSCSI multipathing	68
4.8.5. iSCSI custom initiator IQN	69
4.9. PERSISTENT STORAGE USING NFS	69
4.9.1. Provisioning persistent storage using NFS	70
4.9.2. Enforce disk quotas	71
4.9.3. NFS volume security	72
4.9.3.1. Group IDs	72
4.9.3.2. User IDs	73
4.9.3.3. SELinux	74
4.9.3.4. Export settings	75
4.9.4. Resource reclamation	75

4.9.5. Additional configuration and troubleshooting	76
4.10. RED HAT OPENSIFT DATA FOUNDATION	77
4.11. PERSISTENT STORAGE USING VMWARE VSPHERE VOLUMES	77
4.11.1. Dynamically provisioning VMware vSphere volumes using the UI	78
4.11.2. Dynamically provisioning VMware vSphere volumes using the CLI	79
4.11.3. Statically provisioning VMware vSphere volumes	80
4.11.3.1. Formatting VMware vSphere volumes	82
CHAPTER 5. PERSISTENT STORAGE USING LOCAL STORAGE	83
5.1. LOCAL STORAGE OVERVIEW	83
5.1.1. Overview of local storage options	83
5.1.1.1. Overview of HostPath Provisioner functionality	83
5.1.1.2. Overview of Local Storage Operator functionality	84
5.1.1.3. Overview of LVM Storage functionality	84
5.1.2. Comparison of LVM Storage, LSO, and HPP	84
5.1.2.1. Comparison of the support for storage types and filesystems	84
5.1.2.2. Comparison of the support for core functionalities	85
5.1.2.3. Comparison of performance and isolation capabilities	86
5.1.2.4. Comparison of the support for additional functionalities	86
5.2. PERSISTENT STORAGE USING LOCAL VOLUMES	87
5.2.1. Local Storage Operator overview	87
5.2.2. Installing the Local Storage Operator	87
5.2.2.1. Creating a local project for Local Storage Operator	88
5.2.2.2. Installing the Local Storage Operator by using the CLI	88
5.2.2.3. Installing the Local Storage Operator by using the OpenShift Container Platform web console	90
5.2.3. Provisioning local volumes by using the Local Storage Operator	90
5.2.4. Provisioning local volumes without the Local Storage Operator	94
5.2.5. Creating the local volume persistent volume claim	97
5.2.6. Attach the local claim	97
5.2.7. Automating discovery and provisioning for local storage devices	98
5.2.8. Local Storage Operator symlinks management	101
5.2.8.1. Local Storage Operator symlinks overview	101
5.2.8.2. Responding to symlinks alerts for the Local Storage Operator	102
5.2.9. Using tolerations with Local Storage Operator pods	105
5.2.10. Local Storage Operator Metrics	106
5.2.10.1. Enabling Local Storage Operator Metrics	107
5.2.11. Deleting the Local Storage Operator resources	107
5.2.11.1. Removing a local volume or local volume set	107
5.2.11.2. Uninstalling the Local Storage Operator	109
5.2.12. Additional resources	110
5.3. PERSISTENT STORAGE USING HOSTPATH	110
5.3.1. Overview of hostPath	110
5.3.2. Statically provisioning hostPath volumes	111
5.3.3. Mounting the hostPath share in a privileged pod	112
5.4. PERSISTENT STORAGE USING LOGICAL VOLUME MANAGER STORAGE	113
5.4.1. Logical Volume Manager Storage installation	113
5.4.1.1. Prerequisites to install LVM Storage	114
5.4.1.2. Installing LVM Storage by using the CLI	114
5.4.1.3. Installing LVM Storage by using the web console	116
5.4.1.4. Installing LVM Storage in a disconnected environment	117
5.4.1.5. Installing LVM Storage by using RHACM	119
5.4.2. Static and dynamic device discovery in LVM Storage	121
5.4.2.1. Static mode enforcement	123

5.4.2.2. Validation rules for device discovery policy	123
5.4.2.3. LVMCluster custom resource examples	124
5.4.2.4. LVM cluster custom resource status reporting	125
5.4.3. About the LVMCluster custom resource	125
5.4.3.1. Explanation of fields in the LVMCluster CR	127
5.4.3.2. Limitations to configure the size of the devices used in LVM Storage	131
5.4.3.3. About adding devices to a volume group	132
5.4.3.4. About removing devices and device classes from a volume group	133
5.4.3.4.1. Removing the device paths in the deviceSelector.paths field	134
5.4.3.4.2. Removing the deviceClass from the LVMCluster	134
5.4.3.5. Devices not supported by LVM Storage	134
5.4.4. Ways to create an LVMCluster custom resource	135
5.4.4.1. Reusing a volume group from the previous LVM Storage installation	136
5.4.4.2. Creating an LVMCluster CR by using the CLI	137
5.4.4.3. Creating an LVMCluster CR by using the web console	140
5.4.4.4. Creating an LVMCluster CR by using RHACM	141
5.4.5. Ways to delete an LVMCluster custom resource	143
5.4.5.1. Deleting an LVMCluster CR by using the CLI	143
5.4.5.2. Deleting an LVMCluster CR by using the web console	144
5.4.5.3. Deleting an LVMCluster CR by using RHACM	144
5.4.5.4. Deleting an LVMCluster	148
5.4.6. Provisioning storage	149
5.4.7. StorageClass customization for LVMS device classes	151
5.4.7.1. StorageClass options	151
5.4.7.2. Updating LVM cluster labels	154
5.4.7.3. Sample LVM cluster configuration with storage class option	155
5.4.7.4. Immutable fields of the storage class options	157
5.4.7.5. Behaviors not controlled by StorageClass options	158
5.4.8. Ways to scale up the storage of clusters	159
5.4.8.1. Scaling up the storage of clusters by using the CLI	159
5.4.8.2. Scaling up the storage of clusters by using the web console	160
5.4.8.3. Scaling up the storage of clusters by using RHACM	162
5.4.9. Expanding a persistent volume claim	164
5.4.10. Deleting a persistent volume claim	164
5.4.11. About volume snapshots	165
5.4.11.1. Limitations for creating volume snapshots in multi-node topology	165
5.4.11.2. Creating volume snapshots	166
5.4.11.3. Restoring volume snapshots	167
5.4.11.4. Deleting volume snapshots	168
5.4.12. About volume clones	169
5.4.12.1. Limitations for creating volume clones in multi-node topology	169
5.4.12.2. Creating volume clones	169
5.4.12.3. Deleting volume clones	170
5.4.13. Updating LVM Storage	171
5.4.14. Monitoring LVM Storage	172
5.4.15. Uninstalling LVM Storage by using the CLI	173
5.4.16. Uninstalling LVM Storage by using the web console	174
5.4.17. Uninstalling LVM Storage installed using RHACM	174
5.4.18. Downloading log files and diagnostic information using must-gather	177
5.4.19. Troubleshooting persistent storage	177
5.4.19.1. Investigating a PVC stuck in the Pending state	177
5.4.19.2. Recovering from a missing storage class	178
5.4.19.3. Recovering from node failure	179

5.4.19.4. Recovering from disk failure	180
5.4.19.5. Performing a forced clean-up	181
CHAPTER 6. USING CONTAINER STORAGE INTERFACE (CSI)	184
6.1. CONFIGURING CSI VOLUMES	184
6.1.1. CSI architecture	184
6.1.1.1. External CSI controllers	185
6.1.1.2. CSI driver daemon set	185
6.1.2. CSI drivers supported by OpenShift Container Platform	186
6.1.3. Dynamic provisioning	188
6.1.4. Example using the CSI driver	189
6.2. CSI INLINE EPHEMERAL VOLUMES	190
6.2.1. Overview of CSI inline ephemeral volumes	190
6.2.1.1. Support limitations for CSI inline ephemeral volumes	190
6.2.2. CSI Volume Admission plugin	190
6.2.2.1. Overview of CSI Volume Admission plugin	191
6.2.2.2. Pod security profile enforcement	191
6.2.2.3. Pod security profile warning	192
6.2.2.4. Pod security profile audit	192
6.2.2.5. Default behavior for the CSI Volume Admission plugin	192
6.2.3. Embedding a CSI inline ephemeral volume in the pod specification	193
6.2.4. Additional resources	193
6.3. CSI VOLUME SNAPSHOTS	193
6.3.1. Overview of CSI volume snapshots	193
6.3.2. CSI snapshot controller and sidecar	194
6.3.3. About the CSI Snapshot Controller Operator	195
6.3.3.1. Volume snapshot CRDs	195
6.3.4. Volume snapshot provisioning	196
6.3.4.1. Dynamic provisioning	196
6.3.4.2. Static provisioning	197
6.3.5. Dynamically creating a volume snapshot	197
6.3.6. Statically creating a volume snapshot	199
6.3.7. Deleting a volume snapshot	201
6.3.8. Restoring a volume snapshot	203
6.3.9. Changing the maximum number of snapshots for vSphere	204
6.3.10. Additional resources	206
6.4. CSI VOLUME GROUP SNAPSHOTS	206
6.4.1. Overview of CSI volume group snapshots	206
6.4.2. CSI volume group snapshots limitations	207
6.4.3. Creating a volume group snapshot class	208
6.4.4. Creating a volume group snapshot	208
6.4.5. Restoring a volume group snapshot	211
6.4.6. Additional resources	212
6.5. CSI VOLUME CLONING	212
6.5.1. Overview of CSI volume cloning	212
6.5.1.1. Support limitations	212
6.5.2. Provisioning a CSI volume clone	213
6.6. VOLUME POPULATORS	214
6.6.1. Volume populators overview	215
6.6.2. Volume populators creation	215
6.6.2.1. Creating CRDs for volume populators	215
6.6.2.2. Creating prepopulated volumes using volume populators	219
6.6.3. Uninstalling volume populators	221

6.7. MANAGING THE DEFAULT STORAGE CLASS	221
6.7.1. Overview	221
6.7.2. Managing the default storage class using the web console	222
6.7.3. Managing the default storage class using the CLI	223
6.7.4. Absent or multiple default storage classes	224
6.7.4.1. Multiple default storage classes	224
6.7.4.2. Absent default storage class	224
6.7.5. Changing the default storage class	224
6.8. CSI AUTOMATIC MIGRATION	225
6.8.1. Overview	226
6.8.2. Storage class implications	226
6.9. AWS ELASTIC BLOCK STORE CSI DRIVER OPERATOR	226
6.9.1. Overview of the AWS EBS CSI Driver Operator	227
6.9.2. About CSI	227
6.9.3. User-managed encryption	228
6.9.4. Support for European Sovereign Cloud (EUSC) region	228
6.10. AWS ELASTIC FILE SERVICE CSI DRIVER OPERATOR	229
6.10.1. Overview	229
6.10.2. About CSI	230
6.10.3. Setting up the AWS EFS CSI Driver Operator	230
6.10.3.1. Obtaining a role Amazon Resource Name for Security Token Service	230
6.10.3.2. Installing the AWS EFS CSI Driver Operator	232
6.10.3.3. Installing the AWS EFS CSI Driver	233
6.10.4. About the AWS EFS storage class	234
6.10.4.1. Creating the AWS EFS storage class using the console	234
6.10.4.2. Creating the AWS EFS storage class using the CLI	234
6.10.5. AWS EFS CSI cross account support	235
6.10.6. One Zone file systems	243
6.10.6.1. One Zone file systems overview	244
6.10.6.2. Setting up One Zone file systems with STS	244
6.10.7. Dynamic provisioning for Amazon Elastic File Storage	246
6.10.8. Creating static PVs with Amazon Elastic File Storage	247
6.10.9. Amazon Elastic File Storage security	247
6.10.10. AWS EFS storage CSI usage metrics	248
6.10.10.1. Enabling or disabling usage metrics using the web console	248
6.10.10.2. Enabling or disabling usage metrics using the CLI	249
6.10.11. Amazon Elastic File Storage troubleshooting	250
6.10.12. Uninstalling the AWS EFS CSI Driver Operator	251
6.10.13. Additional resources	252
6.11. AZURE DISK CSI DRIVER OPERATOR	252
6.11.1. Overview of Azure Disk CSI Driver Operator	252
6.11.2. About CSI	253
6.11.3. Creating a storage class with storage account type	253
6.11.4. Performance plus for Azure Disk	255
6.11.4.1. Overview of performance plus	255
6.11.4.2. Limitations for performance plus	255
6.11.4.3. Creating a storage class to use performance plus enhanced disks	255
6.11.4.4. Enabling performance plus by snapshot or cloning	257
6.11.5. User-managed encryption	257
6.11.6. Machine sets that deploy machines with ultra disks using PVCs	258
6.11.6.1. Creating machines with ultra disks by using machine sets	258
6.11.6.2. Troubleshooting resources for machine sets that enable ultra disks	262
6.11.6.2.1. Unable to mount a persistent volume claim backed by an ultra disk	262

6.11.7. Additional resources	262
6.12. AZURE FILE CSI DRIVER OPERATOR	262
6.12.1. Overview	262
6.12.2. About CSI	263
6.12.3. NFS support	263
6.12.4. Azure File cross-subscription support	264
6.12.4.1. Dynamic provisioning across subscriptions for Azure File	264
6.12.4.2. Static provisioning across subscriptions for Azure File by creating a PV and PVC:	267
6.12.5. Static provisioning for Azure File	269
6.13. AZURE STACK HUB CSI DRIVER OPERATOR	272
6.13.1. Overview of Azure Stack Hub	272
6.13.2. About CSI	273
6.13.3. Additional resources	273
6.14. GCP PD CSI DRIVER OPERATOR	273
6.14.1. Overview of GCP PD CSI Driver Operator	273
6.14.2. About CSI	274
6.14.3. Reducing permissions while using the GCP PD CSI Driver Operator	274
6.14.4. GCP PD CSI driver storage class parameters	275
6.14.5. C3 instance type for bare metal and N4 machines series	276
6.14.5.1. C3 and N4 instance type limitations	276
6.14.5.2. Hyperdisk-balanced high availability disks overview	277
6.14.5.3. Storage pools for hyperdisk-balanced disks overview	277
6.14.5.4. Setting up hyperdisk-balanced disks	277
6.14.6. Creating a custom-encrypted persistent volume	282
6.14.7. User-managed encryption	284
6.14.8. Volume snapshot class csi-gce-pd-vsc-images	285
6.14.9. Additional resources	286
6.15. GOOGLE CLOUD FILESTORE CSI DRIVER OPERATOR	286
6.15.1. Overview	286
6.15.2. About CSI	287
6.15.3. Installing the Google Cloud Filestore CSI Driver Operator	287
6.15.3.1. Preparing to install the Google Cloud Filestore CSI Driver Operator with Workload Identity	287
6.15.3.2. Installing the Google Cloud Filestore CSI Driver Operator	289
6.15.4. Creating a storage class for GCP Filestore Storage	291
6.15.5. NFS export options	292
6.15.6. Destroying clusters and GCP Filestore	293
6.15.7. Additional resources	294
6.16. IBM CLOUD(R) VPC BLOCK CSI DRIVER OPERATOR	294
6.16.1. Overview of IBM Cloud(R) VPC Block CSI Driver Operator	294
6.16.2. About CSI	294
6.16.3. User-managed encryption	294
6.17. IBM POWER VIRTUAL SERVER BLOCK CSI DRIVER OPERATOR	295
6.17.1. Introduction to IBM Power(R) Virtual Server Block CSI Driver Operator	296
6.17.2. IBM Power(R) Virtual Server Block CSI Driver Operator overview	296
6.17.3. About CSI	296
6.18. OPENSTACK CINDER CSI DRIVER OPERATOR	296
6.18.1. Overview	296
6.18.2. About CSI	297
6.18.3. Making OpenStack Cinder CSI the default storage class	297
6.19. OPENSTACK MANILA CSI DRIVER OPERATOR	299
6.19.1. Overview of Manila CSI Driver Operator	299
6.19.2. About CSI	299
6.19.3. Manila CSI Driver Operator limitations	300

6.19.4. Manila CSI volumes dynamic provisioning	300
6.19.4.1. Dynamically provisioning Manila CSI volumes by using the web console	301
6.19.4.2. Dynamically provisioning Manila CSI volumes by by using the CLI	302
6.19.5. Customizing Manila share access rules	303
6.20. SECRETS STORE CONTAINER STORAGE INTERFACE DRIVER OPERATOR	305
6.20.1. Overview of Secrets Store CSI Driver Operator	305
6.20.1.1. Secrets store providers	305
6.20.2. About CSI	306
6.20.3. Support for disconnected environments	306
6.20.4. Support for network policies	307
6.20.5. Installing the Secrets Store CSI driver	307
6.20.6. Installing the Secrets Store CSI driver by using the CLI	308
6.20.7. Uninstalling the Secrets Store CSI Driver Operator	309
6.20.8. Additional resources	310
6.21. CIFS/SMB CSI DRIVER OPERATOR	310
6.21.1. Overview of the CIFS/SMB Driver Operator	310
6.21.2. About CSI	311
6.21.3. Limitations of the CIFS/SMB Driver Operator	311
6.21.4. Installing the CIFS/SMB CSI Driver Operator	312
6.21.5. Installing the CIFS/SMB CSI Driver	312
6.21.6. Dynamic provisioning for CIFS/SMB	313
6.21.7. Static provisioning for CIFS/SMB	315
6.22. VMWARE VSPHERE CSI DRIVER OPERATOR	319
6.22.1. Overview of vSphere CSI Driver Operator	320
6.22.2. About CSI	320
6.22.3. vSphere CSI limitations	321
6.22.4. vSphere storage policy	321
6.22.5. ReadWriteMany vSphere volume support	321
6.22.6. VMware vSphere CSI Driver Operator requirements	322
6.22.7. Removing a third-party vSphere CSI Driver Operator	323
6.22.8. vSphere persistent disks encryption	323
6.22.8.1. Using datastore URL	324
6.22.8.2. Using tag-based placement	325
6.22.9. Multiple vCenter support for vSphere CSI	325
6.22.9.1. Configuring multiple vCenters during installation	326
6.22.10. vSphere CSI topology overview	326
6.22.10.1. vSphere CSI topology requirements	326
6.22.10.2. Creating vSphere storage topology during installation	327
6.22.10.3. Creating vSphere storage topology postinstallation	327
6.22.10.4. Creating vSphere storage topology without an infra topology	329
6.22.10.5. vSphere topology results	332
6.22.11. Changing the maximum number of snapshots for vSphere	333
6.22.12. Migrating CNS volumes between datastores for vSphere	335
6.22.12.1. Limitations	335
6.22.12.2. Additional limitations	336
6.22.13. Disabling and enabling storage on vSphere	336
6.22.13.1. Consequences of disabling and enabling storage on vSphere	336
6.22.13.2. Disabling and enabling storage on vSphere	337
6.22.14. Adding bare-metal nodes	338
6.22.15. Increasing maximum volumes per node for vSphere	338
6.22.15.1. Increasing the maximum allowable volumes per node for vSphere	339
6.22.16. Additional resources	340

CHAPTER 7. GENERIC EPHEMERAL VOLUMES	341
7.1. OVERVIEW OF GENERIC EPHEMERAL VOLUMES	341
7.2. LIFECYCLE AND PERSISTENT VOLUME CLAIMS	341
7.3. SECURITY	342
7.4. PERSISTENT VOLUME CLAIM NAMING	342
7.5. CREATING GENERIC EPHEMERAL VOLUMES	342
CHAPTER 8. EXPANDING PERSISTENT VOLUMES	344
8.1. ENABLING VOLUME EXPANSION SUPPORT	344
8.2. EXPANDING CSI VOLUMES	344
8.3. EXPANDING FLEXVOLUME WITH A SUPPORTED DRIVER	345
8.4. EXPANDING LOCAL VOLUMES	346
8.5. EXPANDING PERSISTENT VOLUME CLAIMS (PVCS) WITH A FILE SYSTEM	346
8.6. RECOVERING FROM FAILURE WHEN EXPANDING VOLUMES	347
8.7. VIEWING THE STATUS OF VOLUME RESIZE	347
8.8. ADDITIONAL RESOURCES	348
CHAPTER 9. DYNAMIC PROVISIONING	349
9.1. ABOUT DYNAMIC PROVISIONING	349
9.2. AVAILABLE DYNAMIC PROVISIONING PLUGINS	349
9.3. DEFINING A STORAGE CLASS	350
9.3.1. Basic StorageClass object definition	351
9.3.2. RHOSP Cinder StorageObject object definition	351
9.3.3. RHOSP Manila Container Storage Interface (CSI) object definition	352
9.3.4. AWS Elastic Block Store (EBS) StorageObject object definition	352
9.3.5. Azure Disk StorageClass object definition	353
9.3.6. Azure File object definition	354
9.3.6.1. Considerations when using Azure File	355
9.3.7. GCE PersistentDisk (gcePD) object definition	356
9.3.8. VMware vSphere object definition	357
9.4. SETTING THE DEFAULT STORAGE CLASS	357
9.5. CHANGING THE DEFAULT STORAGE CLASS	358
CHAPTER 10. DETACH VOLUMES AFTER NON-GRACEFUL NODE SHUTDOWN	360
10.1. OVERVIEW	360
10.2. ADDING AN OUT-OF-SERVICE TAINT MANUALLY FOR AUTOMATIC VOLUME DETACHMENT	360

CHAPTER 1. OPENSIFT CONTAINER PLATFORM STORAGE OVERVIEW

OpenShift Container Platform supports multiple types of storage, both for on-premise and cloud providers. You can manage container storage for persistent and non-persistent data in an OpenShift Container Platform cluster.

1.1. GLOSSARY OF COMMON TERMS FOR OPENSIFT CONTAINER PLATFORM STORAGE

This glossary defines common terms that are used in the storage content.

Access modes

Volume access modes describe volume capabilities. You can use access modes to match persistent volume claim (PVC) and persistent volume (PV). The following are the examples of access modes:

- `ReadWriteOnce (RWO)`
- `ReadOnlyMany (ROX)`
- `ReadWriteMany (RWX)`
- `ReadWriteOncePod (RWOP)`

Cinder

The Block Storage service for Red Hat OpenStack Platform (RHOSP) which manages the administration, security, and scheduling of all volumes.

Config map

A config map provides a way to inject configuration data into pods. You can reference the data stored in a config map in a volume of type **ConfigMap**. Applications running in a pod can use this data.

Container Storage Interface (CSI)

An API specification for the management of container storage across different container orchestration (CO) systems.

Dynamic Provisioning

The framework allows you to create storage volumes on-demand, eliminating the need for cluster administrators to pre-provision persistent storage.

Ephemeral storage

Pods and containers can require temporary or transient local storage for their operation. The lifetime of this ephemeral storage does not extend beyond the life of the individual pod, and this ephemeral storage cannot be shared across pods.

Fiber channel

A networking technology that is used to transfer data among data centers, computer servers, switches and storage.

FlexVolume

FlexVolume is an out-of-tree plugin interface that uses an exec-based model to interface with storage drivers. You must install the FlexVolume driver binaries in a pre-defined volume plugin path on each node and in some cases the control plane nodes.

fsGroup

The fsGroup defines a file system group ID of a pod.

iSCSI

Internet Small Computer Systems Interface (iSCSI) is an Internet Protocol-based storage networking standard for linking data storage facilities. An iSCSI volume allows an existing iSCSI (SCSI over IP) volume to be mounted into your Pod.

hostPath

A hostPath volume in an OpenShift Container Platform cluster mounts a file or directory from the host node's filesystem into your pod.

KMS key

The Key Management Service (KMS) helps you achieve the required level of encryption of your data across different services. you can use the KMS key to encrypt, decrypt, and re-encrypt data.

Local volumes

A local volume represents a mounted local storage device such as a disk, partition or directory.

Nested mount points

A nested mount point is a mount point that attempts to use a mount point created by a previous volume.

Example pod definition with nested mount points

```
kind: Pod
apiVersion: v1
metadata:
  name: webapp
  labels:
    name: webapp
spec:
  containers:
    - name: webapp
      image: nginx
      ports:
        - containerPort: 80
          name: "http-server"
      volumeMounts:
        - mountPath: /mnt/web
          name: web
        - mountPath: /mnt/web/redis 1
          name: redis
  volumes:
    - name: redis
      persistentVolumeClaim:
        claimName: "redis"
    - name: web
      persistentVolumeClaim:
        claimName: "web"
```

1 Nested mount point

Do *not* use nested mount points because OpenShift Container Platform does not guarantee the order in which mount points are created. Such usage is prone to race conditions and undefined behavior.

NFS

A Network File System (NFS) that allows remote hosts to mount file systems over a network and interact with those file systems as though they are mounted locally. This enables system administrators to consolidate resources onto centralized servers on the network.

OpenShift Data Foundation

A provider of agnostic persistent storage for OpenShift Container Platform supporting file, block, and object storage, either in-house or in hybrid clouds

Persistent storage

Pods and containers can require permanent storage for their operation. OpenShift Container Platform uses the Kubernetes persistent volume (PV) framework to allow cluster administrators to provision persistent storage for a cluster. Developers can use PVC to request PV resources without having specific knowledge of the underlying storage infrastructure.

Persistent volumes (PV)

OpenShift Container Platform uses the Kubernetes persistent volume (PV) framework to allow cluster administrators to provision persistent storage for a cluster. Developers can use PVC to request PV resources without having specific knowledge of the underlying storage infrastructure.

Persistent volume claims (PVCs)

You can use a PVC to mount a PersistentVolume into a Pod. You can access the storage without knowing the details of the cloud environment.

Pod

One or more containers with shared resources, such as volume and IP addresses, running in your OpenShift Container Platform cluster. A pod is the smallest compute unit defined, deployed, and managed.

Reclaim policy

A policy that tells the cluster what to do with the volume after it is released. A volume's reclaim policy can be **Retain**, **Recycle**, or **Delete**.

Role-based access control (RBAC)

Role-based access control (RBAC) is a method of regulating access to computer or network resources based on the roles of individual users within your organization.

Stateless applications

A stateless application is an application program that does not save client data generated in one session for use in the next session with that client.

Stateful applications

A stateful application is an application program that saves data to persistent disk storage. A server, client, and applications can use a persistent disk storage. You can use the **Statefulset** object in OpenShift Container Platform to manage the deployment and scaling of a set of Pods, and provides guarantee about the ordering and uniqueness of these Pods.

Static provisioning

A cluster administrator creates a number of PVs. PVs contain the details of storage. PVs exist in the Kubernetes API and are available for consumption.

Storage

OpenShift Container Platform supports many types of storage, both for on-premise and cloud providers. You can manage container storage for persistent and non-persistent data in an OpenShift Container Platform cluster.

Storage class

A storage class provides a way for administrators to describe the classes of storage they offer. Different classes might map to quality of service levels, backup policies, arbitrary policies determined by the cluster administrators.

VMware vSphere's Virtual Machine Disk (VMDK) volumes

Virtual Machine Disk (VMDK) is a file format that describes containers for virtual hard disk drives that is used in virtual machines.

1.2. STORAGE TYPES

OpenShift Container Platform storage is broadly classified into two categories, namely ephemeral storage and persistent storage.

1.2.1. Ephemeral storage

Pods and containers are ephemeral or transient in nature and designed for stateless applications. Ephemeral storage allows administrators and developers to better manage the local storage for some of their operations. For more information about ephemeral storage overview, types, and management, see [Understanding ephemeral storage](#).

1.2.2. Persistent storage

Stateful applications deployed in containers require persistent storage. OpenShift Container Platform uses a pre-provisioned storage framework called persistent volumes (PV) to allow cluster administrators to provision persistent storage. The data inside these volumes can exist beyond the lifecycle of an individual pod. Developers can use persistent volume claims (PVCs) to request storage requirements. For more information about persistent storage overview, configuration, and lifecycle, see [Understanding persistent storage](#).

1.3. CONTAINER STORAGE INTERFACE (CSI)

CSI is an API specification for the management of container storage across different container orchestration (CO) systems. You can manage the storage volumes within the container native environments, without having specific knowledge of the underlying storage infrastructure. With the CSI, storage works uniformly across different container orchestration systems, regardless of the storage vendors you are using. For more information about CSI, see [Using Container Storage Interface \(CSI\)](#).

1.4. DYNAMIC PROVISIONING

Dynamic Provisioning allows you to create storage volumes on-demand, eliminating the need for cluster administrators to pre-provision storage. For more information about dynamic provisioning, see [Dynamic provisioning](#).

CHAPTER 2. UNDERSTANDING EPHEMERAL STORAGE

Ephemeral storage provides temporary per-pod storage for scratch data, caches, and logs that do not persist beyond the pod's lifetime. Understanding different ephemeral storage types and resource management helps you choose options for stateless workloads while preventing node storage exhaustion.

2.1. OVERVIEW OF EPHEMERAL STORAGE

Use ephemeral storage to provide temporary local storage for stateless applications that only need data for the duration of the pod lifecycle, such as caches, scratch files, and logs that do not need to persist after the pod terminates.

Both developers and administrators can use this feature.

Pods and containers can require ephemeral or transient local storage for their operation. The lifetime of this ephemeral storage does not extend beyond the life of the individual pod, and this ephemeral storage cannot be shared across pods.

Issues related to the lack of local storage accounting and isolation include the following:

- Pods cannot detect how much local storage is available to them.
- Pods cannot request guaranteed local storage.
- Local storage is a best-effort resource.
- Pods can be evicted due to other pods filling the local storage, after which new pods are not admitted until sufficient storage is reclaimed.

Unlike persistent volumes, ephemeral storage is unstructured and the space is shared between all pods running on a node, in addition to other uses by the system, the container runtime, and OpenShift Container Platform. The ephemeral storage framework allows pods to specify their transient local storage needs. It also allows OpenShift Container Platform to schedule pods where appropriate, and to protect the node against excessive use of local storage.

While the ephemeral storage framework allows administrators and developers to better manage local storage, I/O throughput and latency are not directly affected.

2.2. TYPES OF EPHEMERAL STORAGE

Provision ephemeral local storage by creating the primary partition using either root or runtime methods. Choose the method that aligns with your node configuration to ensure temporary storage is available for your workloads.

Root

This partition holds the kubelet root directory, `/var/lib/kubelet/` by default, and `/var/log/` directory. This partition can be shared between user pods, the operating system, and Kubernetes system daemons. This partition can be consumed by pods through **EmptyDir** volumes, container logs, image layers, and container-writable layers. Kubelet manages shared access and isolation of this partition. This partition is ephemeral, and applications cannot expect any performance SLAs, such as disk IOPS, from this partition.

Runtime

This is an optional partition that runtimes can use for overlay file systems. OpenShift Container

Platform attempts to identify and provide shared access along with isolation to this partition. Container image layers and writable layers are stored here. If the runtime partition exists, the **root** partition does not hold any image layer or other writable storage.

2.3. EPHEMERAL STORAGE MANAGEMENT OVERVIEW

Cluster administrators can manage ephemeral storage within a project by setting quotas that define limit ranges and request counts for all pods in a non-terminal state. Developers can also set requests and limits on this resource at the pod and container level.

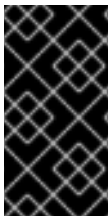
You can manage local ephemeral storage by specifying requests and limits. Each container in a pod can specify the following:

- **`spec.containers[].resources.limits.ephemeral-storage`**
- **`spec.containers[].resources.requests.ephemeral-storage`**

2.3.1. Ephemeral storage management limits and requests

Express ephemeral storage limits and requests using byte quantities with suffixes like G, M, K or power-of-two equivalents Gi, Mi, Ki. Pod-level limits aggregate all container limits plus **emptyDir** volumes, enabling proper scheduling and preventing pods from exhausting node storage.

Limits and requests for ephemeral storage are measured in byte quantities. You can express storage as a plain integer or as a fixed-point number by using one of these suffixes: E, P, T, G, M, K. You can also use the power-of-two equivalents: Ei, Pi, Ti, Gi, Mi, Ki. For example, the following quantities all represent approximately the same value: 128974848, 129e6, 129M, and 123Mi. Pod-level limits aggregate all container limits plus **emptyDir** volumes, enabling proper scheduling and preventing pods from exhausting node storage.



IMPORTANT

The suffixes for each byte quantity are case-sensitive. Be sure to use the correct case. Use the case-sensitive "M", such as used in "400M", to set the request at 400 megabytes. Use the case-sensitive "400Mi" to request 400 mebibytes. If you specify "400m" of ephemeral storage, the storage request is only 0.4 bytes.

The following example configuration file shows a pod with two containers:

- Each container requests 2GiB of local ephemeral storage.
- Each container has a limit of 4GiB of local ephemeral storage.
- At the pod level, kubelet works out an overall pod storage limit by adding up the limits of all the containers in that pod.
 - In this case, the total storage usage at the pod level is the sum of the disk usage from all containers plus the **emptyDir** volumes of a pod.
 - Therefore, the pod has a request of 4GiB of local ephemeral storage, and a limit of 8GiB of local ephemeral storage.

Example ephemeral storage configuration with quotas and limits

apiVersion: v1

```

kind: Pod
metadata:
  name: frontend
spec:
  containers:
  - name: app
    image: images.my-company.example/app:v4
    resources:
      requests:
        ephemeral-storage: "2Gi"
      limits:
        ephemeral-storage: "4Gi"
    volumeMounts:
      - name: ephemeral
        mountPath: "/tmp"
  - name: log-aggregator
    image: images.my-company.example/log-aggregator:v6
    resources:
      requests:
        ephemeral-storage: "2Gi"
      limits:
        ephemeral-storage: "4Gi"
    volumeMounts:
      - name: ephemeral
        mountPath: "/tmp"
  volumes:
  - name: ephemeral
    emptyDir: {}

```

- **spec.containers.name.resources.requests.ephemeral-storage:** Specifies the container request for local ephemeral storage.
- **spec.containers.name.resources.limits.ephemeral-storage:** Specifies the container limit for local ephemeral storage.

Additional resources

- [Resources managed by quotas](#)

2.3.2. Ephemeral storage management configuration affects pod scheduling and eviction

Configure ephemeral storage requests and limits in the pod spec to control how the scheduler places pods on nodes and when kubelet evicts pods that exceed their allocated storage.

- First, the scheduler ensures that the sum of the resource requests of the scheduled containers is less than the capacity of the node. In this case, the pod can be assigned to a node only if the node's available ephemeral storage (allocatable resource) is more than 4GiB.
- Second, at the container level, because the first container sets a resource limit, kubelet eviction manager measures the disk usage of this container and evicts the pod if the storage usage of the container exceeds its limit (4GiB). The kubelet eviction manager also marks the pod for eviction if the total usage exceeds the overall pod storage limit (8GiB).

2.4. MONITORING EPHEMERAL STORAGE

Monitor ephemeral storage usage with the **/bin/df** utility to track disk space consumption on **/var/lib/kubelet** and **/var/lib/containers**. Regular monitoring helps you identify storage-hungry workloads and adjust resource limits before kubelet evicts pods due to storage exhaustion.

When you use the **df** command, the available space for only **/var/lib/kubelet** is shown if **/var/lib/containers** is placed on a separate disk by the cluster administrator.

You can use **/bin/df** as a tool to monitor ephemeral storage usage on the volume where ephemeral container data is located, which is **/var/lib/kubelet** and **/var/lib/containers**. The available space for only **/var/lib/kubelet** is shown when you use the **df** command if **/var/lib/containers** is placed on a separate disk by the cluster administrator.

Procedure

- To show the human-readable values of used and available space in **/var/lib**, run the following command:

```
$ df -h /var/lib
```

The output shows the ephemeral storage usage in **/var/lib**:

Example output

```
Filesystem Size Used Avail Use% Mounted on
/dev/disk/by-partuuid/4cd1448a-01 69G 32G 34G 49% /
```

CHAPTER 3. UNDERSTANDING PERSISTENT STORAGE

Persistent storage decouples data from pod lifecycles, allowing stateful applications to retain data across restarts and failures. Administrators provision persistent volumes (PVs), and developers create persistent volume claims (PVCs) to request storage without infrastructure knowledge.

3.1. PERSISTENT STORAGE OVERVIEW

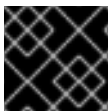
Persistent volumes (PVs) and persistent volume claims (PVCs) separate storage provisioning from consumption, so that administrators can manage storage resources while developers request capacity and access modes independently.

Managing storage is a distinct problem from managing compute resources. OpenShift Container Platform uses the Kubernetes persistent volume (PV) framework to allow cluster administrators to provision persistent storage for a cluster. Developers can use persistent volume claims (PVCs) to request PV resources without having specific knowledge of the underlying storage infrastructure.

PVCs are specific to a project, and are created and used by developers as a means to use a PV. PV resources on their own are not scoped to any single project; they can be shared across the entire OpenShift Container Platform cluster and claimed from any project. After a PV is bound to a PVC, that PV cannot then be bound to additional PVCs. This has the effect of scoping a bound PV to a single namespace, that of the binding project.

PVs are defined by a **PersistentVolume** API object, which represents a piece of existing storage in the cluster that was either statically provisioned by the cluster administrator or dynamically provisioned using a **StorageClass** object. It is a resource in the cluster in the same way that a node is a cluster resource.

PVs are volume plugins similar to **Volumes**, but have a lifecycle that is independent of any individual pod that uses the PV. PV objects capture the details of the implementation of the storage, be that NFS, iSCSI, or a cloud-provider-specific storage system.



IMPORTANT

High availability of storage in the infrastructure is left to the underlying storage provider.

PVCs are defined by a **PersistentVolumeClaim** API object, which represents a request for storage by a developer. It is similar to a pod in that pods consume node resources and PVCs consume PV resources. For example, pods can request specific levels of resources, such as CPU and memory, while PVCs can request specific storage capacity and access modes. For example, they can be mounted once read/write or many times read-only.

3.2. LIFECYCLE OF A VOLUME AND CLAIM

The persistent volume (PV) lifecycle follows five phases: provision, bind, use, release, and reclaim. Each phase has behaviors that affect storage availability and data retention. Understanding the lifecycle helps you choose reclaim policies, troubleshoot binding failures, and prevent data loss.

PVs are resources in the cluster. Persistent volume claims (PVCs) are requests for those resources and also act as claim checks to the resource.

The interaction between PVs and PVCs has the following lifecycle.

3.2.1. Provision storage

In response to requests from a developer defined in a PVC, a cluster administrator configures one or more dynamic provisioners that provision storage and a matching PV.

Alternatively, a cluster administrator can create a number of PVs in advance that carry the details of the real storage that is available for use. PVs exist in the API and are available for use.

3.2.2. Bind claims

When you create a PVC, you request a specific amount of storage, specify the required access mode, and create a storage class to describe and classify the storage. The control loop in the master watches for new PVCs and binds the new PVC to an appropriate PV. If an appropriate PV does not exist, a provisioner for the storage class creates one.

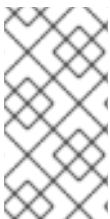
The size of all PVs might exceed your PVC size. This is especially true with manually provisioned PVs. To minimize the excess, OpenShift Container Platform binds to the smallest PV that matches all other criteria.

Claims remain unbound indefinitely if a matching volume does not exist or cannot be created with any available provisioner servicing a storage class. Claims are bound as matching volumes become available. For example, a cluster with many manually provisioned 50Gi volumes would not match a PVC requesting 100Gi. The PVC can be bound when a 100Gi PV is added to the cluster.

3.2.3. Use pods and claimed PVs

Pods use claims as volumes. The cluster inspects the claim to find the bound volume and mounts that volume for a pod. For those volumes that support multiple access modes, you must specify which mode applies when you use the claim as a volume in a pod.

After you have a claim, and that claim is bound, the bound PV belongs to you for as long as you need it. You can schedule pods and access claimed PVs by including **persistentVolumeClaim** in the pod's volumes block.



NOTE

If you attach persistent volumes that have high file counts to pods, those pods can fail or can take a long time to start. For more information, see the Red Hat Knowledgebase article "When using Persistent Volumes with high file counts in OpenShift, why do pods fail to start or take an excessive amount of time to achieve "Ready" state?".

3.2.4. Storage Object in Use Protection

The Storage Object in Use Protection feature ensures that PVCs in active use by a pod and PVs that are bound to PVCs are not removed from the system, as this can result in data loss.

Storage Object in Use Protection is enabled by default.



NOTE

A PVC is in active use by a pod when a **Pod** object exists that uses the PVC.

If a user deletes a PVC that is in active use by a pod, the PVC is not removed immediately. PVC removal is postponed until the PVC is no longer actively used by any pods. Also, if a cluster admin deletes a PV

that is bound to a PVC, the PV is not removed immediately. PV removal is postponed until the PV is no longer bound to a PVC.

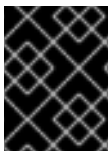
3.2.5. Release a persistent volume

When you are finished with a volume, you can delete the PVC object from the API, which allows reclamation of the resource. The volume is considered released when the claim is deleted, but it is not yet available for another claim. The previous claimant's data remains on the volume and must be handled according to policy.

3.2.6. Reclaim policy for persistent volumes

The reclaim policy of a persistent volume tells the cluster what to do with the volume after it is released. A volume's reclaim policy can be **Retain**, **Recycle**, or **Delete**.

- **Retain** reclaim policy allows manual reclamation of the resource for those volume plugins that support it.
- **Recycle** reclaim policy recycles the volume back into the pool of unbound persistent volumes once it is released from its claim.



IMPORTANT

The **Recycle** reclaim policy is deprecated in OpenShift Container Platform 4. Dynamic provisioning is recommended for equivalent and better functionality.

- **Delete** reclaim policy deletes both the **PersistentVolume** object from OpenShift Container Platform and the associated storage asset in external infrastructure, such as Amazon Elastic Block Store (Amazon EBS) or VMware vSphere.



NOTE

Dynamically provisioned volumes are always deleted.

3.2.7. Reclaiming a persistent volume manually

Manually reclaim released persistent volumes (PVs) to make them available for new claims or to properly clean up storage assets.

When a persistent volume claim (PVC) is deleted, the persistent volume (PV) still exists and is considered "released". However, the PV is not yet available for another claim because the data of the previous claimant remains on the volume.

Procedure

1. Delete the persistent volume (PV) by running the following command:

```
$ oc delete pv <pv_name>
```

The associated storage asset in the external infrastructure, such as an AWS EBS, GCE PD, Azure Disk, or Cinder volume, still exists after the PV is deleted.

2. Clean up the data on the associated storage asset.

3. Delete the associated storage asset. Alternately, to reuse the same storage asset, create a new PV with the storage asset definition.

Result

The reclaimed PV is now available for use by another PVC.

3.2.8. Changing the reclaim policy of a persistent volume

Change a persistent volume's reclaim policy to control whether storage is automatically deleted or retained when claims are removed. Switching from Delete to Retain protects data from accidental loss, while changing to Delete enables automatic cleanup of unused volumes.

Procedure

1. List the persistent volumes in your cluster:

```
$ oc get pv
```

Example output

NAME		CAPACITY	ACCESSMODES	RECLAIMPOLICY	STATUS
CLAIM	STORAGECLASS	REASON	AGE		
pvc-b6efd8da-b7b5-11e6-9d58-0ed433a7dd94		4Gi	RWO	Delete	Bound
default/claim1	manual	10s			
pvc-b95650f8-b7b5-11e6-9d58-0ed433a7dd94		4Gi	RWO	Delete	Bound
default/claim2	manual	6s			
pvc-bb3ca71d-b7b5-11e6-9d58-0ed433a7dd94		4Gi	RWO	Delete	Bound
default/claim3	manual	3s			

2. Choose one of your persistent volumes and change its reclaim policy:

```
$ oc patch pv <your-pv-name> -p '{"spec":{"persistentVolumeReclaimPolicy":"Retain"}}'
```

3. Verify that your chosen persistent volume has the right policy:

```
$ oc get pv
```

Example output

NAME		CAPACITY	ACCESSMODES	RECLAIMPOLICY	STATUS
CLAIM	STORAGECLASS	REASON	AGE		
pvc-b6efd8da-b7b5-11e6-9d58-0ed433a7dd94		4Gi	RWO	Delete	Bound
default/claim1	manual	10s			
pvc-b95650f8-b7b5-11e6-9d58-0ed433a7dd94		4Gi	RWO	Delete	Bound
default/claim2	manual	6s			
pvc-bb3ca71d-b7b5-11e6-9d58-0ed433a7dd94		4Gi	RWO	Retain	Bound
default/claim3	manual	3s			

In the preceding output, the volume bound to claim **default/claim3** now has a **Retain** reclaim policy. The volume will not be automatically deleted when a user deletes claim **default/claim3**.

Additional resources

- [When using Persistent Volumes with high file counts in OpenShift, why do pods fail to start or take an excessive amount of time to achieve "Ready" state? \(Red Hat Knowledgebase\)](#)

3.3. PERSISTENT VOLUMES

Configure persistent volumes (PVs) with capacity, access modes, mount options, and reclaim policies to manage cluster-wide storage resources across their lifecycle phases.

Each storage backend supports different access mode combinations, and volumes transition through phases (Available, Bound, Released, Failed) affecting claim availability.

Each PV contains a **spec** and **status**, which is the specification and status of the volume, for example:

Example PersistentVolume object definition

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv0001
spec:
  capacity:
    storage: 5Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  ...
status:
  ...
```

- **metadata.name:** Specifies the name of the persistent volume.
- **spec.storage:** Specifies the amount of storage available to the volume.
- **spec.accessModes:** Specifies the access mode, defining the read/write and mount permissions.
- **spec.persistentVolumeReclaimPolicy:** Specifies the reclaim policy, indicating how the resource should be handled once it is released.

You can view the name of a PVC that is bound to a PV by running the following command:

```
$ oc get pv <pv_name> -o jsonpath='{.spec.claimRef.name}'
```

3.3.1. Types of PVs

+ OpenShift Container Platform supports the following persistent volume plugins:

- AWS Elastic Block Store (EBS), which is installed by default.
- AWS Elastic File Store (EFS)
- Azure Disk
- Azure File

- Cinder
- Fibre Channel
- GCP Persistent Disk
- GCP Filestore
- IBM Power Virtual Server Block
- IBM Cloud® VPC Block
- HostPath
- iSCSI
- Local volume
- LVM Storage
- NFS
- OpenStack Manila
- Red Hat OpenShift Data Foundation
- CIFS/SMB
- VMware vSphere

3.3.2. Capacity

Generally, a persistent volume (PV) has a specific storage capacity. This is set by using the **capacity** attribute of the PV.

Currently, storage capacity is the only resource that can be set or requested. Future attributes may include IOPS, throughput, and so on.

3.3.3. Access modes

A persistent volume can be mounted on a host in any way supported by the resource provider. Providers have different capabilities and each PV's access modes are set to the specific modes supported by that particular volume. For example, NFS can support multiple read/write clients, but a specific NFS PV might be exported on the server as read-only. Each PV gets its own set of access modes describing that specific PV's capabilities.

Claims are matched to volumes with similar access modes. The only two matching criteria are access modes and size. A claim's access modes represent a request. Therefore, you might be granted more, but never less. For example, if a claim requests RWO, but the only volume available is an NFS PV (RWO+ROX+RWX), the claim would then match NFS because it supports RWO.

Direct matches are always attempted first. The volume's modes must match or contain more modes than you requested. The size must be greater than or equal to what is expected. If two types of volumes, such as NFS and iSCSI, have the same set of access modes, either of them can match a claim with those modes. There is no ordering between types of volumes and no way to choose one type over another.

All volumes with the same modes are grouped, and then sorted by size, smallest to largest. The binder gets the group with matching modes and iterates over each, in size order, until one size matches.



IMPORTANT

Volume access modes describe volume capabilities. They are not enforced constraints. The storage provider is responsible for runtime errors resulting from invalid use of the resource. Errors in the provider show up at runtime as mount errors.

For example, NFS offers **ReadWriteOnce** access mode. If you want to use the volume's ROX capability, mark the claims as **ReadOnlyMany**.

iSCSI and Fibre Channel volumes do not currently have any fencing mechanisms. You must ensure the volumes are only used by one node at a time. In certain situations, such as draining a node, the volumes can be used simultaneously by two nodes. Before draining the node, delete the pods that use the volumes.

The following table lists the access modes:

Table 3.1. Access modes

Access Mode	CLI abbreviation	Description
ReadWriteOnce	RWO	The volume can be mounted as read/write by a single node.
ReadWriteOncePod	RWOP	The volume can be mounted as read/write by a single pod on a single node.
ReadOnlyMany	ROX	The volume can be mounted as read-only by many nodes.
ReadWriteMany	RWX	The volume can be mounted as read/write by many nodes.

Table 3.2. Supported access modes for persistent volumes

Volume plugin	ReadWriteOnce [1]	ReadWriteOncePod	ReadOnlyMany	ReadWriteMany
AWS EBS [2]	■	■		
AWS EFS	■	■	■	■
Azure File	■	■	■	■
Azure Disk	■	■		
CIFS/SMB	■	■	■	■

Volume plugin	ReadWriteOnce [1]	ReadWriteOncePod	ReadOnlyMany	ReadWriteMany
Cinder	■	■		
Fibre Channel	■	■	■	■ [3]
GCP Persistent Disk	■ [4]	■	■	■ [4]
GCP Filestore	■	■	■	■
HostPath	■	■		
IBM Power Virtual Server Disk	■	■	■	■
IBM Cloud® VPC Disk	■	■		
iSCSI	■	■	■	■ [3]
Local volume	■	■		
LVM Storage	■	■		
NFS	■	■	■	■
OpenStack Manila		■		■
Red Hat OpenShift Data Foundation	■	■		■
VMware vSphere	■	■		■ [5]

1. ReadWriteOnce (RWO) volumes cannot be mounted on multiple nodes. If a node fails, the system does not allow the attached RWO volume to be mounted on a new node because it is already assigned to the failed node. If you encounter a multi-attach error message as a result, force delete the pod on a shutdown or crashed node to avoid data loss in critical workloads, such as when dynamic persistent volumes are attached.

2. Use a recreate deployment strategy for pods that rely on AWS EBS.
3. Only raw block volumes support the **ReadWriteMany** (RWX) access mode for Fibre Channel and iSCSI. For more information, see "Block volume support".
4. For GCP hyperdisk-balanced disks:
 - The supported access modes are:
 - **ReadWriteOnce**
 - **ReadWriteMany**
 - Cloning and snapshotting is disabled for disks with **ReadWriteMany** access mode enabled.
 - You can attach a single hyperdisk-balanced disk volume in **ReadWriteMany** to a maximum of 8 instances.
 - You can only resize a disk in **ReadWriteMany** if you detach the disk from all instances.
 - For additional limitations, see Google Cloud documentation "GCP hyperdisk-balanced disk additional limitations".
5. If the underlying vSphere environment supports the vSAN file service, the vSphere Container Storage Interface (CSI) Driver Operator installed by OpenShift Container Platform supports provisioning of ReadWriteMany (RWX) volumes. If you do not have vSAN file service configured, and you request RWX, the volume fails to get created and an error is logged. For more information, see "VMware vSphere CSI Driver Operator".

3.3.4. Phase

Volumes can be found in one of the following phases:

+ .Volume phases

Phase	Description
Available	A free resource not yet bound to a claim.
Bound	The volume is bound to a claim.
Released	The claim was deleted, but the resource is not yet reclaimed by the cluster.
Failed	The volume has failed its automatic reclamation.

Last phase transition time

The **LastPhaseTransitionTime** field has a timestamp that updates every time a persistent volume (PV) transitions to a different phase (**pv.Status.Phase**). To find the time of the last phase transition for a PV, run the following command:

```
$ oc get pv <pv_name> -o json | jq '.status.lastPhaseTransitionTime'
```

For `'status.lastPhaseTransitionTime'` specify the name of the PV that you want to see the last phase transition.

Mount options

You can specify mount options while mounting a PV by using the attribute **mountOptions**.

For example:

Mount options example

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv0001
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  mountOptions:
    - nfsvers=4.1
  nfs:
    path: /tmp
    server: 172.17.0.2
  persistentVolumeReclaimPolicy: Retain
  claimRef:
    name: claim1
    namespace: default
```

spec.mountOptions: Specified mount options are used while mounting the PV to the disk.

The following PV types support mount options:

- AWS Elastic Block Store (EBS)
- AWS Elastic File Storage (EFS)
- Azure Disk
- Azure File
- Cinder
- GCE Persistent Disk
- iSCSI
- Local volume
- NFS
- Red Hat OpenShift Data Foundation (Ceph RBD only)
- CIFS/SMB
- VMware vSphere

**NOTE**

Fibre Channel and HostPath PVs do not support mount options.

Additional resources

- [Block volume support](#)
- [GCP hyperdisk-balanced disk additional limitations](#)
- [VMware vSphere CSI Driver Operator](#)

3.4. PERSISTENT VOLUME CLAIMS

Persistent volume claims (PVCs) are namespace-scoped storage requests that specify capacity, access modes, and storage class requirements. Each claim contains a `spec` field defining the storage request parameters and a `status` field tracking the binding state and current conditions of the claim.

Example `PersistentVolumeClaim` object definition

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: myclaim
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 8Gi
  storageClassName: gold
status:
# ...
```

- **apiVersion:** Specifies the name of the PVC.
- **spec.accessModes:** Specifies the access mode, defining the read/write and mount permissions.
- **requests.storage:** Specifies the amount of storage available to the PVC.
- **storageClassName:** Specifies the name of the **StorageClass** required by the claim.

3.4.1. Volume Attributes Classes

Volume Attributes Classes enable dynamic modification of storage performance, such as IOPS and throughput, on persistent volume claims without re-provisioning or downtime. Administrators define classes representing quality-of-service levels, and users apply them for on-demand performance tuning.

3.4.1.1. Volume Attributes Classes Limitations

Volume Attributes Classes have the following limitations:

- Only supported with AWS Elastic Block Storage (EBS) and Google Cloud Platform (GCP) persistent disk (PD) Container Storage Interface (CSI).
- With GCP PD, volume modification using Volume Attributes Classes is only possible for hyperdisk-balanced disk types.
- No more than 512 parameters can be defined for a **VolumeAttributesClass**.
- The total length of the parameter's object, including its keys and values, cannot exceed 256 KiB.
- If you apply a Volume Attributes Class to a PVC, you can change the applied Volume Attributes Class for that PVC, but you cannot delete it from the PVC. To delete the Volume Attributes Class from the PVC, you must delete the PVC, and then re-create the PVC.
- Volume Attributes Class parameters cannot be edited. If you need to change Volume Attributes Class parameters, create a new Volume Attributes Class with the desired parameters, and then apply it to a PVC.

3.4.1.2. Defining Volume Attributes Classes

When defining Volume Attributes Classes, you must verify supported parameters with your storage provider and explicitly declare all parameters in every Volume Attributes Classes definition.

If there are multiple Volume Attributes Classes on one driver with different parameters sets, and you change from one Volume Attributes Classes to another, you might expect every non-defined parameter to be set back to the default. However, that does not occur. The non-defined parameters stay the same.

For example, if you have:

- Volume Attributes Class A defines throughput **125** and iops **300**.
- Volume Attributes Class B only defines iops to **400**.

If a persistent volume claim (PVC) changes its Volume Attributes Class from A to B, the resulting volume's parameters are changed to throughput **125** and iops **400**.

The metric, **openshift_cluster_storage_vac_mismatch_parameters**, helps identify mismatched parameters that might lead to the persistence issues:

- **0**: All Volume Attributes Classes define the same set of parameters.
- **1**: Mismatch detected. At least one Volume Attributes Class defines a different set of parameters than the others.

You can modify a Volume Attributes Class on a PVC at any time. However, if a Volume Attributes Class is changed multiple times in a short time period, the underlying storage provider might enforce a delay of several hours before the most recent change is fully reconciled on the disk.

The following is an example Volume Attributes Class YAML file for AWS EBS.

Example **VolumeAttributesClass** AWS EBS definition

```
apiVersion: storage.k8s.io/v1
kind: VolumeAttributesClass
metadata:
  name: silver
```

```
driverName: ebs.csi.aws.com
parameters:
  iops: "300"
  throughput: "125"
  type: io2
...
```

- **kind:** Defines object as Volumes Attributes Classes.
- **metadata.Name:** Specifies the name of the **VolumeAttributesClass**. In this example, it is **silver**.
- **driverName:** Specifies the provisioner that determines what volume plugin is used for provisioning persistent volumes (PVs). In this example, it is **ebs.csi.aws.com** for AWS EBS.
- **parameters.type:** Defines the disk type.

The following is an example Volume Attributes Class YAML file for GPC PD.

Example VolumeAttributesClass GPC PD definition

```
apiVersion: storage.k8s.io/v1
kind: VolumeAttributesClass
metadata:
  name: silver
driverName: pd.csi.storage.gke.io
parameters:
  iops: "3000"
  throughput: "150Mi"
...
```

- **kind:** Defines object as Volumes Attributes Classes.
- **metadata.Name:** Specifies the name of the **VolumeAttributesClass**. In this example, it is **silver**.
- **driverName:** Specifies the provisioner that determines what volume plugin is used for provisioning persistent volumes (PVs). In this example, it is "pd.csi.storage.gke.io" for GPC PD.

3.4.1.3. Applying a Volume Attributes Class to a PVC

Apply a Volume Attributes Class to a new or existing persistent volume claim (PVC) by setting the **volumeAttributesClassName** parameter to dynamically configure storage attributes, such as performance tiers without recreating the volume.

Procedure

- Set the PVC's **volumeAttributesClassName** parameter to the Volume Attributes Class's name:

Example

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: test-pv-claim
```

```
spec:
  ...
  volumeAttributesClassName: silver
```

Where **spec.volumeAttributesClassName** specifies using the Volume Attributes Class **silver** for this PVC.

3.4.1.4. Deleting Volume Attributes Classes

Delete a Volume Attributes Class that is no longer needed by first removing or reassigning all persistent volume claims (PVCs) that reference it, since a Volume Attributes Class cannot be deleted while it is still in use.

If you try to delete a Volume Attributes Class while it is still being used by a PVC, the command does not complete until all resources that use the Volume Attributes Class are updated to not use it.

Procedure

1. Search for PVCs that are using Volume Attributes Classes by running the following command:

```
$ oc get pvc -A -o jsonpath='{range .items[?(@.spec.volumeAttributesClassName=="<vac-name>")]}{.metadata.name}{"\n"}}{end}'
```

Where **<vac-name>** is the Volume Attributes Class name.

Example command output

```
$ mypvc
```

2. Complete one of the following steps:
 - Specify a different Volume Attributes Class name in the PVC's **volumeAttributesClassName** parameter:

Example PVC definition specifying a Volume Attributes Class

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mypvc
spec:
  ...
  volumeAttributesClassName: silver
```

Where **spec.volumeAttributesClassName** specifies a different Volume Attributes Class. In this example, **silver**.

- Delete all PVCs that specify the Volume Attributes Class by running the following command:

```
$ oc delete pvc <pvc-name>
```

Where **<pvc-name>** is the name of the PVC that you want to delete.

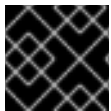
- Now that the Volume Attributes Class is no longer being used by any PVC, delete the Volume Attributes Class by running the following command:

```
$ oc delete vac <vac-name>
```

Where **<pvc-name>** is the name of the Volume Attributes Class that you want to delete.

3.4.2. Block volume support

Raw block volumes are filesystem-free storage that applications access directly for improved performance. Specify **volumeMode: Block** in persistent volumes and claims, and configure privileged containers. Storage provider support varies: static only, dynamic only, both, or none.



IMPORTANT

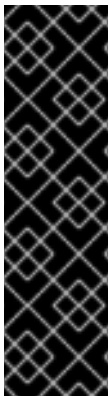
Pods using raw block volumes must be configured to allow privileged containers.

The following table displays which volume plugins support block volumes.

Table 3.3. Block volume support

Volume Plugin	Manually provisioned	Dynamically provisioned	Fully supported
Amazon Elastic Block Store (Amazon EBS)	■	■	■
Amazon Elastic File Storage (Amazon EFS)			
Azure Disk	■	■	■
Azure File			
Cinder	■	■	■
Fibre Channel	■		■
GCP	■	■	■
HostPath			
IBM Cloud Block Storage volume	■	■	■
iSCSI	■		■
Local volume	■		■
LVM Storage	■	■	■

Volume Plugin	Manually provisioned	Dynamically provisioned	Fully supported
NFS			
Red Hat OpenShift Data Foundation	■	■	■
CIFS/SMB			
VMware vSphere	■	■	■



IMPORTANT

Using any of the block volumes that can be provisioned manually, but are not provided as fully supported, is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

3.4.2.1. Block volume examples

Raw block volume examples demonstrate configurations for applications requiring direct access to block storage devices without a filesystem. This approach is commonly used by databases and other applications that need low-level storage control, bypassing traditional filesystem layers.

The examples show how to configure three essential components for block volume storage:

- **PersistentVolume** (PV) with **volumeMode: Block** to define the raw block storage resource.
- **PersistentVolumeClaim** (PVC) that requests block storage by setting **volumeMode: Block**.
- Pod specification that mounts the block device by using **volumeDevices** and **devicePath** instead of the typical **volumeMounts** and **mountPath** used for filesystem volumes.

The following reference tables show the accepted values for **volumeMode** (Filesystem is the default, Block must be explicitly set) and the binding scenarios between PVs and PVCs. Understanding these binding rules is critical. PVs and PVCs must both specify **volumeMode: Block** to bind successfully. Mismatched volume modes, such as a Block PV with a Filesystem PVC, prevent binding, which can cause pod scheduling failures.

PV example

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: block-pv
spec:
  capacity:
```

```

storage: 10Gi
accessModes:
- ReadWriteOnce
volumeMode: Block
persistentVolumeReclaimPolicy: Retain
fc:
  targetWWNs: ["50060e801049cfd1"]
  lun: 0
  readOnly: false

```

spec.volumeMode must be set to **Block** to indicate that this PV is a raw block volume.

PVC example

```

apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: block-pvc
spec:
  accessModes:
  - ReadWriteOnce
  volumeMode: Block
  resources:
    requests:
      storage: 10Gi

```

spec.volumeMode must be set to **Block** to indicate that a raw block PVC is requested.

Pod specification example

```

apiVersion: v1
kind: Pod
metadata:
  name: pod-with-block-volume
spec:
  containers:
  - name: fc-container
    image: fedora:26
    command: ["/bin/sh", "-c"]
    args: [ "tail -f /dev/null" ]
    volumeDevices:
    - name: data
      devicePath: /dev/xvda
  volumes:
  - name: data
    persistentVolumeClaim:
      claimName: block-pvc

```

- **spec.container.volumeDevices**: Instead of **volumeMounts**, this parameter is used for block devices. Only **PersistentVolumeClaim** sources can be used with raw block volumes.
- **spec.container.volumeDevices.devicePath**: Instead of **mountPath**, this parameter represents the path to the physical device where the raw block is mapped to the system.

- **spec.volumes.persistentVolumeClaim.claimName:** The volume source must be of type **persistentVolumeClaim** and must match the name of the PVC as expected.

Table 3.4. Accepted values for **volumeMode**

Value	Default
Filesystem	Yes
Block	No

Table 3.5. Binding scenarios for block volumes

PV volumeMode	PVC volumeMode	Binding result
Filesystem	Filesystem	Bind
Unspecified	Unspecified	Bind
Filesystem	Unspecified	Bind
Unspecified	Filesystem	Bind
Block	Block	Bind
Unspecified	Block	No Bind
Block	Unspecified	No Bind
Filesystem	Block	No Bind
Block	Filesystem	No Bind

**IMPORTANT**

Unspecified values result in the default value of **Filesystem**.

3.4.3. Reduce pod timeouts by using fsGroup

To reduce pod timeouts when using a storage volume with many files, configure the **fsGroup** field. By specifying this field, you can manage how file ownership and permissions are applied, preventing delays caused by the default recursive permission changes on large volumes.

This can occur because, by default, OpenShift Container Platform recursively changes ownership and permissions for the contents of each volume to match the **fsGroup** specified in the **securityContext** of the pod when that volume is mounted. For volumes with many files, checking and changing ownership

and permissions can be time consuming, slowing pod startup. You can use the **fsGroupChangePolicy** field inside a **securityContext** to control the way that OpenShift Container Platform checks and manages ownership and permissions for a volume.

fsGroupChangePolicy defines behavior for changing ownership and permission of the volume before being exposed inside a pod. This field only applies to volume types that support **fsGroup**-controlled ownership and permissions. This field has two possible values:

- **OnRootMismatch**: Only change permissions and ownership if permission and ownership of root directory does not match with expected permissions of the volume. This can help shorten the time it takes to change ownership and permission of a volume to reduce pod timeouts.
- **Always**: (Default) Always change permission and ownership of the volume when a volume is mounted.



NOTE

The **fsGroupChangePolicy** field has no effect on ephemeral volume types, such as secret, configMap, and emptydir.

You can set **fsGroupChangePolicy** at either the namespace or pod level.

3.4.3.1. Changing fsGroup at the namespace level

You can change **fsGroupChangePolicy** at the namespace level to establish a default permission-change behavior for all pods in that namespace, reducing per-pod configuration overhead.

After applying the desired setting for **fsGroupChangePolicy** at the namespace level, all subsequently created pods in that namespace inherit the setting. However, if desired, you can override the inherited **fsGroupChangePolicy** setting for individual pods. Setting **fsGroupChangePolicy** at the pod level overrides inheritance from the namespace level setting for that pod.

Prerequisites

- You are logged in to a running OpenShift Container Platform cluster with administrator privileges.
- You have access to the OpenShift Container Platform console.

Procedure

1. Select the desired namespace:
 - a. Click **Administration** > **Namespaces**.
 - b. On the **Namespaces** page, click the desired namespace. The **Namespace details** page appears.
2. Add the **fsGroupChangePolicy** label to the namespace:
 - a. On the **Namespace details** page, next to **Labels**, click **Edit**.
 - b. In the **Edit labels** dialog, add the label **storage.openshift.io/fsgroup-change-policy** and set it equal to either:

- **OnRootMismatch:** Specifies only changing permissions and ownership if the permission and the ownership of root directory does not match with expected permissions of the volume, thus helping to avoid pod timeout problems.
- **Always:** (Default) Specifies always changing permission and ownership of the volume when a volume is mounted.

c. Click **Save**.

Verification

- Start up a pod in the previously edited namespace and observe that the parameter **spec.securityContext.fsGroupChangePolicy** contains the value that you set for the namespace.

Example pod YAML file showing **fsGroupChangePolicy** setting

```
securityContext:
  seLinuxOptions:
    level: 's0:c27,c24'
  runAsNonRoot: true
  fsGroup: 1000750000
  fsGroupChangePolicy: OnRootMismatch
...
```

The value for **securityContext.fsGroupChangePolicy** is inherited from the namespace.

3.4.3.2. Changing **fsGroup** at the pod level

You can set **fsGroupChangePolicy** parameter in new or existing deployments and stateful sets, and then the pods that it manages will have this parameter value. You cannot edit **fsGroupChangePolicy** on an existing pod; however, you can set this parameter when creating a new pod.

This procedure describes how to set the **fsGroupChangePolicy** parameter in an existing deployment.

Prerequisites

- Access to the OpenShift Container Platform console.

Procedure

1. Click **Workloads > Deployments**.
2. On the **Deployment** page, click the desired deployment.
3. On the **Deployment details** page, click the **YAML** tab.
4. Edit the deployment's YAML file under **spec.template.spec.securityContext** using the following example file:

Example deployment YAML file setting **fsGroupChangePolicy**

```
...
spec:
```

```

replicas: 3
selector:
  matchLabels:
    app: my-app
template:
  metadata:
    creationTimestamp: null
  labels:
    app: my-app
  spec:
    containers:
      - name: container
        image: 'image-registry.openshift-image-registry.svc:5000/openshift/httpd:latest'
        ports:
          - containerPort: 8080
        protocol: TCP
    resources: {}
    terminationMessagePath: /dev/termination-log
    terminationMessagePolicy: File
    imagePullPolicy: Always
    restartPolicy: Always
    terminationGracePeriodSeconds: 30
    dnsPolicy: ClusterFirst
    securityContext:
      fsGroupChangePolicy: OnRootMismatch
    ...

```

spec.securityContext.fsGroupChangePolicy, OnRootMismatch, specifies skipping recursive permission change, thus helping to avoid pod timeout problems. The default value is **Always**, which always changes permission and ownership of the volume when a volume is mounted.

5. Click **Save**.

3.4.4. Reducing pod timeouts using selinuxChangePolicy

The SELinux mount option applies security contexts during mount without recursive relabeling, reducing pod startup times on volumes with many files. This optimization is enabled by default for ReadWriteOncePod volumes and will become the default for ReadWriteOnce and ReadWriteMany volumes.

SELinux (Security-Enhanced Linux) is a security mechanism that assigns security labels (contexts) to all objects (files, processes, network ports, and so on) on a system. These labels determine what a process can access. In OpenShift Container Platform, SELinux helps prevent containers from escaping and accessing the host system or other containers.

When a pod starts, the container runtime recursively relabels all files on a volume to match the pod's SELinux context. For volumes with many files, this can significantly increase pod startup times.

Mount option specifies avoiding recursive relabeling of all files by attempting to mount the volume with the correct SELinux label directly using the `-o context` mount option, thus helping to avoid pod timeout problems.

3.4.4.1. RWOP and SELinux mount option

ReadWriteOncePod (RWOP) persistent volumes use the SELinux mount feature by default.

The mount option feature is driver dependent, and enabled by default in AWS EBS , Azure Disk, GCP PD, IBM Cloud Block Storage volume, Cinder, vSphere, and Red Hat OpenShift Data Foundation. For third-party drivers, contact your storage vendor.

3.4.4.2. RWO and RWX and SELinux mount option

ReadWriteOnce (RWO) and ReadWriteMany (RWX) volumes use recursive relabeling by default.



IMPORTANT

In a future OpenShift Container Platform version, RWO and RWX volumes will use **mount option by default**.

To assist you with the upcoming move to the mount option default, OpenShift Container Platform 4.20 reports SELinux-related conflicts when creating pods, and on running pods, to make you aware of potential conflicts, and to help you resolve them. For more information about this reporting, see the Red Hat Knowledgebase article "OpenShift reports SELinux-related conflicts when creating Pods".

If you are unable to resolve the SELinux-related conflicts, you can proactively opt-out of the future move to mount option as default for selected pods or namespaces. To opt out, see "Opting out of the SELinux mount option default".

Additional resources

- [OpenShift reports SELinux-related conflicts when creating Pods \(Red Hat Knowledgebase\)](#)
- [Opting out of the SELinux mount option default](#)

3.4.4.3. Testing the RWO and RWX and SELinux mount option feature

The SELinux mount option applies the correct security context during mount without recursive relabeling, reducing pod startup times on volumes with many files. In OpenShift Container Platform 4.21, you can evaluate the mount option feature for RWO and RWX volumes as a Technology Preview feature.



IMPORTANT

RWO/RWX SELinux mount is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

Procedure

- Enable Feature Gates. For information about enabling Feature Gates, see "Enabling features using feature gates".
RWO and RWX volumes now have mount option as the default behavior.

Next step

Carefully test your applications and observe how they are using storage. For more information, see the Red Hat Knowledgebase article "OpenShift reports SELinux-related conflicts when creating Pods", and

consider opting out from using mount option if you are experiencing issues. For more information, see "Opting out of the SELinux mount option default".

Additional resources

- [OpenShift reports SELinux-related conflicts when creating Pods \(Red Hat Knowledgebase\)](#)
- [Opting out of the SELinux mount option default](#)

3.4.4.4. Opting out of the SELinux mount option default

If you want to opt out of the future move to mount option as default, you can affirmatively set the **seLinuxChangePolicy** parameter to **Recursive** at either the individual pod or namespace level.

3.4.4.4.1. Changing seLinuxChangePolicy at the namespace level

Configure **seLinuxChangePolicy** to **Recursive** at the namespace level to opt out of the SELinux mount option default for all pods in that namespace. This setting applies automatically to new pods while allowing pod-level overrides when workloads require different SELinux relabeling behavior.

Prerequisites

- Logged in to a running OpenShift Container Platform cluster with administrator privileges.
- Access to the OpenShift Container Platform console.

Procedure

1. Select the needed namespace:
 - a. Click **Administration** > **Namespaces**.
 - b. On the **Namespaces** page, click the desired namespace. The **Namespace details** page appears.
2. Add the **seLinuxChangePolicy** label to the namespace:
 - a. On the **Namespace details** page, next to **Labels**, click **Edit**.
 - b. In the **Edit labels** dialog, add the label **storage.openshift.io/selinux-change-policy=Recursive**.
This specifies recursively relabeling all files on pod volumes to the appropriate SELinux context.
 - c. Click **Save**.
3. Verify the results by starting up a pod in the previously edited namespace and observe that the parameter **spec.securityContext.seLinuxChangePolicy** is set to **Recursive**.

Example pod YAML file showing seLinuxChangePolicy setting

```
securityContext:
  seLinuxOptions:
    level: 's0:c27,c19'
  runAsNonRoot: true
```

```

fsGroup: 1000740000
seccompProfile:
  type: RuntimeDefault
seLinuxChangePolicy: Recursive
...

```

- The value for **securityContext.seLinuxChangePolicy** is inherited from the namespace.

3.4.4.4.2. Changing seLinuxChangePolicy at the pod level

Set **seLinuxChangePolicy** to **Recursive** at the pod level to override namespace defaults or opt out of the SELinux mount option default for specific workloads.

Configure the **seLinuxChangePolicy** parameter in deployment or statefulset specifications to apply it to managed pods, or set it directly when creating individual pods. You cannot modify this parameter on existing pods.

This procedure describes how to set the **seLinuxChangePolicy** parameter in an existing deployment.

Prerequisites

- Access to the OpenShift Container Platform console.

Procedure

1. Click **Workloads > Deployments**.
2. On the **Deployment** page, click the required deployment.
3. On the **Deployment details** page, click the **YAML** tab.
4. Edit the deployment's YAML file under **spec.template.spec.securityContext** similar to the following example file:

Example deployment YAML file setting seLinuxChangePolicy

```

...
securityContext:
  seLinuxChangePolicy: Recursive
...

```

- **securityContext.seLinuxChangePolicy**: When set to **Recursive**, specifies recursively relabeling all files on all pod volumes to the appropriate SELinux context.

5. Click **Save**.

3.4.5. Additional resources

- [Enabling features using feature gates](#)

CHAPTER 4. CONFIGURING PERSISTENT STORAGE

4.1. PERSISTENT STORAGE USING AWS ELASTIC BLOCK STORE

OpenShift Container Platform supports Amazon Elastic Block Store (EBS) volumes. You can provision your OpenShift Container Platform cluster with persistent storage by using Amazon EC2.

The Kubernetes persistent volume framework allows administrators to provision a cluster with persistent storage and gives users a way to request those resources without having any knowledge of the underlying infrastructure. You can dynamically provision Amazon EBS volumes. Persistent volumes are not bound to a single project or namespace; they can be shared across the OpenShift Container Platform cluster. Persistent volume claims are specific to a project or namespace and can be requested by users. You can define a KMS key to encrypt container-persistent volumes on AWS. By default, newly created clusters by using OpenShift Container Platform version 4.10 and later use gp3 storage and the AWS EBS CSI driver.



IMPORTANT

High-availability of storage in the infrastructure is left to the underlying storage provider.



IMPORTANT

OpenShift Container Platform 4.12 and later provides automatic migration for the AWS Block in-tree volume plugin to its equivalent CSI driver.

CSI automatic migration should be seamless. Migration does not change how you use all existing API objects, such as persistent volumes, persistent volume claims, and storage classes. For more information about migration, see [CSI automatic migration](#).

4.1.1. About the EBS storage class

To enable dynamic provisioning of persistent volumes, create a storage class that defines storage characteristics and allows users to automatically provision volumes on-demand.

4.1.2. Creating the persistent volume claim

You can create a persistent volume claim to dynamically provision and bind storage from a pre-configured storage class, so that your applications can consume persistent storage in OpenShift Container Platform.

Prerequisites

- Storage must exist in the underlying infrastructure before it can be mounted as a volume in OpenShift Container Platform.

Procedure

1. In the OpenShift Container Platform web console, click **Storage → Persistent Volume Claims**
2. In the persistent volume claims overview, click **Create Persistent Volume Claim**
3. Define the required options on the page that is displayed.
 - a. Select the previously-created storage class from the drop-down menu.

- b. Enter a unique name for the storage claim.
 - c. Select the access mode. This selection determines the read and write access for the storage claim.
 - d. Define the size of the storage claim.
4. Click **Create** to create the persistent volume claim and generate a persistent volume.

4.1.3. Volume format

You can use unformatted AWS volumes as persistent volumes, because OpenShift Container Platform automatically formats the device before mounting it to a container.

Before OpenShift Container Platform mounts the volume and passes it to a container, it checks that the volume contains a file system as specified by the **fsType** parameter in the persistent volume definition. If the device is not formatted with the file system, all data from the device is erased and the device is automatically formatted with the given file system.

4.1.4. Maximum number of EBS volumes on a node

By default, you can attach a maximum of 39 EBS volumes attached to one node. This limit is consistent with the AWS volume limits. The volume limit depends on the instance type.



IMPORTANT

As a cluster administrator, you must use either in-tree or Container Storage Interface (CSI) volumes and their respective storage classes, but never both volume types at the same time. The maximum attached EBS volume number is counted separately for in-tree and CSI volumes, which means you could have up to 39 EBS volumes of each type.

4.1.5. Encrypting container persistent volumes on AWS with a KMS key

You can define a KMS key to encrypt container-persistent volumes on AWS if you have explicit compliance and security guidelines when deploying to AWS.

Prerequisites

- Underlying infrastructure must contain storage.
- You must create a customer KMS key on AWS.

Procedure

1. Create a storage class:

```
$ cat << EOF | oc create -f -
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: <storage-class-name>
parameters:
  fsType: ext4
  encrypted: "true"
  kmsKeyId: keyvalue
```



```
provisioner: ebs.csi.aws.com
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer
EOF
```

where:

metadata.name

Specifies the name of the storage class.

parameters.fsType

Specifies the file system that is created on provisioned volumes.

parameters.kmsKeyId

Specifies the full Amazon Resource Name (ARN) of the key to use when encrypting the container-persistent volume. If you do not provide any key, but the **encrypted** field is set to **true**, then the default KMS key is used.

2. Create a persistent volume claim (PVC) with the storage class specifying the KMS key:

```
$ cat << EOF | oc create -f -
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mypvc
spec:
  accessModes:
    - ReadWriteOnce
  volumeMode: Filesystem
  storageClassName: <storage-class-name>
resources:
  requests:
    storage: 1Gi
EOF
```

3. Create workload containers to consume the PVC:

```
$ cat << EOF | oc create -f -
kind: Pod
metadata:
  name: mypod
spec:
  containers:
    - name: httpd
      image: quay.io/centos7/httpd-24-centos7
      ports:
        - containerPort: 80
      volumeMounts:
        - mountPath: /mnt/storage
          name: data
  volumes:
    - name: data
      persistentVolumeClaim:
        claimName: mypvc
EOF
```

4.1.6. Additional resources

- [Amazon EC2 documentation](#)
- [AWS EBS CSI driver](#)
- [CSI automatic migration](#)
- [AWS Elastic Block Store CSI Driver Operator](#)

4.2. PERSISTENT STORAGE USING AZURE

OpenShift Container Platform supports Microsoft Azure Disk volumes. You can provision your OpenShift Container Platform cluster with persistent storage by using Azure. Some familiarity with Kubernetes and Azure is assumed.

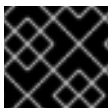
The Kubernetes persistent volume framework allows administrators to provision a cluster with persistent storage and gives users a way to request those resources without having any knowledge of the underlying infrastructure. Azure Disk volumes can be provisioned dynamically. Persistent volumes are not bound to a single project or namespace; they can be shared across the OpenShift Container Platform cluster. Persistent volume claims are specific to a project or namespace and can be requested by users.



IMPORTANT

OpenShift Container Platform 4.11 and later provides automatic migration for the Azure Disk in-tree volume plugin to its equivalent CSI driver.

CSI automatic migration should be seamless. Migration does not change how you use all existing API objects, such as persistent volumes, persistent volume claims, and storage classes. For more information about migration, see [CSI automatic migration](#).



IMPORTANT

High availability of storage in the infrastructure is left to the underlying storage provider.

Additional resources

- [CSI automatic migration](#)
- [Microsoft Azure Disk](#)

4.2.1. Creating the Azure storage class

You can use storage classes to differentiate and delineate storage levels and usages. By defining a storage class, you can obtain dynamically provisioned persistent volumes.

Procedure

1. In the OpenShift Container Platform web console, click **Storage → Storage Classes**.
2. In the storage class overview, click **Create Storage Class**
3. Define the desired options on the page that appears.

- a. Enter a name to reference the storage class.
- b. Enter an optional description.
- c. Select the reclaim policy.
- d. Select **kubernetes.io/azure-disk** from the drop down list.
- i. Enter the storage account type. This corresponds to your Azure storage account SKU tier. Valid options are **Premium_LRS**, **PremiumV2_LRS**, **Standard_LRS**, **StandardSSD_LRS**, and **UltraSSD_LRS**.



IMPORTANT

The skuname **PremiumV2_LRS** is not supported in all regions, and in some supported regions, not all of the availability zones are supported. For more information, see [Azure doc](#).

- ii. Enter the kind of account. Valid options are **shared**, **dedicated**, and **managed**.



IMPORTANT

Red Hat only supports the use of **kind: Managed** in the storage class.

With **Shared** and **Dedicated**, Azure creates unmanaged disks, while OpenShift Container Platform creates a managed disk for machine OS (root) disks. But because Azure Disk does not allow the use of both managed and unmanaged disks on a node, unmanaged disks created with **Shared** or **Dedicated** cannot be attached to OpenShift Container Platform nodes.

- e. Enter additional parameters for the storage class as desired.
4. Click **Create** to create the storage class.

Additional resources

- [Azure Disk Storage Class](#)

4.2.2. Creating the persistent volume claim

You can create a persistent volume claim to dynamically provision and bind storage from a pre-configured storage class, so that your applications can consume persistent storage in OpenShift Container Platform.

Prerequisites

- Storage must exist in the underlying infrastructure before it can be mounted as a volume in OpenShift Container Platform.

Procedure

1. In the OpenShift Container Platform web console, click **Storage → Persistent Volume Claims**

2. In the persistent volume claims overview, click **Create Persistent Volume Claim**
3. Define the required options on the page that is displayed.
 - a. Select the previously-created storage class from the drop-down menu.
 - b. Enter a unique name for the storage claim.
 - c. Select the access mode. This selection determines the read and write access for the storage claim.
 - d. Define the size of the storage claim.
4. Click **Create** to create the persistent volume claim and generate a persistent volume.

4.2.3. Volume format

You can use unformatted Azure volumes as persistent volumes, because OpenShift Container Platform automatically formats the device before mounting it to a container.

OpenShift Container Platform verifies that a volume contains the file system specified by the **fsType** parameter in the persistent volume definition before mounting it to a container. An unformatted device is erased and automatically formatted with the specified file system.

4.2.4. Machine sets that deploy machines with ultra disks using PVCs

You can create a machine set running on Microsoft Azure that deploys machines with ultra disks. Ultra disks are high-performance storage that are intended for use with the most demanding data workloads.

Both the in-tree plugin and CSI driver support using PVCs to enable ultra disks. You can also deploy machines with ultra disks as data disks without creating a PVC.

Additional resources

- [Microsoft Azure ultra disks documentation](#)
- [Machine sets that deploy machines on ultra disks using CSI PVCs](#)
- [Machine sets that deploy machines on ultra disks as data disks](#)

4.2.4.1. Creating machines with ultra disks by using machine sets

You can deploy machines with ultra disks on Microsoft Azure by editing your machine set YAML file.

Prerequisites

- Have an existing Microsoft Azure cluster.

Procedure

1. Copy an existing Azure **MachineSet** custom resource (CR) and edit it by running the following command:

```
$ oc edit machineset <machine_set_name>
```

where:

<machine_set_name>

Indicates the machine set that you want to provision machines with ultra disks.

2. Add the following lines in the positions indicated:

```
apiVersion: machine.openshift.io/v1beta1
kind: MachineSet
spec:
  template:
    spec:
      metadata:
        labels:
          disk: ultrassd
      providerSpec:
        value:
          ultraSSDCapability: Enabled
```

where:

spec.template.spec.metadata.labels.disk

Specifies a label to use to select a node that is created by this machine set. The example uses **disk.ultrassd** for this value.

spec.template.spec.providerSpec.value.ultraSSDCapability

Enables the use of ultra disks.

3. Create a machine set by using the updated configuration by running the following command:

```
$ oc create -f <machine_set_name>.yaml
```

4. Create a storage class that contains the following YAML definition:

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: ultra-disk-sc
parameters:
  cachingMode: None
  diskIopsReadWrite: "2000"
  diskMbpsReadWrite: "320"
  kind: managed
  skuName: UltraSSD_LRS
provisioner: disk.csi.azure.com
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer
```

where:

metadata.name

Specifies the name of the storage class. The example uses **ultra-disk-sc** for this value.

parameters.diskIopsReadWrite

Specifies the number of Input/Output Operations Per Second (IOPS) for the storage class.

parameters.diskMbpsReadWrite

Specifies the throughput in MBps for the storage class.

provisioner

For Microsoft Azure Kubernetes Service (AKS) version 1.21 or later, use **disk.csi.azure.com**.

For earlier versions of AKS, use **kubernetes.io/azure-disk**.

volumeBindingMode

Optional parameter. Specifies this parameter to wait for the creation of the pod that will use the disk.

5. Create a persistent volume claim (PVC) to reference the **ultra-disk-sc** storage class that contains the following YAML definition:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: ultra-disk
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: ultra-disk-sc
  resources:
    requests:
      storage: 4Gi
```

where:

metadata.name

Specifies the name of the PVC. The example uses **ultra-disk** for this value.

spec.storageClassName

Specifies the name of the storage class to use. The example uses **ultra-disk-sc** storage class.

spec.resources.requests.storage

Specifies the size for the storage class. The minimum value is **4Gi**.

6. Create a pod that contains the following YAML definition:

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-ultra
spec:
  nodeSelector:
    disk: ultrasdd
  containers:
    - name: nginx-ultra
      image: alpine:latest
      command:
        - "sleep"
        - "infinity"
      volumeMounts:
```

```

- mountPath: "/mnt/azure"
  name: volume
volumes:
- name: volume
  persistentVolumeClaim:
    claimName: ultra-disk

```

where:

spec.nodeSelector.disk

Specifies the label of the machine set that enables the use of ultra disks. The example uses **disk.ultrassd** for this value.

spec.volumes.persistentVolumeClaim.claimName

Specifies the name of the PVC to attach. This pod references the **ultra-disk** PVC.

Verification

1. Validate that the machines are created by running the following command:

```
$ oc get machines
```

The machines should be in the **Running** state.

2. For a machine that is running and has a node attached, validate the partition by running the following command:

```
$ oc debug node/<node_name> -- chroot /host lsblk
```

In this command, **oc debug node/<node_name>** starts a debugging shell on the node **<node_name>** and passes a command with **--**. The passed command **chroot /host** provides access to the underlying host OS binaries, and **lsblk** shows the block devices that are attached to the host OS machine.

Next steps

- To use an ultra disk from within a pod, create a workload that uses the mount point. Create a YAML file similar to the following example:

```

apiVersion: v1
kind: Pod
metadata:
  name: ssd-benchmark1
spec:
  containers:
  - name: ssd-benchmark1
    image: nginx
    ports:
    - containerPort: 80
      name: "http-server"
    volumeMounts:
    - name: lun0p1
      mountPath: "/tmp"
  volumes:
  - name: lun0p1

```

```

hostPath:
  path: /var/lib/lunOp1
  type: DirectoryOrCreate
nodeSelector:
  disktype: ultrasd

```

4.2.4.2. Troubleshooting resources for machine sets that enable ultra disks

You can recover from issues that you might encounter when you enable ultra disks for machine sets. Review fields, such as disk settings, and ensure that the parameters are correctly configured.

4.2.4.2.1. Unable to mount a persistent volume claim backed by an ultra disk

If there is an issue mounting a persistent volume claim backed by an ultra disk, the pod becomes stuck in the **ContainerCreating** state and an alert is triggered.

For example, if the **additionalCapabilities.ultraSSDEnabled** parameter is not set on the machine that backs the node that hosts the pod, the following error message appears:

```
StorageAccountType UltraSSD_LRS can be used only when additionalCapabilities.ultraSSDEnabled is set.
```

- To resolve this issue, describe the pod by running the following command:

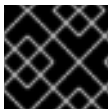
```
$ oc -n <stuck_pod_namespace> describe pod <stuck_pod_name>
```

4.3. PERSISTENT STORAGE USING AZURE FILE

OpenShift Container Platform supports Microsoft Azure File volumes. You can provision your OpenShift Container Platform cluster with persistent storage using Azure. Some familiarity with Kubernetes and Azure is assumed.

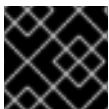
The Kubernetes persistent volume framework allows administrators to provision a cluster with persistent storage and gives users a way to request those resources without having any knowledge of the underlying infrastructure. You can provision Azure File volumes dynamically.

Persistent volumes are not bound to a single project or namespace, and you can share them across the OpenShift Container Platform cluster. Persistent volume claims are specific to a project or namespace, and can be requested by users for use in applications.



IMPORTANT

High availability of storage in the infrastructure is left to the underlying storage provider.



IMPORTANT

Azure File volumes use Server Message Block.



IMPORTANT

OpenShift Container Platform 4.13 and later provides automatic migration for the Azure File in-tree volume plugin to its equivalent CSI driver.

CSI automatic migration should be seamless. Migration does not change how you use all existing API objects, such as persistent volumes, persistent volume claims, and storage classes. For more information about migration, see "CSI automatic migration".

Additional resources

- [CSI automatic migration](#)
- [Azure Files](#)

4.3.1. Create the Azure File share persistent volume claim

To create the persistent volume claim, you must first define a **Secret** object that contains the Azure account and key. This secret is used in the **PersistentVolume** definition, and will be referenced by the persistent volume claim for use in applications.

Prerequisites

- An Azure File share exists.
- The credentials to access this share, specifically the storage account and key, are available.

Procedure

1. Create a **Secret** object that contains the Azure File credentials:

```
$ oc create secret generic __<secret-name>__ --from-
literal=azurestorageaccountname=__<storage-account> --from-
literal=azurestorageaccountkey=__<storage-account-key>
```

where:

<secret-name>

Specifies the Azure File storage account name.

<storage-account-key>

Specifies the Azure File storage account key.

2. Create a **PersistentVolume** object that references the **Secret** object you created:

```
apiVersion: "v1"
kind: "PersistentVolume"
metadata:
  name: "pv0001"
spec:
  capacity:
    storage: "5Gi"
  accessModes:
    - "ReadWriteOnce"
  storageClassName: azure-file-sc
```

```
azureFile:
  secretName: <secret-name>
  shareName: <share-name>
  readOnly: false
```

where:

metadata.name

Specifies the name of the persistent volume.

spec.capacity.storage

Specifies the size of this persistent volume, for example **5Gi**.

spec.azureFile.secretName

Specifies the name of the secret that contains the Azure File share credentials.

spec.azureFile.shareName

Specifies the name of the Azure File share.

3. Create a **PersistentVolumeClaim** object that maps to the persistent volume you created:

```
apiVersion: "v1"
kind: "PersistentVolumeClaim"
metadata:
  name: "claim1"
spec:
  accessModes:
    - "ReadWriteOnce"
  resources:
    requests:
      storage: "5Gi"
  storageClassName: azure-file-sc
  volumeName: "pv0001"
```

where:

metadata.name

Specifies the name of the persistent volume claim.

spec.resources.requests.storage

Specifies the size of this persistent volume claim, for example **5Gi**.

spec.storageClassName

Specifies the name of the existing **PersistentVolume** object that references the Azure File share. Specify the storage class used in the **PersistentVolume** definition.

spec.volumeName

Specifies the name of the existing **PersistentVolume** object that references the Azure File share.

4.3.2. Mount the Azure File share in a pod

After you create a persistent volume (PV), you can use the PV inside by an application.

The following example demonstrates mounting this share inside of a pod.

Prerequisites

- A persistent volume claim exists that is mapped to the underlying Azure File share.

Procedure

- Create a pod that mounts the existing persistent volume claim:

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-name
spec:
  containers:
    ...
  volumeMounts:
    - mountPath: "/data"
      name: azure-file-share
  volumes:
    - name: azure-file-share
      persistentVolumeClaim:
        claimName: claim1
```

where:

metadata.name

Specifies the name of the pod.

spec.containers.volumeMounts.mountPath

Specifies the path to mount the Azure File share inside the pod, for example **/data**. Do not mount to the container root, **/**, or any path that is the same in the host and the container. This can corrupt your host system if the container is sufficiently privileged, such as the host **/dev/pts** files. It is safe to mount the host by using **/host**.

spec.volumes.persistentVolumeClaim.claimName

Specifies the name of the **PersistentVolumeClaim** object that has been previously created.

4.4. PERSISTENT STORAGE USING CINDER

OpenShift Container Platform supports OpenStack Cinder volumes. You can provision your OpenShift Container Platform cluster with persistent storage using OpenStack Cinder. Some familiarity with Kubernetes and OpenStack is assumed.

Persistent volumes are not bound to a single project or namespace; they can be shared across the OpenShift Container Platform cluster. Persistent volume claims are specific to a project or namespace and can be requested by users.



IMPORTANT

OpenShift Container Platform 4.11 and later provides automatic migration for the Cinder in-tree volume plugin to its equivalent CSI driver.

CSI automatic migration should be seamless. Migration does not change how you use all existing API objects, such as persistent volumes, persistent volume claims, and storage classes. For more information about migration, see "CSI automatic migration".

Additional resources

- [CSI automatic migration](#)
- [OpenStack Cinder](#)

4.4.1. Manual provisioning with Cinder

Storage must exist in the underlying infrastructure before it can be mounted as a volume in OpenShift Container Platform.

Manual provisioning requires that OpenShift Container Platform is configured for Red Hat OpenStack Platform (RHOSP) and that you have the Cinder volume ID.

4.4.1.1. Creating the persistent volume

You can create a persistent volume (PV) that provisions storage from an Red Hat OpenStack Platform (RHOSP) Cinder volume for use with OpenShift Container Platform.

Prerequisites

- You have defined your PV in an object definition before creating it in OpenShift Container Platform.

Procedure

1. Save your object definition to a file.

cinder-persistentvolume.yaml

```
apiVersion: "v1"
kind: "PersistentVolume"
metadata:
  name: "pv0001"
spec:
  capacity:
    storage: "5Gi"
  accessModes:
    - "ReadWriteOnce"
  cinder:
    fsType: "ext3"
    volumeID: "f37a03aa-6212-4c62-a805-9ce139fab180"
```

where:

metadata.name

Specifies the name of the volume that is used by persistent volume claims or pods.

spec.capacity.storage

Specifies the amount of storage allocated to this volume.

spec.cinder

Indicates **cinder** for Red Hat OpenStack Platform (RHOSP) Cinder volumes.

spec.cinder.fsType

Specifies the file system that is created when the volume is mounted for the first time.

spec.cinder.volumeID

Specifies the Cinder volume to use.



IMPORTANT

Do not change the **fstype** parameter value after the volume is formatted and provisioned. Changing this value can result in data loss and pod failure.

2. Create the object definition file you saved in the previous step.

```
$ oc create -f cinder-persistentvolume.yaml
```

4.4.1.2. Persistent volume formatting

You can use unformatted Cinder volumes as PVs because OpenShift Container Platform formats them before the first use.

Before OpenShift Container Platform mounts the volume and passes it to a container, the system checks that it contains a file system as specified by the **fstype** parameter in the PV definition. If the device is not formatted with the file system, all data from the device is erased and the device is automatically formatted with the given file system.

4.4.1.3. Configuring Cinder volume security

If you use Cinder PVs in your application, configure security for their deployment resources.

Prerequisites

- An SCC must be created that uses the appropriate **fsGroup** strategy.

Procedure

1. Create a service account and add it to the SCC:

```
$ oc create serviceaccount <service_account>
```

```
$ oc adm policy add-scc-to-user <new_scc> -z <service_account> -n <project>
```

2. In your application's deployment resource, provide the service account name and **securityContext**:

```
apiVersion: v1
kind: ReplicationController
metadata:
  name: frontend-1
spec:
  replicas: 1
  selector:
    name: frontend
  template:
```

```

metadata:
  labels:
    name: frontend
spec:
  containers:
    - image: openshift/hello-openshift
      name: helloworld
      ports:
        - containerPort: 8080
          protocol: TCP
      restartPolicy: Always
      serviceAccountName: <service_account>
      securityContext:
        fsGroup: 7777

```

where:

spec.replicas

Specifies the number of copies of the pod to run.

spec.selector

Specifies the label selector of the pod to run.

spec.template

Specifies a template for the pod that the controller creates.

spec.template.metadata.labels

Specifies the labels on the pod. They must include labels from the label selector.

spec.template.metadata.labels.name

Specifies the maximum name length after expanding any parameters is 63 characters.

spec.template.spec.serviceAccountName

Specifies the service account you created.

spec.template.spec.securityContext.fsGroup

Specifies an **fsGroup** for the pods.

4.5. PERSISTENT STORAGE USING FIBRE CHANNEL

You can provision your OpenShift Container Platform cluster with persistent storage by using Fibre Channel volumes for workloads that require high-speed, reliable block-level storage.



IMPORTANT

Persistent storage using Fibre Channel is not supported on ARM architecture based infrastructures.

The Kubernetes persistent volume framework allows administrators to provision a cluster with persistent storage and gives users a way to request those resources without having any knowledge of the underlying infrastructure. Persistent volumes are not bound to a single project or namespace; they can be shared across the OpenShift Container Platform cluster. Persistent volume claims are specific to a project or namespace and can be requested by users.

**IMPORTANT**

High availability of storage in the infrastructure is left to the underlying storage provider.

Additional resources

- [Using Fibre Channel devices](#)

4.5.1. About provisioning storage using Fibre Channel

You can provision Fibre Channel volumes by using the **PersistentVolume** API.

Before provisioning the volumes, the following items must be available.

- The **targetWWNs** (array of Fibre Channel target's World Wide Names).
- A valid LUN number.
- The filesystem type.

A persistent volume and a LUN have a one-to-one mapping between them.

**NOTE**

Fibre Channel LUNs must exist in the underlying infrastructure.

PersistentVolume object definition

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv0001
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  fc:
    wwids: [3600508b400105e210000900000490000]
    targetWWNs: ['500a0981891b8dc5', '500a0981991b8dc5']
    lun: 2
    fsType: ext4
```

where:

spec.fc.wwids

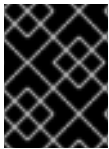
Specifies the world wide identifiers (WWIDs). Either FC **wwids** or a combination of FC **targetWWNs** and **lun** must be set, but not both simultaneously. The FC WWID identifier is recommended over the WWNs target because it is guaranteed to be unique for every storage device, and independent of the path that is used to access the device. The WWID identifier can be obtained by issuing a SCSI Inquiry to retrieve the Device Identification Vital Product Data (**page 0x83**) or Unit Serial Number (**page 0x80**). FC WWIDs are identified as **/dev/disk/by-id/** to reference the data on the disk, even if the path to the device changes and even when accessing the device from different systems.

spec.fc.targetWWNs

Specifies the Fibre Channel World Wide Names (WWNs). Fibre Channel WWNs are identified as `/dev/disk/by-path/pci-<IDENTIFIER>-fc-0x<WWN>-lun-<LUN#>`, but you do not need to provide any part of the path leading up to the **WWN**, including the **0x**, and anything after, including the **-** (hyphen).

spec.fc.lun

Specifies the LUN number.

**IMPORTANT**

Changing the value of the **fstype** parameter after the volume has been formatted and provisioned can result in data loss and pod failure.

4.5.1.1. Enforcing disk quotas

You can use LUN partitions to enforce disk quotas and size constraints. Each LUN is mapped to a single persistent volume, and unique names must be used for persistent volumes.

Enforcing quotas in this way allows the user to request persistent storage by a specific amount, such as 10Gi, and be matched with a corresponding volume of equal or greater capacity.

4.5.1.2. Fibre Channel volume security

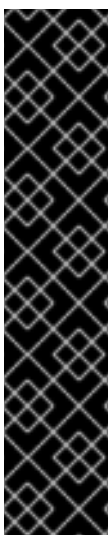
You can request storage with a persistent volume claim. This claim only lives in the user's namespace, and can only be referenced by a pod within that same namespace. Any attempt to access a persistent volume across a namespace causes the pod to fail.

Each Fibre Channel LUN must be accessible by all nodes in the cluster.

4.6. PERSISTENT STORAGE USING FLEXVOLUME

To use storage from a back-end that does not have a built-in plugin, you can extend OpenShift Container Platform through FlexVolume drivers and provide persistent storage to applications.

FlexVolume is an out-of-tree plugin that uses an executable model to interface with drivers.

**IMPORTANT**

FlexVolume is a deprecated feature. Deprecated functionality is still included in OpenShift Container Platform and continues to be supported; however, it will be removed in a future release of this product and is not recommended for new deployments.

Out-of-tree Container Storage Interface (CSI) driver is the recommended way to write volume drivers in OpenShift Container Platform. Maintainers of FlexVolume drivers should implement a CSI driver and move users of FlexVolume to CSI. Users of FlexVolume should move their workloads to CSI driver.

For the most recent list of major functionality that has been deprecated or removed within OpenShift Container Platform, refer to the *Deprecated and removed features* section of the OpenShift Container Platform release notes.

Pods interact with FlexVolume drivers through the **flexvolume** in-tree plugin.

Additional resources

- [Expanding persistent volumes](#)

4.6.1. About FlexVolume drivers

When working with FlexVolume drivers, it is helpful to understand how OpenShift Container Platform interact with the drivers.

A FlexVolume driver is an executable file that resides in a well-defined directory on all nodes in the cluster. OpenShift Container Platform calls the FlexVolume driver whenever it needs to mount or unmount a volume represented by a **PersistentVolume** object with **flexVolume** as the source.



IMPORTANT

Attach and detach operations are not supported in OpenShift Container Platform for FlexVolume.

4.6.2. FlexVolume driver example

When working with FlexVolume drivers, it is helpful to become familiar with structure of the driver.

The first command-line argument of the FlexVolume driver is always an operation name. Other parameters are specific to each operation. Most of the operations take a JavaScript Object Notation (JSON) string as a parameter. This parameter is a complete JSON string, and not the name of a file with the JSON data.

The FlexVolume driver contains:

- All **flexVolume.options**.
- Some options from **flexVolume** prefixed by **kubernetes.io/**, such as **fsType** and **readwrite**.
- The content of the referenced secret, if specified, prefixed by **kubernetes.io/secret/**.

FlexVolume driver JSON input example

```
{
  "fooServer": "192.168.0.1:1234",
  "fooVolumeName": "bar",
  "kubernetes.io/fsType": "ext4",
  "kubernetes.io/readwrite": "ro",
  "kubernetes.io/secret/<key name>": "<key value>",
  "kubernetes.io/secret/<another key name>": "<another key value>",
}
```

where:

fooServer

Specifies all options from **flexVolume.options**.

kubernetes.io/fsType

Specifies the value of **flexVolume.fsType**.

kubernetes.io/readwrite

Specifies the value **ro** or **rw** based on **flexVolume.readOnly**.

kubernetes.io/secret/<key name>

Specifies all keys and their values from the secret referenced by **flexVolume.secretRef**.

OpenShift Container Platform expects JSON data on standard output of the driver. When not specified, the output describes the result of the operation.

FlexVolume driver default output example

```
{
  "status": "<Success/Failure/Not supported>",
  "message": "<Reason for success/failure>"
}
```

Exit code of the driver should be **0** for success and **1** for error.

Operations should be idempotent, which means that the mounting of an already mounted volume should result in a successful operation.

4.6.3. Installing FlexVolume drivers

You can implement FlexVolumes by using a list of operations to call and the installation path are all that is required. FlexVolume drivers that are used to extend OpenShift Container Platform are executed only on the node.

Prerequisites

- FlexVolume drivers must implement these operations:

init

Initializes the driver. It is called during initialization of all nodes.

- Arguments: none
- Executed on: node
- Expected output: default JSON

mount

Mounts a volume to directory. This can include anything that is necessary to mount the volume, including finding the device and then mounting the device.

- Arguments: **<mount-dir>** **<json>**
- Executed on: node
- Expected output: default JSON

unmount

Unmounts a volume from a directory. This can include anything that is necessary to clean up the volume after unmounting.

- Arguments: **<mount-dir>**

- Executed on: node
- Expected output: default JSON

mountdevice

Mounts a volume's device to a directory where individual pods can then bind mount. This call-out does not pass "secrets" specified in the FlexVolume spec. If your driver requires secrets, do not implement this call-out.

- Arguments: **<mount-dir>** **<json>**
- Executed on: node
- Expected output: default JSON

unmountdevice

Unmounts a volume's device from a directory.

- Arguments: **<mount-dir>**
- Executed on: node
- Expected output: default JSON

- All other operations should return JSON with **{"status": "Not supported"}** and exit code **1**.

Procedure

To install the FlexVolume driver:

1. Ensure that the executable file exists on all nodes in the cluster.
2. Place the executable file at the volume plugin path: **/etc/kubernetes/kubelet-plugins/volume/exec/<vendor>~<driver>/<driver>**.

For example, to install the FlexVolume driver for the storage **foo**, place the executable file at: **/etc/kubernetes/kubelet-plugins/volume/exec/openshift.com~foo/foo**.

4.6.4. Consuming storage using FlexVolume drivers

You can consume a Fibre Channel volume by using a **PersistentVolume** object.

Each **PersistentVolume** object in OpenShift Container Platform represents one storage asset in the storage back-end, such as a volume.

Procedure

- Use the **PersistentVolume** object to reference the installed storage.

Persistent volume object definition using FlexVolume drivers example

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv0001
```

```
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  flexVolume:
    driver: openshift.com/foo
    fsType: "ext4"
    secretRef: foo-secret
    readOnly: true
    options:
      fooServer: 192.168.0.1:1234
      fooVolumeName: bar
```

where:

metadata.name

Specifies the name of the volume. This is how it is identified through persistent volume claims or from pods. This name can be different from the name of the volume on back-end storage.

spec.capacity.storage

Specifies the amount of storage allocated to this volume.

spec.flexVolume.driver

Specifies the name of the driver. This field is mandatory.

spec.flexVolume.fsType

Specifies the file system that is present on the volume. This field is optional.

spec.flexVolume.secretRef

Specifies the reference to a secret. Keys and values from this secret are provided to the FlexVolume driver on invocation. This field is optional.

spec.flexVolume.readOnly

Specifies the read-only flag. This field is optional.

spec.flexVolume.options

Specifies the additional options for the FlexVolume driver. In addition to the flags specified by the user in the **options** field, the following flags are also passed to the executable:

"fsType": "<FS type>", "readwrite": "<rw>", "secret/key1": "<secret1>" "secret/keyN": "<secretN>"



NOTE

Secrets are passed only to mount or unmount call-outs.

4.7. PERSISTENT STORAGE USING GCE PERSISTENT DISK

OpenShift Container Platform supports GCE Persistent Disk volumes (gcePD). You can provision your OpenShift Container Platform cluster with persistent storage using GCE. Some familiarity with Kubernetes and GCE is assumed.

The Kubernetes persistent volume framework allows administrators to provision a cluster with persistent storage and gives users a way to request those resources without having any knowledge of the underlying infrastructure.

GCE Persistent Disk volumes can be provisioned dynamically.

Persistent volumes are not bound to a single project or namespace; they can be shared across the OpenShift Container Platform cluster. Persistent volume claims are specific to a project or namespace and can be requested by users.



IMPORTANT

OpenShift Container Platform 4.12 and later provides automatic migration for the GCE Persistent Disk in-tree volume plugin to its equivalent CSI driver.

CSI automatic migration should be seamless. Migration does not change how you use all existing API objects, such as persistent volumes, persistent volume claims, and storage classes.

For more information about migration, see [CSI automatic migration](#).



IMPORTANT

High availability of storage in the infrastructure is left to the underlying storage provider.

Additional resources

- [GCE Persistent Disk](#)
- [CSI automatic migration](#)

4.7.1. About the GCE storage class

To enable dynamic provisioning of persistent volumes, create a storage class that defines storage characteristics and allows users to automatically provision volumes on-demand.

4.7.2. Creating the persistent volume claim

You can create a persistent volume claim to dynamically provision and bind storage from a pre-configured storage class, so that your applications can consume persistent storage in OpenShift Container Platform.

Prerequisites

- Storage must exist in the underlying infrastructure before it can be mounted as a volume in OpenShift Container Platform.

Procedure

1. In the OpenShift Container Platform web console, click **Storage → Persistent Volume Claims**
2. In the persistent volume claims overview, click **Create Persistent Volume Claim**
3. Define the required options on the page that is displayed.
 - a. Select the previously-created storage class from the drop-down menu.
 - b. Enter a unique name for the storage claim.

- c. Select the access mode. This selection determines the read and write access for the storage claim.
 - d. Define the size of the storage claim.
4. Click **Create** to create the persistent volume claim and generate a persistent volume.

4.7.3. Volume format

You can use unformatted GCE volumes as persistent volumes, because OpenShift Container Platform automatically formats the device before mounting it to a container.

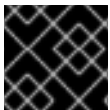
Before OpenShift Container Platform mounts the volume and passes it to a container, it checks that the volume contains a file system as specified by the **fsType** parameter in the persistent volume definition. If the device is not formatted with the file system, all data from the device is erased and the device is automatically formatted with the given file system.

4.8. PERSISTENT STORAGE USING ISCSI

You can provision your OpenShift Container Platform cluster with persistent storage by creating persistent volumes that pods can use to store and access data.

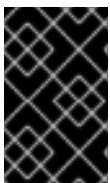
Some familiarity with Kubernetes and iSCSI is assumed.

The Kubernetes persistent volume framework allows administrators to provision a cluster with persistent storage and gives users a way to request those resources without having any knowledge of the underlying infrastructure.



IMPORTANT

High-availability of storage in the infrastructure is left to the underlying storage provider.



IMPORTANT

When you use iSCSI on Amazon Web Services, you must update the default security policy to include TCP traffic between nodes on the iSCSI ports. By default, they are ports **860** and **3260**.



IMPORTANT

Users must ensure that the iSCSI initiator is already configured on all OpenShift Container Platform nodes by installing the **iscsi-initiator-utils** package and configuring their initiator name in **/etc/iscsi/initiatorname.iscsi**. The **iscsi-initiator-utils** package is already installed on deployments that use Red Hat Enterprise Linux CoreOS (RHCOS). For more information, see "Managing Storage Devices".

Additional resources

- [Getting started with iSCSI](#)
- [Configuring an iSCSI initiator](#)

4.8.1. Provisioning

You can verify that the storage exists in the underlying infrastructure before mounting it as a volume in OpenShift Container Platform. All that is required for the iSCSI is the iSCSI target portal, a valid iSCSI Qualified Name (IQN), a valid LUN number, the filesystem type, and the **PersistentVolume** API.

Procedure

- Verify that the storage exists in the underlying infrastructure before mounting it as a volume in OpenShift Container Platform by creating the following **PersistentVolume** object definition:

Example **PersistentVolume** object definition

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: iscsi-pv
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  iscsi:
    targetPortal: 10.16.154.81:3260
    iqn: iqn.2014-12.example.server:storage.target00
    lun: 0
    fsType: 'ext4'
```

4.8.2. Enforce disk quotas

You can use LUN partitions to enforce disk quotas and size constraints. Each LUN is one persistent volume. Kubernetes enforces unique names for persistent volumes.

Enforcing quotas in this way allows the user to request persistent storage by a specific amount (for example, **10Gi**) and be matched with a corresponding volume of equal or greater capacity.

4.8.3. iSCSI volume security

For security purposes, when you can request storage with a **PersistentVolumeClaim** object, the claim lives in the user's namespace only and can only be referenced by a pod within that same namespace. Any attempt to access a persistent volume claim across a namespace causes the pod to fail.

Each iSCSI LUN must be accessible by all nodes in the cluster.

4.8.3.1. Challenge Handshake Authentication Protocol (CHAP) configuration

Optionally, OpenShift Container Platform can use CHAP to authenticate itself to iSCSI targets:

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: iscsi-pv
spec:
  capacity:
    storage: 1Gi
  accessModes:
```

```

- ReadWriteOnce
iscsi:
  targetPortal: 10.0.0.1:3260
  iqn: iqn.2016-04.test.com:storage.target00
  lun: 0
  fsType: ext4
  chapAuthDiscovery: true
  chapAuthSession: true
  secretRef:
    name: chap-secret

```

where:

spec.iscsi.chapAuthDiscovery

When set to **true**, enables CHAP authentication of iSCSI discovery.

spec.iscsi.chapAuthSession

When set to **true**, enables CHAP authentication of iSCSI session.

spec.iscsi.secretRef.name

Specifies the name of **Secret** object with user name and password. This **Secret** object must be available in all namespaces that can use the referenced volume.

4.8.4. iSCSI multipathing

For iSCSI-based storage, you can configure multiple paths by using the same IQN for more than one target portal IP address. Multipathing ensures access to the persistent volume when one or more of the components in a path fail.

Procedure

- To specify multi-paths in the pod specification, specify a value in the **portals** field of the **PersistentVolume** definition object.

Example PersistentVolume object with a value specified in the portals field.

```

apiVersion: v1
kind: PersistentVolume
metadata:
  name: iscsi-pv
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  iscsi:
    targetPortal: 10.0.0.1:3260
    portals: ['10.0.2.16:3260', '10.0.2.17:3260', '10.0.2.18:3260']
    iqn: iqn.2016-04.test.com:storage.target00
    lun: 0
    fsType: ext4
    readOnly: false

```

where:

spec.iscsi.portals

Add additional target portals by using the **portals** field.

4.8.5. iSCSI custom initiator IQN

You can configure the custom initiator iSCSI Qualified Name (IQN) if the iSCSI targets are restricted to certain IQNs, but the nodes that the iSCSI PVs are attached to are not guaranteed to have these IQNs.

Procedure

- To specify a custom initiator IQN, update the **initiatorName** field in the **PersistentVolume** definition object.

Example PersistentVolume object with a value specified in the `initiatorName` field.

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: iscsi-pv
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  iscsi:
    targetPortal: 10.0.0.1:3260
    portals: ['10.0.2.16:3260', '10.0.2.17:3260', '10.0.2.18:3260']
    iqn: iqn.2016-04.test.com:storage.target00
    lun: 0
    initiatorName: iqn.2016-04.test.com:custom.iqn
    fsType: ext4
    readOnly: false
```

where:

spec.iscsi.initiatorName

Specifies the name of the initiator.

4.9. PERSISTENT STORAGE USING NFS

You can provision OpenShift Container Platform clusters with persistent storage using NFS.

Persistent volumes (PVs) and persistent volume claims (PVCs) provide a convenient method for sharing a volume across a project. While the NFS-specific information contained in a PV definition could also be defined directly in a pod definition, doing so does not create the volume as a distinct cluster resource, making the volume more susceptible to conflicts.

**NOTE**

The in-tree NFS provisioner does not support user namespaces.

Additional resources

- [Mounting NFS shares](#)

4.9.1. Provisioning persistent storage using NFS

You can provision persistent storage for OpenShift Container Platform by creating persistent volume (PV) and persistent volume claim (PVC) objects that reference your NFS servers and export paths.

Prerequisites

- You have NFS storage available in the underlying infrastructure with the appropriate export paths configured.

Procedure

1. Create an object definition for the PV:

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv0001
spec:
  capacity:
    storage: 5Gi
  accessModes:
    - ReadWriteOnce
  nfs:
    path: /tmp
    server: 172.17.0.2
  persistentVolumeReclaimPolicy: Retain
```

where:

metadata.name

Specifies the name of the volume. This is the PV identity in various **oc** commands.

spec.capacity.storage

Specifies the amount of storage allocated to this volume.

spec.accessModes.ReadWriteOnce

Though this appears to be related to controlling access to the volume, it is actually used similarly to labels and used to match a PVC to a PV. Currently, no access rules are enforced based on the **accessModes**.

spec.nfs

Specifies the volume type being used, in this case the **nfs** plugin.

spec.nfs.path

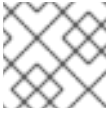
Specifies the path that is exported by the NFS server.

spec.nfs.server

Specifies the hostname or IP address of the NFS server.

spec.persistentVolumeReclaimPolicy

Specifies the reclaim policy for the PV. This defines what happens to a volume when released.

**NOTE**

Each NFS volume must be mountable by all schedulable nodes in the cluster.

2. Verify that the PV was created:

```
$ oc get pv
```

Example output

NAME	LABELS	CAPACITY	ACCESSMODES	STATUS	CLAIM REASON	AGE
pv0001	<none>	5Gi	RWO	Available		31s

3. Create a persistent volume claim that binds to the new PV:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: nfs-claim1
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
  volumeName: pv0001
  storageClassName: ""
```

where:

spec.accessModes.ReadWriteOnce

Specifies the access modes do not enforce security, but rather act as labels to match a PV to a PVC.

spec.resources.requests.storage

This claim looks for PVs offering **5Gi** or greater capacity.

4. Verify that the persistent volume claim was created:

```
$ oc get pvc
```

Example output

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS	AGE
nfs-claim1	Bound	pv0001	5Gi	RWO		2m

4.9.2. Enforce disk quotas

You can enforce disk quotas for NFS volumes by allocating individual persistent volumes for each project, allowing you to control storage capacity per namespace.

You can use disk partitions to enforce disk quotas and size constraints. Each partition can be its own export. Each export is one PV. OpenShift Container Platform enforces unique names for PVs, but the uniqueness of the NFS volume's server and path is up to the administrator.

Enforcing quotas in this way allows the developer to request persistent storage by a specific amount, such as 10Gi, and be matched with a corresponding volume of equal or greater capacity.

4.9.3. NFS volume security

To understand NFS volume security, including matching permissions and SELinux considerations, you should understand the basics of POSIX permissions, process UIDs, supplemental groups, and SELinux.

Developers request NFS storage by referencing either a PVC by name or the NFS volume plugin directly in the **volumes** section of their **Pod** definition.

The **/etc/exports** file on the NFS server contains the accessible NFS directories. The target NFS directory has POSIX owner and group IDs. The OpenShift Container Platform NFS plugin mounts the container's NFS directory with the same POSIX ownership and permissions found on the exported NFS directory. However, the container is not run with its effective UID equal to the owner of the NFS mount, which is the desired behavior.

As an example, if the target NFS directory appears on the NFS server as:

```
$ ls -lZ /opt/nfs -d
```

Example output

```
drwxrws---. nfsnobody 5555 unconfined_u:object_r:usr_t:s0 /opt/nfs
```

```
$ id nfsnobody
```

Example output

```
uid=65534(nfsnobody) gid=65534(nfsnobody) groups=65534(nfsnobody)
```

Then the container must match SELinux labels, and either run with a UID of **65534**, the **nfsnobody** owner, or with **5555** in its supplemental groups to access the directory.



NOTE

The owner ID of **65534** is used as an example. Even though NFS's **root_squash** maps **root**, uid **0**, to **nfsnobody**, uid **65534**, NFS exports can have arbitrary owner IDs. Owner **65534** is not required for NFS exports.

4.9.3.1. Group IDs

You can use supplemental groups to manage NFS access in OpenShift Container Platform when you cannot change permissions on the NFS export. Supplemental groups manage shared storage such as NFS, while block storage such as iSCSI uses the **fsGroup** SCC strategy and **fsGroup** value in the pod **securityContext**.

**NOTE**

To gain access to persistent storage, it is generally preferable to use supplemental group IDs versus user IDs.

Because the group ID on the example target NFS directory is **5555**, the pod can define that group ID using **supplementalGroups** under the **securityContext** definition of the pod. For example:

```
spec:
  containers:
    - name:
      ...
      securityContext:
        supplementalGroups: [5555]
```

where:

spec.securityContext

Must be defined at the pod level, not under a specific container.

spec.securityContext.supplementalGroups

Specifies an array of GIDs defined for the pod. In this case, there is one element in the array. Additional GIDs would be comma-separated.

Assuming there are no custom SCCs that might satisfy the pod requirements, the pod likely matches the **restricted** SCC. This SCC has the **supplementalGroups** strategy set to **RunAsAny**, meaning that any supplied group ID is accepted without range checking.

As a result, the above pod passes admissions and is launched. However, if group ID range checking is desired, a custom SCC is the preferred solution. A custom SCC can be created such that minimum and maximum group IDs are defined, group ID range checking is enforced, and a group ID of **5555** is allowed.

**NOTE**

To use a custom SCC, you must first add it to the appropriate service account. For example, use the **default** service account in the given project unless another has been specified on the **Pod** specification.

4.9.3.2. User IDs

You can define user IDs in the container image or in the pod definition.

**NOTE**

It is generally preferable to use supplemental group IDs to gain access to persistent storage versus using user IDs.

In the example target NFS directory shown above, the container needs its UID set to **65534**, ignoring group IDs for the moment, so the following can be added to the **Pod** definition:

```
spec:
  containers:
    - name:
```

```
...
securityContext:
  runAsUser: 65534
```

where:

spec.containers

Pods contain a **securityContext** definition specific to each container and a pod's **securityContext**, which applies to all containers defined in the pod.

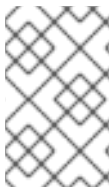
spec.securityContext.runAsUser

Specifies the user ID to run the container as. In this example, **65534** is the **nfsnobody** user.

Assuming that the project is **default** and the SCC is **restricted**, the user ID of **65534** as requested by the pod is not allowed. Therefore, the pod fails for the following reasons:

- It requests **65534** as its user ID.
- All SCCs available to the pod are examined to see which SCC allows a user ID of **65534**. While all policies of the SCCs are checked, the focus here is on user ID.
- Because all available SCCs use **MustRunAsRange** for their **runAsUser** strategy, UID range checking is required.
- **65534** is not included in the SCC or project's user ID range.

It is generally considered a good practice not to modify the predefined SCCs. The preferred way to fix this situation is to create a custom SCC. A custom SCC can be created such that minimum and maximum user IDs are defined, UID range checking is still enforced, and the UID of **65534** is allowed.



NOTE

To use a custom SCC, you must first add it to the appropriate service account. For example, use the **default** service account in the given project unless another has been specified on the **Pod** specification.

4.9.3.3. SELinux

For non-RHEL and non-RHCOS systems, SELinux does not allow writing from a pod to a remote NFS server. The NFS volume mounts correctly but it is read-only. You need to manually enable the correct SELinux permissions.

Red Hat Enterprise Linux (RHEL) and Red Hat Enterprise Linux CoreOS (RHCOS) systems are configured to use SELinux on remote NFS servers by default.

The following procedure shows how to enable the correct SELinux permissions.

Prerequisites

- The **container-selinux** package must be installed. This package provides the **virt_use_nfs** SELinux boolean.

Procedure

- Enable the **virt_use_nfs** boolean using the following command. The **-P** option makes this boolean persistent across reboots.

```
# setsebool -P virt_use_nfs 1
```

4.9.3.4. Export settings

Before you can enable arbitrary container users to read and write the volume, check that each exported volume on the NFS server meets the required conditions.

Prerequisites

- You have access to the NFS server with root permissions.
- You have installed and configured an NFS server on your system.

Procedure

- Every export must be exported using the following format:

```
/<example_fs> *(rw,root_squash)
```

- The firewall must be configured to allow traffic to the mount point.
 - For NFSv4, configure the default port **2049** (**nfs**).

NFSv4

```
# iptables -I INPUT 1 -p tcp --dport 2049 -j ACCEPT
```

- For NFSv3, there are three ports to configure: **2049** (**nfs**), **20048** (**mountd**), and **111** (**portmapper**).

NFSv3

```
# iptables -I INPUT 1 -p tcp --dport 2049 -j ACCEPT
```

```
# iptables -I INPUT 1 -p tcp --dport 20048 -j ACCEPT
```

```
# iptables -I INPUT 1 -p tcp --dport 111 -j ACCEPT
```

- The NFS export and directory must be set up so that they are accessible by the target pods. Either set the export to be owned by the container's primary UID, or supply the pod group access using **supplementalGroups**, as shown in the group IDs above.

4.9.4. Resource reclamation

You can release NFS shares to allow them to be reclaimed.

NFS implements the OpenShift Container Platform **Recyclable** plugin interface. Automatic processes handle reclamation tasks based on policies set on each persistent volume.

By default, PVs are set to **Retain**.

Once claim to a PVC is deleted, and the PV is released, the PV object should not be reused. Instead, a new PV should be created with the same basic volume details as the original.

For example, the administrator creates a PV named **nfs1**:

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: nfs1
spec:
  capacity:
    storage: 1Mi
  accessModes:
    - ReadWriteMany
  nfs:
    server: 192.168.1.1
    path: "/"
```

The user creates **PVC1**, which binds to **nfs1**. The user then deletes **PVC1**, releasing claim to **nfs1**. This results in **nfs1** being **Released**. If the administrator wants to make the same NFS share available, they should create a new PV with the same NFS server details, but a different PV name:

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: nfs2
spec:
  capacity:
    storage: 1Mi
  accessModes:
    - ReadWriteMany
  nfs:
    server: 192.168.1.1
    path: "/"
```

Deleting the original PV and re-creating it with the same name is discouraged. Attempting to manually change the status of a PV from **Released** to **Available** causes errors and potential data loss.

4.9.5. Additional configuration and troubleshooting

You can configure additional NFS mount options to customize volume behavior and optimize performance for your specific storage requirements.

Depending on what version of NFS is being used and how it is configured, there may be additional configuration steps needed for proper export and security mapping. The following are some that may apply:

NFSv4 mount incorrectly shows all files with ownership of **nobody:nobody**

- Could be attributed to the ID mapping settings, found in **/etc/idmapd.conf** on your NFS.
- See [this Red Hat Solution](#)

Disabling ID mapping on NFSv4

- On the NFS server, run the following command:

```
# echo 'Y' >
/sys/module/nfsd/parameters/nfs4_disable_idmapping
```

4.10. RED HAT OPENSIFT DATA FOUNDATION

You can deploy Red Hat OpenShift Data Foundation in your OpenShift Container Platform cluster as software-defined storage for containers.

Red Hat OpenShift Data Foundation is a provider of agnostic persistent storage for OpenShift Container Platform supporting file, block, and object storage, either in-house or in hybrid clouds. As a Red Hat storage solution, Red Hat OpenShift Data Foundation is completely integrated with OpenShift Container Platform for deployment, management, and monitoring. For more information, see Red Hat OpenShift Data Foundation documentation.



IMPORTANT

OpenShift Data Foundation on top of Red Hat Hyperconverged Infrastructure (RHHi) for Virtualization, which uses hyperconverged nodes that host virtual machines installed with OpenShift Container Platform, is not a supported configuration. For more information about supported platforms, see the Red Hat OpenShift Data Foundation Supportability and Interoperability Guide.

Additional resources

- [Red Hat OpenShift Data Foundation documentation](#)
- [Red Hat OpenShift Data Foundation Supportability and Interoperability Guide](#)

4.11. PERSISTENT STORAGE USING VMWARE VSPHERE VOLUMES

OpenShift Container Platform allows use of VMware vSphere's Virtual Machine Disk (VMDK) volumes. You can provision your OpenShift Container Platform cluster with persistent storage using VMware vSphere. Some familiarity with Kubernetes and VMware vSphere is assumed.

VMware vSphere volumes can be provisioned dynamically. OpenShift Container Platform creates the disk in vSphere and attaches this disk to the correct image.



NOTE

OpenShift Container Platform provisions new volumes as independent persistent disks that can freely attach and detach the volume on any node in the cluster. Consequently, you cannot back up volumes that use snapshots, or restore volumes from snapshots. For more information, see "Snapshot Limitations".

The Kubernetes persistent volume framework allows administrators to provision a cluster with persistent storage and gives users a way to request those resources without having any knowledge of the underlying infrastructure.

Persistent volumes are not bound to a single project or namespace; they can be shared across the OpenShift Container Platform cluster. Persistent volume claims are specific to a project or namespace and can be requested by users.



IMPORTANT

For new installations, OpenShift Container Platform 4.13 and later provides automatic migration for the vSphere in-tree volume plugin to its equivalent CSI driver. Updating to OpenShift Container Platform 4.15 and later also provides automatic migration. For more information about updating and migration, see "CSI automatic migration".

CSI automatic migration should be seamless. Migration does not change how you use all existing API objects, such as persistent volumes, persistent volume claims, and storage classes.

You can provision VMware vSphere volumes dynamically or statically. However, dynamically provisioning VMware vSphere volumes is the recommended method.

Additional resources

- [VMware vSphere](#)
- [Installing a cluster on vSphere](#)

4.11.1. Dynamically provisioning VMware vSphere volumes using the UI

You can dynamically provision VMware vSphere volumes by using the OpenShift Container Platform web console to create persistent volume claims with the default **thin** storage class, so that your applications have on-demand access to vSphere storage without manual volume creation.

OpenShift Container Platform installs a default storage class, named **thin**, that uses the **thin** disk format for provisioning volumes.

You can use the following procedure to dynamically provision these volumes using the default storage class.

Prerequisites

- An OpenShift Container Platform cluster installed on a VMware vSphere version that meets the requirements for the components that you use. For more information, see "Installing a cluster on vSphere".
- Storage must exist in the underlying infrastructure before it can be mounted as a volume in OpenShift Container Platform.

Procedure

1. In the OpenShift Container Platform console, click **Storage → Persistent Volume Claims**
2. In the persistent volume claims overview, click **Create Persistent Volume Claim**

3. Define the required options on the resulting page.
 - a. Select the **thin** storage class.
 - b. Enter a unique name for the storage claim.
 - c. Select the access mode to determine the read and write access for the created storage claim.
 - d. Define the size of the storage claim.
4. Click **Create** to create the persistent volume claim and generate a persistent volume.

4.11.2. Dynamically provisioning VMware vSphere volumes using the CLI

You can dynamically provision VMware vSphere volumes from the CLI to provide persistent storage for your applications on-demand. OpenShift Container Platform installs a default StorageClass, named **thin**, that uses the **thin** disk format for provisioning.

Prerequisites

- An OpenShift Container Platform cluster installed on a VMware vSphere version that meets the requirements for the components that you use. For more information, see "Installing a cluster on vSphere".
- Storage must exist in the underlying infrastructure before it can be mounted as a volume in OpenShift Container Platform.

Procedure

1. You can define a VMware vSphere PersistentVolumeClaim by creating a file, **pvc.yaml**, with the following contents:

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

where:

metadata.name

Specifies a unique name that represents the persistent volume claim.

spec.accessModes.ReadWriteOnce

Specifies the access mode of the persistent volume claim. With **ReadWriteOnce**, the volume can be mounted with read and write permissions by a single node.

spec.resources.requests.storage

Specifies the size of the persistent volume claim.

2. Enter the following command to create the **PersistentVolumeClaim** object from the file:

```
$ oc create -f pvc.yaml
```

4.11.3. Statically provisioning VMware vSphere volumes

To statically provision VMware vSphere volumes you must create the virtual machine disks for reference by the persistent volume framework.

Prerequisites

- Storage must exist in the underlying infrastructure before it can be mounted as a volume in OpenShift Container Platform.

Procedure

1. Create the virtual machine disks. Virtual machine disks (VMDKs) must be created manually before statically provisioning VMware vSphere volumes. Use either of the following methods:

- Create using **vmkfstools**. Access ESX through Secure Shell (SSH) and then use following command to create a VMDK volume:

```
$ vmkfstools -c <size> /vmfs/volumes/<datastore-name>/volumes/<disk-name>.vmdk
```

- Create using **vmware-diskmanager**:

```
$ shell vmware-vdiskmanager -c -t 0 -s <size> -a lsilogic <disk-name>.vmdk
```

2. Create a persistent volume that references the VMDKs. Create a file, **pv1.yaml**, with the **PersistentVolume** object definition:

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv1
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  vsphereVolume:
    volumePath: "[datastore1] volumes/myDisk"
    fsType: ext4
```

where:

metadata.name

Specifies the name of the volume. This name is how it is identified by persistent volume claims or pods.

spec.capacity.storage

Specifies the amount of storage allocated to this volume.

spec.vsphereVolume

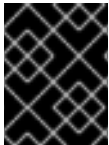
Specifies the volume type used, with **vsphereVolume** for vSphere volumes. The label is used to mount a vSphere VMDK volume into pods. The contents of a volume are preserved when it is unmounted. The volume type supports VMFS and VSAN datastore.

spec.vsphereVolume.volumePath

Specifies the existing VMDK volume to use. If you used **vmkfstools**, you must enclose the datastore name in square brackets, **[]**, in the volume definition, as shown previously.

spec.vsphereVolume.fsType

Specifies the file system type to mount. For example, ext4, xfs, or other file systems.

**IMPORTANT**

Changing the value of the fsType parameter after the volume is formatted and provisioned can result in data loss and pod failure.

1. Create the **PersistentVolume** object from the file:

```
$ oc create -f pv1.yaml
```

2. Create a persistent volume claim that maps to the persistent volume you created in the previous step. Create a file, **pvc1.yaml**, with the **PersistentVolumeClaim** object definition:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc1
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: "1Gi"
  volumeName: pv1
```

where:

metadata.name

Specifies a unique name that represents the persistent volume claim.

spec.accessModes.ReadWriteOnce

Specifies the access mode of the persistent volume claim. With **ReadWriteOnce**, the volume can be mounted with read and write permissions by a single node.

spec.resources.requests.storage

Specifies the size of the persistent volume claim.

spec.volumeName

Specifies the name of the existing persistent volume.

3. Create the **PersistentVolumeClaim** object from the file:

```
$ oc create -f pvc1.yaml
```

4.11.3.1. Formatting VMware vSphere volumes

You can use unformatted vSphere volumes as PVs because OpenShift Container Platform formats them before the first use.

Before OpenShift Container Platform mounts the volume and passes it to a container, it checks that the volume contains a file system that is specified by the **fsType** parameter value in the **PersistentVolume** (PV) definition. If the device is not formatted with the file system, all data from the device is erased, and the device is automatically formatted with the specified file system.

CHAPTER 5. PERSISTENT STORAGE USING LOCAL STORAGE

5.1. LOCAL STORAGE OVERVIEW

Local storage provides direct access to disks attached to cluster nodes, delivering lower latency and higher throughput than network-attached or cloud-based storage. Use local storage for performance-sensitive workloads, single-node clusters, or environments without cloud storage infrastructure.

5.1.1. Overview of local storage options

OpenShift Container Platform provides three solutions for provisioning node-local storage. Each offers different capabilities for dynamic provisioning and topology awareness. Node-local storage binds workloads to specific nodes.

You can use any of the following solutions to provision local storage:

- HostPath Provisioner (HPP)
- Local Storage Operator (LSO)
- Logical Volume Manager (LVM) Storage



WARNING

These solutions support provisioning only node-local storage. The workloads are bound to the nodes that provide the storage. If the node becomes unavailable, the workload also becomes unavailable. To maintain workload availability despite node failures, you must ensure storage data replication through active or passive replication mechanisms.

5.1.1.1. Overview of HostPath Provisioner functionality

You can perform the following actions using HostPath Provisioner (HPP):

- Map the host filesystem paths to storage classes for provisioning local storage.
- Statically create storage classes to configure filesystem paths on a node for storage consumption.
- Statically provision Persistent Volumes (PVs) based on the storage class.
- Create workloads and PersistentVolumeClaims (PVCs) while being aware of the underlying storage topology.



NOTE

HPP is available in upstream Kubernetes. However, it is not recommended to use HPP from upstream Kubernetes.

5.1.1.2. Overview of Local Storage Operator functionality

You can perform the following actions using Local Storage Operator (LSO):

- Assign the storage devices (disks or partitions) to the storage classes without modifying the device configuration.
- Statically provision PVs and storage classes by configuring the **LocalVolume** custom resource (CR).
- Create workloads and PVCs while being aware of the underlying storage topology.



NOTE

LSO is developed and delivered by Red Hat.

5.1.1.3. Overview of LVM Storage functionality

You can perform the following actions using Logical Volume Manager (LVM) Storage:

- Configure storage devices (disks or partitions) as lvm2 volume groups and expose the volume groups as storage classes.
- Create workloads and request storage by using PVCs without considering the node topology.

LVM Storage uses the TopoLVM CSI driver to dynamically allocate storage space to the nodes in the topology and provision PVs.



NOTE

LVM Storage is developed and maintained by Red Hat. The CSI driver provided with LVM Storage is the upstream project "topolvm".

5.1.2. Comparison of LVM Storage, LSO, and HPP

Compare LVM Storage, Local Storage Operator (LSO), and HostPath Provisioner (HPP) across storage types, core features, performance, and isolation to determine the best local storage provisioning solution for your cluster.

5.1.2.1. Comparison of the support for storage types and filesystems

The following table compares the support for storage types and filesystems provided by LVM Storage, Local Storage Operator (LSO), and HostPath Provisioner (HPP) to provision local storage:

Table 5.1. Comparison of the support for storage types and filesystems

Functionality	LVM Storage	LSO	HPP
Support for block storage	Yes	Yes	No
Support for file storage	Yes	Yes	Yes
Support for object storage ^[1]	No	No	No

Functionality	LVM Storage	LSO	HPP
Available filesystems	ext4, xfs	ext4, xfs	Any mounted system available on the node is supported.

1. None of the solutions (LVM Storage, LSO, and HPP) provide support for object storage. Therefore, if you want to use object storage, you need an S3 object storage solution, such as **MultiClusterGateway** from the Red Hat OpenShift Data Foundation. All of the solutions can serve as underlying storage providers for the S3 object storage solutions.

5.1.2.2. Comparison of the support for core functionalities

The following table compares how LVM Storage, Local Storage Operator (LSO), and HostPath Provisioner (HPP) support core functionalities for provisioning local storage:

Table 5.2. Comparison of the support for core functionalities

Functionality	LVM Storage	LSO	HPP
Support for automatic file system formatting	Yes	Yes	N/A
Support for dynamic provisioning	Yes	No	No
Support for using software Redundant Array of Independent Disks (RAID) arrays	Yes Supported on 4.15 and later.	Yes	Yes
Support for transparent disk encryption	Yes Supported on 4.16 and later.	Yes	Yes
Support for volume based disk encryption	No	No	No
Support for disconnected installation	Yes	Yes	Yes
Support for PVC expansion	Yes	No	No
Support for volume snapshots and volume clones	Yes	No	No

Functionality	LVM Storage	LSO	HPP
Support for thin provisioning	Yes Devices are thin-provisioned by default.	Yes You can configure the devices to point to the thin-provisioned volumes	Yes You can configure a path to point to the thin-provisioned volumes.
Support for automatic disk discovery and setup	Yes Automatic disk discovery is available during installation and runtime. You can also dynamically add the disks to the LVMCluster custom resource (CR) to increase the storage capacity of the existing storage classes.	Technology Preview Automatic disk discovery is available during installation.	No

5.1.2.3. Comparison of performance and isolation capabilities

The following table compares the performance and isolation capabilities of LVM Storage, Local Storage Operator (LSO), and HostPath Provisioner (HPP) in provisioning local storage.

Table 5.3. Comparison of performance and isolation capabilities

Functionality	LVM Storage	LSO	HPP
Performance	I/O speed is shared for all workloads that use the same storage class. Block storage allows direct I/O operations. Thin provisioning can affect the performance.	I/O depends on the LSO configuration. Block storage allows direct I/O operations.	I/O speed is shared for all workloads that use the same storage class. The restrictions imposed by the underlying filesystem can affect the I/O speed.
Isolation boundary ^[1]	LVM Logical Volume (LV) It provides higher level of isolation compared to HPP.	LVM Logical Volume (LV) It provides higher level of isolation compared to HPP	Filesystem path It provides lower level of isolation compared to LSO and LVM Storage.

1. Isolation boundary refers to the level of separation between different workloads or applications that use local storage resources.

5.1.2.4. Comparison of the support for additional functionalities

The following table compares the additional features provided by LVM Storage, Local Storage Operator (LSO), and HostPath Provisioner (HPP) to provision local storage:

Table 5.4. Comparison of the support for additional functionalities

Functionality	LVM Storage	LSO	HPP
Support for generic ephemeral volumes	Yes	No	No
Support for CSI inline ephemeral volumes	No	No	No
Support for storage topology	Yes Supports CSI node topology	Yes LSO provides partial support for storage topology through node tolerations.	No
Support for ReadWriteMany (RWX) access mode ^[1]	No	No	No

1. All of the solutions (LVM Storage, LSO, and HPP) have the **ReadWriteOnce** (RWO) access mode. RWO access mode allows access from multiple pods on the same node.

5.2. PERSISTENT STORAGE USING LOCAL VOLUMES

OpenShift Container Platform can be provisioned with persistent storage by using local volumes. Local persistent volumes allow you to access local storage devices, such as a disk or partition, by using the standard persistent volume claim interface.

5.2.1. Local Storage Operator overview

Local volumes can be used without manually scheduling pods to nodes because the system is aware of the volume node constraints. However, local volumes are still subject to the availability of the underlying node and are not suitable for all applications.



NOTE

Local volumes can only be used as a statically created persistent volume.

5.2.2. Installing the Local Storage Operator

You can install the Local Storage Operator (LSO) to provision and manage local persistent storage volumes in your cluster.

1. Create the **openshift-local-storage** project.
2. Install and configure the LSO by using either the CLI or OpenShift Container Platform web console.

5.2.2.1. Creating a local project for Local Storage Operator

To install the Local Storage Operator (LSO) to provision and manage local persistent storage volumes in your cluster, first create the **openshift-local-storage** project.

Prerequisites

- Access to the OpenShift Container Platform web console or command-line interface (CLI).

Procedure

1. Create the **openshift-local-storage** project by running the following command:

```
$ oc adm new-project openshift-local-storage
```

2. Optional: Allow local storage creation on infrastructure nodes.
You might want to use the LSO to create volumes on infrastructure nodes in support of components such as logging and monitoring.

You must adjust the default node selector so that the LSO includes the infrastructure nodes, and not just worker nodes.

To block the LSO from inheriting the cluster-wide default selector, run the following command:

```
$ oc annotate namespace openshift-local-storage openshift.io/node-selector=
```

3. Optional: Allow local storage to run on the management pool of CPUs in single-node deployment.
Use the LSO in single-node deployments and allow the use of CPUs that belong to the **management** pool. Perform this step on single-node installations that use management workload partitioning.

To allow LSO to run on the management CPU pool, run following command:

```
$ oc annotate namespace openshift-local-storage  
workload.openshift.io/allowed='management'
```

Next steps

Install the LSO Operator.

5.2.2.2. Installing the Local Storage Operator by using the CLI

Install the Local Storage Operator (LSO) to provision and manage local persistent storage volumes in your cluster using the command-line interface (CLI).

The LSO is not installed in OpenShift Container Platform by default. Use the following procedure to install and configure this Operator to enable local volumes in your cluster.

Prerequisites

- Access to an **openshift-local-storage** project.
- Access to the OpenShift Container Platform command-line interface (CLI).

Procedure

1. Create an object YAML file to define an Operator group and subscription for the LSO, such as **openshift-local-storage.yaml**:

Example openshift-local-storage.yaml

```
apiVersion: operators.coreos.com/v1
kind: OperatorGroup
metadata:
  name: local-operator-group
  namespace: openshift-local-storage
spec:
  targetNamespaces:
    - openshift-local-storage
---
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: local-storage-operator
  namespace: openshift-local-storage
spec:
  channel: stable
  installPlanApproval: Automatic
  name: local-storage-operator
  source: redhat-operators
  sourceNamespace: openshift-marketplace
```

spec.installPlanApproval is the user approval policy for an install plan.

2. Create the LSO object by running the following command:

```
$ oc apply -f openshift-local-storage.yaml
```

The Operator Lifecycle Manager (OLM) is now aware of the LSO. A ClusterServiceVersion (CSV) for the Operator appears in the target namespace, and APIs provided by the Operator should be available for creation.

3. Verify local storage installation by checking that all pods and the LSO have been created:
 - a. Check that all the required pods have been created by running the following command:

```
$ oc -n openshift-local-storage get pods
```

Example output

```
NAME                                READY STATUS  RESTARTS  AGE
local-storage-operator-746bf599c9-vlt5t  1/1   Running  0         19m
```

- b. Check the ClusterServiceVersion (CSV) YAML manifest to see that the LSO is available in the **openshift-local-storage** project:

```
$ oc get csvs -n openshift-local-storage
```

Example output

NAME	DISPLAY	VERSION	REPLACES	PHASE
local-storage-operator.4.2.26-202003230335	Local Storage	4.2.26-202003230335		Succeeded

Result

After all checks have passed, the LSO is installed successfully.

5.2.2.3. Installing the Local Storage Operator by using the OpenShift Container Platform web console

Install the Local Storage Operator (LSO) to provision and manage local persistent storage volumes in your cluster using the OpenShift Container Platform web console.

The LSO is not installed in OpenShift Container Platform by default. Use the following procedure to install and configure this Operator to enable local volumes in your cluster.

Prerequisites

- Access to an **openshift-local-storage** project.
- Access to the OpenShift Container Platform web console.

Procedure

1. Log in to the OpenShift Container Platform web console.
2. Navigate to **Ecosystem → Software Catalog**.
3. Type **Local Storage** into the filter box to locate the LSO.
4. Click **Install**.
5. On the **Install Operator** page, select **A specific namespace on the cluster**. Select **openshift-local-storage** from the drop-down menu.
6. Adjust the values for **Update Channel** and **Approval Strategy** to the values that you want.
7. Click **Install**.

Result

After finishing, the LSO is listed in the **Installed Operators** section of the OpenShift Container Platform web console.

5.2.3. Provisioning local volumes by using the Local Storage Operator

Provision local volumes for persistent storage by creating a **LocalVolume** resource that defines nodes and device paths, because local volumes cannot be created by dynamic provisioning and must be statically provisioned by the Local Storage Operator (LSO).

Prerequisites

- The LSO is installed.
- You have a local disk that meets the following conditions:
 - It is attached to a node.
 - It is not mounted.
 - It does not contain partitions.

Procedure

1. Create the local volume resource. This resource must define the nodes and paths to the local volumes.



IMPORTANT

- Editing the **LocalVolume** object does not change the **fsType** or **volumeMode** of existing persistent volumes because doing so might result in a destructive operation.
- Do not use different storage class names for the same device. Doing so creates multiple persistent volumes (PVs).

Example: Filesystem

```
apiVersion: "local.storage.openshift.io/v1"
kind: "LocalVolume"
metadata:
  name: "local-disks"
  namespace: "openshift-local-storage"
spec:
  nodeSelector:
    nodeSelectorTerms:
      - matchExpressions:
          - key: kubernetes.io/hostname
            operator: In
            values:
              - ip-10-0-140-183
              - ip-10-0-158-139
              - ip-10-0-164-33
  storageClassDevices:
    - storageClassName: "local-sc"
      forceWipeDevicesAndDestroyAllData: false
      volumeMode: Filesystem
      fsType: xfs
      devicePaths:
        - /path/to/device
```

- **metadata.namespace:** Specifies the namespace where the LSO is installed.
- **spec.nodeSelector:** (Optional) A node selector containing a list of nodes where the local storage volumes are attached. This example uses the node hostnames, obtained from **oc get node**. If a value is not defined, then the LSO will attempt to find matching disks on all available nodes.

- **spec.storageClassDevices.storageClassName:** Specifies the name of the storage class to use when creating persistent volume objects. The LSO automatically creates the storage class if it does not exist. Be sure to use a storage class that uniquely identifies this set of local volumes.
- **spec.storageClassDevices.forceWipeDevicesAndDestroyAllData:** This setting defines whether or not to call **wipefs**, which removes partition table signatures (magic strings) making the disk ready to use for LSO provisioning. No other data besides signatures is erased. The default is "false" (**wipefs** is not invoked). Setting **forceWipeDevicesAndDestroyAllData** to "true" can be useful in scenarios where previous data can remain on disks that need to be re-used. In these scenarios, setting this field to true eliminates the need for administrators to erase the disks manually. Such cases can include single-node OpenShift cluster environments where a node can be redeployed multiple times or when using OpenShift Data Foundation, where previous data can remain on the disks planned to be consumed as object storage devices (OSDs).
- **spec.storageClassDevices.volumeMode:** Specifies the volume mode, either **Filesystem** or **Block**, that defines the type of local volumes.

**NOTE**

A raw block volume (**volumeMode: Block**) is not formatted with a file system. Use this mode only if any application running on the pod can use raw block devices.

- **spec.storageClassDevices.fsType:** The file system that is created when the local volume is mounted for the first time.
- **spec.storageClassDevices.devicePaths:** The path containing a list of local storage devices to choose from. Replace this value with your actual local disks filepath to the **LocalVolume** resource **by-id**, such as **/dev/disk/by-id/wwn**. PVs are created for these local disks when the provisioner is deployed successfully.

**NOTE**

If you are running OpenShift Container Platform with RHEL KVM, you must assign a serial number to your VM disk. Otherwise, the VM disk can not be identified after reboot. You can use the **virsh edit <VM>** command to add the **<serial>mydisk</serial>** definition.

Example: Block

```
apiVersion: "local.storage.openshift.io/v1"
kind: "LocalVolume"
metadata:
  name: "local-disks"
  namespace: "openshift-local-storage"
spec:
  nodeSelector:
    nodeSelectorTerms:
      - matchExpressions:
          - key: kubernetes.io/hostname
            operator: In
            values:
```



```

- ip-10-0-136-143
- ip-10-0-140-255
- ip-10-0-144-180
storageClassDevices:
- storageClassName: "local-sc"
  forceWipeDevicesAndDestroyAllData: false
  volumeMode: Block
  devicePaths:
  - /path/to/device

```

- **metadata.namespace:** Specifies the namespace where the LSO is installed.
- **spec.nodeSelector:** (Optional) A node selector containing a list of nodes where the local storage volumes are attached. This example uses the node hostnames, obtained from **oc get node**. If a value is not defined, then the LSO will attempt to find matching disks on all available nodes.
- **spec.storageClassDevices.storageClassName:** Specifies the name of the storage class to use when creating persistent volume objects.
- **spec.storageClassDevices.forceWipeDevicesAndDestroyAllData:** This setting defines whether or not to call **wipefs**, which removes partition table signatures (magic strings) making the disk ready to use for LSO provisioning. No other data besides signatures is erased. The default is "false" (**wipefs** is not invoked). Setting **forceWipeDevicesAndDestroyAllData** to "true" can be useful in scenarios where previous data can remain on disks that need to be re-used. In these scenarios, setting this field to true eliminates the need for administrators to erase the disks manually. Such cases can include single-node OpenShift cluster environments where a node can be redeployed multiple times or when using OpenShift Data Foundation, where previous data can remain on the disks planned to be consumed as object storage devices (OSDs).
- **spec.storageClassDevices.volumeMode:** Specifies the volume mode, either **Filesystem** or **Block**, that defines the type of local volumes.
- **spec.storageClassDevices.devicePaths:** Specifies the path containing a list of local storage devices to choose from. Replace this value with your actual local disks filepath to the **LocalVolume** resource **by-id**, such as **dev/disk/by-id/wwn**. PVs are created for these local disks when the provisioner is deployed successfully.



NOTE

If you are running OpenShift Container Platform with RHEL KVM, you must assign a serial number to your VM disk. Otherwise, the VM disk can not be identified after reboot. You can use the **virsh edit <VM>** command to add the **<serial>mydisk</serial>** definition.

2. Create the local volume resource in your OpenShift Container Platform cluster. Specify the file you just created:

```
$ oc create -f <local-volume>.yaml
```

3. Verify that the provisioner was created and that the corresponding daemon sets were created:

```
$ oc get all -n openshift-local-storage
```

Example output

```

NAME                                READY STATUS  RESTARTS  AGE
pod/diskmaker-manager-9wzms         1/1   Running  0         5m43s
pod/diskmaker-manager-jgvjp         1/1   Running  0         5m43s
pod/diskmaker-manager-tbdsj         1/1   Running  0         5m43s
pod/local-storage-operator-7db4bd9f79-t6k87 1/1   Running  0         14m

NAME                                TYPE      CLUSTER-IP  EXTERNAL-IP  PORT(S)
AGE
service/local-storage-operator-metrics ClusterIP  172.30.135.36 <none>
8383/TCP,8686/TCP 14m

NAME            DESIRED  CURRENT  READY  UP-TO-DATE  AVAILABLE
NODE SELECTOR  AGE
daemonset.apps/diskmaker-manager 3        3        3      3           3        <none>
5m43s

NAME            READY  UP-TO-DATE  AVAILABLE  AGE
deployment.apps/local-storage-operator 1/1    1           1          14m

NAME            DESIRED  CURRENT  READY  AGE
replicaset.apps/local-storage-operator-7db4bd9f79 1      1        1      14m

```

Note the desired and current number of daemon set processes. A desired count of **0** indicates that the label selectors were invalid.

4. Verify that the persistent volumes were created by running the following command:

```
$ oc get pv
```

Example output

```

NAME            CAPACITY  ACCESS MODES  RECLAIM POLICY  STATUS  CLAIM
STORAGECLASS  REASON  AGE
local-pv-1cec77cf 100Gi    RWO           Delete          Available  local-sc      88m
local-pv-2ef7cd2a 100Gi    RWO           Delete          Available  local-sc
82m
local-pv-3fa1c73 100Gi    RWO           Delete          Available  local-sc      48m

```

5.2.4. Provisioning local volumes without the Local Storage Operator

Provision local volumes manually by defining **PersistentVolume** objects without using the Local Storage Operator (LSO), though this approach includes risk of potential data leaks and the operator is recommended for automating device lifecycle.

**IMPORTANT**

Manual provisioning of persistent volumes (PVs) includes the risk of potential data leaks across PV reuse when persistent volume claims (PVCs) are deleted. The Local Storage Operator is recommended for automating the life cycle of devices when provisioning local PVs.

Prerequisites

- Local disks are attached to the OpenShift Container Platform nodes.

Procedure

1. Define the PV. Create a file, such as **example-pv-filesystem.yaml** or **example-pv-block.yaml**, with the **PersistentVolume** object definition. This resource must define the nodes and paths to the local volumes.



NOTE

Do not use different storage class names for the same device. Doing so creates multiple PVs.

Example-pv-filesystem.yaml

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: example-pv-filesystem
spec:
  capacity:
    storage: 100Gi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Delete
  storageClassName: local-sc
  local:
    path: /dev/xvdf
  nodeAffinity:
    required:
      nodeSelectorTerms:
        - matchExpressions:
            - key: kubernetes.io/hostname
              operator: In
              values:
                - example-node
```

- **spec.volumeMode:** Specifies the volume mode, either **Filesystem** or **Block**, that defines the type of PVs.
- **spec.storageClassName:** Specifies the name of the storage class to use when creating PV resources. Use a storage class that uniquely identifies this set of PVs.
- **spec.local.path:** Specifies the path containing a list of local storage devices to choose from, or a directory. You can only specify a directory with **Filesystem volumeMode**.



NOTE

A raw block volume (**volumeMode: block**) is not formatted with a file system. Use this mode only if any application running on the pod can use raw block devices.

Example-pv-block.yaml

```

apiVersion: v1
kind: PersistentVolume
metadata:
  name: example-pv-block
spec:
  capacity:
    storage: 100Gi
  volumeMode: Block
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Delete
  storageClassName: local-sc
  local:
    path: /dev/xvdf
  nodeAffinity:
    required:
      nodeSelectorTerms:
        - matchExpressions:
            - key: kubernetes.io/hostname
              operator: In
              values:
                - example-node

```

- `spec.volumeMode`: Specifies the volume mode, either **Filesystem** or **Block**, that defines the type of PVs.
 - **`spec.storageClassName`**: Specifies the name of the storage class to use when creating PV resources. Be sure to use a storage class that uniquely identifies this set of PVs.
 - **`spec.local.path`**: Specifies the path containing a list of local storage devices to choose from.
2. Specifying the file that you just created, create the PV resource in your OpenShift Container Platform cluster by running the following command:

```
$ oc create -f <example-pv>.yaml
```

3. Verify that the local PV was created:

```
$ oc get pv
```

Example output

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS	CLAIM
STORAGECLASS	REASON	AGE			
example-pv-filesystem	100Gi	RWO	Delete	Available	local-sc
example-pv1	1Gi	RWO	Delete	Bound	local-storage/pvc1
sc 12h					
example-pv2	1Gi	RWO	Delete	Bound	local-storage/pvc2
					local-

sc	12h						
example-pv3		1Gi	RWO	Delete	Bound	local-storage/pvc3	local-
sc	12h						

5.2.5. Creating the local volume persistent volume claim

Create a persistent volume claim (PVC) to access local volumes in your pod, because local volumes must be statically created and cannot use dynamic provisioning.

Prerequisites

- Persistent volumes have been created using the local volume provisioner.

Procedure

1. Create the PVC using the corresponding storage class:

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: local-pvc-name
spec:
  accessModes:
    - ReadWriteOnce
  volumeMode: Filesystem
  resources:
    requests:
      storage: 100Gi
  storageClassName: local-sc
```

- **metadata.name:** Specifies the name of the PVC.
 - **spec.volumeMode:** Specifies the type of the PVC. Defaults to **Filesystem**.
 - **spec.resources.requests.storage:** Specifies the amount of storage available to the PVC.
 - **spec.storageClassName:** Specifies the name of the storage class required by the claim.
2. Create the PVC in the OpenShift Container Platform cluster, specifying the file you just created, by running the following command:

```
$ oc create -f <local-pvc>.yaml
```

5.2.6. Attach the local claim

After a local volume has been mapped to a persistent volume claim (PVC), attach the claim to a pod by specifying it in the pod specification to make the local storage available to the application.

Prerequisites

- A persistent volume claim exists in the same namespace.

Procedure

1. Include the defined claim in the resource spec. The following example declares the persistent volume claim inside a pod:

```
apiVersion: v1
kind: Pod
spec:
# ...
  containers:
    volumeMounts:
      - name: local-disks
        mountPath: /data
    volumes:
      - name: local-disks
        persistentVolumeClaim:
          claimName: local-pvc-name
# ...
```

- **spec...containers.volumeMounts.name:** Specifies the name of the volume to mount.
 - **spec...containers.volumeMounts.mountPath:** Specifies the path inside the pod where the volume is mounted. Do not mount to the container root, /, or any path that is the same in the host and the container. This can corrupt your host system if the container is sufficiently privileged, such as the host **/dev/pts** files. It is safe to mount the host by using **/host**.
 - **spec...volumes.persistentVolumeClaim.claimName:** Specifies the name of the existing persistent volume claim to use.
2. Create the resource in the OpenShift Container Platform cluster, specifying the file you just created, by running the following command:

```
$ oc create -f <local-pod>.yaml
```

5.2.7. Automating discovery and provisioning for local storage devices

Automate local storage discovery and provisioning using the Local Storage Operator (LSO) to simplify installation when dynamic provisioning is not available, such as with bare metal, VMware vSphere, or Amazon Web Services (AWS) instances with attached devices.

IMPORTANT

Automatic discovery and provisioning is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

IMPORTANT

Automatic discovery and provisioning is fully supported when used to deploy Red Hat OpenShift Data Foundation on-premise or with platform-agnostic deployment.

Use the following procedure to automatically discover local devices, and to automatically provision local volumes for selected devices.



WARNING

Use the **LocalVolumeSet** object with caution. When you automatically provision persistent volumes (PVs) from local disks, the local PVs might claim all devices that match. If you are using a **LocalVolumeSet** object, make sure the LSO is the only entity managing local devices on the node. Creating multiple instances of a **LocalVolumeSet** that target a node more than once is not supported.

Prerequisites

- You have cluster administrator permissions.
- You have installed the LSO.
- You have attached local disks to OpenShift Container Platform nodes.
- You have access to the OpenShift Container Platform web console and the **oc** command-line interface (CLI).

Procedure

1. Enable automatic discovery of local devices from the OpenShift Container Platform web console:
 - a. Click **Ecosystem** → **Installed Operators**.
 - b. In the **openshift-local-storage** namespace, click **Local Storage**.
 - c. Click the **Local Volume Discovery** tab.
 - d. Click **Create Local Volume Discovery** and then select either **Form view** or **YAML view**.
 - e. Configure the **LocalVolumeDiscovery** object parameters.
 - f. Click **Create**.
The LSO creates a local volume discovery instance named **auto-discover-devices**.
2. Display a continuous list of available devices on a node:
 - a. Log in to the OpenShift Container Platform web console.
 - b. Click **Compute** → **Nodes**.
 - c. Click the node name that you want to open. The "Node Details" page is displayed.
 - d. Click the **Disks** tab to display the list of the selected devices.
The device list updates continuously as local disks are added or removed. You can filter the devices by name, status, type, model, capacity, and mode.

3. Automatically provision local volumes for the discovered devices from the OpenShift Container Platform web console:
 - a. Click **Ecosystem** → **Installed Operators** and select **Local Storage** from the list of Operators.
 - b. Click **Local Volume Set** → **Create Local Volume Set**
 - c. Enter a volume set name and a storage class name.
 - d. Click **All nodes** or **Select nodes** to apply filters accordingly.

**NOTE**

Only worker nodes are available, regardless of whether you filter using **All nodes** or **Select nodes**.

- e. Select the disk type, mode, size, and limit that you want to apply to the local volume set, and then click **Create**.
A message is displayed after several minutes, indicating that the "Operator reconciled successfully."
4. Alternatively, provision local volumes for the discovered devices from the CLI:
 - a. Create an object YAML file to define the local volume set, such as **local-volume-set.yaml**, as shown in the following example:

Example local volume set YAML file

```

apiVersion: local.storage.openshift.io/v1alpha1
kind: LocalVolumeSet
metadata:
  name: example-autodetect
spec:
  nodeSelector:
    nodeSelectorTerms:
      - matchExpressions:
          - key: kubernetes.io/hostname
            operator: In
            values:
              - worker-0
              - worker-1
  storageClassName: local-sc
  volumeMode: Filesystem
  fsType: ext4
  maxDeviceCount: 10
  deviceInclusionSpec:
    deviceTypes:
      - disk
      - part
    deviceMechanicalProperties:
      - NonRotational
  minSize: 10G
  maxSize: 100G
  models:
    - SAMSUNG

```



```
- Crucial_CT525MX3
vendors:
- ATA
- ST2000LM
```

- **spec.storageClassName:** Determines the storage class that is created for persistent volumes that are provisioned from discovered devices. The LSO automatically creates the storage class if it does not exist. Be sure to use a storage class that uniquely identifies this set of local volumes.
- **spec.deviceInclusionSpec.deviceTypes:** When using the local volume set feature, the LSO does not support the use of logical volume management (LVM) devices.

b. Create the local volume set object:

```
$ oc apply -f local-volume-set.yaml
```

c. Verify that the local persistent volumes were dynamically provisioned based on the storage class:

```
$ oc get pv
```

Example output

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS
CLAIM STORAGECLASS	REASON	AGE		
local-pv-1cec77cf	100Gi	RWO	Delete	Available
88m				local-sc
local-pv-2ef7cd2a	100Gi	RWO	Delete	Available
82m				local-sc
local-pv-3fa1c73	100Gi	RWO	Delete	Available
48m				local-sc

Next steps

Results are deleted after they are removed from the node. Symlinks must be manually removed.

5.2.8. Local Storage Operator symlinks management

To prevent storage breakage during OpenShift Container Platform upgrades, OpenShift Container Platform provides a mechanism, the **LocalVolumeDeviceLink** Custom Resource Definition, to detect, alert, and remap broken symlinks without manual node-level intervention.

5.2.8.1. Local Storage Operator symlinks overview

The Local Storage Operator (LSO) traditionally creates persistent volumes (PVs) based on **/dev/disk/by-id/** paths, following the assumption that they are stable. However, Linux kernel updates, firmware updates, or **udev** rule changes can cause these supposedly stable names to change or disappear.

Administrator options

Administrators have the following notification and correction options to deal with symlink disruptions:

- **Monitoring:** (default) If the current and preferred path do not match, an alert occurs, but no changes occur to the current path.
- **Use existing path:** Alerts are silenced and LSO uses the existing path.
- **Recreate symlinks:** Symlinks are re-created to point to the new, updated device path.

5.2.8.2. Responding to symlinks alerts for the Local Storage Operator

To prevent storage breakage during OpenShift Container Platform upgrades, an administrator can elect to detect, alert, and remap broken symlinks without manual node-level intervention.

By default, LSO engages in link monitoring and generates an alert if the current and preferred paths do not match.

If an alert occurs, an administrator can choose to either have LSO:

- Use the existing path.
- Re-create the symlink to point to the new, updated device path.

Prerequisites

- Access to the OpenShift Container Platform web console or command-line interface (CLI) with administrative privileges.
- Install the Local Storage Operator (LSO). For more information, see "Installing the Local Storage Operator".

Procedure

1. Log in to the OpenShift Container Platform web console.
2. To view volumes generating alerts:
 - a. On the left navigation menu, click **Observe > Alerting**.
 - b. In the **Alert Name** filter box, search for the required LSO alerts:

Table 5.5. Symlink alerts for LSO

Alert	Description
Iso_no_stable_volume_path	Device does not have a stable path and is being referenced by device name, which can change between reboots
Iso_device_link_mismatch	Device has mismatching preferred and current symlink
Iso_lv_missing_device_path	LV object has missing devicePath on actual node

Alternatively, you can list volumes triggering device link alerts by running the following command:

```
$ oc get localvolumedevice link -A -o json | jq -r '
  .items[] |
  select(
    .spec.policy == "None" and
    (.status.currentLinkTarget != .status.preferredLinkTarget or .status.currentLinkTarget
    == "")
  ) |
  [.metadata.namespace, .metadata.name, .spec.persistentVolumeName,
  .status.currentLinkTarget // "MISSING", .status.preferredLinkTarget // "NONE"] |
  @tsv' |
  column -t -s $'\t' -N "NAMESPACE,NAME,PV,CURRENT-TARGET,PREFERRED-
  TARGET"
```

Example output

```
NAMESPACE      NAME                      PV                      CURRENT-TARGET
PREFERRED-TARGET
openshift-storage local-pv-1a2b3c-worker-0-block local-pv-1a2b3c /dev/disk/by-
id/scsi-0NVME_MODEL_abcde /dev/disk/by-id/scsi-2ace42e0035eabcde
openshift-storage local-pv-4d5e6f-worker-1-block local-pv-4d5e6f /dev/disk/by-
id/scsi-0NVME_MODEL_fghij /dev/disk/by-id/scsi-2ace42e0035efghij
openshift-storage local-pv-7g8h9i-worker-2-block local-pv-7g8h9i MISSING
/dev/disk/by-id/scsi-35000c500a1b2c3d4
```

3. Open the Custom Resource Definition, **localVolumeDeviceLink**:

- a. Click **Ecosystem > Installed Operators**.
- b. On the **Installed Operators** page, in the **Search by name** box, type "LocalVolumeDeviceLink".
- c. Click **LocalVolumeDeviceLink**.
Local Storage Operator creates **LocalVolumeDeviceLink** objects for each individual device (or partition or volume) it is managing. A cluster administrator has the option of specifying how LSO should handle symlinks for that particular device when underlying symlinks change because of unforeseen circumstances.
- d. On the **Operator details** page, click the **YAML** tab.
- e. Go to the **localVolumeDeviceLink.status** field and view its nested fields that are shown in the following table for a list of valid symlink targets, current link (**by-id**), and the generated preferred symlink.

Table 5.6. localVolumeDeviceLink.status nested fields

Status fields	Description
validLinkTargets	The full list of valid symlink targets, since there might be multiple by-id symlinks pointing to the same physical device.

Status fields	Description
currentLinkTarget	The by-id symlink currently used by the PV.
preferredLinkTarget	The preferred symlink LSO has determined that is less likely to change because of udev rules updates, firmware updates, and so on.
filesystemUUID	The corresponding UUID of the filesystem, if one is found.

Alternatively, you can view the status from the command line by running the following command:

```
$ oc get localvolumedevicelink local-pv-1a2b3c-worker-0-block -n openshift-storage -o jsonpath='{.status}' | jq
```

Example

```
{
  "currentLinkTarget": "/dev/disk/by-id/scsi-0NVME_MODEL_abcde",
  "preferredLinkTarget": "/dev/disk/by-id/scsi-2ace42e0035eabcde",
  "validLinkTargets": [
    "/dev/disk/by-id/nvme-eui.ace42e0035eabcde",
    "/dev/disk/by-id/scsi-0NVME_MODEL_abcde",
    "/dev/disk/by-id/scsi-2ace42e0035eabcde"
  ],
  "filesystemUUID": "a1b2c3d4-e5f6-7890-abcd-ef1234567890"
}
```

- f. Got to the **localVolumeDeviceLink.spec.policy** field and set its value to one of the following options:

- **CurrentLinkTarget:** Silences alerts and tells LSO to use the existing path.
- **PreferredLinkTarget:** LSO recreates the symlink to point to the new, updated device path. Note that selecting this option does not mean that all symlinks are fixed for LV objects.

Alternatively, you can set the policy from the command line:

- To set the policy to **CurrentLinkTarget**, run the following command:

```
$ oc patch localvolumedevicelink local-pv-1a2b3c-worker-0-block -n openshift-storage \
--type merge -p '{"spec":{"policy":"CurrentLinkTarget"}}'
```

- To set the policy to **PreferredLinkTarget**, run the following command:

```
$ oc patch localvolumedevicelink local-pv-1a2b3c-worker-0-block -n openshift-storage \
--type merge -p '{"spec":{"policy":"PreferredLinkTarget"}}'
```

The following example output applies to both preceding commands:

Example

```
localvolumedevicelink.local.storage.openshift.io/local-pv-1a2b3c-worker-0-block
patched
```

4. Click **Save**.
5. (Optional) Set **localVolumeDeviceLink.spec.policy** back to null.
The default value of **localVolumeDeviceLink.spec.policy** is none, which means LSO is monitoring the volumes used by the OpenShift cluster, but does not automatically change symlinks for PVs if existing symlinks are broken or LSO finds a better symlink to use. In this state, LSO just alerts you if any anomaly is detected.
6. Click **Save**.

5.2.9. Using tolerations with Local Storage Operator pods

Configure tolerations in the **LocalVolume** resource to allow Local Storage Operator (LSO) pods to run on tainted nodes, enabling local storage provisioning on nodes that would otherwise repel all pods.

Taints can be applied to nodes to prevent them from running general workloads. To allow the LSO to use tainted nodes, you must add tolerations to the **Pod** or **DaemonSet** definition. This allows the created resources to run on these tainted nodes.

You apply tolerations to the LSO pod through the **LocalVolume** resource and apply taints to a node through the node specification. A taint on a node instructs the node to repel all pods that do not tolerate the taint. Using a specific taint that is not on other pods ensures that the LSO pod can also run on that node.



IMPORTANT

Taints and tolerations consist of a key, value, and effect. As an argument, it is expressed as **key=value:effect**. An operator allows you to leave one of these parameters empty.

Prerequisites

- The LSO is installed.
- Local disks are attached to OpenShift Container Platform nodes with a taint.
- Tainted nodes are expected to provision local storage.

Procedure

1. Modify the YAML file that defines the **Pod** and add the **LocalVolume** spec, as shown in the following example:

```
apiVersion: "local.storage.openshift.io/v1"
kind: "LocalVolume"
metadata:
  name: "local-disks"
  namespace: "openshift-local-storage"
```

```
spec:
  tolerations:
    - key: localstorage
      operator: Equal
      value: "localstorage"
  storageClassDevices:
    - storageClassName: "local-sc"
      volumeMode: Block
      devicePaths:
        - /dev/xvdg
```

- **spec.tolerations.key**: Specifies the key that you added to the node.
 - **spec.tolerations.operator**: Specifies the **Equal** operator to require the **key/value** parameters to match. If operator is **Exists**, the system checks that the key exists and ignores the value. If operator is **Equal**, then the key and value must match.
 - **spec.tolerations.value**: Specifies the value **local** of the tainted node.
 - **spec.storageClassDevices.volumeMode**: Specifies the volume mode, either **Filesystem** or **Block**, defining the type of the local volumes.
 - **spec.storageClassDevices.devicePaths**: Specifies the path containing a list of local storage devices to choose from.
2. Optional: To create local persistent volumes on only tainted nodes, modify the YAML file and add the **LocalVolume** spec, as shown in the following example:

```
spec:
  tolerations:
    - key: node-role.kubernetes.io/master
      operator: Exists
```

Results

The defined tolerations is passed to the resulting daemon sets, allowing the diskmaker and provisioner pods to be created for nodes that contain the specified taints.

5.2.10. Local Storage Operator Metrics

To understand storage utilization and troubleshoot provisioning issues, you can track Local Storage Operator (LSO) operations with available metrics that monitor disk discovery, persistent volume creation, unmatched disks, and orphaned symlinks.

OpenShift Container Platform provides the following metrics for the LSO:

- **iso_discovery_disk_count**: total number of discovered devices on each node
- **iso_lvset_provisioned_PV_count**: total number of PVs created by **LocalVolumeSet** objects
- **iso_lvset_unmatched_disk_count**: total number of disks that LSO did not select for provisioning because of mismatching criteria
- **iso_lvset_orphaned_symlink_count**: number of devices with PVs that no longer match **LocalVolumeSet** object criteria

- **iso_lv_orphaned_symlink_count**: number of devices with PVs that no longer match **LocalVolume** object criteria
- **iso_lv_provisioned_PV_count**: total number of provisioned PVs for **LocalVolume**

To use metrics, you must enable them first. For more information, see "Enabling Local Storage Operator Metrics".

For more information about metrics, see "Accessing metrics as an administrator".

5.2.10.1. Enabling Local Storage Operator Metrics

To monitor Local Storage Operator (LSO) disk discovery, volume provisioning, and storage utilization, enable LSO metrics by configuring cluster monitoring on the Operator namespace.

Procedure

1. Enable local metrics by doing one of the following:
 - When installing the LSO from the software catalog in the web console, select the **Enable Operator recommended cluster monitoring on this Namespace** checkbox.
 - Manually add the **openshift.io/cluster-monitoring=true** label to the Operator namespace by running the following command:

```
$ oc label ns/openshift-local-storage openshift.io/cluster-monitoring=true
```

5.2.11. Deleting the Local Storage Operator resources

Occasionally you need to delete local volumes (LVs) and local volume sets (LVSs) when no longer needed or uninstall the Local Storage Operator, ensuring persistent volumes are properly released and backed up before removal to prevent data loss.

5.2.11.1. Removing a local volume or local volume set

Remove local volumes (LVs) and local volume sets (LVSs) when no longer needed by releasing bound persistent volumes (PV), deleting the LV or LVS, and cleaning up any PVs with **Retain** reclaim policy.

Prerequisites

- The PV must be in a **Released** or **Available** state.



WARNING

Deleting a PV that is still in use can result in data loss or corruption.

Procedure

1. If there are any bound PVs owned by the LV or LVS that is being deleted, delete the corresponding persistent volume claims (PVCs) to release the PVs:
 - a. To find bound PVs owned by a particular LV or LVS, run the following command:

```
$ oc get pv --selector storage.openshift.com/owner-name=<LV_LVS_name>
```

<LV_LVS_name> is the name of the LV or LVS.

Example output

```
NAME                CAPACITY  ACCESS MODES  RECLAIM POLICY  STATUS  CLAIM                STORAGECLASS  VOLUMEATTRIBUTESCLASS  REASON  AGE
local-pv-3fa1c73    5Gi      RWO           Delete          Available                slow
<unset>                28s
local-pv-1cec77cf   30Gi     RWX           Retain          Bound    openshift/storage
my-sc                <unset>                168d
```

Bound PVs have a status of **Bound** and their corresponding PVCs appear in the **CLAIM** column. In the preceding example, PV **local-pv-1cec77cf** is bound, and its PVC is **openshift/storage**.

- b. Delete corresponding PVCs of bound PVs owned by the LV or LVS being deleted by running the following command:

```
$ oc delete pvc <name>
```

In this example, you would delete PVC **openshift/storage**.

2. Delete the LVs or LVSs by running the applicable following command:

- Command for deleting LV:

```
$ oc delete lv <name>
```

or

- Command for deleting LVS:

```
$ oc delete lvs <name>
```

3. If any PV owned by the LV or LVS has a **Retain** reclaim policy, back up any important data, and then delete the PV:



NOTE

PVs with a **Delete** policy are automatically deleted when you delete the LVs or LVS.

- a. To find PVs with **Retain** reclaim policy, run the following command:

```
$ oc get pv
```


Example output

NAME	CAPACITY	ACCESS MODES	RECLAIM POLICY	STATUS
CLAIM	STORAGECLASS	REASON	AGE	
local-pv-1cec77cf	30Gi	RWX	Retain	Available
168d				my-sc

In this example, PV **local-pv-1cec77cf** has a **Retain** reclaim policy and needs to be manually deleted.

- b. Back up any important data on this volume.
- c. Delete the PV by running the following command:

```
$ oc delete pv <name>
```

In this example, delete PV **local-pv-1cec77cf**.

5.2.11.2. Uninstalling the Local Storage Operator

Safely uninstall the Local Storage Operator (LSO) when local storage persistent volumes (PVs) are no longer in use by removing all local volume resources, uninstalling the operator, deleting remaining PVs, and removing the project.

**WARNING**

Uninstalling the LSO while local storage PVs are still in use is not recommended. Although the PVs remain after removing the Operator, there might be indeterminate behavior if the Operator is uninstalled and reinstalled without removing the PVs and local storage resources.

Prerequisites

- Access to the OpenShift Container Platform web console.

Procedure

1. Delete any local volume resources installed in the project, such as **localvolume**, **localvolumeset**, and **localvolumediscovery** by running the following commands:

```
$ oc delete localvolume --all --all-namespaces
```

```
$ oc delete localvolumeset --all --all-namespaces
```

```
$ oc delete localvolumediscovery --all --all-namespaces
```

2. Uninstall the LSO from the OpenShift Container Platform web console.

- a. Log in to the OpenShift Container Platform web console.
 - b. Go to **Ecosystem → Installed Operators**.
 - c. Type **Local Storage** into the filter box to locate the LSO.
 - d. Click the **Options** menu  at the end of the LSO.
 - e. Click **Uninstall Operator**.
 - f. Click **Remove** in the window that appears.
3. The PVs created by the LSO remain in the cluster until deleted. After these volumes are no longer in use, delete them by running the following command:

```
$ oc delete pv <pv-name>
```

4. Delete the **openshift-local-storage** project by running the following command:

```
$ oc delete project openshift-local-storage
```

5.2.12. Additional resources

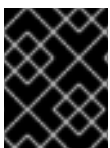
- [Installing the Local Storage Operator](#)
- [Enabling Local Storage Operator Metric](#)
- [Accessing metrics as an administrator](#)

5.3. PERSISTENT STORAGE USING HOSTPATH

A hostPath volume mounts a file or directory from the host node's filesystem into your pod. Use hostPath volumes primarily for testing or development, as they require privileged pods and grant access to the host node's filesystem.

5.3.1. Overview of hostPath

A hostPath volume mounts files or directories from the host node's filesystem into pods for development and testing on single-node clusters. Because hostPath requires privileged pods and must be statically provisioned, it is not recommended for production. Use network storage with dynamic provisioning instead.



IMPORTANT

The cluster administrator must configure pods to run as privileged. This grants access to pods in the same node.

Most pods do not need a hostPath volume, but it does offer a quick option for testing should an application require it.

In a production cluster, you would not use `hostPath`. Instead, a cluster administrator would provision a network resource, such as a GCE Persistent Disk volume, an NFS share, or an Amazon EBS volume. Network resources support the use of storage classes to set up dynamic provisioning.

A `hostPath` volume must be provisioned statically.

IMPORTANT

Do not mount to the container root, `/`, or any path that is the same in the host and the container. This can corrupt your host system if the container is sufficiently privileged. It is safe to mount the host by using **`/host`**. The following example shows the `/` directory from the host being mounted into the container at **`/host`**.

```
apiVersion: v1
kind: Pod
metadata:
  name: test-host-mount
spec:
  containers:
    - image: registry.access.redhat.com/ubi9/ubi
      name: test-container
      command: ['sh', '-c', 'sleep 3600']
      volumeMounts:
        - mountPath: /host
          name: host-slash
  volumes:
    - name: host-slash
      hostPath:
        path: /
        type: "
```

5.3.2. Statically provisioning `hostPath` volumes

Statically provision `hostPath` volumes by creating a persistent volume (PV) that maps to a path on the host node's filesystem, then creating a persistent volume claim (PVC) that binds to the PV. Dynamic provisioning is not supported for `hostPath` volumes.

A pod that uses a `hostPath` volume must be referenced by manual (static) provisioning.

Procedure

1. Define the persistent volume (PV) by creating a **`pv.yaml`** file with the **`PersistentVolume`** object definition:

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: task-pv-volume
  labels:
    type: local
spec:
  storageClassName: manual
  capacity:
    storage: 5Gi
  accessModes:
```

```
- ReadWriteOnce
persistentVolumeReclaimPolicy: Retain
hostPath:
  path: "/mnt/data"
```

- **metadata.name:** Specifies the name of the volume. This name is how the volume is identified by persistent volume (PV) claims or pods.
- **spec.storageClassName:** The storage class is used to bind persistent volume claim (PVC) requests to the PV.
- **spec.accessModes:** Specifies the access mode. The volume can be mounted as **read-write** by a single node.
- **spec.hostPath.path:** The configuration file specifies that the volume is at **/mnt/data** on the cluster's node. To avoid corrupting your host system, do not mount to the container root, **/**, or any path that is the same in the host and the container. You can safely mount the host by using **/host**

2. Create the PV from the file:

```
$ oc create -f pv.yaml
```

3. Define the PVC by creating a **pvc.yaml** file with the **PersistentVolumeClaim** object definition:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: task-pvc-volume
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: manual
```

4. Create the PVC from the file:

```
$ oc create -f pvc.yaml
```

5.3.3. Mounting the hostPath share in a privileged pod

Mount a hostPath share in a privileged pod by referencing an existing persistent volume claim (PVC) in the pod specification. The pod must run as privileged to access the node's storage. Do not mount to the container root or any path that matches the host to avoid corrupting the host system.

After the PVC has been created, it can be used inside by an application. The following example demonstrates mounting this share inside of a pod.

Prerequisites

- A persistent volume claim exists that is mapped to the underlying hostPath share.

Procedure

- Create a privileged pod that mounts the existing persistent volume claim:

```
apiVersion: v1
kind: Pod
metadata:
  name: pod-name
spec:
  containers:
    ...
    securityContext:
      privileged: true
    volumeMounts:
      - mountPath: /data
        name: hostpath-privileged
    ...
  securityContext: {}
  volumes:
    - name: hostpath-privileged
      persistentVolumeClaim:
        claimName: task-pvc-volume
```

- **metadata.name:** Specifies the name of the pod.
- **spec.containers.securityContext.privileged:** The pod must run as privileged to access the node's storage.
- **spec.containers.volumeMounts.mountPath:** The path to mount the host path share inside the privileged pod. Do not mount to the container root, /, or any path that is the same in the host and the container. This can corrupt your host system if the container is sufficiently privileged, such as the host **/dev/pts** files. It is safe to mount the host by using **/host**.
- **spec.volumes.persistentVolumeClaim.claimName:** Specifies the name of the **PersistentVolumeClaim** object that has been previously created.

5.4. PERSISTENT STORAGE USING LOGICAL VOLUME MANAGER STORAGE

Logical Volume Manager (LVM) Storage uses LVM2 through the **TopoLVM CSI** driver to dynamically provision local storage on a cluster with limited resources.

You can create volume groups, persistent volume claims (PVCs), volume snapshots, and volume clones by using LVM Storage.

5.4.1. Logical Volume Manager Storage installation

You can install LVM Storage on an OpenShift Container Platform cluster and configure it to dynamically provision storage for your workloads.

You can install LVM Storage by using the OpenShift Container Platform CLI (**oc**), OpenShift Container Platform web console, or Red Hat Advanced Cluster Management (RHACM).

**WARNING**

When using LVM Storage on multi-node clusters, LVM Storage only supports provisioning local storage. LVM Storage does not support storage data replication mechanisms across nodes. You must ensure storage data replication through active or passive replication mechanisms to avoid a single point of failure.

5.4.1.1. Prerequisites to install LVM Storage

The prerequisites to install LVM Storage are as follows:

- Ensure that you have a minimum of 10 milliCPU and 100 MiB of RAM.
- Ensure that every managed cluster has dedicated disks that are used to provision storage. LVM Storage uses only those disks that are empty and do not contain file system signatures. To ensure that the disks are empty and do not contain file system signatures, wipe the disks before using them.
- Before installing LVM Storage in a private CI environment where you can reuse the storage devices that you configured in the previous LVM Storage installation, ensure that you have wiped the disks that are not in use. If you do not wipe the disks before installing LVM Storage, you cannot reuse the disks without manual intervention.

**NOTE**

You cannot wipe the disks that are in use.

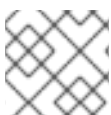
- If you want to install LVM Storage by using Red Hat Advanced Cluster Management (RHACM), ensure that you have installed RHACM on an OpenShift Container Platform cluster. For more information, see "Installing LVM Storage by using RHACM".

Additional resources

- [Red Hat Advanced Cluster Management for Kubernetes: Installing while connected online](#)

5.4.1.2. Installing LVM Storage by using the CLI

You can install LVM Storage by using the OpenShift CLI (**oc**) to dynamically provision local storage on clusters with limited resources.

**NOTE**

The default namespace for the LVM Storage Operator is **openshift-lvm-storage**.

Prerequisites

- You have installed the OpenShift CLI (**oc**).
- You have logged in to OpenShift Container Platform as a user with **cluster-admin** and Operator installation permissions.

Procedure

1. Create a YAML file with the configuration for creating a namespace:

Example YAML configuration for creating a namespace

```
apiVersion: v1
kind: Namespace
metadata:
  labels:
    openshift.io/cluster-monitoring: "true"
    pod-security.kubernetes.io/enforce: privileged
    pod-security.kubernetes.io/audit: privileged
    pod-security.kubernetes.io/warn: privileged
name: openshift-lvm-storage
```

2. Create the namespace by running the following command:

```
$ oc create -f <file_name>
```

3. Create an **OperatorGroup** CR YAML file:

Example OperatorGroup CR

```
apiVersion: operators.coreos.com/v1
kind: OperatorGroup
metadata:
  name: openshift-storage-operatorgroup
  namespace: openshift-lvm-storage
spec:
  targetNamespaces:
    - openshift-storage
```

4. Create the **OperatorGroup** CR by running the following command:

```
$ oc create -f <file_name>
```

5. Create a **Subscription** CR YAML file:

Example Subscription CR

```
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: lvms
  namespace: openshift-lvm-storage
spec:
  installPlanApproval: Automatic
  name: lvms-operator
  source: redhat-operators
  sourceNamespace: openshift-marketplace
```

6. Create the **Subscription** CR by running the following command:

```
$ oc create -f <file_name>
```

Verification

1. To verify that LVM Storage is installed, run the following command:

```
$ oc get csv -n openshift-lvm-storage -o custom-  
columns=Name:.metadata.name,Phase:.status.phase
```

Example output

Name	Phase
4.13.0-202301261535	Succeeded

5.4.1.3. Installing LVM Storage by using the web console

Install LVM Storage from the OpenShift Container Platform web console to dynamically provision local storage on clusters with limited resources.



NOTE

The default namespace for the LVM Storage Operator is **openshift-lvm-storage**.

Prerequisites

- You have access to the cluster.
- You have access to OpenShift Container Platform with **cluster-admin** and Operator installation permissions.

Procedure

1. Log in to the OpenShift Container Platform web console.
2. Click **Ecosystem** → **Software Catalog**.
3. Click **LVM Storage** on the software catalog page.
4. Set the following options on the **Operator Installation** page:
 - a. **Update Channel** as **stable-4.22**.
 - b. **Installation Mode** as **A specific namespace on the cluster**
 - c. **Installed Namespace** as **Operator recommended namespace openshift-storage**. If the **openshift-lvm-storage** namespace does not exist, it is created during the operator installation.
 - d. **Update approval** as **Automatic** or **Manual**.



NOTE

If you select **Automatic** updates, the Operator Lifecycle Manager (OLM) automatically updates the running instance of LVM Storage without any intervention.

If you select **Manual** updates, the OLM creates an update request. As a cluster administrator, you must manually approve the update request to update LVM Storage to a newer version.

5. Optional: Select the **Enable Operator recommended cluster monitoring on this Namespace** checkbox.
6. Click **Install**.

Verification

- Verify that LVM Storage shows a green tick, indicating successful installation.

5.4.1.4. Installing LVM Storage in a disconnected environment

Install LVM Storage in a disconnected environment where your cluster has no internet access, such as air-gapped networks, high-security facilities, or regulated industries requiring network isolation for security and compliance.

Prerequisites

- You read "About disconnected installation mirroring".
- You have access to the OpenShift Container Platform image repository.
- You created a mirror registry (see "Creating a mirror registry with mirror registry for Red Hat OpenShift").

Procedure

1. Follow the steps in the "Creating the image set configuration" procedure. To create an **ImageSetConfiguration** custom resource (CR) for LVM Storage, you can use the following example **ImageSetConfiguration** CR configuration:

Example ImageSetConfiguration CR for LVM Storage

```
kind: ImageSetConfiguration
apiVersion: mirror.openshift.io/v1alpha2
archiveSize: 4
storageConfig:
  registry:
    imageURL: example.com/mirror/oc-mirror-metadata
    skipTLS: false
  mirror:
    platform:
      channels:
        - name: stable-4.22
          type: ocp
      graph: true
```

```

operators:
- catalog: registry.redhat.io/redhat/redhat-operator-index:v4.22
  packages:
  - name: lvms-operator
  channels:
  - name: stable
additionalImages:
- name: registry.redhat.io/ubi9/ubi:latest
helm: {}

```

- **archiveSize:** Specifies the maximum size (in GiB) of each file within the image set.
 - **storageConfig:** Specifies the location in which you want to save the image set. This location can be a registry or a local directory. You must configure the **storageConfig** field unless you are using the Technology Preview OCI feature.
 - **storageConfig.registry.imageURL:** Specifies the storage URL for the image stream when using a registry. For more information, see "Why use imagestreams".
 - **mirror.platform.name:** Specifies the channel from which you want to retrieve the OpenShift Container Platform images.
 - **mirror.platform.channels[].name:** Set this field to **true** to generate the OpenShift Update Service (OSUS) graph image. For more information, see "About the OpenShift Update Service".
 - **mirror.operators.catalog:** Specifies the Operator catalog from which you want to retrieve the OpenShift Container Platform images.
 - **mirror.operators.packages.name:** Specifies the Operator packages to include in the image set. If this field is empty, all packages in the catalog are retrieved.
 - **mirror.operators[].packages[].channels.name:** Specifies the channels of the Operator packages to include in the image set. You must include the default channel for the Operator package even if you do not use the bundles in that channel. You can find the default channel by running the following command: **\$ oc mirror list operators --catalog=<catalog_name> --package=<package_name>**.
 - **mirror.additionalImages.name:** Specifies any additional images to include in the image set.
2. Follow the procedure in "Mirroring an image set to a mirror registry".
 3. Follow the procedure in "Configuring image registry repository mirroring".

Additional resources

- [About disconnected installation mirroring](#)
- [Creating a mirror registry with mirror registry for Red Hat OpenShift](#)
- [Mirroring the OpenShift Container Platform image repository](#)
- [Creating the image set configuration](#)
- [Mirroring an image set to a mirror registry](#)

- [Configuring image registry repository mirroring](#)
- [Why use imagestreams](#)

5.4.1.5. Installing LVM Storage by using RHACM

To install LVM Storage on clusters by using Red Hat Advanced Cluster Management (RHACM), you must create a **Policy** custom resource (CR) and configure the criteria to select the target clusters.



NOTE

The **Policy** CR that is created to install LVM Storage is also applied to the clusters that are imported or created after creating the **Policy** CR.

Prerequisites

- You have access to the RHACM cluster using an account with **cluster-admin** and Operator installation permissions.
- You have dedicated disks that LVM Storage can use on each cluster.
- The cluster must be managed by RHACM.

Procedure

1. Log in to the RHACM CLI using your OpenShift Container Platform credentials.
2. Create a namespace.

```
$ oc create ns <namespace>
```

3. Create a **Policy** CR YAML file:

Example Policy CR to install and configure LVM Storage

```
apiVersion: apps.open-cluster-management.io/v1
kind: PlacementRule
metadata:
  name: placement-install-lvms
spec:
  clusterConditions:
  - status: "True"
    type: ManagedClusterConditionAvailable
  clusterSelector:
    matchExpressions:
    - key: mykey
      operator: In
      values:
      - myvalue
---
apiVersion: policy.open-cluster-management.io/v1
kind: PlacementBinding
metadata:
  name: binding-install-lvms
```

```

placementRef:
  apiGroup: apps.open-cluster-management.io
  kind: PlacementRule
  name: placement-install-lvms
subjects:
- apiGroup: policy.open-cluster-management.io
  kind: Policy
  name: install-lvms
---
apiVersion: policy.open-cluster-management.io/v1
kind: Policy
metadata:
  annotations:
    policy.open-cluster-management.io/categories: CM Configuration Management
    policy.open-cluster-management.io/controls: CM-2 Baseline Configuration
    policy.open-cluster-management.io/standards: NIST SP 800-53
  name: install-lvms
spec:
  disabled: false
  remediationAction: enforce
  policy-templates:
  - objectDefinition:
      apiVersion: policy.open-cluster-management.io/v1
      kind: ConfigurationPolicy
      metadata:
        name: install-lvms
      spec:
        object-templates:
        - complianceType: musthave
          objectDefinition:
            apiVersion: v1
            kind: Namespace
            metadata:
              labels:
                openshift.io/cluster-monitoring: "true"
                pod-security.kubernetes.io/enforce: privileged
                pod-security.kubernetes.io/audit: privileged
                pod-security.kubernetes.io/warn: privileged
            name: openshift-lvm-storage
        - complianceType: musthave
          objectDefinition:
            apiVersion: operators.coreos.com/v1
            kind: OperatorGroup
            metadata:
              name: openshift-storage-operatorgroup
              namespace: openshift-lvm-storage
            spec:
              targetNamespaces:
              - openshift-lvm-storage
        - complianceType: musthave
          objectDefinition:
            apiVersion: operators.coreos.com/v1alpha1
            kind: Subscription
            metadata:
              name: lvms
              namespace: openshift-lvm-storage

```

```
spec:
  installPlanApproval: Automatic
  name: lvms-operator
  source: redhat-operators
  sourceNamespace: openshift-marketplace
  remediationAction: enforce
  severity: low
```

- **spec.clusterSelector:** Set the **key** field and **values** field in **PlacementRule.spec.clusterSelector** to match the labels that are configured in the clusters on which you want to install LVM Storage.
- **spec.policy-templates[0].objectDefinition[0].spec.object-templates[0].objectDefinition:** Specifies the namespace configuration.
- **spec.policy-templates[0].objectDefinition[1].spec.object-templates[1].objectDefinition:** Specifies the **OperatorGroup** CR configuration.
- **spec.policy-templates[0].objectDefinition.spec.object-templates[2].objectDefinition:** Specifies the **Subscription** CR configuration.

4. Create the **Policy** CR by running the following command:

```
$ oc create -f <file_name> -n <namespace>
```

Upon creating the **Policy** CR, the following custom resources are created on the clusters that match the selection criteria configured in the **PlacementRule** CR:

- **Namespace**
- **OperatorGroup**
- **Subscription**



NOTE

The default namespace for the LVM Storage Operator is **openshift-lvm-storage**.

Additional resources

- [Red Hat Advanced Cluster Management for Kubernetes: Installing while connected online](#)
- [About the **LVMCluster** custom resource](#)

5.4.2. Static and dynamic device discovery in LVM Storage

You can use static or dynamic discovery policies to manage how block devices join your volume groups. Selecting the appropriate policy helps you automate storage expansion safely or preserve a locked, predictable storage footprint over time.

Static

The Operator creates the volume group by using devices it finds at installation time. The Operator ignores devices discovered after the volume group exists.

Static discovery is the default policy for new volume groups. It eliminates operational risk by locking the device set after the Operator creates the volume group.

Combined with explicit device paths, it provides a fully deterministic storage configuration.

Without explicit paths, the Operator discovers devices only at creation time and then stops the set.

Dynamic

The Operator continuously discovers and adds devices to the volume group on each reconciliation cycle.

Dynamic discovery remains the default for existing volume groups where the policy field is nil to maintain backward compatibility.

However, this policy can lead to unexpected behavior in production environments. Devices that appear after the initial setup because of hardware changes, driver reloads, or kernel device renaming are automatically added to the volume group.

This creates operational risk because the volume group composition becomes non-deterministic and depends on the runtime state of the node rather than explicit administrator intent.



NOTE

The Operator adds the **DeviceDiscoveryPolicy** field to the **DeviceClass** specification. If you explicitly set device paths in **deviceSelector.paths** or **deviceSelector.optionalPaths**, the cluster always uses those exact paths, and ignores your discovery policy setting.

The cluster status reports the effective policy by using **DeviceDiscoveryPolicyStatus**, which distinguishes three runtime states:

Table 5.7. Effective policy status values

Status value	Description
Preconfigured	Explicit device paths configuration by using deviceSelector . Discovery policy is not applicable.
RuntimeDynamic	No explicit paths. Discovery policy is Dynamic. The Operator continuously discovers devices.
RuntimeStatic	No explicit paths. Discovery policy is Static. The Operator discovers devices once at creation time.

The following table shows the behavior matrix:

Table 5.8. Device discovery behavior by configuration

Explicit paths	Discovery policy	Effective behavior
----------------	------------------	--------------------

Explicit paths	Discovery policy	Effective behavior
Yes	Any / nil	Preconfigured: The Operator honors the specified paths and ignores the discovery policy.
No	Static	RuntimeStatic: The Operator locks the device set immediately after creating the volume group
No	Dynamic	RuntimeDynamic: continuous discovery every 30 seconds
No	nil (new volume group)	RuntimeStatic: defaults to Static
No	nil (existing volume group)	RuntimeDynamic: defaults to Dynamic for backward compatibility

5.4.2.1. Static mode enforcement

In static mode, the system locks the device set after initial discovery. If a volume group lacks explicit paths, newly attached devices are automatically excluded to prevent unintended volume expansions.

This strict filtering behavior does not apply during the very first reconciliation cycle. During this initial pass, the Operator discovers all available devices to successfully create the volume group. Once created, the Operator locks the device set during all subsequent reconciliations.

The discovery policy also controls whether the controller re-queues for periodic device scanning:

Table 5.9. Requeue behavior by configuration

Configuration	Periodic requeue
Explicit paths	No: paths define the exact device set; changes trigger reconciliation by using the LVMVolumeGroup watch
Dynamic without explicit paths	Yes: every 30 seconds
Static without explicit paths	No: device set is locked after creation

5.4.2.2. Validation rules for device discovery policy

To ensure your storage cluster deploys successfully and avoids misconfiguration errors, the validating webhook enforces strict rules when you create or update an **LVMCluster** custom resource.

Creation

- If you define one device class without paths, a webhook warning appears. Avoid the default **Static** policy in production. Set **deviceDiscoveryPolicy** explicitly.

- If multiple device classes are defined, every device class must specify device paths. Auto-discovery without paths is not allowed with many device classes. The cluster cannot determine which devices belong to which class.
- If the **deviceDiscoveryPolicy** is empty and paths are missing, a webhook warning appears. Administrators must define the policy explicitly.

Updates

No specific update restrictions apply to the **deviceDiscoveryPolicy** field. You can change it at any time.

The following table shows how the device discovery policy feature interacts with other features:

Table 5.10. Device discovery policy feature interactions

Feature	Interaction
forceWipeDevicesAndDestroyAllData	Works independently of the discovery policy. Devices are wiped before being added to the volume group, regardless of how they were discovered.
Node selector	Works independently. The discovery policy applies only to the set of devices found on nodes matching the selector.

5.4.2.3. LVMCluster custom resource examples

You can configure the **deviceDiscoveryPolicy** field in your **LVMCluster** custom resource (CR) by using these examples to meet your specific storage requirements.

Explicit device paths (recommended for production)

```
apiVersion: lvm.topolvm.io/v1alpha1
kind: LVMCluster
metadata:
  name: my-lvmcluster
spec:
  storage:
    deviceClasses:
      - name: vg1
    deviceSelector:
      paths:
        - /dev/disk/by-id/scsi-SATA_VBOX_HARDDISK_VB12345678-90abcdef
        - /dev/disk/by-id/scsi-SATA_VBOX_HARDDISK_VBabcdef01-23456789
    thinPoolConfig:
      name: thin-pool-1
      sizePercent: 90
```

The discovery policy is not relevant here. Explicit paths always define the device set.

Static discovery without explicit paths

```
apiVersion: lvm.topolvm.io/v1alpha1
kind: LVMCluster
```



```

metadata:
  name: my-lvmcluster
spec:
  storage:
    deviceClasses:
      - name: vg1
    deviceDiscoveryPolicy: Static
    thinPoolConfig:
      name: thin-pool-1
      sizePercent: 90

```

The Operator discovers and adds all available devices to the volume group during the initial reconciliation. After the Operator creates the volume group, it adds no new devices.

Dynamic discovery without explicit paths (not recommended for production)

```

apiVersion: lvm.topolvm.io/v1alpha1
kind: LVMCluster
metadata:
  name: my-lvmcluster
spec:
  storage:
    deviceClasses:
      - name: vg1
    deviceDiscoveryPolicy: Dynamic
    thinPoolConfig:
      name: thin-pool-1
      sizePercent: 90

```

The Operator continuously discovers and adds devices to the volume group every 30 seconds. This setting is useful for development and testing. However, it might introduce operational risks in production environments.

5.4.2.4. LVM cluster custom resource status reporting

To view a list of excluded devices and the reason for their exclusion, use the **LVMVolumeGroupNodeStatus** custom resource (CR).

If static device discovery excludes a device, the status report displays the error in the following format:

```
<device> was not part of <vg_name> at creation (static device discovery enabled)
```

The **VGStatus.DeviceDiscoveryPolicy** parameter reports the effective discovery policy as one of the following values:

- **Preconfigured**
- **RuntimeDynamic**
- **RuntimeStatic.**

5.4.3. About the LVMCluster custom resource

The **LVMCluster** custom resource (CR) is the primary configuration for LVM Storage deployment, defining how storage is provisioned across your cluster by specifying volume groups, devices, node selection, and thin pool settings to meet your workload requirements.

You can configure the **LVMCluster** CR to perform the following actions:

- Create LVM volume groups that you can use to provision persistent volume claims (PVCs).
- Configure a list of devices that you want to add to the LVM volume groups.
- Configure the requirements to select the nodes on which you want to create an LVM volume group, and the thin pool configuration for the volume group.
- Force wipe the selected devices.

After you have installed LVM Storage, you must create an **LVMCluster** custom resource (CR).

Example LVMCluster CR YAML file

```
apiVersion: lvm.topolvm.io/v1alpha1
kind: LVMCluster
metadata:
  name: my-lvmcluster
spec:
  tolerations:
    - effect: NoSchedule
      key: xyz
      operator: Equal
      value: "true"
  storage:
    deviceClasses:
      - name: vg1
        fstype: ext4
        default: true
    nodeSelector:
      nodeSelectorTerms:
        - matchExpressions:
            - key: mykey
              operator: In
              values:
                - ssd
    deviceSelector:
      paths:
        - /dev/disk/by-path/pci-0000:87:00.0-nvme-1
        - /dev/disk/by-path/pci-0000:88:00.0-nvme-1
      optionalPaths:
        - /dev/disk/by-path/pci-0000:89:00.0-nvme-1
        - /dev/disk/by-path/pci-0000:90:00.0-nvme-1
      forceWipeDevicesAndDestroyAllData: true
  thinPoolConfig:
    name: thin-pool-1
    sizePercent: 90
    overprovisionRatio: 10
    chunkSize: 128Ki
```

```
chunkSizeCalculationPolicy: Static
metadataSize: 1Gi
metadataSizeCalculationPolicy: Host
```

The following are optional fields: **fstype**, **nodeSelector**, **deviceSelector**, **sizePercent**, **chunkSize**, **chunkSizeCalculationPolicy**, **metadataSize**, **metadataSizeCalculationPolicy**.

5.4.3.1. Explanation of fields in the LVMCluster CR

The **LVMCluster** CR fields are described in the following table:

Table 5.11. LVMCluster CR fields

Field	Type	Description
spec.storage.deviceClasses	array	Contains the configuration to assign the local storage devices to the LVM volume groups. LVM Storage creates a storage class and volume snapshot class for each device class that you create.
deviceClasses.name	string	Specify a name for the LVM volume group (VG). You can also configure this field to reuse a volume group that you created in the previous installation. For more information, see "Reusing a volume group from the previous LVM Storage installation".
deviceClasses.fstype	string	Set this field to ext4 or xfs . By default, this field is set to xfs .
deviceClasses.default	boolean	Set this field to true to indicate that a device class is the default. Otherwise, you can set it to false . You can only configure a single default device class.
deviceClasses.nodeSelector	object	Contains the configuration to choose the nodes on which you want to create the LVM volume group. If this field is empty, all nodes without no-schedule taints are considered. On the control-plane node, LVM Storage detects and uses the additional worker nodes when the new nodes become active in the cluster.
nodeSelector.nodeSelectorTerms	array	Configure the requirements that are used to select the node.

Field	Type	Description
deviceClasses.deviceSelector	object	Contains the configuration to perform the following actions: - Specify the paths to the devices that you want to add to the LVM volume group. - Force wipe the devices that are added to the LVM volume group. For more information, see "About adding devices to a volume group".
deviceSelector.paths	array	Specify the device paths. If the device path specified in this field does not exist, or the device is not supported by LVM Storage, the LVMCluster CR moves to the Failed state.
deviceSelector.optionalPaths	array	Specify the optional device paths. If the device path specified in this field does not exist, or the device is not supported by LVM Storage, LVM Storage ignores the device without causing an error.
deviceSelector.forceWipeDevicesAndDestroyAllData	boolean	LVM Storage uses only those disks that are empty and do not contain file system signatures. To ensure that the disks are empty and do not contain file system signatures, wipe the disks before using them. To force wipe the selected devices, set this field to true. By default, this field is set to false.  WARNING If this field is set to true, LVM Storage wipes all previous data on the devices. Use this feature with caution. Wiping the device can lead to inconsistencies in data integrity if any of the following conditions are met: - The device is being used as swap space. - The device is part of a RAID array. - The device is mounted. If any of these conditions are true, do not force wipe the disk. Instead, you must manually wipe the disk.

Field	Type	Description
deviceClasses.storageClassOptions	object	Optional. Allows customization of the StorageClass created for this device class, including reclaim policy, volume binding mode, additional parameters, and labels. For more information, see "StorageClass customization for LVMS device classes".
deviceClasses.thinPoolConfig	object	Contains the configuration to create a thin pool in the LVM volume group. If you exclude this field, logical volumes are thick provisioned. Using thick-provisioned storage includes the following limitations: • No copy-on-write support for volume cloning. • No support for snapshot class. • No support for over-provisioning. As a result, the provisioned capacity of PersistentVolumeClaims (PVCs) is immediately reduced from the volume group. • No support for thin metrics. Thick-provisioned devices only support volume group metrics.
thinPoolConfig.name	string	Specify a name for the thin pool.
thinPoolConfig.sizePercent	integer	Specify the percentage of space in the LVM volume group for creating the thin pool. By default, this field is set to 90. The minimum value that you can set is 10, and the maximum value is 90.
thinPoolConfig.overprovisionRatio	integer	Specify a factor by which you can provision additional storage based on the available storage in the thin pool. For example, if this field is set to 10, you can provision up to 10 times the amount of available storage in the thin pool. You can modify this field after the LVM cluster has been created. To update the parameter, do any of the following tasks: • To edit the LVM Cluster, run the following command: <pre>\$ oc edit lvmlcluster <lvmlcluster_name></pre> • To apply a patch, run the following command: <pre>\$ oc patch lvmlcluster <lvmlcluster_name> -p <patch_file.yaml></pre> To disable over-provisioning, set this field to 1.

Field	Type	Description
thinPoolConfig.chunkSize	integer	Specifies the statically calculated chunk size for the thin pool. This field is only used when the ChunkSizeCalculationPolicy field is set to Static. The value for this field must be configured in the range of 64 KiB to 1 GiB because of the underlying limitations of lvm2. If you do not configure this field and the ChunkSizeCalculationPolicy field is set to Static, the default chunk size is set to 128 KiB. For more information, see "Overview of chunk size".
thinPoolConfig.chunkSizeCalculationPolicy	string	Specifies the policy to calculate the chunk size for the underlying volume group. You can set this field to either Static or Host. By default, this field is set to Static. If this field is set to Static, the chunk size is set to the value of the chunkSize field. If the chunkSize field is not configured, chunk size is set to 128 KiB. If this field is set to Host, the chunk size is calculated based on the configuration in the lvm.conf file. For more information, see "Limitations to configure the size of the devices used in LVM Storage".
thinPoolConfig.metadataSize	integer	Specifies the metadata size for the thin pool. You can configure this field only when the MetadataSizeCalculationPolicy field is set to Static. If this field is not configured, and the MetadataSizeCalculationPolicy field is set to Static, the default metadata size is set to 1 GiB. The value for this field must be configured in the range of 2 MiB to 16 GiB due to the underlying limitations of lvm2. You can only increase the value of this field during updates.
thinPoolConfig.metadataSizeCalculationPolicy	string	Specifies the policy to calculate the metadata size for the underlying volume group. You can set this field to either Static or Host. By default, this field is set to Host. If this field is set to Static, the metadata size is calculated based on the value of the thinPoolConfig.metadataSize field. If this field is set to Host, the metadata size is calculated based on the lvm2 settings.

Additional resources

- [Overview of chunk size](#)
- [Limitations to configure the size of the devices used in LVM Storage](#)
- [Reusing a volume group from the previous LVM Storage installation](#)
- [About adding devices to a volume group](#)
- [Adding worker nodes to single-node OpenShift clusters](#)

5.4.3.2. Limitations to configure the size of the devices used in LVM Storage

To ensure your devices are compatible with storage operations, review the size configuration limitations in LVM Storage. Adhering to these constraints prevents provisioning failures by ensuring selected devices meet the required capacity specifications.

When provisioning storage by using LVM Storage, the following factors limit device size:

- The total storage size that you can provision is limited by the size of the underlying Logical Volume Manager (LVM) thin pool and the over-provisioning factor.
- The size of the logical volume depends on the size of the Physical Extent (PE) and the Logical Extent (LE).
 - You can define the size of PE and LE during the physical and logical device creation.
 - The default PE and LE size is 4 MiB.
 - If the size of the PE is increased, the maximum size of the LVM is determined by the kernel limits and your disk space.

The following tables describe the chunk size and volume size limits for static and host configurations:

Table 5.12. Tested configuration

Parameter	Value
Chunk size	128 KiB
Maximum volume size	32 TiB

Table 5.13. Theoretical size limits for static configuration

Parameter	Minimum value	Maximum value
Chunk size	64 KiB	1 GiB
Volume size	Minimum size of the underlying Red Hat Enterprise Linux CoreOS (RHCOS) system.	Maximum size of the underlying RHCOS system.

Table 5.14. Theoretical size limits for a host configuration

Parameter	Value
Chunk size	This value is based on the configuration in the lvm.conf file. By default, the configuration sets the value to 128 KiB.
Maximum volume size	Equal to the maximum volume size of the underlying RHCOS system.
Minimum volume size	Equal to the minimum volume size of the underlying RHCOS system.

5.4.3.3. About adding devices to a volume group

To add devices to the Logical Volume Manager (LVM) volume group, use the **deviceSelector** field in the **LVMCluster** Custom Resource (CR) to specify the paths to the devices.

You can specify the device paths in the **deviceSelector.paths** field, the **deviceSelector.optionalPaths** field, or both. If you do not specify the device paths in both the **deviceSelector.paths** field and the **deviceSelector.optionalPaths** field, LVM Storage adds the supported unused devices to the volume group (VG).



IMPORTANT

It is recommended to avoid referencing disks using symbolic naming, such as **/dev/sdX**, as these names may change across reboots within RHCOS. Instead, you must use stable naming schemes, such as **/dev/disk/by-path/** or **/dev/disk/by-id/**, to ensure consistent disk identification.

With this change, you might need to adjust existing automation workflows in the cases where monitoring collects information about the install device for each node.

For more information, see the "RHEL documentation".

You can add the path to the Redundant Array of Independent Disks (RAID) arrays in the **deviceSelector** field to integrate the RAID arrays with LVM Storage. You can create the RAID array by using the **mdadm** utility. LVM Storage does not support creating a software RAID.



NOTE

You can create a RAID array only during an OpenShift Container Platform installation. For information on creating a RAID array, see:

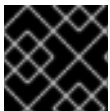
- "Configuring a RAID-enabled data volume"
- "Creating a software RAID on an installed system"
- "Replacing a failed disk in RAID"
- "Repairing RAID disks"

You can also add encrypted devices to the volume group. You can enable disk encryption on the cluster nodes during an OpenShift Container Platform installation. After encrypting a device, you can specify the path to the LUKS encrypted device in the **deviceSelector** field. For information on disk encryption, see "About disk encryption" and "Configuring disk encryption and mirroring".

The devices that you want to add to the VG must be supported by LVM Storage. For information about unsupported devices, see "Devices not supported by LVM Storage".

LVM Storage adds the devices to the VG only if the following conditions are met:

- The device path exists.
- The device is supported by LVM Storage.



IMPORTANT

After a device is added to the VG, you cannot remove the device.

LVM Storage supports dynamic device discovery. If you do not add the **deviceSelector** field in the **LVMCluster** CR, LVM Storage automatically adds the new devices to the VG when the devices are available.



WARNING

It is not recommended to add the devices to the VG through dynamic device discovery due to the following reasons:

- When you add a new device that you do not intend to add to the VG, LVM Storage automatically adds this device to the VG through dynamic device discovery.
- If LVM Storage adds a device to the VG through dynamic device discovery, LVM Storage does not restrict you from removing the device from the node. Removing or updating the devices that are already added to the VG can disrupt the VG. This can also lead to data loss and necessitate manual node remediation.

Additional resources

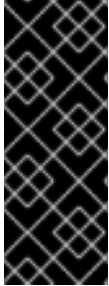
- [Configuring a RAID-enabled data volume](#)
- [About disk encryption](#)
- [Configuring disk encryption and mirroring](#)
- [Devices not supported by LVM Storage](#)

5.4.3.4. About removing devices and device classes from a volume group

You can remove devices and device classes from a Logical Volume Manager (LVM) volume group to decommission storage hardware or reorganize your storage configuration by updating the **deviceSelector** field in the **LVMCluster** CR.

5.4.3.4.1. Removing the device paths in the deviceSelector.paths field

You can remove the device paths in the **deviceSelector.paths** field.



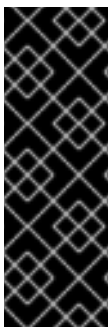
IMPORTANT

Ensure that the following criteria are met before removing device paths:

- The device that you want to remove is empty. You can use the **pvdisk** command to see attributes of physical volumes (PVs) used in LVM.
- At least one additional device is specified in the **deviceSelector.paths** field.

5.4.3.4.2. Removing the deviceClass from the LVMCluster

You can also remove the **deviceClass** object from the **LVMCluster** resource. For device class deletion, there is no need to delete **deviceSelector.paths** object.



IMPORTANT

Ensure that the following criteria are met before removing a device class:

- The **deviceClasses.default** field is set to **false**.
- The disks specified in the **deviceSelector.paths** field are empty.
- At least one additional device class is specified in the **storage** field.

5.4.3.5. Devices not supported by LVM Storage

When adding device paths to the **LVMCluster** custom resource (CR), ensure devices are supported by LVM Storage. LVM Storage excludes unsupported devices to avoid complexity in managing logical volumes.

If you do not specify any device path in the **deviceSelector** field, LVM Storage adds only the unused devices that it supports.



NOTE

To get information about the devices, run the following command:

```
$ lsblk --paths --json -o \
NAME,ROTA,TYPE,SIZE,MODEL,VENDOR,RO,STATE,KNAME,SERIAL,PARTLABEL
,FSTYPE
```

LVM Storage does not support the following devices:

Read-only devices

Devices with the **ro** parameter set to **true**.

Suspended devices

Devices with the **state** parameter set to **suspended**.

ROM devices

Devices with the **type** parameter set to **rom**.

LVM partition devices

Devices with the **type** parameter set to **lvm**.

Devices with invalid partition labels

Devices with the **partlabel** parameter set to **bios**, **boot**, or **reserved**.

Devices with an invalid filesystem

Devices with the **fstype** parameter set to any value other than **null** or **LVM2_member**.



IMPORTANT

LVM Storage supports devices with **fstype** parameter set to **LVM2_member** only if the devices do not contain children devices.

Devices that are part of another volume group

To get the information about the volume groups of the device, run the following command:

```
$ pvs <device-name>
```

Where **<device-name>** is the device name.

Devices with bind mounts

To get the mount points of a device, run the following command:

```
$ cat /proc/1/mountinfo | grep <device-name>
```

Where **<device-name>** is the device name.

Devices that contain children devices



NOTE

It is recommended to wipe the device before using it in LVM Storage to prevent unexpected behavior.

5.4.4. Ways to create an LVMCluster custom resource

You can create an **LVMCluster** custom resource (CR) to configure LVM Storage deployment and provision storage for your workloads by using the OpenShift CLI (**oc**), OpenShift Container Platform web console, or Red Hat Advanced Cluster Management (RHACM).

You must install LVM Storage by using RHACM if you want to create an **LVMCluster** CR by using RHACM.

**IMPORTANT**

You must create the **LVMCluster** CR in the same namespace where you installed the LVM Storage Operator, which is **openshift-storage** by default.

After creating the **LVMCluster** CR, LVM Storage creates the following system-managed CRs:

- A **storageClass** and **volumeSnapshotClass** for each device class.

**NOTE**

LVM Storage configures the name of the storage class and volume snapshot class in the format **lvms-<device_class_name>**, where, **<device_class_name>** is the value of the **deviceClasses.name** field in the **LVMCluster** CR. For example, if the **deviceClasses.name** field is set to **vg1**, the name of the storage class and volume snapshot class is **lvms-vg1**.

- **LVMVolumeGroup**: This CR is a specific type of persistent volume (PV) that is backed by an LVM volume group. It tracks the individual volume groups across multiple nodes.
- **LVMVolumeGroupNodeStatus**: This CR tracks the status of the volume groups on a node.

5.4.4.1. Reusing a volume group from the previous LVM Storage installation

You can reuse an existing volume group (VG) from a previous LVM Storage installation to preserve your existing storage configuration and avoid recreating VGs when reinstalling or upgrading LVM Storage.

You can only reuse a VG, but not the logical volume associated with the VG.

**IMPORTANT**

You can perform this procedure only while creating an **LVMCluster** custom resource (CR).

Prerequisites

- The VG that you want to reuse must not be corrupted.
- The VG that you want to reuse must have the **lvms** tag. For more information on adding tags to LVM objects, see "Grouping LVM objects with tags".

Procedure

1. Open the **LVMCluster** CR YAML file.
2. Configure the **LVMCluster** CR parameters as described in the following example:

Example LVMCluster CR YAML file

```
apiVersion: lvm.topolvm.io/v1alpha1
kind: LVMCluster
metadata:
  name: my-lvmcluster
```

```
spec:
# ...
storage:
  deviceClasses:
    - name: vg1
      fstype: ext4
      default: true
  deviceSelector:
# ...
    forceWipeDevicesAndDestroyAllData: false
  thinPoolConfig:
# ...
  nodeSelector:
# ...
```

- **spec.storage.deviceClasses.name:** Specifies the name of a VG from the previous LVM Storage installation.
- **spec.storage.deviceClasses.fstype:** Set this field to **ext4** or **xfs**. By default, this field is set to **xfs**.
- **spec.storage.deviceClasses.name.deviceSelector:** You can add new devices to the VG that you want to reuse by specifying the new device paths in the **deviceSelector** field. If you do not want to add new devices to the VG, ensure that the **deviceSelector** configuration in the current LVM Storage installation is same as that of the previous LVM Storage installation.
- **spec...forceWipeDevicesAndDestroyAllData:** If this field is set to **true**, LVM Storage wipes all the data on the devices that are added to the VG.
- **spec....thinPoolConfig:** To retain the **thinPoolConfig** configuration of the VG that you want to reuse, ensure that the **thinPoolConfig** configuration in the current LVM Storage installation is same as that of the previous LVM Storage installation. Otherwise, you can configure the **thinPoolConfig** field as required.
- **spec...nodeSelector:** Configure the requirements to choose the nodes on which you want to create the LVM volume group. If this field is empty, all nodes without no-schedule taints are considered.

3. Save the **LVMCluster** CR YAML file.

Verification

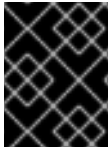
To view the devices that are part a volume group, run the following command:

```
$ pvs -S vgname=<vg_name>
```

Replace **<vg_name>** with the name of the volume group.

5.4.4.2. Creating an LVMCluster CR by using the CLI

You can create an **LVMCluster** custom resource (CR) on a worker node by using the OpenShift CLI (**oc**) to configure storage deployment and provision local storage for your workloads.



IMPORTANT

You can only create a single instance of the **LVMCluster** custom resource (CR) on an OpenShift Container Platform cluster.

Prerequisites

- You have installed the OpenShift CLI (**oc**).
- You have logged in to OpenShift Container Platform as a user with **cluster-admin** privileges.
- You have installed LVM Storage.
- You have installed a worker node in the cluster.
- You read "About the LVMCluster custom resource".

Procedure

1. Create an **LVMCluster** custom resource (CR) YAML file:

Example LVMCluster CR YAML file

```
apiVersion: lvm.topolvm.io/v1alpha1
kind: LVMCluster
metadata:
  name: my-lvmcluster
  namespace: openshift-lvm-storage
spec:
  # ...
  storage:
    deviceClasses:
  # ...
    nodeSelector:
  # ...
    deviceSelector:
  # ...
    thinPoolConfig:
  # ...
```

- **spec.storage.deviceClasses:** Specifies the configuration to assign the local storage devices to the LVM volume groups.
- **spec...nodeSelector:** Specifies the configuration to choose the nodes on which you want to create the LVM volume group. If this field is empty, all nodes without no-schedule taints are considered.
- **spec...deviceSelector:** Specifies the configuration to specify the paths to the devices that you want to add to the LVM volume group, and force wipe the devices that are added to the LVM volume group.
- **spec...thinPoolConfig:** Specifies the configuration to create a thin pool in the LVM volume group. If you exclude this field, logical volumes are thick provisioned.

2. Create the **LVMCluster** CR by running the following command:

```
$ oc create -f <file_name>
```

Example output

```
lvmcluster/lvmcluster created
```

Verification

1. Check that the **LVMCluster** CR is in the **Ready** state by running the following command:

```
$ oc get lvmclusters.lvm.topolvm.io -o jsonpath='{.items[*].status}' -n <namespace>
```

Example output

```
{
  "deviceClassStatuses": [
    {
      "name": "vg1",
      "nodeStatus": [
        {
          "devices": [
            "/dev/nvme0n1",
            "/dev/nvme1n1",
            "/dev/nvme2n1"
          ],
          "node": "kube-node",
          "status": "Ready"
        }
      ]
    }
  ],
  "state": "Ready"
}
```

- **deviceClassStatuses**: Specifies the status of the device class.
- **nodeStatus**: Specifies the status of the LVM volume group on each node.
- **devices**: Specifies the list of devices used to create the LVM volume group.
- **node**: Specifies the node on which the device class is created.
- **status**: Specifies the status of the LVM volume group on the node.
- **state**: Specifies the status of the **LVMCluster** CR.

**NOTE**

If the **LVMCluster** CR is in the **Failed** state, you can view the reason for failure in the **status** field.

+ Example of **status** field with the reason for failure:

```
status:
  deviceClassStatuses:
    - name: vg1
      nodeStatus:
        - node: my-node-1.example.com
          reason: no available devices found for volume group
          status: Failed
      state: Failed
```

- To view the storage classes created by LVM Storage for each device class, run the following command:

```
$ oc get storageclass
```

Example output

NAME	PROVISIONER	RECLAIMPOLICY	VOLUMEBINDINGMODE	ALLOWVOLUMEEXPANSION	AGE
lvms-vg1	topolvm.io	Delete	WaitForFirstConsumer	true	31m

- To view the volume snapshot classes created by LVM Storage for each device class, run the following command:

```
$ oc get volumesnapshotclass
```

Example output

NAME	DRIVER	DELETIONPOLICY	AGE
lvms-vg1	topolvm.io	Delete	24h

Additional resources

- [About the **LVMCluster** custom resource](#)

5.4.4.3. Creating an LVMCluster CR by using the web console

You can create an **LVMCluster** custom resource (CR) on a worker node by using the OpenShift Container Platform web console to configure storage deployment and provision local storage for your workloads.

**IMPORTANT**

You can only create a single instance of the **LVMCluster** custom resource (CR) on an OpenShift Container Platform cluster.

Prerequisites

- You have access to the OpenShift Container Platform cluster with **cluster-admin** privileges.
- You have installed LVM Storage.
- You have installed a worker node in the cluster.
- You read the "About the LVMCluster custom resource" section.

Procedure

1. Log in to the OpenShift Container Platform web console.
2. Click **Ecosystem → Installed Operators**.
3. In the **openshift-lvm-storage** namespace, click **LVM Storage**.
4. Click **Create LVMCluster** and select either **Form view** or **YAML view**.
5. Configure the required **LVMCluster** CR parameters.
6. Click **Create**.
7. Optional: If you want to edit the **LVMCluster** CR, perform the following actions:
 - a. Click the **LVMCluster** tab.
 - b. From the **Actions** menu, select **Edit LVMCluster**.
 - c. Click **YAML** and edit the required **LVMCluster** CR parameters.
 - d. Click **Save**.

Verification

1. On the **LVMCluster** page, check that the **LVMCluster** CR is in the **Ready** state.
2. Optional: To view the available storage classes created by LVM Storage for each device class, click **Storage → StorageClasses**.
3. Optional: To view the available volume snapshot classes created by LVM Storage for each device class, click **Storage → VolumeSnapshotClasses**.

Additional resources

- [About the LVMCluster custom resource](#)

5.4.4.4. Creating an LVMCluster CR by using RHACM

After installing Logical Volume Manager (LVM) Storage by using RHACM, create an **LVMCluster** custom resource (CR) to configure storage deployment, specify devices and volume groups, and provision storage for your workloads.

Prerequisites

- You have installed LVM Storage by using RHACM.
- You have access to the RHACM cluster using an account with **cluster-admin** permissions.
- You read the "About the LVMCluster custom resource" section.

Procedure

1. Log in to the RHACM CLI using your OpenShift Container Platform credentials.
2. Create a **ConfigurationPolicy** CR YAML file with the configuration to create an **LVMCluster** CR:

Example ConfigurationPolicy CR YAML file to create an LVMCluster CR

```
apiVersion: policy.open-cluster-management.io/v1
kind: ConfigurationPolicy
metadata:
  name: lvms
  namespace: openshift-lvm-storage
spec:
  object-templates:
  - complianceType: musthave
    objectDefinition:
      apiVersion: lvm.topolvm.io/v1alpha1
      kind: LVMCluster
      metadata:
        name: my-lvmcluster
        namespace: openshift-lvm-storage
      spec:
        storage:
          deviceClasses:
            # ...
          deviceSelector:
            # ...
          thinPoolConfig:
            # ...
          nodeSelector:
            # ...
        remediationAction: enforce
        severity: low
```

- **spec.object-templates.objectDefinition.spec.storage.deviceClasses:** Specifies the configuration to assign the local storage devices to the LVM volume groups.
- **spec...deviceSelector:** Contains the configuration to specify the paths to the devices that you want to add to the LVM volume group, and force wipe the devices that are added to the LVM volume group.
- **spec...thinPoolConfig:** Contains the configuration to create a thin pool in the LVM volume group. If you exclude this field, logical volumes are thick provisioned.
- **spec...nodeSelector:** Contains the configuration to choose the nodes on which you want to create the LVM volume groups. If this field is empty, then all nodes without no-schedule taints are considered.

3. Create the **ConfigurationPolicy** CR by running the following command:

```
$ oc create -f <file_name> -n <cluster_namespace>
```

<cluster_namespace> is the namespace of the OpenShift Container Platform cluster on which LVM Storage is installed.

Additional resources

- [Red Hat Advanced Cluster Management for Kubernetes: Installing while connected online](#)
- [About the **LVMCluster** custom resource](#)

5.4.5. Ways to delete an LVMCluster custom resource

Delete an **LVMCluster** custom resource (CR) when decommissioning LVM Storage or reconfiguring storage by using the OpenShift CLI (**oc**), OpenShift Container Platform web console, or Red Hat Advanced Cluster Management (RHACM).

You must have installed LVM Storage by using RHACM to delete an **LVMCluster** CR by using RHACM.

After deleting the **LVMCluster** CR, LVM Storage deletes the following CRs:

- **storageClass**
- **volumeSnapshotClass**
- **LVMVolumeGroup**
- **LVMVolumeGroupNodeStatus**

5.4.5.1. Deleting an LVMCluster CR by using the CLI

You can delete an **LVMCluster** custom resource (CR) when decommissioning LVM Storage or reconfiguring storage by using the OpenShift CLI (**oc**).

Prerequisites

- You have access to OpenShift Container Platform as a user with **cluster-admin** permissions.
- You have deleted the persistent volume claims (PVCs), volume snapshots, and volume clones provisioned by LVM Storage. You have also deleted the applications that are using these resources.

Procedure

1. Log in to the OpenShift CLI (**oc**).
2. Delete the **LVMCluster** CR by running the following command:

```
$ oc delete lvmcluster <lvm_cluster_name> -n <namespace>
```

Verification

- To verify that the **LVMCluster** CR has been deleted, run the following command:

```
$ oc get lvmcluster -n <namespace>
```

Example output

```
No resources found in openshift-lvm-storage namespace.
```

5.4.5.2. Deleting an LVMCluster CR by using the web console

You can delete an **LVMCluster** custom resource (CR) when decommissioning LVM Storage or reconfiguring storage by using the OpenShift Container Platform web console.

Prerequisites

- You have access to OpenShift Container Platform as a user with **cluster-admin** permissions.
- You have deleted the persistent volume claims (PVCs), volume snapshots, and volume clones provisioned by LVM Storage. You have also deleted the applications that are using these resources.

Procedure

1. Log in to the OpenShift Container Platform web console.
2. Click **Ecosystem** → **Installed Operators** to view all the installed Operators.
3. Click **LVM Storage** in the **openshift-lvm-storage** namespace.
4. Click the **LVMCluster** tab.
5. From the **Actions**, select **Delete LVMCluster**.
6. Click **Delete**.

Verification

- On the **LVMCluster** page, check that the **LVMCluster** CR has been deleted.

5.4.5.3. Deleting an LVMCluster CR by using RHACM

You can delete an **LVMCluster** custom resource (CR) when decommissioning LVM Storage or reconfiguring storage by using Red Hat Advanced Cluster Management (RHACM).

You can only delete an **LVMCluster** CR by using RHACM if you installed LVM Storage by using Red Hat Advanced Cluster Management (RHACM).

Prerequisites

- You have access to the RHACM cluster as a user with **cluster-admin** permissions.
- You have deleted the persistent volume claims (PVCs), volume snapshots, and volume clones provisioned by LVM Storage. You have also deleted the applications that are using these resources.

Procedure

1. Log in to the RHACM CLI using your OpenShift Container Platform credentials.
2. Delete the **ConfigurationPolicy** CR YAML file that was created for the **LVMCluster** CR:

```
$ oc delete -f <file_name> -n <cluster_namespace>
```

<cluster_namespace> is the namespace of the OpenShift Container Platform cluster on which LVM Storage is installed.

3. Create a **Policy** CR YAML file to delete the **LVMCluster** CR:

Example Policy CR to delete the LVMCluster CR

```
apiVersion: policy.open-cluster-management.io/v1
kind: Policy
metadata:
  name: policy-lvmcluster-delete
  annotations:
    policy.open-cluster-management.io/standards: NIST SP 800-53
    policy.open-cluster-management.io/categories: CM Configuration Management
    policy.open-cluster-management.io/controls: CM-2 Baseline Configuration
spec:
  remediationAction: enforce
  disabled: false
  policy-templates:
    - objectDefinition:
        apiVersion: policy.open-cluster-management.io/v1
        kind: ConfigurationPolicy
        metadata:
          name: policy-lvmcluster-removal
        spec:
          remediationAction: enforce
          severity: low
          object-templates:
            - complianceType: mustnothave
              objectDefinition:
                kind: LVMCluster
                apiVersion: lvm.topolvm.io/v1alpha1
                metadata:
                  name: my-lvmcluster
                  namespace: openshift-lvm-storage
  ---
  apiVersion: policy.open-cluster-management.io/v1
  kind: PlacementBinding
  metadata:
    name: binding-policy-lvmcluster-delete
  placementRef:
    apiGroup: apps.open-cluster-management.io
    kind: PlacementRule
    name: placement-policy-lvmcluster-delete
  subjects:
    - apiGroup: policy.open-cluster-management.io
      kind: Policy
      name: policy-lvmcluster-delete
```

```

---
apiVersion: apps.open-cluster-management.io/v1
kind: PlacementRule
metadata:
  name: placement-policy-lvmcluster-delete
spec:
  clusterConditions:
    - status: "True"
      type: ManagedClusterConditionAvailable
  clusterSelector:
    matchExpressions:
      - key: mykey
        operator: In
        values:
          - myvalue

```

- **spec.policy-templates.spec.remediationAction:** This field is overridden by the preceding parameter value for **spec.remediationAction**.
- **spec.policy-templates.objectDefinition.spec.objectDefinition.metadata.namespace:** This **namespace** field must have the **openshift-lvm-storage** value.
- **spec.clusterSelector:** Configures the requirements to select the clusters. LVM Storage is uninstalled on the clusters that match the selection criteria.

4. Create the **Policy** CR by running the following command:

```
$ oc create -f <file_name> -n <namespace>
```

5. Create a **Policy** CR YAML file to check if the **LVMCluster** CR has been deleted:

Example Policy CR to check if the LVMCluster CR has been deleted

```

apiVersion: policy.open-cluster-management.io/v1
kind: Policy
metadata:
  name: policy-lvmcluster-inform
  annotations:
    policy.open-cluster-management.io/standards: NIST SP 800-53
    policy.open-cluster-management.io/categories: CM Configuration Management
    policy.open-cluster-management.io/controls: CM-2 Baseline Configuration
spec:
  remediationAction: inform
  disabled: false
  policy-templates:
    - objectDefinition:
        apiVersion: policy.open-cluster-management.io/v1
        kind: ConfigurationPolicy
        metadata:
          name: policy-lvmcluster-removal-inform
        spec:
          remediationAction: inform
          severity: low
          object-templates:
            - complianceType: mustnothave

```

```

      objectDefinition:
        kind: LVMCluster
        apiVersion: lvm.topolvm.io/v1alpha1
        metadata:
          name: my-lvmcluster
          namespace: openshift-lvm-storage
    ---
  apiVersion: policy.open-cluster-management.io/v1
  kind: PlacementBinding
  metadata:
    name: binding-policy-lvmcluster-check
  placementRef:
    apiGroup: apps.open-cluster-management.io
    kind: PlacementRule
    name: placement-policy-lvmcluster-check
  subjects:
    - apiGroup: policy.open-cluster-management.io
      kind: Policy
      name: policy-lvmcluster-inform
    ---
  apiVersion: apps.open-cluster-management.io/v1
  kind: PlacementRule
  metadata:
    name: placement-policy-lvmcluster-check
  spec:
    clusterConditions:
      - status: "True"
        type: ManagedClusterConditionAvailable
    clusterSelector:
      matchExpressions:
        - key: mykey
          operator: In
          values:
            - myvalue

```

- **spec.policy-templates.objectDefinition.spec.remediationAction:** This field is overridden by the preceding parameter value for **spec.remediationAction**.
- **spec.policy-templates.objectDefinition.spec.object-templates.objectDefinition.metadata.namespace:** This **namespace** field must have the **openshift-lvm-storage** value.

6. Create the **Policy** CR by running the following command:

```
$ oc create -f <file_name> -n <namespace>
```

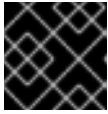
Verification

- Check the status of the **Policy** CRs by running the following command:

```
$ oc get policy -n <namespace>
```

Example output

NAME	REMEDIATION ACTION	COMPLIANCE STATE	AGE
policy-lvmcluster-delete	enforce	Compliant	15m
policy-lvmcluster-inform	inform	Compliant	15m

**IMPORTANT**

The **Policy** CRs must be in **Compliant** state.

5.4.5.4. Deleting an LVMCluster

When you delete an **LVMCluster** custom resource (CR), the Operator enforces deletion gates to prevent data loss. The gates that apply depend on the reclaim policy that is configured for the storage class.

Prerequisites

- You have administrative access to the cluster.
- You have identified the reclaim policy in use: **Delete** or **Retain**.

Procedure

1. Delete all Persistent Volume Claims (PVCs) that reference LVM **StorageClass** resources. If PVCs that reference LVM StorageClasses still exist, the Operator blocks **LVMCluster** deletion and generates a **DeletionPending** event:

found PVCs provisioned by LVMS, waiting 10s for their deletion

2. Back up any data before deleting PVCs.
 - a. List the PVCs that use the LVM StorageClass by running the following command:

```
$ oc get pvc -A -o custom-
columns='NAMESPACE:.metadata.namespace,NAME:.metadata.name,SC:.spec.storageCl
assName' | grep lvms-vg1
```

- b. Delete the PVCs by running the following command:

```
$ oc delete pvc <pvc_name> -n <namespace>
```

With the **Delete** reclaim policy, deleting the PVCs automatically removes the persistent volumes (PVs) and on-disk logical volumes. After all PVCs are removed, **LVMCluster** deletion completes automatically. No further action is required.

3. If you use the **Retain** reclaim policy, delete the retained PVs. After you delete PVCs, if the reclaim policy is **Retain**, the Operator blocks **LVMCluster** deletion and generates a **DeletionPending** event:

found PVs with Retain policy from LVMS, waiting 10s for manual cleanup

- a. List the retained PVs by running the following command:


```
$ oc get pv -o custom-columns='NAME:.metadata.name,SC:.spec.storageClassName' |
grep lvms-vg1
```

- b. Delete the PVs by running the following command:

```
$ oc delete pv <pv_name>
```

4. If you are using the **Retain** reclaim policy, delete the TopoLVM **LogicalVolume** custom resources.

After you delete PV objects from Kubernetes, the underlying logical volumes remain on disk because the **Retain** policy preserved them. The VG Manager detects this and generates a **ManualCleanupRequired** event:

```
Warning ManualCleanupRequired volume group vg1 has retained logical volumes [pvc-
abc123]; manual cleanup required before deletion can proceed
```

5. Deleting the **LogicalVolume** custom resources triggers on-disk logical volume cleanup.

- a. List the **LogicalVolume** custom resources by running the following command:

```
$ oc get logicalvolumes
```

- b. Delete the **LogicalVolume** custom resources for your device class by running the following command:

```
$ oc delete logicalvolume <lv_name>
```

Verification

- Verify that the **LVMCluster** deletion completed by confirming the resource no longer exists by running the following command:

```
$ oc get lvmcluster -A
```

5.4.6. Provisioning storage

After you have created the LVM volume groups using the **LVMCluster** custom resource (CR), you can provision the storage by creating persistent volume claims (PVCs).

The following are the minimum storage sizes that you can request for each file system type:

- **block**: 8 MiB
- **xfs**: 300 MiB
- **ext4**: 32 MiB

To create a PVC, you must create a **PersistentVolumeClaim** object.

Prerequisites

- You have created an **LVMCluster** CR.

Procedure

1. Log in to the OpenShift CLI (**oc**).
2. Create a **PersistentVolumeClaim** object:

Example PersistentVolumeClaim object

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: lvm-block-1 ❶
  namespace: default
spec:
  accessModes:
    - ReadWriteOnce
  volumeMode: Filesystem ❷
  resources:
    requests:
      storage: 10Gi ❸
    limits:
      storage: 20Gi ❹
  storageClassName: lvms-vg1 ❺
```

- ❶ Specify a name for the PVC.
- ❷ To create a file PVC, set this field to **Filesystem**. To create a block PVC, set this field to **Block**.
- ❸ Specify the storage size. If the value is less than the minimum storage size, the requested storage size is rounded to the minimum storage size. The total storage size you can provision is limited by the size of the Logical Volume Manager (LVM) thin pool and the over-provisioning factor.
- ❹ Optional: Specify the storage limit. Set this field to a value that is greater than or equal to the minimum storage size. Otherwise, PVC creation fails with an error.
- ❺ The value of the **storageClassName** field must be in the format **lvms-
<device_class_name>** where **<device_class_name>** is the value of the **deviceClasses.name** field in the **LVMCluster** CR. For example, if the **deviceClasses.name** field is set to **vg1**, you must set the **storageClassName** field to **lvms-vg1**.



NOTE

The **volumeBindingMode** field of the storage class is set to **WaitForFirstConsumer**.

3. Create the PVC by running the following command:

```
# oc create -f <file_name> -n <application_namespace>
```

**NOTE**

The created PVCs remain in **Pending** state until you deploy the pods that use them.

Verification

- To verify that the PVC is created, run the following command:

```
$ oc get pvc -n <namespace>
```

Example output

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES
STORAGECLASS	AGE			
lvm-block-1	Bound	pvc-e90169a8-fd71-4eea-93b8-817155f60e47	1Gi	RWO
lvms-vg1	5s			

5.4.7. StorageClass customization for LVMS device classes

You can customize the StorageClass for each device class by using the optional `storageClassOptions` field in the **LVMCluster** custom resource (CR).

Before, Logical Volume Manager Storage (LVMS) automatically created a StorageClass for each device class without allowing modification. If you attempted to manually edit a generated StorageClass, the Operator overwrote your changes during the next reconciliation loop.

The **storageClassOptions** field lets you control four properties of the generated StorageClass:

- **reclaimPolicy**
- **volumeBindingMode**
- **additionalParameters**
- **additionalLabels**

If you omit **storageClassOptions**, LVMS creates the StorageClass with the same defaults as in previous versions. Existing **LVMCluster** configurations are fully compatible with earlier versions.

**NOTE**

No user action is required after upgrading. The **storageClassOptions** field is optional, and default values match the behavior before this feature was introduced.

5.4.7.1. StorageClass options

You can configure custom StorageClass behaviors by defining the **storageClassOptions** field in your device class specification.

If you set an empty configuration (`storageClassOptions: {}`) or omit the field entirely, the Operator uses the following default settings:

Table 5.15. StorageClass Options Reference

Field	Type	Immutable	Description	Example
reclaimPolicy	string	Yes	Controls what happens to the PersistentVolume (PV) and its underlying logical volume when the PersistentVolumeClaim (PVC) is deleted. Allowed values: Delete (default), Retain When set to Retain, deleting a PVC does not delete the PV or the underlying logical volume on disk. Data is preserved, useful for data protection scenarios where accidental PVC deletion must not cause data loss. Manual cleanup is required before you can delete the LVMCluster. When set to Delete, both the PV and the on-disk logical volume are removed when the PVC is deleted.	<pre>storageClassOptions: reclaimPolicy: Retain</pre>
volumeBindingMode	string	Yes	Controls when volume binding and dynamic provisioning occur. Allowed values: WaitForFirstConsumer (default), Immediate WaitForFirstConsumer delays PV provisioning until a pod that uses the PVC is scheduled, enabling topology-aware scheduling where LVMS creates the PV on the node where the pod will run. Immediate provisions and binds the PV as soon as the PVC is created, without waiting for a consumer pod. On multi-node clusters, PVs might be provisioned on nodes where the consuming pod cannot run. Use Immediate only on single-node clusters or when node affinity is managed externally.	<pre>storageClassOptions: volumeBindingMode: Immediate</pre>

Field	Type	Immutable	Description	Example
additionalParameters	map[string]string	Yes	Adds custom key-value pairs to the StorageClass .parameters map. Default: {} (empty). Maximum entries: 16. StorageClass parameters are passed to the CSI driver (TopoLVM) during volume provisioning. TopoLVM recognizes only topolvm.io/device-class and csi.storage.k8s.io/fstype. Use additionalParameters for forward-compatibility or for parameters consumed by other Kubernetes components. The following keys are managed by LVMS and are rejected at admission: ● topolvm.io/device-class – automatically set to the device class name ● csi.storage.k8s.io/fstype – automatically set from the fstype field on the device class  IMPORTANT To change the filesystem type, use the fstype field on the device class directly. Do not use additionalParameters.	<pre>storageClassOptions: additionalParameters: custom-param-key: custom-param-value</pre>

Field	Type	Immutable	Description	Example
additionalLabels	map[string]string	No	Adds custom labels to the StorageClass metadata. Default: none. Maximum entries: 16. Use for organizational tagging, cluster policy integration, or monitoring. When you remove a label from additionalLabels, the operator removes it from the StorageClass during the next reconciliation. Labels added directly by other tools are not affected. The following label keys are reserved and cannot be set through additionalLabels: • app.kubernetes.io/managed-by • app.kubernetes.io/part-of • app.kubernetes.io/name • app.kubernetes.io/component • Any key with the prefix owned-by.topolvm.io/	<pre>storageClassOptions: additionalLabels: environment: production team: storage</pre>

5.4.7.2. Updating LVM cluster labels

To organize and categorize your storage resources, you can update, remove, or clear custom storage class labels by patching the **LVMCluster** custom resource. Labels are the only configuration field that you can modify after cluster creation.

Procedure

1. Patch the **LVMCluster** resource to update **additionalLabels** by running the following command:

```
$ oc -n openshift-lvm-storage patch lvmcluster <name> --type=json \
-p
'[{"op":"replace","path":"/spec/storage/deviceClasses/0/storageClassOptions/additionalLabels","value":{"environment":"staging"}}]'
```

2. To remove a specific label, update **additionalLabels** without the label you want to remove. The Operator removes the label from the **StorageClass** during the next reconciliation.

- To remove all custom labels, set **additionalLabels** to an empty map {}.



NOTE

The Operator preserves labels that you add directly to the **StorageClass**, for example with **oc label storageclass lvms-vg1 my-label=value**. The Operator prunes only the labels that you manage through the **additionalLabels** field in the **LVMCluster** custom resource (CR) when you remove them from the CR.

5.4.7.3. Sample LVM cluster configuration with storage class option

Use these examples to configure **storageClassOptions** in your **LVMCluster** custom resource (CR) to meet your specific storage requirements.

Default StorageClass behavior (no options)

```
apiVersion: lvm.topolvm.io/v1alpha1
kind: LVMCluster
metadata:
  name: my-lvmcluster
  namespace: openshift-lvm-storage
spec:
  storage:
    deviceClasses:
      - name: vg1
        default: true
    thinPoolConfig:
      name: thin-pool-1
      sizePercent: 90
      overprovisionRatio: 10
```

This produces a **StorageClass** with **reclaimPolicy: Delete** and **volumeBindingMode: WaitForFirstConsumer**, which is the same as the behavior before this feature.

Retain policy for data protection

```
apiVersion: lvm.topolvm.io/v1alpha1
kind: LVMCluster
metadata:
  name: my-lvmcluster
  namespace: openshift-lvm-storage
spec:
  storage:
    deviceClasses:
      - name: vg1
        default: true
    thinPoolConfig:
      name: thin-pool-1
      sizePercent: 90
      overprovisionRatio: 10
    storageClassOptions:
      reclaimPolicy: Retain
```

Immediate binding for pre-provisioning

```
apiVersion: lvm.topolvm.io/v1alpha1
kind: LVMCluster
metadata:
  name: my-lvmcluster
  namespace: openshift-lvm-storage
spec:
  storage:
    deviceClasses:
      - name: vg1
        default: true
    thinPoolConfig:
      name: thin-pool-1
      sizePercent: 90
      overprovisionRatio: 10
    storageClassOptions:
      volumeBindingMode: Immediate
```

All options configured together

```
apiVersion: lvm.topolvm.io/v1alpha1
kind: LVMCluster
metadata:
  name: my-lvmcluster
  namespace: openshift-lvm-storage
spec:
  storage:
    deviceClasses:
      - name: vg1
        default: true
    thinPoolConfig:
      name: thin-pool-1
      sizePercent: 90
      overprovisionRatio: 10
    storageClassOptions:
      reclaimPolicy: Retain
      volumeBindingMode: WaitForFirstConsumer
      additionalParameters:
        custom-key: custom-value
      additionalLabels:
        environment: production
        team: storage
```

Multiple device classes with different options

```
apiVersion: lvm.topolvm.io/v1alpha1
kind: LVMCluster
metadata:
  name: my-lvmcluster
  namespace: openshift-lvm-storage
spec:
  storage:
    deviceClasses:
      - name: vg-fast
        default: true
    thinPoolConfig:
```



```

    name: thin-pool-1
    sizePercent: 90
    overprovisionRatio: 10
  deviceSelector:
    paths:
    - /dev/nvme0n1
  storageClassOptions:
    reclaimPolicy: Delete
    volumeBindingMode: WaitForFirstConsumer
    additionalLabels:
      tier: fast
- name: vg-archive
  thinPoolConfig:
    name: thin-pool-1
    sizePercent: 90
    overprovisionRatio: 10
  deviceSelector:
    paths:
    - /dev/sda
  storageClassOptions:
    reclaimPolicy: Retain
    volumeBindingMode: WaitForFirstConsumer
    additionalLabels:
      tier: archive

```

For a device class named **vg1** with the full configuration, LVMS generates a **StorageClass** named **lvms-vg1** with the following structure:

```

apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: lvms-vg1
  annotations:
    description: "Provides RWO and RWOP Filesystem & Block volumes"
    storageclass.kubernetes.io/is-default-class: "true"
  labels:
    environment: production
    team: storage
provisioner: topolvm.io
reclaimPolicy: Retain
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true
parameters:
  custom-key: custom-value
  topolvm.io/device-class: vg1
  csi.storage.k8s.io/fstype: xfs

```

The **StorageClass** name always follows the convention **lvms-<device_class_name>**.

5.4.7.4. Immutable fields of the storage class options

After you create the **LVMCluster**, you cannot change the value of the some of the **storageClassOptions** fields such as **reclaimPolicy**, **volumeBindingMode**, and **additionalParameters**. This mirrors the behavior of Kubernetes StorageClasses, which do not allow changes to these fields after creation.

If you attempt to modify an immutable field, the API server rejects the request:

Invalid value: "object": reclaimPolicy is immutable once set

There is no way to patch or update immutable fields in place. To change an immutable field, you must delete the **LVMCluster** and recreate it with the new values.

For example, you cannot change the filesystem type through **additionalParameters**. The **csi.storage.k8s.io/fstype** parameter is managed by LVMS and is rejected at admission if set through **additionalParameters**. To use **ext4** instead of the default **xfs**, use the **fstype** field on the device class:

```
deviceClasses:
- name: vg1
  fstype: ext4
```

However, the **fstype** field is also immutable after creation.



NOTE

The deletion gates require all PVCs and, for the **Retain** policy, all PVs to be removed before the **LVMCluster** can be deleted. After you recreate the **LVMCluster** with the new values, new PVCs use the updated StorageClass configuration.

5.4.7.5. Behaviors not controlled by StorageClass options

Review these behaviors before you delete an LVMCluster. Although these behaviors relate to **storageClassOptions**, the **storageClassOptions** field does not control them.

Volume expansion behavior

Logical Volume Manager Storage (LVMS) always enables volume expansion by setting **allowVolumeExpansion: true** on generated StorageClasses. You cannot control this setting by using the **storageClassOptions** field. All LVMS volumes support online expansion.

VolumeSnapshotClass management

The **storageClassOptions** field only affects StorageClasses. When you configure thin provisioning, LVMS generates a **VolumeSnapshotClass** for each device class. This generated class always uses a fixed value **deletionPolicy: Delete**, regardless of the reclaimPolicy that you set in **storageClassOptions**.

Additionally, LVMS does not apply the **additionalParameters** and **additionalLabels** fields to **VolumeSnapshotClasses**. If you need to retain snapshot data, you must manage it separately from the StorageClass reclaim policy.

Default StorageClass annotation behavior

The default field on a device class controls the **storageclass.kubernetes.io/is-default-class** annotation on the generated StorageClass.

Setting **default: true** does not guarantee that the LVMS StorageClass becomes the cluster default. If another default StorageClass already exists on the cluster, for example, gp3-csi on AWS-based OpenShift Container Platform clusters, LVMS sets the annotation to **false** to prevent many cluster-wide defaults. Because the Operator actively manages this annotation, it reverts any manual, out-of-band changes during the next reconciliation loop.

5.4.8. Ways to scale up the storage of clusters

OpenShift Container Platform supports additional worker nodes for clusters on bare metal user-provisioned infrastructure. You can scale up the storage of clusters either by adding new worker nodes with available storage or by adding new devices to the existing worker nodes.

Logical Volume Manager (LVM) Storage detects and uses additional worker nodes when the nodes become active.

To add a new device to the existing worker nodes on a cluster, you must add the path to the new device in the **deviceSelector** field of the **LVMCluster** custom resource (CR).



IMPORTANT

You can add the **deviceSelector** field in the **LVMCluster** CR only while creating the **LVMCluster** CR. If you have not added the **deviceSelector** field while creating the **LVMCluster** CR, you must delete the **LVMCluster** CR and create a new **LVMCluster** CR containing the **deviceSelector** field.

If you do not add the **deviceSelector** field in the **LVMCluster** CR, LVM Storage automatically adds the new devices when the devices are available.



NOTE

LVM Storage adds only the supported devices. For information about unsupported devices, see "Devices not supported by LVM Storage".

Additional resources

- [Adding worker nodes to single-node OpenShift clusters](#)
- [Devices not supported by LVM Storage](#)

5.4.8.1. Scaling up the storage of clusters by using the CLI

You can scale up the storage capacity of the worker nodes on a cluster by using the OpenShift CLI (**oc**).

Prerequisites

- You have additional unused devices on each cluster to be used by Logical Volume Manager (LVM) Storage.
- You have installed the OpenShift CLI (**oc**).
- You have created an **LVMCluster** custom resource (CR).

Procedure

1. Edit the **LVMCluster** CR by running the following command:

```
$ oc edit <lvmlcluster_file_name> -n <namespace>
```

2. Add the path to the new device in the **deviceSelector** field.

Example LVMCluster CR

```
apiVersion: lvm.topolvm.io/v1alpha1
kind: LVMCluster
metadata:
  name: my-lvmcluster
spec:
  storage:
    deviceClasses:
# ...
    deviceSelector: ❶
    paths: ❷
    - /dev/disk/by-path/pci-0000:87:00.0-nvme-1
    - /dev/disk/by-path/pci-0000:88:00.0-nvme-1
    optionalPaths: ❸
    - /dev/disk/by-path/pci-0000:89:00.0-nvme-1
    - /dev/disk/by-path/pci-0000:90:00.0-nvme-1
# ...
```

- ❶ Contains the configuration to specify the paths to the devices that you want to add to the LVM volume group. You can specify the device paths in the **paths** field, the **optionalPaths** field, or both. If you do not specify the device paths in both **paths** and **optionalPaths**, Logical Volume Manager (LVM) Storage adds the supported unused devices to the LVM volume group. LVM Storage adds the devices to the LVM volume group only if the following conditions are met:
 - The device path exists.
 - The device is supported by LVM Storage. For information about unsupported devices, see "Devices not supported by LVM Storage".
- ❷ Specify the device paths. If the device path specified in this field does not exist, or the device is not supported by LVM Storage, the **LVMCluster** CR moves to the **Failed** state.
- ❸ Specify the optional device paths. If the device path specified in this field does not exist, or the device is not supported by LVM Storage, LVM Storage ignores the device without causing an error.



IMPORTANT

After a device is added to the LVM volume group, it cannot be removed.

3. Save the **LVMCluster** CR.

Additional resources

- [About the LVMCluster custom resource](#)
- [Devices not supported by LVM Storage](#)
- [About adding devices to a volume group](#)

5.4.8.2. Scaling up the storage of clusters by using the web console

You can scale up the storage capacity of the worker nodes on a cluster by using the OpenShift Container Platform web console.

Prerequisites

- You have additional unused devices on each cluster to be used by Logical Volume Manager (LVM) Storage.
- You have created an **LVMCluster** custom resource (CR).

Procedure

1. Log in to the OpenShift Container Platform web console.
2. Click **Ecosystem** → **Installed Operators**.
3. Click **LVM Storage** in the **openshift-lvm-storage** namespace.
4. Click the **LVMCluster** tab to view the **LVMCluster** CR created on the cluster.
5. From the **Actions** menu, select **Edit LVMCluster**.
6. Click the **YAML** tab.
7. Edit the **LVMCluster** CR to add the new device path in the **deviceSelector** field:

Example LVMCluster CR

```
apiVersion: lvm.topolvm.io/v1alpha1
kind: LVMCluster
metadata:
  name: my-lvmcluster
spec:
  storage:
    deviceClasses:
# ...
    deviceSelector: ❶
    paths: ❷
      - /dev/disk/by-path/pci-0000:87:00.0-nvme-1
      - /dev/disk/by-path/pci-0000:88:00.0-nvme-1
    optionalPaths: ❸
      - /dev/disk/by-path/pci-0000:89:00.0-nvme-1
      - /dev/disk/by-path/pci-0000:90:00.0-nvme-1
# ...
```

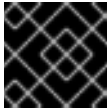
- ❶ Contains the configuration to specify the paths to the devices that you want to add to the LVM volume group. You can specify the device paths in the **paths** field, the **optionalPaths** field, or both. If you do not specify the device paths in both **paths** and **optionalPaths**, Logical Volume Manager (LVM) Storage adds the supported unused devices to the LVM volume group. LVM Storage adds the devices to the LVM volume group only if the following conditions are met:

- The device path exists.

❷ The paths are specified in the LVMCluster CR. For example, the following paths are specified:

- The device is supported by LVM Storage. For information about unsupported devices, see "Devices not supported by LVM Storage".

- 2 Specify the device paths. If the device path specified in this field does not exist, or the device is not supported by LVM Storage, the **LVMCluster** CR moves to the **Failed** state.
- 3 Specify the optional device paths. If the device path specified in this field does not exist, or the device is not supported by LVM Storage, LVM Storage ignores the device without causing an error.



IMPORTANT

After a device is added to the LVM volume group, it cannot be removed.

8. Click **Save**.

Additional resources

- [About the **LVMCluster** custom resource](#)
- [Devices not supported by LVM Storage](#)
- [About adding devices to a volume group](#)

5.4.8.3. Scaling up the storage of clusters by using RHACM

You can scale up the storage capacity of worker nodes on the clusters by using RHACM.

Prerequisites

- You have access to the RHACM cluster using an account with **cluster-admin** privileges.
- You have created an **LVMCluster** custom resource (CR) by using RHACM.
- You have additional unused devices on each cluster to be used by Logical Volume Manager (LVM) Storage.

Procedure

1. Log in to the RHACM CLI using your OpenShift Container Platform credentials.
2. Edit the **LVMCluster** CR that you created using RHACM by running the following command:

```
$ oc edit -f <file_name> -n <namespace> 1
```

- 1 Replace **<file_name>** with the name of the **LVMCluster** CR.

3. In the **LVMCluster** CR, add the path to the new device in the **deviceSelector** field.

Example LVMCluster CR

```
apiVersion: policy.open-cluster-management.io/v1
```

```

kind: ConfigurationPolicy
metadata:
  name: lvms
spec:
  object-templates:
    - complianceType: musthave
      objectDefinition:
        apiVersion: lvm.topolvm.io/v1alpha1
        kind: LVMCluster
        metadata:
          name: my-lvmcluster
          namespace: openshift-lvm-storage
        spec:
          storage:
            deviceClasses:
# ...
              deviceSelector: ❶
              paths: ❷
              - /dev/disk/by-path/pci-0000:87:00.0-nvme-1
              optionalPaths: ❸
              - /dev/disk/by-path/pci-0000:89:00.0-nvme-1
# ...

```

- ❶ Contains the configuration to specify the paths to the devices that you want to add to the LVM volume group. You can specify the device paths in the **paths** field, the **optionalPaths** field, or both. If you do not specify the device paths in both **paths** and **optionalPaths**, Logical Volume Manager (LVM) Storage adds the supported unused devices to the LVM volume group. LVM Storage adds the devices to the LVM volume group only if the following conditions are met:
 - The device path exists.
 - The device is supported by LVM Storage. For information about unsupported devices, see "Devices not supported by LVM Storage".
- ❷ Specify the device paths. If the device path specified in this field does not exist, or the device is not supported by LVM Storage, the **LVMCluster** CR moves to the **Failed** state.
- ❸ Specify the optional device paths. If the device path specified in this field does not exist, or the device is not supported by LVM Storage, LVM Storage ignores the device without causing an error.



IMPORTANT

After a device is added to the LVM volume group, it cannot be removed.

4. Save the **LVMCluster** CR.

Additional resources

- [Red Hat Advanced Cluster Management for Kubernetes: Installing while connected online](#)
- [About the **LVMCluster** custom resource](#)

- [Devices not supported by LVM Storage](#)
- [About adding devices to a volume group](#)

5.4.9. Expanding a persistent volume claim

After scaling up the storage of a cluster, you can expand the existing persistent volume claims (PVCs).

To expand a PVC, you must update the **storage** field in the PVC.

Prerequisites

- Dynamic provisioning is used.
- The **StorageClass** object associated with the PVC has the **allowVolumeExpansion** field set to **true**.

Procedure

1. Log in to the OpenShift CLI (**oc**).
2. Update the value of the **spec.resources.requests.storage** field to a value that is greater than the current value by running the following command:

```
$ oc patch pvc <pvc_name> -n <application_namespace> \ 1
--type=merge -p '{ "spec": { "resources": { "requests": { "storage": "<desired_size>" }}}}' 2
```

1 Replace **<pvc_name>** with the name of the PVC that you want to expand.

2 Replace **<desired_size>** with the new size to expand the PVC.

Verification

- To verify that resizing is completed, run the following command:

```
$ oc get pvc <pvc_name> -n <application_namespace> -o=jsonpath=
{.status.capacity.storage}
```

LVM Storage adds the **Resizing** condition to the PVC during expansion. It deletes the **Resizing** condition after the PVC expansion.

Additional resources

- [Ways to scale up the storage of clusters](#)
- [Enabling volume expansion support](#)

5.4.10. Deleting a persistent volume claim

You can delete a persistent volume claim (PVC) by using the OpenShift CLI (**oc**).

Prerequisites

- You have access to OpenShift Container Platform as a user with **cluster-admin** permissions.

Procedure

1. Log in to the OpenShift CLI (**oc**).
2. Delete the PVC by running the following command:

```
$ oc delete pvc <pvc_name> -n <namespace>
```

Verification

- To verify that the PVC is deleted, run the following command:

```
$ oc get pvc -n <namespace>
```

The deleted PVC must not be present in the output of this command.

5.4.11. About volume snapshots

You can create snapshots of persistent volume claims (PVCs) that are provisioned by LVM Storage.

You can perform the following actions using the volume snapshots:

- Back up your application data.



IMPORTANT

Volume snapshots are located on the same devices as the original data. To use the volume snapshots as backups, you must move the snapshots to a secure location. You can use OpenShift API for Data Protection (OADP) backup and restore solutions. For information about OADP, see "OADP features".

- Revert to a state at which the volume snapshot was taken.



NOTE

You can also create volume snapshots of the volume clones.

5.4.11.1. Limitations for creating volume snapshots in multi-node topology

LVM Storage has the following limitations for creating volume snapshots in multi-node topology:

- Creating volume snapshots is based on the LVM thin pool capabilities.
- After creating a volume snapshot, the node must have additional storage space for further updating the original data source.
- You can create volume snapshots only on the node where you have deployed the original data source.
- Pods relying on the PVC that uses the snapshot data can be scheduled only on the node where you have deployed the original data source.

Additional resources

- [OADP features](#)

5.4.11.2. Creating volume snapshots

You can create volume snapshots based on the available capacity of the thin pool and the over-provisioning limits. To create a volume snapshot, you must create a **VolumeSnapshotClass** object.

Prerequisites

- You have access to OpenShift Container Platform as a user with **cluster-admin** permissions.
- You ensured that the persistent volume claim (PVC) is in **Bound** state. This is required for a consistent snapshot.
- You stopped all the I/O to the PVC.

Procedure

1. Log in to the OpenShift CLI (**oc**).
2. Create a **VolumeSnapshot** object:

Example VolumeSnapshot object

```
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshot
metadata:
  name: lvm-block-1-snap 1
spec:
  source:
    persistentVolumeClaimName: lvm-block-1 2
    volumeSnapshotClassName: lvms-vg1 3
```

- 1 Specify a name for the volume snapshot.
- 2 Specify the name of the source PVC. LVM Storage creates a snapshot of this PVC.
- 3 Set this field to the name of a volume snapshot class.



NOTE

To get the list of available volume snapshot classes, run the following command:

```
$ oc get volumesnapshotclass
```

3. Create the volume snapshot in the namespace where you created the source PVC by running the following command:

```
$ oc create -f <file_name> -n <namespace>
```

LVM Storage creates a read-only copy of the PVC as a volume snapshot.

Verification

- To verify that the volume snapshot is created, run the following command:

```
$ oc get volumesnapshot -n <namespace>
```

Example output

```
NAME          READYTOUSE SOURCEPVC  SOURCESNAPSHOTCONTENT
RESTORESIZE  SNAPSHOTCLASS SNAPSHOTCONTENT
CREATIONTIME AGE
lvms-test-1  true          lvms-test-1          1Gi          lvms-vg1
snapcontent-af409f97-55fc-40cf-975f-71e44fa2ca91  19s          19s
```

The value of the **READYTOUSE** field for the volume snapshot that you created must be **true**.

5.4.11.3. Restoring volume snapshots

To restore a volume snapshot, you must create a persistent volume claim (PVC) with the **dataSource.name** field set to the name of the volume snapshot.

The restored PVC is independent of the volume snapshot and the source PVC.

Prerequisites

- You have access to OpenShift Container Platform as a user with **cluster-admin** permissions.
- You have created a volume snapshot.

Procedure

- Log in to the OpenShift CLI (**oc**).
- Create a **PersistentVolumeClaim** object with the configuration to restore the volume snapshot:

Example PersistentVolumeClaim object to restore a volume snapshot

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: lvms-test-1-restore
spec:
  accessModes:
    - ReadWriteOnce
  volumeMode: Block
  Resources:
    Requests:
      storage: 2Gi 1
  storageClassName: lvms-vg1 2
  dataSource:
```

```
name: lvm-block-1-snap 3
kind: VolumeSnapshot
apiGroup: snapshot.storage.k8s.io
```

- 1** Specify the storage size of the restored PVC. The storage size of the requested PVC must be greater than or equal to the storage size of the volume snapshot that you want to restore. If a larger PVC is required, you can also resize the PVC after restoring the volume snapshot.
 - 2** Set this field to the value of the **storageClassName** field in the source PVC of the volume snapshot that you want to restore.
 - 3** Set this field to the name of the volume snapshot that you want to restore.
3. Create the PVC in the namespace where you created the volume snapshot by running the following command:

```
$ oc create -f <file_name> -n <namespace>
```

Verification

- To verify that the volume snapshot is restored, run the following command:

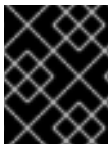
```
$ oc get pvc -n <namespace>
```

Example output

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES
lvm-block-1-restore	Bound	pvc-e90169a8-fd71-4eea-93b8-817155f60e47	1Gi	RWO
lvms-vg1	5s			

5.4.11.4. Deleting volume snapshots

You can delete the volume snapshots of the persistent volume claims (PVCs).



IMPORTANT

When you delete a persistent volume claim (PVC), LVM Storage deletes only the PVC, but not the snapshots of the PVC.

Prerequisites

- You have access to OpenShift Container Platform as a user with **cluster-admin** permissions.
- You have ensured that the volume snapshot that you want to delete is not in use.

Procedure

1. Log in to the OpenShift CLI (**oc**).
2. Delete the volume snapshot by running the following command:

```
$ oc delete volumesnapshot <volume_snapshot_name> -n <namespace>
```

Verification

- To verify that the volume snapshot is deleted, run the following command:

```
$ oc get volumesnapshot -n <namespace>
```

The deleted volume snapshot must not be present in the output of this command.

5.4.12. About volume clones

A volume clone is a duplicate of an existing persistent volume claim (PVC). You can create a volume clone to make a point-in-time copy of the data.

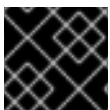
5.4.12.1. Limitations for creating volume clones in multi-node topology

LVM Storage has the following limitations for creating volume clones in multi-node topology:

- Creating volume clones is based on the LVM thin pool capabilities.
- The node must have additional storage after creating a volume clone for further updating the original data source.
- You can create volume clones only on the node where you have deployed the original data source.
- Pods relying on the PVC that uses the clone data can be scheduled only on the node where you have deployed the original data source.

5.4.12.2. Creating volume clones

To create a clone of a persistent volume claim (PVC), you must create a **PersistentVolumeClaim** object in the namespace where you created the source PVC.



IMPORTANT

The cloned PVC has write access.

Prerequisites

- You ensured that the source PVC is in **Bound** state. This is required for a consistent clone.

Procedure

1. Log in to the OpenShift CLI (**oc**).
2. Create a **PersistentVolumeClaim** object:

Example **PersistentVolumeClaim** object to create a volume clone

```
kind: PersistentVolumeClaim
apiVersion: v1
```

```

metadata:
  name: lvm-pvc-clone
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: lvms-vg1 1
  volumeMode: Filesystem 2
  dataSource:
    kind: PersistentVolumeClaim
    name: lvm-pvc 3
  resources:
    requests:
      storage: 1Gi 4

```

- 1** Set this field to the value of the **storageClassName** field in the source PVC.
- 2** Set this field to the **volumeMode** field in the source PVC.
- 3** Specify the name of the source PVC.
- 4** Specify the storage size for the cloned PVC. The storage size of the cloned PVC must be greater than or equal to the storage size of the source PVC.

3. Create the PVC in the namespace where you created the source PVC by running the following command:

```
$ oc create -f <file_name> -n <namespace>
```

Verification

- To verify that the volume clone is created, run the following command:

```
$ oc get pvc -n <namespace>
```

Example output

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES
STORAGECLASS	AGE			
lvm-block-1-clone	Bound	pvc-e90169a8-fd71-4eea-93b8-817155f60e47	1Gi	RWO
lvms-vg1	5s			

5.4.12.3. Deleting volume clones

You can delete volume clones.



IMPORTANT

When you delete a persistent volume claim (PVC), LVM Storage deletes only the source persistent volume claim (PVC) but not the clones of the PVC.

Prerequisites

- You have access to OpenShift Container Platform as a user with **cluster-admin** permissions.

Procedure

1. Log in to the OpenShift CLI (**oc**).
2. Delete the cloned PVC by running the following command:

```
# oc delete pvc <clone_pvc_name> -n <namespace>
```

Verification

- To verify that the volume clone is deleted, run the following command:

```
$ oc get pvc -n <namespace>
```

The deleted volume clone must not be present in the output of this command.

5.4.13. Updating LVM Storage

You can update LVM Storage to ensure compatibility with the OpenShift Container Platform version after upgrading your cluster.



NOTE

The default namespace for the LVM Storage Operator is **openshift-lvm-storage**.

Prerequisites

- You have updated your OpenShift Container Platform cluster.
- You have installed a previous version of LVM Storage.
- You have installed the OpenShift CLI (**oc**).
- You have access to the cluster using an account with **cluster-admin** permissions.

Procedure

1. Log in to the OpenShift CLI (**oc**).
2. Update the **Subscription** custom resource (CR) that you created while installing LVM Storage by running the following command:

```
$ oc patch subscription lvms-operator -n openshift-lvm-storage --type merge --patch '{"spec": {"channel": "<update_channel>"}}'
```

Replace **<update_channel>** with the version of LVM Storage that you want to install. For example, **stable-4.22**.

3. View the update events to check that the installation is complete by running the following command:

```
$ oc get events -n openshift-lvm-storage
```

Example output

```
...
8m13s    Normal  RequirementsUnknown  clusterserviceversion/lvms-operator.v4.22
requirements not yet checked
8m11s    Normal  RequirementsNotMet   clusterserviceversion/lvms-operator.v4.22  one
or more requirements couldn't be found
7m50s    Normal  AllRequirementsMet   clusterserviceversion/lvms-operator.v4.22  all
requirements found, attempting install
7m50s    Normal  InstallSucceeded     clusterserviceversion/lvms-operator.v4.22  waiting
for install components to report healthy
7m49s    Normal  InstallWaiting       clusterserviceversion/lvms-operator.v4.22  installing:
waiting for deployment lvms-operator to become ready: deployment "lvms-operator" waiting
for 1 outdated replica(s) to be terminated
7m39s    Normal  InstallSucceeded     clusterserviceversion/lvms-operator.v4.22  install
strategy completed with no errors
...
```

Verification

- Verify the LVM Storage version by running the following command:

```
$ oc get subscription lvms-operator -n openshift-lvm-storage -o
jsonpath='{.status.installedCSV}'
```

Example output

```
lvms-operator.v4.22
```

5.4.14. Monitoring LVM Storage

You can monitor LVM Storage by enabling cluster monitoring with a namespace label, then viewing metrics to track storage usage and receiving alerts when thin pool and volume group capacity reaches critical thresholds to prevent data loss.

To enable cluster monitoring, you must add a label in the namespace where you have installed LVM Storage.



IMPORTANT

For information about enabling cluster monitoring in RHACM, see "Observability" and "Adding custom metrics".

Procedure

- To enable cluster monitoring, add the following label in the namespace where you have installed LVM Storage:

```
openshift.io/cluster-monitoring=true
```


5.4.15. Uninstalling LVM Storage by using the CLI

Uninstall LVM Storage when it is no longer needed or before upgrading to a different storage solution by using the OpenShift CLI (**oc**) after removing all provisioned storage resources.

Prerequisites

- You have logged in to **oc** as a user with **cluster-admin** permissions.
- You deleted the persistent volume claims (PVCs), volume snapshots, and volume clones provisioned by LVM Storage. You have also deleted the applications that are using these resources.
- You deleted the **LVMCluster** custom resource (CR).

Procedure

1. Get the **currentCSV** value for the LVM Storage Operator by running the following command:

```
$ oc get subscription.operators.coreos.com lvms-operator -n <namespace> -o yaml | grep currentCSV
```

Example output

```
currentCSV: lvms-operator.v4.15.3
```

2. Delete the subscription by running the following command:

```
$ oc delete subscription.operators.coreos.com lvms-operator -n <namespace>
```

Example output

```
subscription.operators.coreos.com "lvms-operator" deleted
```

3. Delete the CSV for the LVM Storage Operator in the target namespace by running the following command:

```
$ oc delete clusterserviceversion <currentCSV> -n <namespace>
```

Replace **<currentCSV>** with the **currentCSV** value for the LVM Storage Operator.

Example output

```
clusterserviceversion.operators.coreos.com "lvms-operator.v4.15.3" deleted
```

Verification

- To verify that the LVM Storage Operator is uninstalled, run the following command:

```
$ oc get csv -n <namespace>
```

If the LVM Storage Operator was successfully uninstalled, it does not appear in the output of this command.

5.4.16. Uninstalling LVM Storage by using the web console

Uninstall LVM Storage when it is no longer needed or before upgrading to a different storage solution by using the OpenShift Container Platform web console after removing all provisioned storage resources.

Prerequisites

- You have access to OpenShift Container Platform as a user with **cluster-admin** permissions.
- You have deleted the persistent volume claims (PVCs), volume snapshots, and volume clones provisioned by LVM Storage. You have also deleted the applications that are using these resources.
- You have deleted the **LVMCluster** custom resource (CR).

Procedure

1. Log in to the OpenShift Container Platform web console.
2. Click **Ecosystem** → **Installed Operators**.
3. Click **LVM Storage** in the **openshift-lvm-storage** namespace.
4. Click the **Details** tab.
5. From the **Actions** menu, select **Uninstall Operator**.
6. Optional: When prompted, select the **Delete all operand instances for this operator** checkbox to delete the operand instances for LVM Storage.
7. Click **Uninstall**.

5.4.17. Uninstalling LVM Storage installed using RHACM

To uninstall LVM Storage that you installed by using RHACM when it is no longer needed or before switching to a different storage solution, delete the RHACM **Policy** custom resource (CR) that you created for installation after removing all provisioned storage resources.

Prerequisites

- You have access to the RHACM cluster as a user with **cluster-admin** permissions.
- You have deleted the persistent volume claims (PVCs), volume snapshots, and volume clones provisioned by LVM Storage. You have also deleted the applications that are using these resources.
- You have deleted the **LVMCluster** CR that you created using RHACM.

Procedure

1. Log in to the OpenShift CLI (**oc**).

2. Delete the RHACM **Policy** CR that you created for installing and configuring LVM Storage by using the following command:

```
$ oc delete -f <policy> -n <namespace>
```

Replace **<policy>** with the name of the **Policy** CR YAML file.

3. Create a **Policy** CR YAML file with the configuration to uninstall LVM Storage:

Example Policy CR to uninstall LVM Storage

```
apiVersion: apps.open-cluster-management.io/v1
kind: PlacementRule
metadata:
  name: placement-uninstall-lvms
spec:
  clusterConditions:
    - status: "True"
      type: ManagedClusterConditionAvailable
  clusterSelector:
    matchExpressions:
      - key: mykey
        operator: In
        values:
          - myvalue
---
apiVersion: policy.open-cluster-management.io/v1
kind: PlacementBinding
metadata:
  name: binding-uninstall-lvms
placementRef:
  apiGroup: apps.open-cluster-management.io
  kind: PlacementRule
  name: placement-uninstall-lvms
subjects:
  - apiGroup: policy.open-cluster-management.io
    kind: Policy
    name: uninstall-lvms
---
apiVersion: policy.open-cluster-management.io/v1
kind: Policy
metadata:
  annotations:
    policy.open-cluster-management.io/categories: CM Configuration Management
    policy.open-cluster-management.io/controls: CM-2 Baseline Configuration
    policy.open-cluster-management.io/standards: NIST SP 800-53
  name: uninstall-lvms
spec:
  disabled: false
  policy-templates:
    - objectDefinition:
        apiVersion: policy.open-cluster-management.io/v1
        kind: ConfigurationPolicy
        metadata:
          name: uninstall-lvms
```

```

spec:
  object-templates:
  - complianceType: mustnothave
    objectDefinition:
      apiVersion: v1
      kind: Namespace
      metadata:
        name: openshift-lvm-storage
  - complianceType: mustnothave
    objectDefinition:
      apiVersion: operators.coreos.com/v1
      kind: OperatorGroup
      metadata:
        name: openshift-storage-operatorgroup
        namespace: openshift-lvm-storage
    spec:
      targetNamespaces:
      - openshift-lvm-storage
  - complianceType: mustnothave
    objectDefinition:
      apiVersion: operators.coreos.com/v1alpha1
      kind: Subscription
      metadata:
        name: lvms-operator
        namespace: openshift-lvm-storage
    remediationAction: enforce
    severity: low
- objectDefinition:
  apiVersion: policy.open-cluster-management.io/v1
  kind: ConfigurationPolicy
  metadata:
    name: policy-remove-lvms-crds
  spec:
    object-templates:
    - complianceType: mustnothave
      objectDefinition:
        apiVersion: apiextensions.k8s.io/v1
        kind: CustomResourceDefinition
        metadata:
          name: logicalvolumes.topolvm.io
    - complianceType: mustnothave
      objectDefinition:
        apiVersion: apiextensions.k8s.io/v1
        kind: CustomResourceDefinition
        metadata:
          name: lvmclusters.lvm.topolvm.io
    - complianceType: mustnothave
      objectDefinition:
        apiVersion: apiextensions.k8s.io/v1
        kind: CustomResourceDefinition
        metadata:
          name: lvmvolumegroupnodestatuses.lvm.topolvm.io
    - complianceType: mustnothave
      objectDefinition:
        apiVersion: apiextensions.k8s.io/v1
        kind: CustomResourceDefinition

```

```

metadata:
  name: lvmvolumeegroups.lvm.topolvm.io
remediationAction: enforce
severity: high

```

4. Create the **Policy** CR by running the following command:

```
$ oc create -f <policy> -ns <namespace>
```

5.4.18. Downloading log files and diagnostic information using must-gather

Use the must-gather tool to collect log files and diagnostic information when LVM Storage cannot automatically resolve a problem. You or Red Hat Support can then review the collected data to troubleshoot the issue.

Procedure

- Run the **must-gather** command from the client connected to the LVM Storage cluster:

```
$ oc adm must-gather --image=registry.redhat.io/lvms4/lvms-must-gather-rhel9:v4.22 --dest-dir=<directory_name>
```

Additional resources

- [About the must-gather tool](#)

5.4.19. Troubleshooting persistent storage

If persistent storage issues occur with Logical Volume Manager (LVM) Storage, such as persistent volume claims (PVCs) stuck in a pending state, missing components, or node and disk failures, you can diagnose and resolve the problem by reviewing logs and recovering affected resources.

5.4.19.1. Investigating a PVC stuck in the Pending state

Investigate persistent volume claims (PVCs) stuck in a **Pending** state to determine whether the cause is insufficient resources, network problems, mismatched storage classes, or unavailable persistent volumes (PVs).

A persistent volume claim (PVC) can get stuck in the **Pending** state for the following reasons:

- Insufficient computing resources.
- Network problems.
- Mismatched storage class or node selector.
- No available persistent volumes (PVs).
- The node with the PV is in the **Not Ready** state.

Prerequisites

- You have installed the OpenShift CLI (**oc**).

- You have logged in to the OpenShift CLI (**oc**) as a user with **cluster-admin** permissions.

Procedure

- Retrieve the list of PVCs by running the following command:

```
$ oc get pvc
```

Example output

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS	AGE
lvms-test	Pending			lvms-vg1		11s

- Inspect the events associated with a PVC stuck in the **Pending** state by running the following command:

```
$ oc describe pvc <pvc_name>
```

Replace **<pvc_name>** with the name of the PVC. For example, **lvms-vg1**.

Example output

Type	Reason	Age	From	Message
Warning	ProvisioningFailed	4s (x2 over 17s)	persistentvolume-controller	storageclass.storage.k8s.io "lvms-vg1" not found

5.4.19.2. Recovering from a missing storage class

Resolve the "storage class not found" error by verifying that the LVMCluster custom resource (CR) exists and all Logical Volume Manager (LVM) Storage pods are running, then reviewing logs to identify configuration issues.

Prerequisites

- You have installed the OpenShift CLI (**oc**).
- You have logged in to the OpenShift CLI (**oc**) as a user with **cluster-admin** permissions.

Procedure

- Verify that the **LVMCluster** CR is present by running the following command:

```
$ oc get lvmcluster -n <namespace>
```

Example output

NAME	AGE
my-lvmcluster	65m

- If the **LVMCluster** CR is not present, create an **LVMCluster** CR. For more information, see "Ways to create an LVMCluster custom resource".

3. In the namespace where the operator is installed, check that all the LVM Storage pods are in the **Running** state by running the following command:

```
$ oc get pods -n <namespace>
```

Example output

NAME	READY	STATUS	RESTARTS	AGE
lvms-operator-7b9fb858cb-6nsm1	3/3	Running	0	70m
topolvm-controller-5dd9cf78b5-7wwr2	5/5	Running	0	66m
topolvm-node-dr26h	4/4	Running	0	66m
vg-manager-r6zdv	1/1	Running	0	66m

The output of this command must contain a running instance of the following pods:

- **lvms-operator**
- **vg-manager**

If the **vg-manager** pod is stuck while loading a configuration file, it is due to a failure to locate an available disk for LVM Storage to use. To retrieve the necessary information to troubleshoot this issue, review the logs of the **vg-manager** pod by running the following command:

```
$ oc logs -l app.kubernetes.io/component=vg-manager -n <namespace>
```

Additional resources

- [About the **LVMCluster** custom resource](#)
- [Ways to create an **LVMCluster** custom resource](#)

5.4.19.3. Recovering from node failure

Identify failed nodes causing persistent volume claims (PVCs) to remain in pending state by examining the restart count of the **topolvm-node** pod, which indicates potential underlying node problems requiring investigation.

Prerequisites

- You have installed the OpenShift CLI (**oc**).
- You have logged in to the OpenShift CLI (**oc**) as a user with **cluster-admin** permissions.

Procedure

- Examine the restart count of the **topolvm-node** pod instances by running the following command:

```
$ oc get pods -n <namespace>
```

Example output

NAME	READY	STATUS	RESTARTS	AGE
lvms-operator-7b9fb858cb-6nsm1	3/3	Running	0	70m
topolvm-controller-5dd9cf78b5-7wwr2	5/5	Running	0	66m
topolvm-node-dr26h	4/4	Running	0	66m
topolvm-node-54as8	4/4	Running	0	66m
topolvm-node-78fft	4/4	Running	17 (8s ago)	66m
vg-manager-r6zdv	1/1	Running	0	66m
vg-manager-990ut	1/1	Running	0	66m
vg-manager-an118	1/1	Running	0	66m

Next steps

If the PVC is stuck in the **Pending** state even after you have resolved any issues with the node, you must perform a forced clean-up. For more information, see "Performing a forced clean-up".

Additional resources

- [Performing a forced clean-up](#)

5.4.19.4. Recovering from disk failure

Diagnose and resolve disk and volume provisioning failures by inspecting persistent volume claim (PVC) events to identify specific error messages, then connecting to the affected host to fix the underlying disk issue.

Disk and volume provisioning issues result with a generic error message such as **Failed to provision volume with storage class <storage_class_name>**. The generic error message is followed by a specific volume failure error message.

The following table describes the volume failure error messages:

Table 5.16. Volume failure error messages

Error message	Description
Failed to check volume existence	Indicates a problem in verifying whether the volume already exists. Volume verification failure can be caused by network connectivity problems or other failures.
Failed to bind volume	Failure to bind a volume can happen if the persistent volume (PV) that is available does not match the requirements of the PVC.
FailedMount or FailedAttachVolume	This error indicates problems when trying to mount the volume to a node. If the disk has failed, this error can appear when a pod tries to use the PVC.
FailedUnMount	This error indicates problems when trying to unmount a volume from a node. If the disk has failed, this error can appear when a pod tries to use the PVC.

Error message	Description
Volume is already exclusively attached to one node and cannot be attached to another	This error can appear with storage solutions that do not support ReadWriteMany access modes.

Prerequisites

- You have installed the OpenShift CLI (**oc**).
- You have logged in to the OpenShift CLI (**oc**) as a user with **cluster-admin** permissions.

Procedure

1. Inspect the events associated with a PVC by running the following command:

```
$ oc describe pvc <pvc_name>
```

Replace **<pvc_name>** with the name of the PVC.

2. Establish a direct connection to the host where the problem is occurring.
3. Resolve the disk issue.

Next steps

If the volume failure messages persist or recur even after you have resolved the issue with the disk, you must perform a forced clean-up. For more information, see "Performing a forced clean-up".

Additional resources

- [Performing a forced clean-up](#)

5.4.19.5. Performing a forced clean-up

Perform a forced clean-up by removing all Logical Volume Manager (LVM) Storage custom resources (CRs) when disk or node-related problems continue after standard troubleshooting, to restore proper storage functioning.

If the disk or node-related problems persist even after you have completed the troubleshooting procedures, you must perform a forced clean-up. A forced clean-up is used to address persistent issues and ensure the proper functioning of LVM Storage.

Prerequisites

- You have installed the OpenShift CLI (**oc**).
- You have logged in to the OpenShift CLI (**oc**) as a user with **cluster-admin** permissions.
- You have deleted all the persistent volume claims (PVCs) that were created by using LVM Storage.
- You have stopped the pods that are using the PVCs that were created by using LVM Storage.

Procedure

Procedure

1. Switch to the namespace where you have installed the LVM Storage Operator by running the following command:

```
$ oc project <namespace>
```

2. Check if the **LogicalVolume** custom resources are present by running the following command:

```
$ oc get logicalvolume
```

- a. If the **LogicalVolume** CRs are present, delete them by running the following command:

```
$ oc delete logicalvolume <name>
```

Replace **<name>** with the name of the **LogicalVolume** CR.

- b. After deleting the **LogicalVolume** CRs, remove their finalizers by running the following command:

```
$ oc patch logicalvolume <name> -p '{"metadata":{"finalizers":[]}}' --type=merge
```

Replace **<name>** with the name of the **LogicalVolume** CR.

3. Check if the **LVMVolumeGroup** CRs are present by running the following command:

```
$ oc get lvmvolumegroup
```

- a. If the **LVMVolumeGroup** CRs are present, delete them by running the following command:

```
$ oc delete lvmvolumegroup <name>
```

Replace **<name>** with the name of the **LVMVolumeGroup** CR.

- b. After deleting the **LVMVolumeGroup** CRs, remove their finalizers by running the following command:

```
$ oc patch lvmvolumegroup <name> -p '{"metadata":{"finalizers":[]}}' --type=merge
```

Replace **<name>** with the name of the **LVMVolumeGroup** CR.

4. Delete any **LVMVolumeGroupNodeStatus** CRs by running the following command:

```
$ oc delete lvmvolumegroupnodestatus --all
```

5. Delete the **LVMCluster** CR by running the following command:

```
$ oc delete lvmcluster --all
```

- a. After deleting the **LVMCluster** CR, remove its finalizer by running the following command:

```
$ oc patch lvmcluster <name> -p '{"metadata":{"finalizers":[]}}' --type=merge
```

Replace **<name>** with the name of the **LVMCluster** CR.

CHAPTER 6. USING CONTAINER STORAGE INTERFACE (CSI)

6.1. CONFIGURING CSI VOLUMES

Container Storage Interface (CSI) is a standard specification enabling storage vendors to develop plugins that work across container orchestration systems. OpenShift Container Platform uses CSI drivers to provision and manage persistent storage, replacing in-tree storage plugins.

6.1.1. CSI architecture

Container Storage Interface (CSI) architecture uses containerized drivers and bridge components for communication between OpenShift Container Platform and storage backends. Each driver requires controller deployments and daemon sets for volume operations. Multiple drivers can run simultaneously.

The Container Storage Interface (CSI) allows OpenShift Container Platform to consume storage from storage back ends that implement the CSI interface as persistent storage.



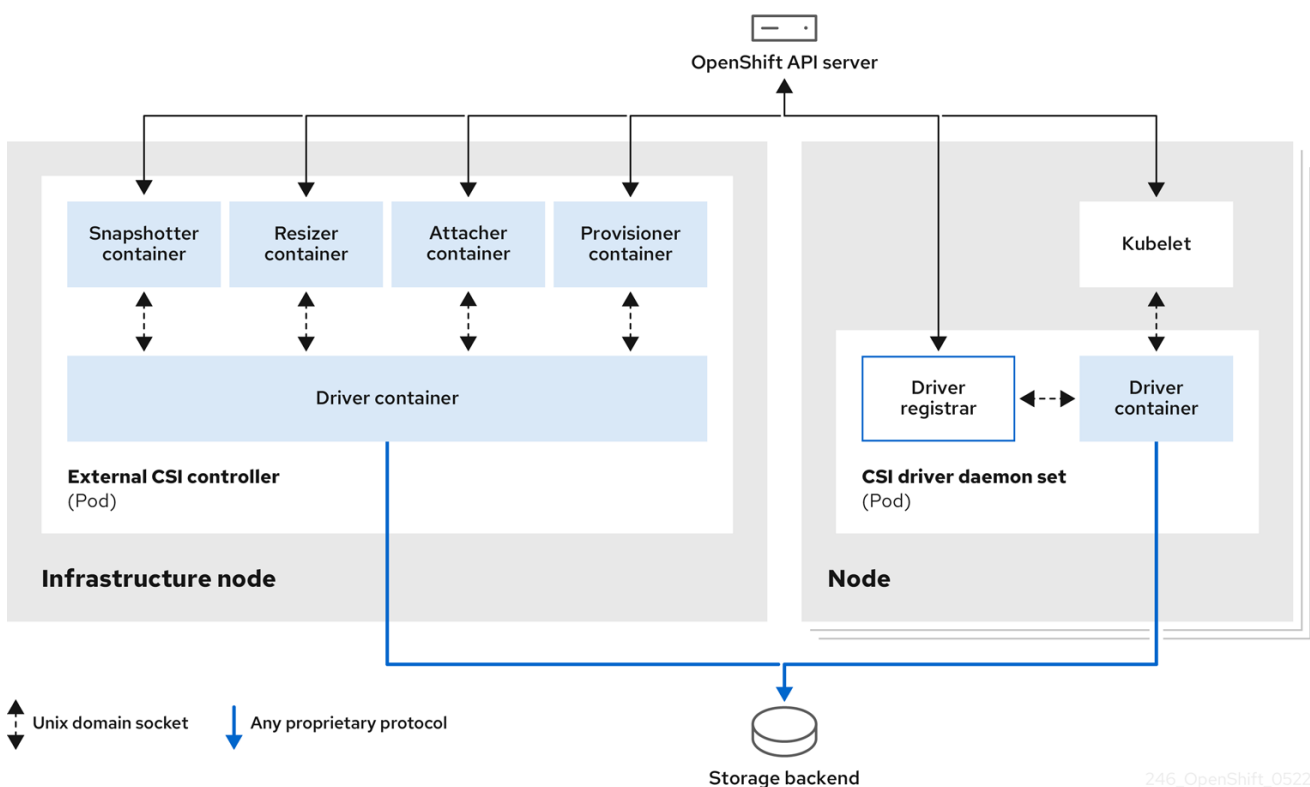
NOTE

OpenShift Container Platform 4.22 supports version 1.6.0 of the CSI specification.

For more information about the CSI spec, see "CSI spec".

CSI drivers are typically shipped as container images. These containers are not aware of OpenShift Container Platform where they run. To use CSI-compatible storage back end in OpenShift Container Platform, the cluster administrator must deploy several components that serve as a bridge between OpenShift Container Platform and the storage driver.

The following diagram provides a high-level overview about the components running in pods in the OpenShift Container Platform cluster.



It is possible to run multiple CSI drivers for different storage back ends. Each driver needs its own external controllers deployment and daemon set with the driver and CSI registrar.

Additional resources

- [CSI spec](#)

6.1.1.1. External CSI controllers

External Container Storage Interface (CSI) controllers run as deployments with containers handling volume provisioning, deletion, attachment, snapshotting, and resizing. Controller pods communicate with CSI drivers using UNIX Domain Sockets and run on infrastructure nodes to protect credentials.

External CSI controllers is a deployment that deploys one or more pods with five containers:

- The snapshotter container watches **VolumeSnapshot** and **VolumeSnapshotContent** objects and is responsible for the creation and deletion of **VolumeSnapshotContent** object.
- The resizer container is a sidecar container that watches for **PersistentVolumeClaim** updates and triggers **ControllerExpandVolume** operations against a CSI endpoint if you request more storage on **PersistentVolumeClaim** object.
- An external CSI attacher container translates **attach** and **detach** calls from OpenShift Container Platform to respective **ControllerPublish** and **ControllerUnpublish** calls to the CSI driver.
- An external CSI provisioner container that translates **provision** and **delete** calls from OpenShift Container Platform to respective **CreateVolume** and **DeleteVolume** calls to the CSI driver.
- A CSI driver container.

The CSI attacher and CSI provisioner containers communicate with the CSI driver container using UNIX Domain Sockets, ensuring that no CSI communication leaves the pod. The CSI driver is not accessible from outside of the pod.



NOTE

The **attach**, **detach**, **provision**, and **delete** operations typically require the CSI driver to use credentials to the storage backend. Run the CSI controller pods on infrastructure nodes so the credentials are never leaked to user processes, even in case of a catastrophic security breach on a compute node.



NOTE

The external attacher must also run for CSI drivers that do not support third-party **attach** or **detach** operations. The external attacher does not issue any **ControllerPublish** or **ControllerUnpublish** operations to the CSI driver. However, it still must run to implement the necessary OpenShift Container Platform attachment API.

6.1.1.2. CSI driver daemon set

CSI driver daemon sets run on every node to enable volume mounting and operations. Each pod contains a driver and registrar communicating with node services using UNIX Domain Sockets. The node driver uses minimal credentials and implements node-specific CSI operations like publish and stage.

The CSI driver daemon set runs a pod on every node that allows OpenShift Container Platform to mount storage provided by the CSI driver to the node and use it in user workloads (pods) as persistent volumes (PVs). The pod with the CSI driver installed contains the following containers:

CSI driver registrar

The CSI driver registrar registers the CSI driver into the **openshift-node** service running on the node. The **openshift-node** process running on the node then directly connects with the CSI driver using the UNIX Domain Socket available on the node.

CSI driver

The CSI driver deployed on the node should have as few credentials to the storage back end as possible. OpenShift Container Platform will only use the node plugin set of CSI calls such as **NodePublish/NodeUnpublish** and **NodeStage/NodeUnstage**, if these calls are implemented.

6.1.2. CSI drivers supported by OpenShift Container Platform

OpenShift Container Platform installs several CSI drivers by default, automatically deploying the driver Operator, driver, and storage class for supported backends. Default drivers provide enhanced features beyond in-tree plugins. Some drivers, such as AWS EFS and GCP Filestore, require manual installation.

To create CSI-provisioned persistent volumes that mount to these supported storage assets, OpenShift Container Platform installs the necessary CSI driver Operator, the CSI driver, and the required storage class by default. For more details about the default namespace of the Operator and driver, see the documentation for the specific CSI Driver Operator.



IMPORTANT

The AWS EFS CSI driver is not installed by default, and must be installed manually. For instructions about installing the AWS EFS CSI driver, see "Setting up the AWS Elastic File Service CSI Driver Operator".

The following table describes the CSI drivers that are installed with OpenShift Container Platform, supported by OpenShift Container Platform, and which CSI features they support, such as volume snapshots and resize.



IMPORTANT

If your CSI driver is not listed in the following table, you must follow the installation instructions provided by your CSI storage vendor to use their supported CSI features.

For a list of third-party-certified CSI drivers, see the "Red Hat ecosystem portal".

Table 6.1. Supported CSI drivers and features in OpenShift Container Platform

CSI driver	CSI volume snapshots	CSI volume group snapshots [1]	CSI cloning	CSI resize	Inline ephemeral volumes	User namespaces
AWS EBS	■			■		■
AWS EFS						

CSI driver	CSI volume snapshots	CSI volume group snapshots [1]	CSI cloning	CSI resize	Inline ephemeral volumes	User namespaces
Google Compute Platform (GCP) persistent disk (PD)	■		■ ^[2]	■		■
GCP Filestore	■			■		
IBM Power® Virtual Server Block				■		■
IBM Cloud® Block	■ ^[3]			■ ^[3]		■
LVM Storage	■		■	■		■
Microsoft Azure Disk	■		■	■		■
Microsoft Azure Stack Hub	■		■	■		■
Microsoft Azure File	■		■	■	■	
OpenStack Cinder	■		■	■		■
OpenShift Data Foundation	■	■	■	■		■ ^[4]
OpenStack Manila	■			■		
CIFS/SMB			■			
VMware vSphere	■ ^[5]			■ ^[6]		■ ^[7]

1.

**IMPORTANT**

CSI volume group snapshots is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

2.

- Cloning is not supported on hyperdisk-balanced disks with storage pools.

3.

- Does not support offline snapshots or resize. Volume must be attached to a running pod.

4.

- RBD supports user namespaces; CephFS does not.

5.

- Requires VMware vSphere version 8.0 Update 1 or later, or VMware vSphere Foundation (VVF) 9, or VMware Cloud Foundation (VCF) 9, for both vCenter Server and ESXi.
- Does not support fileshare volumes.

6.

- Online expansion is supported from VMware vSphere version 8.0 Update 1 and later, or VVF 9, or VCF 9.

7.

- File persistent volumes (PVs), such as vSAN file service, do not support user namespaces.

Additional resources

- [Setting up the AWS EFS CSI Driver Operator](#)
- [Red Hat ecosystem portal](#)
- [Third-party support policy](#)

6.1.3. Dynamic provisioning

Dynamic provisioning creates persistent volumes on-demand from storage class configurations. Container Storage Interface (CSI) drivers support specific parameters determining behavior. Create a default storage class to enable provisioning for claims without a specified class.

Dynamic provisioning of persistent storage depends on the capabilities of the CSI driver and underlying storage back end. The provider of the CSI driver should document how to create a storage class in OpenShift Container Platform and the parameters available for configuration.

The created storage class can be configured to enable dynamic provisioning.

Procedure

- Create a default storage class that ensures all PVCs that do not require any special storage class are provisioned by the installed CSI driver.

```
# oc create -f - << EOF
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: <storage-class>
  annotations:
    storageclass.kubernetes.io/is-default-class: "true"
provisioner: <provisioner-name>
parameters:
  csi.storage.k8s.io/fstype: xfs
EOF
```

- **metadata.name:** Specifies the name of the storage class that will be created.
- **provisioner:** Specifies the name of the CSI driver that has been installed.
- **parameters.csi.storage.k8s.io/fstype:** The vSphere CSI driver supports all of the file systems supported by the underlying Red Hat Core operating system release, including XFS and Ext4.

6.1.4. Example using the CSI driver

Deploy a MySQL application using Container Storage Interface (CSI) persistent storage to demonstrate dynamic volume provisioning. This example shows CSI drivers automatically creating and binding persistent volume claims to dynamically provisioned volumes without manual intervention.

Prerequisites

- The CSI driver has been deployed.
- A storage class has been created for dynamic provisioning.

Procedure

- Create the MySQL template:

```
# oc new-app mysql-persistent
```

Example output

```
--> Deploying template "openshift/mysql-persistent" to project default
...

# oc get pvc
```

■

Example output

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES
STORAGECLASS	AGE			
mysql	Bound	kubernetes-dynamic-pv-3271fcb4e1811e8	1Gi	RWO
gp3-csi	3s			

6.2. CSI INLINE EPHEMERAL VOLUMES

You can provision temporary, pod-specific storage by using Container Storage Interface (CSI) inline ephemeral volumes that are automatically created at pod deployment and removed at pod termination.

6.2.1. Overview of CSI inline ephemeral volumes

Traditionally, volumes that are backed by Container Storage Interface (CSI) drivers can only be used with a **PersistentVolume** and **PersistentVolumeClaim** object combination.

CSI inline ephemeral volumes allow you to specify CSI volumes directly in the **Pod** specification, rather than in a **PersistentVolume** object. Inline volumes are ephemeral and do not persist across pod restarts.

CSI inline ephemeral volumes are only available with the following supported CSI drivers:

- Azure File CSI driver
- Secrets Store CSI driver

6.2.1.1. Support limitations for CSI inline ephemeral volumes



IMPORTANT

The Shared Resource CSI Driver feature is now generally available in Builds for Red Hat OpenShift 1.1. This feature is now removed in OpenShift Container Platform 4.18 and later. To use this feature, ensure that you are using Builds for Red Hat OpenShift 1.1 or later. For information about Builds for Red Hat OpenShift 1.1, see "Builds for Red Hat OpenShift 1.1".

By default, OpenShift Container Platform supports CSI inline ephemeral volumes with these limitations:

- Support is only available for CSI drivers. In-tree and FlexVolumes are not supported.
- Community or storage vendors provide other CSI drivers that support these volumes. Follow the installation instructions provided by the CSI driver provider.

CSI drivers might not have implemented the inline volume functionality, including **Ephemeral** capacity. For details, see the CSI driver documentation.

Additional resources

- [Builds for Red Hat OpenShift 1.1](#)

6.2.2. CSI Volume Admission plugin

To restrict Container Storage Interface (CSI) ephemeral volume usage based on pod security standards, the CSI Volume Admission plugin enforces admission policies by inspecting security profile labels on CSI drivers.

6.2.2.1. Overview of CSI Volume Admission plugin

The CSI Volume Admission plugin allows you to restrict the use of an individual CSI driver capable of provisioning CSI ephemeral volumes on pod admission. Administrators can add a **csi-ephemeral-volume-profile** label, and this label is then inspected by the Admission plugin and used in enforcement, warning, and audit decisions.

To use the CSI Volume Admission plugin, administrators add the **security.openshift.io/csi-ephemeral-volume-profile** label to a **CSIDriver** object, which declares the CSI driver's effective pod security profile when it is used to provide CSI ephemeral volumes, as shown in the following example:

Example CSIDriver YAML file enabling using of the CSI Admission plugin

```
kind: CSIDriver
metadata:
  name: csi.mydriver.company.org
  labels:
    security.openshift.io/csi-ephemeral-volume-profile: restricted
```

Setting **metadata.labels.security.openshift.io/csi-ephemeral-volume-profile** to **restricted** enables use of the CSI Admission plugin.

This “effective profile” communicates that a pod can use the CSI driver to mount CSI ephemeral volumes when the pod's namespace is governed by a pod security standard.

The CSI Volume Admission plugin inspects pod volumes when pods are created; existing pods that use CSI volumes are not affected. If a pod uses a container storage interface (CSI) volume, the plugin looks up the **CSIDriver** object and inspects the **csi-ephemeral-volume-profile** label, and then use the label's value in its enforcement, warning, and audit decisions.

6.2.2.2. Pod security profile enforcement

When a CSI driver has the **csi-ephemeral-volume-profile** label, pods using the CSI driver to mount CSI ephemeral volumes must run in a namespace that enforces a pod security standard of equal or greater permission. If the namespace enforces a more restrictive standard, the CSI Volume Admission plugin denies admission. The following table describes the enforcement behavior for different pod security profiles for given label values.

Table 6.2. Pod security profile enforcement

Pod security profile	Driver label: restricted	Driver label: baseline	Driver label: privileged
Restricted	Allowed	Denied	Denied
Baseline	Allowed	Allowed	Denied
Privileged	Allowed	Allowed	Allowed

6.2.2.3. Pod security profile warning

The CSI Volume Admission plugin can warn you if the CSI driver's effective profile is more permissive than the pod security warning profile for the pod namespace. The following table shows when a warning occurs for different pod security profiles for given label values.

Table 6.3. Pod security profile warning

Pod security profile	Driver label: restricted	Driver label: baseline	Driver label: privileged
Restricted	No warning	Warning	Warning
Baseline	No warning	No warning	Warning
Privileged	No warning	No warning	No warning

6.2.2.4. Pod security profile audit

The CSI Volume Admission plugin can apply audit annotations to the pod if the CSI driver's effective profile is more permissive than the pod security audit profile for the pod namespace. The following table shows the audit annotation applied for different pod security profiles for given label values.

Table 6.4. Pod security profile audit

Pod security profile	Driver label: restricted	Driver label: baseline	Driver label: privileged
Restricted	No audit	Audit	Audit
Baseline	No audit	No audit	Audit
Privileged	No audit	No audit	No audit

6.2.2.5. Default behavior for the CSI Volume Admission plugin

If the referenced CSI driver for a CSI ephemeral volume does not have the **csi-ephemeral-volume-profile** label, the CSI Volume Admission plugin considers the driver to have the privileged profile for enforcement, warning, and audit behaviors. Likewise, if the pod's namespace does not have the pod security admission label set, the Admission plugin assumes the restricted profile is allowed for enforcement, warning, and audit decisions. Therefore, if no labels are set, CSI ephemeral volumes using that CSI driver are only usable in privileged namespaces by default.

The CSI drivers that ship with OpenShift Container Platform and support ephemeral volumes have a reasonable default set for the **csi-ephemeral-volume-profile** label:

- Azure File CSI driver: privileged

If desired, an admin can change the default value of the label.

6.2.3. Embedding a CSI inline ephemeral volume in the pod specification

To provision temporary storage that automatically follows your pod's lifecycle, embed a Container Storage Interface (CSI) inline ephemeral volume in the pod specification so the CSI driver manages volume creation and cleanup as pods start/stop.

Procedure

1. Create the **Pod** object definition and save it to a file.
2. Embed the CSI inline ephemeral volume in the file as in the following pod YAML file:

Example pod YAML file with embedded ephemeral volume

```
kind: Pod
apiVersion: v1
metadata:
  name: my-csi-app
spec:
  containers:
    - name: my-frontend
      image: busybox
      volumeMounts:
        - mountPath: "/data"
          name: my-csi-inline-vol
      command: [ "sleep", "1000000" ]
  volumes:
    - name: my-csi-inline-vol
      csi:
        driver: inline.storage.kubernetes.io
        volumeAttributes:
          foo: bar
```

Where ``spec.volumes.name`` is the name of the volume that is used by pods.

3. Create the object definition file that you saved in the previous step by running the following command.

```
$ oc create -f my-csi-app.yaml
```

6.2.4. Additional resources

- [Pod Security Standards](#)

6.3. CSI VOLUME SNAPSHOTS

Container Storage Interface (CSI) snapshots capture point-in-time copies for data protection and recovery. Snapshots enable restoring volumes to previous states or creating new volumes from existing data using **VolumeSnapshot**, **VolumeSnapshotContent**, and **VolumeSnapshotClass** objects.

6.3.1. Overview of CSI volume snapshots

A snapshot represents the state of the storage volume in a cluster at a particular point in time. Volume snapshots can be used to provision a new volume.

OpenShift Container Platform supports Container Storage Interface (CSI) volume snapshots by default. However, a specific CSI driver is required.

Familiarity with persistent volumes is suggested. For information about persistent volumes, see "Persistent volumes".

With CSI volume snapshots, a cluster administrator can:

- Deploy a third-party CSI driver that supports snapshots.
- Create a new persistent volume claim (PVC) from an existing volume snapshot.
- Take a snapshot of an existing PVC.
- Restore a snapshot as a different PVC.
- Delete an existing volume snapshot.

With CSI volume snapshots, an app developer can:

- Use volume snapshots as building blocks for developing application- or cluster-level storage backup solutions.
- Rapidly rollback to a previous development version.
- Use storage more efficiently by not having to make a full copy each time.

Be aware of the following when using volume snapshots:

- Support is only available for CSI drivers. In-tree and FlexVolumes are not supported.
- OpenShift Container Platform only includes select CSI drivers. For CSI drivers that are not provided by an OpenShift Container Platform Driver Operator, it is recommended to use the CSI drivers provided by the Kubernetes community or known storage vendors. Follow the installation instructions furnished by the CSI driver provider. For information about Kubernetes CSI drivers, see "Kubernetes CSI Developer Documentation".
- CSI drivers may or may not have implemented the volume snapshot functionality. CSI drivers that have provided support for volume snapshots will likely use the **csi-external-snapshotter** sidecar. See documentation provided by the CSI driver for details.

6.3.2. CSI snapshot controller and sidecar

Container Storage Interface (CSI) snapshots require two components: a controller deployed by OpenShift Container Platform to the control plane, and a vendor-provided sidecar with the CSI driver. The controller manages **VolumeSnapshot** bindings while the sidecar triggers create and delete operations.

The CSI snapshot controller and sidecar provide volume snapshotting through the OpenShift Container Platform API. These external components run in the cluster.

The external controller is deployed by the CSI Snapshot Controller Operator.

External controller

The CSI snapshot controller binds **VolumeSnapshot** and **VolumeSnapshotContent** objects. The controller manages dynamic provisioning by creating and deleting **VolumeSnapshotContent** objects.

External sidecar

Your CSI driver vendor provides the **csi-external-snapshotter** sidecar. This is a separate helper container that is deployed with the CSI driver. The sidecar manages snapshots by triggering **CreateSnapshot** and **DeleteSnapshot** operations. Follow the installation instructions provided by your vendor.

6.3.3. About the CSI Snapshot Controller Operator

You can manage volume snapshots in OpenShift Container Platform by using the CSI Snapshot Controller Operator's custom resource definitions (CRDs) for snapshot requests, storage, and configuration.

The Container Storage Interface (CSI) Snapshot Controller Operator runs in the **openshift-cluster-storage-operator** namespace. It is installed by the Cluster Version Operator (CVO) in all clusters by default.

The CSI Snapshot Controller Operator installs the CSI snapshot controller, which runs in the **openshift-cluster-storage-operator** namespace.

6.3.3.1. Volume snapshot CRDs

During OpenShift Container Platform installation, the CSI Snapshot Controller Operator creates the following snapshot custom resource definitions (CRDs) in the **snapshot.storage.k8s.io/v1** API group:

VolumeSnapshotContent

A snapshot taken of a volume in the cluster that has been provisioned by a cluster administrator. Similar to the **PersistentVolume** object, the **VolumeSnapshotContent** CRD is a cluster resource that points to a real snapshot in the storage back end.

For manually pre-provisioned snapshots, a cluster administrator creates a number of **VolumeSnapshotContent** CRDs. These carry the details of the real volume snapshot in the storage system.

The **VolumeSnapshotContent** CRD is not namespaced and is for use by a cluster administrator.

VolumeSnapshot

Similar to the **PersistentVolumeClaim** object, the **VolumeSnapshot** CRD defines a developer request for a snapshot. The CSI Snapshot Controller Operator runs the CSI snapshot controller, which handles the binding of a **VolumeSnapshot** CRD with an appropriate **VolumeSnapshotContent** CRD. The binding is a one-to-one mapping.

The **VolumeSnapshot** CRD is namespaced. A developer uses the CRD as a distinct request for a snapshot.

VolumeSnapshotClass

The **VolumeSnapshotClass** CRD allows a cluster administrator to specify different attributes belonging to a **VolumeSnapshot** object. These attributes may differ among snapshots taken of the same volume on the storage system, in which case they would not be expressed by using the same storage class of a persistent volume claim.

For more information about the CSI Snapshot Controller Operator, see [CSI Snapshot Controller Operator](#).

The **VolumeSnapshotClass** CRD defines the parameters for the **csi-external-snapshotter** sidecar to use when creating a snapshot. This allows the storage back end to know what kind of snapshot to dynamically create if multiple options are supported.

Dynamically provisioned snapshots use the **VolumeSnapshotClass** CRD to specify storage-provider-specific parameters to use when creating a snapshot.

The **VolumeSnapshotContentClass** CRD is not namespaced and is for use by a cluster administrator to enable global configuration options for their storage back end.

For Google Cloud Platform (GCP) persistent disk (PD) storage CSI, there is a non-default **VolumeSnapshotClass**, named **csi-gce-pd-vsc-images**, that uses the **snapshot-type: images** parameter. When using KubeVirt, this allows you to create VMs from "golden images" (templates saved as snapshots).

If you want to use the images volume snapshot class for dynamic snapshot provisioning, you can either:

- Make the images volume snapshot class the default by changing the **snapshot.storage.kubernetes.io/is-default-class** annotation to **true**. Also, for the normal default volume snapshot class, **csi-gce-pd-vsc**, be sure to change this parameter to **false**.
- When creating the snapshot object, be sure to set **volumeSnapshotClassName** to **csi-gce-pd-vsc-images**.
For information about creating volume snapshots, see "Dynamically creating a volume snapshot" and "Statically creating a volume snapshot".

Example images volume snapshot class YAML file

```
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshotClass
metadata:
  name: csi-gce-pd-vsc-images
driver: pd.csi.storage.gke.io
parameters:
  snapshot-type: images
```

- **metadata.name:csi-gce-pd-vsc-images**: Specifies the name for the non-default images volume snapshot class.
- **parameters: snapshot-type: images**: Defines the snapshot as a "golden image" or a bootable template, rather than the standard disk backup.

6.3.4. Volume snapshot provisioning

You can provision volume snapshots in OpenShift Container Platform using either dynamic provisioning to create new snapshots on-demand or static provisioning to reference pre-existing snapshots.

6.3.4.1. Dynamic provisioning

Instead of using a preexisting snapshot, you can request that a snapshot be taken dynamically from a persistent volume claim. Parameters are specified using a **VolumeSnapshotClass** CRD.

6.3.4.2. Static provisioning

As a cluster administrator, you can manually pre-provision a number of **VolumeSnapshotContent** objects. These carry the real volume snapshot details available to cluster users.

6.3.5. Dynamically creating a volume snapshot

To create a point-in-time backup of a persistent volume claim (PVC), dynamically provision a volume snapshot by defining a **VolumeSnapshotClass** and **VolumeSnapshot** that automate the snapshot creation.

When you create a **VolumeSnapshot** object, OpenShift Container Platform creates a volume snapshot.

Prerequisites

- Logged in to a running OpenShift Container Platform cluster.
- A PVC created using a CSI driver that supports **VolumeSnapshot** objects.
- A storage class to provision the storage back end.
- No pods are using the persistent volume claim (PVC) that you want to take a snapshot of.



WARNING

Creating a volume snapshot of a PVC that is in use by a pod can cause unwritten data and cached data to be excluded from the snapshot. To ensure that all data is written to the disk, delete the pod that is using the PVC before creating the snapshot.

Procedure

1. Create a file with the **VolumeSnapshotClass** object described by the following YAML:

Example volumesnapshotclass.yaml

```
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshotClass
metadata:
  name: csi-hostpath-snap
driver: hostpath.csi.k8s.io
deletionPolicy: Delete
```

driver is the name of the CSI driver that is used to create snapshots of this **VolumeSnapshotClass** object. The name must be the same as the **Provisioner** field of the storage class that is responsible for the PVC that is being snapshot.

**NOTE**

Depending on the driver that you used to configure persistent storage, additional parameters might be required. You can also use an existing **VolumeSnapshotClass** object.

2. Create the object you saved in the previous step by entering the following command:

```
$ oc create -f volumesnapshotclass.yaml
```

3. Create a **VolumeSnapshot** object:

Example volumesnapshot-dynamic.yaml

```
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshot
metadata:
  name: mysnap
spec:
  volumeSnapshotClassName: csi-hostpath-snap
  source:
    persistentVolumeClaimName: myclaim
```

- **spec.volumeSnapshotClassName:** Specifies the request for a particular class by the volume snapshot. If the **volumeSnapshotClassName** setting is absent and there is a default volume snapshot class, a snapshot is created with the default volume snapshot class name. But if the field is absent and no default volume snapshot class exists, then no snapshot is created.
- **spec.source.persistentVolumeClaimName:** Specifies the name of the **PersistentVolumeClaim** object bound to a persistent volume. This defines what you want to create a snapshot of. Required for dynamically provisioning a snapshot.

4. Create the object you saved in the previous step by entering the following command:

```
$ oc create -f volumesnapshot-dynamic.yaml
```

Verification

1. After the snapshot has been created in the cluster, additional details about the snapshot are available.

To display details about the volume snapshot that was created, run the following command:

```
$ oc describe volumesnapshot mysnap
```

The following example displays details about the **mysnap** volume snapshot:

Example volumesnapshot.yaml

```
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshot
metadata:
  name: mysnap
```

```
spec:
  source:
    persistentVolumeClaimName: myclaim
    volumeSnapshotClassName: csi-hostpath-snap
status:
  boundVolumeSnapshotContentName: snapcontent-1af4989e-a365-4286-96f8-
d5dcd65d78d6
  creationTime: "2020-01-29T12:24:30Z"
  readyToUse: true
  restoreSize: 500Mi
```

- **status.boundVolumeSnapshotContentName:** This parameter is the pointer to the actual storage content that was created by the controller.
 - **status.creationTime:** Specifies the time when the snapshot was created. The snapshot contains the volume content that was available at this indicated time.
 - **status.readyToUse:** Specifies the readiness of the snapshot. If the value is set to **true**, the snapshot can be used to restore as a new PVC. If the value is set to **false**, the snapshot was created. However, the storage back end needs to perform additional tasks to make the snapshot usable so that it can be restored as a new volume. For example, Amazon Elastic Block Store data might be moved to a different, less expensive location, which can take several minutes.
2. To verify that the volume snapshot was created, enter the following command:

```
$ oc get volumesnapshotcontent
```

The pointer to the actual content is displayed. If the **boundVolumeSnapshotContentName** field is populated, a **VolumeSnapshotContent** object exists and the snapshot was created.

3. To verify that the snapshot is ready, confirm that the **VolumeSnapshot** object has **readyToUse: true**.

6.3.6. Statically creating a volume snapshot

To make a pre-existing storage snapshot available in OpenShift Container Platform, manually create a volume snapshot that references the existing snapshot content by name.

Prerequisites

- Logged in to a running OpenShift Container Platform cluster.
- A PVC created using a CSI driver that supports **VolumeSnapshot** objects.
- A storage class to provision the storage back end.
- No pods are using the persistent volume claim (PVC) that you want to take a snapshot of.

**WARNING**

Creating a volume snapshot of a PVC that is in use by a pod can cause unwritten data and cached data to be excluded from the snapshot. To ensure that all data is written to the disk, delete the pod that is using the PVC before creating the snapshot.

Procedure

1. Create a file with the **VolumeSnapshotClass** object described by the following YAML:

Example volumesnapshotclass.yaml

```
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshotClass
metadata:
  name: csi-hostpath-snap
driver: hostpath.csi.k8s.io
deletionPolicy: Delete
```

driver is the name of the CSI driver that is used to create snapshots of this **VolumeSnapshotClass** object. The name must be the same as the **Provisioner** field of the storage class that is responsible for the PVC that is being snapshot.

**NOTE**

Depending on the driver that you used to configure persistent storage, additional parameters might be required. You can also use an existing **VolumeSnapshotClass** object.

2. Create the object you saved in the previous step by entering the following command:

```
$ oc create -f volumesnapshotclass.yaml
```

3. Provide a value for the **volumeSnapshotContentName** parameter as the source for the snapshot:

Example volumesnapshot-manual.yaml

```
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshot
metadata:
  name: snapshot-demo
spec:
  source:
    volumeSnapshotContentName: mycontent
```

spec.source.volumeSnapshotContentName is required for pre-provisioned snapshots.

4. Create the object you saved in the previous step by entering the following command:

```
$ oc create -f volumesnapshot-manual.yaml
```

Verification

After the snapshot has been created in the cluster, additional details about the snapshot are available.

1. To display details about the volume snapshot that was created, enter the following command:

```
$ oc describe volumesnapshot mysnap
```

The following example displays details about the **mysnap** volume snapshot:

Example `volumesnapshot.yaml`

```
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshot
metadata:
  name: mysnap
spec:
  source:
    persistentVolumeClaimName: myclaim
    volumeSnapshotClassName: csi-hostpath-snap
status:
  boundVolumeSnapshotContentName: snapcontent-1af4989e-a365-4286-96f8-
d5dcd65d78d6
  creationTime: "2020-01-29T12:24:30Z"
  readyToUse: true
  restoreSize: 500Mi
```

- **status.boundVolumeSnapshotContentName:** Specifies the pointer to the actual storage content that was created by the controller.
 - **status.creationTime:** Specifies the time when the snapshot was created. The snapshot contains the volume content that was available at this indicated time.
 - **status.readyToUse:** If the value is set to **true**, the snapshot can be used to restore as a new PVC. If the value is set to **false**, the snapshot was created. However, the storage back end needs to perform additional tasks to make the snapshot usable so that it can be restored as a new volume. For example, Amazon Elastic Block Store data might be moved to a different, less expensive location, which can take several minutes.
2. To verify that the volume snapshot was created, enter the following command:

```
$ oc get volumesnapshotcontent
```

The pointer to the actual content is displayed. If the **boundVolumeSnapshotContentName** field is populated, a **VolumeSnapshotContent** object exists and the snapshot was created.

3. To verify that the snapshot is ready, confirm that the **VolumeSnapshot** object has **readyToUse: true**.

6.3.7. Deleting a volume snapshot

To clean up unneeded snapshots and free storage resources, delete volume snapshots by setting a deletion policy that controls whether the underlying content is retained or removed.

Procedure

1. Specify the deletion policy that you require in the **VolumeSnapshotClass** object, as shown in the following example:

Example volumesnapshotclass.yaml file

```
apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshotClass
metadata:
  name: csi-hostpath-snap
driver: hostpath.csi.k8s.io
deletionPolicy: Delete
# ...
```

When deleting the volume snapshot, if **deletionPolicy** is set to **Delete**, the underlying snapshot is deleted along with the **VolumeSnapshotContent** object. If the **Retain** value is set, both the underlying snapshot and **VolumeSnapshotContent** object remain.



NOTE

If the **Retain** value is set and the **VolumeSnapshot** object is deleted without deleting the corresponding **VolumeSnapshotContent** object, the content remains. The snapshot itself is also retained in the storage back end.

2. Delete the volume snapshot by entering the following command:

```
$ oc delete volumesnapshot <volumesnapshot_name>
```

Replace **<volumesnapshot_name>** with the name of the volume snapshot you want to delete.

Example output

```
volumesnapshot.snapshot.storage.k8s.io "mysnapshot" deleted
```

3. If the deletion policy is set to **Retain**, delete the volume snapshot content by entering the following command:

```
$ oc delete volumesnapshotcontent <volumesnapshotcontent_name>
```

Replace **<volumesnapshotcontent_name>** with the content you want to delete.

4. Optional: If the **VolumeSnapshot** object is not successfully deleted, enter the following command to remove any finalizers for the leftover resource so that the delete operation can continue:



IMPORTANT

Only remove the finalizers if you are confident that there are no existing references from either persistent volume claims or volume snapshot contents to the **VolumeSnapshot** object. Even with the **--force** option, the delete operation does not delete snapshot objects until all finalizers are removed.

```
$ oc patch -n $PROJECT volumesnapshot/$NAME --type=merge -p '{"metadata": {"finalizers": null}}'
```

Example output

```
volumesnapshotclass.snapshot.storage.k8s.io "csi-ocs-rbd-snapclass" deleted
```

The finalizers are removed and the volume snapshot is deleted.

6.3.8. Restoring a volume snapshot

To recover previous data or reuse snapshot data, create a new persistent volume claim (PVC) that is pre-populated with content from an existing volume snapshot.

The **VolumeSnapshot** CRD content can be used to restore the existing volume to a previous state. After your **VolumeSnapshot** CRD is bound and the **readyToUse** value is set to **true**, you can use that resource to provision a new volume that is pre-populated with data from the snapshot.

Prerequisites

- Logged in to a running OpenShift Container Platform cluster.
- A persistent volume claim (PVC) created using a Container Storage Interface (CSI) driver that supports volume snapshots.
- A storage class to provision the storage back end.
- A volume snapshot has been created and is ready to use.

Procedure

1. Specify a **VolumeSnapshot** data source on a PVC as shown in the following:

pvc-restore.yaml

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: myclaim-restore
spec:
  storageClassName: csi-hostpath-sc
  dataSource:
    name: mysnap
    kind: VolumeSnapshot
    apiGroup: snapshot.storage.k8s.io
  accessModes:
    - ReadWriteOnce
```

```
resources:
  requests:
    storage: 1Gi
```

- **spec.dataSource.name:** Specifies the name of the **VolumeSnapshot** object representing the snapshot to use as source.
- **spec.dataSource.kind:** Must be set to the **VolumeSnapshot** value.
- **spec.dataSource.apiGroup:** Must be set to the **snapshot.storage.k8s.io** value.

2. Create a PVC by entering the following command:

```
$ oc create -f pvc-restore.yaml
```

3. Verify that the restored PVC has been created by entering the following command:

```
$ oc get pvc
```

A new PVC such as **myclaim-restore** is displayed.

6.3.9. Changing the maximum number of snapshots for vSphere

Configure the maximum number of snapshots per volume globally or for specific datastore types to balance storage capacity and performance in your vSphere environment.

The default maximum number of snapshots per volume in vSphere Container Storage Interface (CSI) is 3. You can change the maximum number up to 32 per volume. However, be aware that increasing the snapshot maximum involves a performance trade off, so for better performance use only 2 to 3 snapshots per volume.

For more VMware snapshot performance recommendations, see "Best practices for using VMware snapshots in the vSphere environment".

Prerequisites

- Access to the cluster with administrator rights.

Procedure

1. Check the current secret by the running the following command:

```
$ oc -n openshift-cluster-csi-drivers get secret/vsphere-csi-config-secret -o
jsonpath='{.data.cloud\conf}' | base64 -d
```

Example output

```
# Labels with topology values are added dynamically via operator
[Global]
cluster-id = vsphere-01-cwv8p

# Populate VCenters (multi) after here
[VirtualCenter "vcenter.openshift.com"]
insecure-flag      = true
```



```

datacenters      = DEVQEdatacenter
password         = "xxxxxxx"
user             = "xxxxxxx@devcluster.openshift.com"
migration-datastore-url = ds:///vmfs/volumes/vsan:52c842f232751e0d-3253aadeac21ca82/

```

In this example, the global maximum number of snapshots is not configured, so the default value of 3 is applied.

2. Change the snapshot limit by running the following command:

- Set **global** snapshot limit:

```
$ oc patch clustercsidriver/csi.vsphere.vmware.com --type=merge -p '{"spec":
{"driverConfig":{"vSphere":{"globalMaxSnapshotsPerBlockVolume": 10}}}'
```

```
clustercsidriver.operator.openshift.io/csi.vsphere.vmware.com patched
```

In this example, the global limit is being changed to 10 (**globalMaxSnapshotsPerBlockVolume** set to 10).

- Set **Virtual Volume** snapshot limit:

This parameter sets the limit on the Virtual Volumes datastore only. The Virtual Volume maximum snapshot limit overrides the global constraint if set, but defaults to the global limit if it is not set.

```
$ oc patch clustercsidriver/csi.vsphere.vmware.com --type=merge -p '{"spec":
{"driverConfig":{"vSphere":{"granularMaxSnapshotsPerBlockVolumeInVVOL": 5}}}'
```

```
clustercsidriver.operator.openshift.io/csi.vsphere.vmware.com patched
```

In this example, the Virtual Volume limit is being changed to 5 (**granularMaxSnapshotsPerBlockVolumeInVVOL** set to 5).

- Set **vSAN** snapshot limit:

This parameter sets the limit on the vSAN datastore only. The vSAN maximum snapshot limit overrides the global constraint if set, but defaults to the global limit if it is not set. You can set a maximum value of 32 under vSAN ESA setup.

```
$ oc patch clustercsidriver/csi.vsphere.vmware.com --type=merge -p '{"spec":
{"driverConfig":{"vSphere":{"granularMaxSnapshotsPerBlockVolumeInVSAN": 7}}}'
```

```
clustercsidriver.operator.openshift.io/csi.vsphere.vmware.com patched
```

In this example, the vSAN limit is being changed to 7 (**granularMaxSnapshotsPerBlockVolumeInVSAN** set to 7).

Verification

- Verify that any changes you made are reflected in the config map by running the following command:

```
$ oc -n openshift-cluster-csi-drivers get secret/vsphere-csi-config-secret -o
jsonpath='{.data.cloud\.conf}' | base64 -d
```

Example output

```
# Labels with topology values are added dynamically via operator
[Global]
cluster-id = vsphere-01-cwv8p

# Populate VCenters (multi) after here
[VirtualCenter "vcenter.openshift.com"]
insecure-flag      = true
datacenters        = DEVQEdatacenter
password           = "xxxxxxx"
user               = "xxxxxxx@devcluster.openshift.com"
migration-datastore-url = ds:///vmfs/volumes/vsan:52c842f232751e0d-3253aadeac21ca82/

[Snapshot]
global-max-snapshots-per-block-volume = 10
```

The parameter **global-max-snapshots-per-block-volume** is now set to 10.

6.3.10. Additional resources

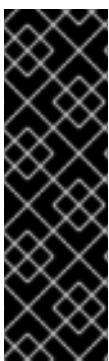
- [Persistent volumes](#)
- [Kubernetes CSI Developer Documentation](#)
- [Dynamically creating a volume snapshot](#)
- [Statically creating a volume snapshot](#)
- [Best practices for using VMware snapshots in the vSphere environment](#)

6.4. CSI VOLUME GROUP SNAPSHOTS

Volume group snapshots capture point-in-time copies of multiple volumes simultaneously, gathering data across related volumes. This enables restoring multi-volume applications to a previous state or provisioning new volume sets with the same data for testing or development purposes.

6.4.1. Overview of CSI volume group snapshots

Volume group snapshots capture point-in-time copies of multiple persistent volume claims using label selectors. Three API objects manage snapshots: `VolumeGroupSnapshot`, `VolumeGroupSnapshotContent`, and `VolumeGroupSnapshotClass`.



IMPORTANT

CSI volume group snapshots is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

To use this Technology Preview feature, you must enable it by using feature gates. For information about using feature gates, see "Enabling features using feature gates".

Snapshot

A snapshot represents the state of the storage volume in a cluster at a particular point in time. Volume snapshots can be used to provision a new volume.

Volume group snapshot

A volume group snapshot uses a label selector to group multiple persistent volume claims for snapshotting. A volume group snapshot represents copies from multiple volumes that are taken at the same point-in-time. This can be useful for applications that contain multiple volumes.

To use volume group snapshots, familiarity with persistent volumes is suggested. For information about persistent volumes, see "Understanding persistent volumes".

Volume group snapshots provide three new API objects for managing snapshots:

VolumeGroupSnapshot

Requests creation of a volume group snapshot for multiple persistent volume claims. It contains information about the volume group snapshot operation, such as the timestamp when the volume group snapshot was taken, and whether it is ready to use.

VolumeGroupSnapshotContent

Created by the snapshot controller for a dynamically created volumeGroupSnapshot. It contains information about the volume group snapshot including the volume group snapshot ID. This object represents a provisioned resource on the cluster (a group snapshot). The **VolumeGroupSnapshotContent** object binds to the volume group snapshot for which it was created with a one-to-one mapping.

VolumeGroupSnapshotClass

Created by cluster administrators to describe how volume group snapshots should be created, including the driver information, the deletion policy, etc.

These three API kinds are defined as **CustomResourceDefinitions** (CRDs). These CRDs must be installed in a OpenShift Container Platform cluster for a CSI driver to support volume group snapshots.

6.4.2. CSI volume group snapshots limitations

Volume group snapshots have operational and consistency limitations that affect how you can use them for data protection and recovery. Understanding these constraints helps you design appropriate backup and recovery strategies for multi-volume applications.

Volume group snapshots have the following limitations:

- Does not support reverting an existing persistent volume claim (PVC) to an earlier state represented by a snapshot. You can only provision a new volume from a snapshot.
- No guarantees of application consistency, for example, crash consistency, are provided beyond those provided by the storage system. For more information about application consistency, see "Quiesce and Unquiesce Hooks".
- Volume group snapshots need to be supported by the Container Storage Interface (CSI) driver. OpenShift Data Foundation supports volume group snapshots.

Additional resources

- [Quiesce and Unquiesce Hooks](#)

6.4.3. Creating a volume group snapshot class

Create a **VolumeGroupSnapshotClass** to define how volume group snapshots are provisioned, including the Container Storage Interface (CSI) driver and deletion policy. Cluster administrators must create this class before users can provision volume group snapshots.

Prerequisites

- Logged in to a running OpenShift Container Platform cluster with administrator privileges.
- Enabled this feature using feature gates. For information about how to use feature gates, see "Enabling features sets by using feature gates".

Procedure

1. Create a **VolumeGroupSnapshotClass** YAML file using the following example file:

Example volume group snapshot class YAML file

```
apiVersion: groupsnapshot.storage.k8s.io/v1beta2
kind: VolumeGroupSnapshotClass
metadata:
  name: csi-hostpath-groupsnapclass
deletionPolicy: Delete
driver: hostpath.csi.k8s.io
.....
```

- **kind**: Specifies the **VolumeGroupSnapshotClass** object.
 - **metadata.name**: Name of the **VolumeGroupSnapshotClass**.
2. Create the 'VolumeGroupSnapshotClass' object by running the following command:

```
$ oc create -f <volume-group-snapshot-class-filename>.yaml
```

6.4.4. Creating a volume group snapshot

Create a **VolumeGroupSnapshot** object to capture point-in-time copies of multiple persistent volume claims (PVCs). Label the PVCs, specify the **VolumeGroupSnapshotClass**, and individual **VolumeSnapshot** objects are created automatically.

When you create a **VolumeGroupSnapshot** object, OpenShift Container Platform creates a volume group snapshot.

Prerequisites

- Logged in to a running OpenShift Container Platform cluster.
- Enabled this feature using feature gates. For information about how to use feature gates, see "Enabling features sets by using feature gates".
- The persistent volume claims (PVCs) that you want to group for the snapshot have been created using a CSI driver that supports **VolumeGroupSnapshot** objects.

- A storage class to provision the storage back end.
- Administrator has created the **VolumeGroupSnapshotClass** object.

Procedure

1. Locate (or create) the PVCs that you want to include in the volume group snapshot by running the following command:

```
$ oc get pvc
```

Example

NAME	STATUS	VOLUME	CAPACITY	ACCESSMODES	AGE
pvc-0	Bound	pvc-a42d7ea2-e3df-11ed-b5ea-0242ac120002	1Gi	RWO	48s
pvc-1	Bound	pvc-a42d81b8-e3df-11ed-b5ea-0242ac120002	1Gi	RWO	48S

This example uses two PVCs

2. Label the PVCs to belong to a snapshot group:
 - a. Label PVC pvc-0 by running the following command:

```
$ oc label pvc pvc-0 group=myGroup
```

Example output

```
persistentvolumeclaim/pvc-0 labeled
```

- b. Label PVC pvc-1 by running the following command:

Example output

```
$ oc label pvc pvc-1 group=myGroup
```

Example output

```
persistentvolumeclaim/pvc-1 labeled
```

In this example, you are labeling PVC "pvc-0" and "pvc-1" to belong to group "myGroup".

3. Create a **VolumeGroupSnapshot** object to specify your volume group snapshot:
 - a. Create a **VolumeGroupSnapshot** object YAML file with the following example file:

Example VolumeGroupSnapshot YAML file

```
apiVersion: groupsnapshot.storage.k8s.io/v1beta2
kind: VolumeGroupSnapshot
metadata:
  name: <volume-group-snapshot-name>
  namespace: <namespace>
spec:
```

```

volumeGroupSnapshotClassName: <volume-group-snapshot-class-name>
source:
  selector:
    matchLabels:
      group: myGroup

```

- **kind:** The **VolumeGroupSnapshot** object requests creation of a volume group snapshot for multiple PVCs.
- **metadata.name:** Name of the volume group snapshot.
- **metadata.namespace:** Namespace for the volume group snapshot.
- **spec.volumeGroupSnapshotClassName:** The **VolumeGroupSnapshotClass** name. This object is created by the administrator and describes how volume group snapshots should be created.
- **spec.source.selector.matchLabels.group:** The name of the label used to group the required PVCs for the snapshot. In this example, it is "myGroup".

b. Create the **VolumeGroupSnapshot** object by running the following command:

```
$ oc create -f <volume-group-snapshot-filename>.yaml
```

Results

Individual volume snapshots are created according to how many PVCs were specified as part of the volume group snapshot.

These individual volume snapshots are named with the following format: <hash of VolumeGroupSnapshotContentUUID+volumeHandle>:

```

apiVersion: snapshot.storage.k8s.io/v1
kind: VolumeSnapshot
metadata:
  name: snapshot-4dc1c53a29538b36e85003503a4bcac5dbde4cff59e81f1e3bb80b6c18c3fd03
  namespace: default
ownerReferences:
- apiVersion: groupsnapshot.storage.k8s.io/v1beta2
  kind: VolumeGroupSnapshot
  name: my-groupsnapshot
  uid: ba2d60c5-5082-4279-80c2-daa85f0af354
resourceVersion: "124503"
uid: c0137282-f161-4e86-92c1-c41d36c6d04c
spec:
  source:
    persistentVolumeClaimName: pvc-1
status:
  volumeGroupSnapshotName: volume-group-snapshot-name

```

In the preceding example, two individual volume snapshots are created as part of the volume group snapshot.

```

snapshot-4dc1c53a29538b36e85003503a4bcac5dbde4cff59e81f1e3bb80b6c18c3fd03
snapshot-fbfe59eff570171765df664280910c3bf1a4d56e233a5364cd8cb0152a35965b

```

6.4.5. Restoring a volume group snapshot

Restore volumes from a volume group snapshot by creating new persistent volume claims (PVCs) from individual **VolumeSnapshot** objects. Each PVC provisions a volume populated with snapshot data. Repeat for each **VolumeSnapshot** in the group to restore all volumes to their previous state.

Prerequisites

- Logged in to a running OpenShift Container Platform cluster.
- PVC has been created using a Container Storage Interface (CSI) driver that supports volume group snapshots.
- A storage class to provision the storage back end.
- A volume group snapshot has been created and is ready to use.

Procedure

1. Specify a **VolumeSnapshot** data source from a volume group snapshot for a PVC as shown in the following example:

Example restore PVC YAML file

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: <pvc-restore-name>
  namespace: <namespace>
spec:
  storageClassName: csi-hostpath-sc
  dataSource:
    name: snapshot-
    fbfe59eff570171765df664280910c3bf1a4d56e233a5364cd8cb0152a35965b
    kind: VolumeSnapshot
    apiGroup: snapshot.storage.k8s.io
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

- **metadata.name**: Name of the restore PVC.
- **metadata.namespace**: Name of the namespace.
- **spec.dataSource.name**: Name of an individual volume snapshot that is part of the volume group snapshot to use as source.
- **spec.dataSource.kind**: Must be set to the **VolumeSnapshot** value.
- **spec.dataSource.apiGroup**: Must be set to the **snapshot.storage.k8s.io** value.

2. Create the PVC by running the following command:

```
$ oc create -f <pvc-restore-filename>.yaml
```

Where **<pvc-restore-filename>.yaml** is the name of the PVC restore file specified in the preceding step.

3. Verify that the restored PVC has been created by running the following command:

```
$ oc get pvc
```

A new PVC with the name you specified in the first step is displayed.

4. Repeat the procedure as needed until all volumes are created from all the snapshots that are part of a volume group snapshot.

6.4.6. Additional resources

- [Understanding persistent volumes](#)
- [Enabling features using feature gates](#)
- [CSI volume snapshots](#)

6.5. CSI VOLUME CLONING

Container Storage Interface (CSI) volume cloning duplicates existing persistent volumes to create independent copies for data protection, testing, or deployment. You can use cloning to create new volumes from existing data without manual copying or backup restoration.

6.5.1. Overview of CSI volume cloning

You can use Container Storage Interface (CSI) volume clones to create point-in-time duplicates of existing persistent volumes.

Volume cloning is similar to volume snapshots, although it is more efficient. For example, a cluster administrator can duplicate a cluster volume by creating another instance of the existing cluster volume.

Cloning creates an exact duplicate of the specified volume on the back-end device, rather than creating a new empty volume. After dynamic provisioning, you can use a volume clone just as you would use any standard volume.

No new API objects are required for cloning. The existing **dataSource** field in the **PersistentVolumeClaim** object is expanded so that it can accept the name of an existing PersistentVolumeClaim in the same namespace.

Before you provision a CSI volume clone, you should be familiar with persistent volumes. For information about persistent volumes, see *Understanding persistent volumes* in *Additional resources*.

6.5.1.1. Support limitations

By default, OpenShift Container Platform supports CSI volume cloning with these limitations:

- The destination persistent volume claim (PVC) must exist in the same namespace as the source PVC.

- Cloning is supported with a different Storage Class.
 - Destination volume can be the same for a different storage class as the source.
 - You can use the default storage class and omit **storageClassName** in the **spec**.
- Support is only available for CSI drivers. In-tree and FlexVolumes are not supported.
- CSI drivers might not have implemented the volume cloning functionality. For details, see the CSI driver documentation.

6.5.2. Provisioning a CSI volume clone

Create a new persistent volume claim (PVC) that specifies an existing PVC as its data source. The new volume automatically populates with a copy of the source PVC's data and must be created in the same namespace as the source.

When you create a cloned persistent volume claim (PVC) API object, you trigger the provisioning of a CSI volume clone. The clone pre-populates with the contents of another PVC, adhering to the same rules as any other persistent volume. The one exception is that you must add a **dataSource** that references an existing PVC in the same namespace.

Prerequisites

- You are logged in to a running OpenShift Container Platform cluster.
- Your PVC is created using a CSI driver that supports volume cloning.
- Your storage back end is configured for dynamic provisioning. Cloning support is not available for static provisioners.

Procedure

1. Create and save a file with the **PersistentVolumeClaim** object described by the following YAML:

Example pvc-clone.yaml

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-1-clone
  namespace: mynamespace
spec:
  storageClassName: csi-cloning
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
  dataSource:
    kind: PersistentVolumeClaim
    name: pvc-1
```

Where **spec.storageClassName** is the name of the storage class that provisions the storage back end. The default storage class can be used and **storageClassName** can be omitted in the spec.

2. Create the object you saved in the previous step by running the following command:

```
$ oc create -f pvc-clone.yaml
```

A new PVC **pvc-1-clone** is created.

3. Verify that the volume clone was created and is ready by running the following command:

```
$ oc get pvc pvc-1-clone
```

The **pvc-1-clone** shows that it is **Bound**.

You are now ready to use the newly cloned PVC to configure a pod.

4. Create and save a file with the **Pod** object described by the YAML. For example:

Example pod YAML file

```
kind: Pod
apiVersion: v1
metadata:
  name: mypod
spec:
  containers:
    - name: myfrontend
      image: dockerfile/nginx
      volumeMounts:
        - mountPath: "/var/www/html"
          name: mypd
  volumes:
    - name: mypd
      persistentVolumeClaim:
        claimName: pvc-1-clone
```

Where **spec.volumes.persistentVolumeClaim.claimName** is the cloned PVC created during the CSI volume cloning operation.

Result

The created **Pod** object is now ready to consume, clone, snapshot, or delete your cloned PVC independently of its original **dataSource** PVC.

Additional resources

- [Understanding persistent volumes](#)

6.6. VOLUME POPULATORS

Volume populators enable the automatic pre-loading of data into a volume during dynamic provisioning, instead of provisioning an empty volume.

6.6.1. Volume populators overview

With volume populators, using the **dataSourceRef** field, you can prepopulate volumes from a Custom Resource Definition (CRD) instead of only persistent volume claims (PVCs) and snapshots.

In OpenShift Container Platform versions 4.12 through 4.19, the **dataSource** field in a PVC spec provides volume populator capability. However, it is limited to using only PVCs and snapshots as the data source for populating volumes.

Starting with OpenShift Container Platform version 4.20, the **dataSourceRef** field is used instead. With the **dataSourceRef** field, you can use any appropriate custom resource (CR) as the data source to prepopulate a new volume.



NOTE

Volume populator functionality using the **dataSource** field is likely to be deprecated in future versions. If you have created any volume populators using this field, consider re-creating your volume populators to use the **dataSourceRef** field to avoid future issues.

Volume population is enabled by default and OpenShift Container Platform includes the installed **volume-data-source-validator** controller. However, OpenShift Container Platform does not ship with any volume populators.

6.6.2. Volume populators creation

To implement custom volume prepopulation behavior, create a volume populator by defining a custom resource definition (CRD) and then using it to create prepopulated volumes.

6.6.2.1. Creating CRDs for volume populators

To enable custom volume prepopulation, create a Custom Resource Definition (CRD) that defines a data source that users can instantiate to populate persistent volume claims (PVCs).

The following procedure explains how to create an example "hello, world" CRD for a volume populator.

Users can then create instances of this CRD to populate PVCs.

Prerequisites

- Access to the OpenShift Container Platform web console.
- Access to the cluster with cluster-admin privileges.

Procedure

1. Create a namespace for the logical grouping and operation of the populator, and related resources, using the following example YAML file:

Example namespace YAML file

```
apiVersion: v1
kind: Namespace
metadata:
  name: hello
```

2. Create a CRD for your data source using the following example YAML file:

Example CRD YAML file

```

apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: hellos.hello.example.com
spec:
  group: hello.example.com
  names:
    kind: Hello
    listKind: HelloList
    plural: hellos
    singular: hello
  scope: Namespaced
  versions:
    - name: v1alpha1
      schema:
        openAPIV3Schema:
          description: Hello is a specification for a Hello resource
          properties:
            apiVersion:
              description: 'APIVersion defines the versioned schema of this representation
                of an object. Servers should convert recognized schemas to the latest
                internal value, and may reject unrecognized values. More info:
                https://git.k8s.io/community/contributors/devel/sig-architecture/api-
                conventions.md#resources'
              type: string
            kind:
              description: 'Kind is a string value representing the REST resource this
                object represents. Servers may infer this from the endpoint the client
                submits requests to. Cannot be updated. In CamelCase. More info:
                https://git.k8s.io/community/contributors/devel/sig-architecture/api-conventions.md#types-
                kinds'
              type: string
            spec:
              description: HelloSpec is the spec for a Hello resource
              properties:
                fileContents:
                  type: string
                fileName:
                  type: string
              required:
                - fileContents
                - fileName
              type: object
            served: true
            storage: true

```

3. Deploy the controller by creating a **ServiceAccount**, **ClusterRole**, **ClusterRoleBinding**, and **Deployment** to run the logic that implements the population:

- a. Create a service account for the populator using the following example YAML file:

Example service account YAML file

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: hello-account
  namespace: hello
```

Where **metadata.namespace** references the namespace that you created earlier.

- b. Create a cluster role for the populator using the following example YAML file:

Example cluster role YAML file

```
kind: ClusterRole
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: hello-role
rules:
  - apiGroups: [hello.example.com]
    resources: [hellos]
    verbs: [get, list, watch]
```

- c. Create a cluster role binding using the following example YAML file:

Example cluster role binding YAML file

```
kind: ClusterRoleBinding
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  name: hello-binding
subjects:
  - kind: ServiceAccount
    name: hello-account
    namespace: hello
roleRef:
  kind: ClusterRole
  name: hello-role
  apiGroup: rbac.authorization.k8s.io
```

- **metadata.name:** Specifies the role binding name.
- **subjects.name:** References the name of the service account that you created earlier.
- **subjects.namespace:** References the name of the namespace for the service account that you created earlier.
- **roleRef.name:** References the cluster role you created earlier.

- d. Create a Deployment for the populator using the following example YAML file:

Example deployment YAML file

```

kind: Deployment
apiVersion: apps/v1
metadata:
  name: hello-populator
  namespace: hello
spec:
  selector:
    matchLabels:
      app: hello
  template:
    metadata:
      labels:
        app: hello
    spec:
      serviceAccount: hello-account
      containers:
        - name: hello
          image: registry.k8s.io/sig-storage/hello-populator:v1.0.1
          imagePullPolicy: IfNotPresent
          args:
            - --mode=controller
            - --image-name=registry.k8s.io/sig-storage/hello-populator:v1.0.1
            - --http-endpoint=:8080
          ports:
            - containerPort: 8080
              name: http-endpoint
              protocol: TCP

```

- **metadata.namespace:** References the namespace that you created earlier.
- **spec.template.spec.serviceAccount:** References the service account that you created earlier.

4. Create a volume populator to register the **kind:Hello** resource as a valid data source for the volume using the following example YAML file:

Example volume populator YAML file

```

kind: VolumePopulator
apiVersion: populator.storage.k8s.io/v1beta1
metadata:
  name: hello-populator
sourceKind:
  group: hello.example.com
  kind: Hello

```

The **metadata.name** field specifies the Volume populator name.

PVCs that use an unregistered populator generate an event: "The datasource for this PVC does not match any registered VolumePopulator", indicating that the PVC might not be provisioned because you are using an unknown (unregistered) populator.

Next steps

- You can now create CR instances of this CRD to populate PVCs

For more information, see "Creating prepopulated volumes using volume populators".

6.6.2.2. Creating prepopulated volumes using volume populators

To create volumes that are automatically filled with data when provisioned, define a Custom Resource Definition (CRD) as a data source and reference it when creating the persistent volume claim (PVC).

The following procedure explains how to create a prepopulated PVC using the example **hellos.hello.example.com** CRD created previously.

In this example, rather than using an actual data source, you are creating a file called "example.txt" that contains the string "Hello, world!" in the root directory of the volume. For a real-world implementation, you need to create your own volume populator.

Prerequisites

- You are logged in to a running OpenShift Container Platform cluster.
- There is an existing CRD for volume populators.
- OpenShift Container Platform does not ship with any volume populators. You **must** create your own volume populator.

Procedure

1. Create a Custom Resource (CR) instance of the **Hello** CRD with the text "Hello, World!" passed in as **fileContents** parameter by running the following command:

```
$ oc apply -f - <<EOF
apiVersion: hello.example.com/v1alpha1
kind: Hello
metadata:
  name: example-hello
spec:
  fileName: example.txt
  fileContents: Hello, world!
EOF
```

2. Create a PVC that references the Hello CR similar to the following example file:

Example PVC YAML file

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: example-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Mi
  dataSourceRef:
    apiGroup: hello.example.com
```

```
kind: Hello
name: example-hello
volumeMode: Filesystem
```

- **spec.dataSourceRef:** Specifies the data source for the PVC.
- **spec.dataSourceRef.name:** Specifies the name of the CR that you are using as the data source. In this example, it is 'example-hello'.

Verification

1. After a few minutes, ensure that the PVC is created and in the **Bound** status by running the following command:

```
$ oc get pvc example-pvc -n hello
```

In this example, the name of the PVC is **example-pvc**.

Example output

```
NAME          STATUS  VOLUME  CAPACITY  ACCESS MODES  STORAGECLASS  VOLUMEATTRIBUTESCLASS  AGE
example-pvc   Bound  my-pv   10Mi     ReadWriteOnce  gp3-csi       <unset>                 14s
```

2. Create a job that reads from the PVC to verify that the data source information was applied using the following example file:

Example job YAML file

```
apiVersion: batch/v1
kind: Job
metadata:
  name: example-job
spec:
  template:
    spec:
      containers:
        - name: example-container
          image: busybox:latest
          command:
            - cat
            - /mnt/example.txt
          volumeMounts:
            - name: vol
              mountPath: /mnt
      restartPolicy: Never
      volumes:
        - name: vol
          persistentVolumeClaim:
            claimName: example-pvc
```

- **spec.template.spec.containers.command:** Specifies the location and name of the file with the "Hello, world!" text. In this example, the location is "/mnt/example.txt".

- **spec.template.spec.volumes.persistentVolumeClaim:** Specifies the name of the PVC you created in Step 2. In this example, it is **example-pvc**.

3. Start the job by running the following command:

```
$ oc run example-job --image=busybox --command -- sleep 30 --restart=OnFailure
```

Example output

```
pod/example-job created
```

4. Wait for the job, and all of its dependencies, to finish by running the following command:

```
$ oc wait --for=condition=Complete pod/example-job
```

5. Verify the contents collected by the job by running the following command:

```
$ oc logs job/example-job
```

Example expected output

```
Hello, world!
```

6.6.3. Uninstalling volume populators

To remove custom volume prepopulation functionality, delete all volume populator resources in reverse order of creation.

Prerequisites

- Access to the OpenShift Container Platform web console.
- Access to the cluster with cluster-admin privileges.

Procedure

- To uninstall volume populators, delete in reverse order all objects installed in the following procedures:
 1. "Creating prepopulated volumes using volume populators".
 2. "Creating CRDs for volume populators".
Be sure to remove the **VolumePopulator** instance.

6.7. MANAGING THE DEFAULT STORAGE CLASS

Many Container Storage Interface (CSI) operators can actively manage default storage classes, removing manual intervention needs and avoiding accidental deletion. Proper management ensures persistent volume claims provision correctly with the appropriate storage backend for your applications.

6.7.1. Overview

You can manage the default storage class by configuring the `spec.storageClassState` field in the `ClusterCSIDriver` object to control dynamic provisioning, prevent automatic recreation, or rename the storage class.

Managing the default storage class allows you to accomplish several different objectives:

- Enforcing static provisioning by disabling dynamic provisioning.
- When you have other preferred storage classes, preventing the storage operator from re-creating the initial default storage class.
- Renaming, or otherwise changing, the default storage class

To accomplish these objectives, you change the setting for the **`spec.storageClassState`** field in the **`ClusterCSIDriver`** object. The possible settings for this field are:

- **Managed:** (Default) The Container Storage Interface (CSI) operator is actively managing its default storage class, so that most manual changes made by a cluster administrator to the default storage class are removed, and the default storage class is continuously re-created if you attempt to manually delete it.
- **Unmanaged:** You can modify the default storage class. The CSI operator is not actively managing storage classes, so that it is not reconciling the default storage class it creates automatically.
- **Removed:** The CSI operators deletes the default storage class.

Managing the default storage classes is supported by the following Container Storage Interface (CSI) driver operators:

- Amazon Web Services (AWS) Elastic Block Storage (EBS)
- Azure Disk
- Azure File
- Google Cloud Platform (GCP) Persistent Disk (PD)
- IBM Cloud® VPC Block
- OpenStack Cinder
- VMware vSphere

6.7.2. Managing the default storage class using the web console

Manage storage class behavior using the web console by configuring the **`ClusterCSIDriver`** object's **`storageClassState`** field. Set the state to `Managed` for operator control, `Unmanaged` for manual control, or `Removed` to delete the storage class, determining how default storage classes are handled.

Prerequisites

- Access to the OpenShift Container Platform web console.
- Access to the cluster with cluster-admin privileges.

Procedure

1. Log in to the web console.
2. Click **Administration** > **CustomResourceDefinitions**.
3. On the **CustomResourceDefinitions** page, type **clustercsidriver** to find the **ClusterCSIDriver** object.
4. Click **ClusterCSIDriver**, and then click the **Instances** tab.
5. Click the name of the desired instance, and then click the **YAML** tab.
6. Add the **spec.storageClassState** field with a value of **Managed**, **Unmanaged**, or **Removed**.

Example

```
...
spec:
  driverConfig:
    driverType: "
  logLevel: Normal
  managementState: Managed
  observedConfig: null
  operatorLogLevel: Normal
  storageClassState: Unmanaged
...
```

For this example, **spec.storageClassState** field is set to "Unmanaged".

7. Click **Save**.

6.7.3. Managing the default storage class using the CLI

Manage storage class behavior using the CLI by configuring the **ClusterCSIDriver** object's **storageClassState** field. Set the state to **Managed** for operator control, **Unmanaged** for manual control, or **Removed** to delete the storage class, determining how default storage classes are handled.

Prerequisites

- Access to the cluster with cluster-admin privileges.

Procedure

- To manage the storage class using the CLI, run the following command:

```
$ oc patch clustercsidriver $DRIVERNAME --type=merge -p '{"spec\":"storageClassState\":"${STATE}\"}'}
```

- Where **\${STATE}** is "Removed" or "Managed" or "Unmanaged".
- Where **\$DRIVERNAME** is the provisioner name. You can find the provisioner name by running the command **oc get sc**.

6.7.4. Absent or multiple default storage classes

Absent or multiple default storage classes cause persistent volume claim issues. Multiple default storage classes might result in unpredictable selection and alerts, while absent default storage classes leave claims pending. Resolve by ensuring exactly one storage class is designated as the default.

6.7.4.1. Multiple default storage classes

Multiple default storage classes can occur if you mark a non-default storage class as default and do not unset the existing default storage class, or you create a default storage class when a default storage class is already present. With multiple default storage classes present, any persistent volume claim (PVC) requesting the default storage class (**pvc.spec.storageClassName=nil**) gets the most recently created default storage class, regardless of the default status of that storage class, and the administrator receives an alert in the alerts dashboard that there are multiple default storage classes, **MultipleDefaultStorageClasses**.

6.7.4.2. Absent default storage class

There are two possible scenarios where PVCs can attempt to use a non-existent default storage class:

- An administrator removes the default storage class or marks it as non-default, and then a user creates a PVC requesting the default storage class.
- During installation, the installer creates a PVC requesting the default storage class, which has not yet been created.

In the preceding scenarios, PVCs remain in the pending state indefinitely. To resolve this situation, create a default storage class or declare one of the existing storage classes as the default. As soon as the default storage class is created or declared, the PVCs get the new default storage class. If possible, the PVCs eventually bind to statically or dynamically provisioned PVs as usual, and move out of the pending state.

6.7.5. Changing the default storage class

Change the default storage class to ensure new persistent volume claims (PVCs) automatically use your preferred storage backend. This helps you optimize costs, align with infrastructure changes, or ensure consistent storage types across new deployments without requiring users to specify a storage class for each claim.

In this example, you have two defined storage classes, **gp3** and **standard**, and you want to change the default storage class from **gp3** to **standard**.

Prerequisites

- Access to the cluster with cluster-admin privileges.

Procedure

1. List the storage classes by running the following command:

```
$ oc get storageclass
```

Example output

NAME	TYPE
gp3 (default)	ebs.csi.aws.com
standard	ebs.csi.aws.com

The text **(default)** indicates the default storage class. In this example **gp3** is the current default storage class.

2. Make the required storage class the default.

For the required storage class, set the **storageclass.kubernetes.io/is-default-class** annotation to **true** by running the following command:

```
$ oc patch storageclass standard -p '{"metadata": {"annotations": {"storageclass.kubernetes.io/is-default-class": "true"}}}'
```



NOTE

You can have many default storage classes for a short time. However, you must ensure that only one default storage class exists eventually.

With many default storage classes present, any persistent volume claim (PVC) requesting the default storage class (**pvc.spec.storageClassName=nil**) gets the most recently created default storage class, regardless of the default status of that storage class. The administrator receives an alert in the alerts dashboard that there are many default storage classes, **MultipleDefaultStorageClasses**.

3. Remove the default storage class setting from the old default storage class.

For the old default storage class, change the value of the **storageclass.kubernetes.io/is-default-class** annotation to **false** by running the following command:

```
$ oc patch storageclass gp3 -p '{"metadata": {"annotations": {"storageclass.kubernetes.io/is-default-class": "false"}}}'
```

4. Verify the changes by running the following command:

```
$ oc get storageclass
```

Example output

NAME	TYPE
gp3	ebs.csi.aws.com
standard (default)	ebs.csi.aws.com

The **standard** storage class is now the default.

6.8. CSI AUTOMATIC MIGRATION

In-tree storage drivers that are traditionally shipped with OpenShift Container Platform are being deprecated and replaced by their equivalent Container Storage Interface (CSI) drivers. OpenShift Container Platform provides automatic migration for in-tree volume plugins to their equivalent CSI drivers.

6.8.1. Overview

Container Storage Interface (CSI) migration transparently translates in-tree storage volumes to CSI equivalents in memory without data migration or API changes.

This process does not perform any data migration; OpenShift Container Platform only translates the persistent volume object in memory. As a result, the translated persistent volume object is not stored on disk, nor is its contents changed. CSI automatic migration should be seamless. This feature does not change how you use all existing API objects: for example, **PersistentVolumes**, **PersistentVolumeClaims**, and **StorageClasses**.

The following in-tree to CSI drivers are automatically migrated:

- Azure Disk
- OpenStack Cinder
- Amazon Web Services (AWS) Elastic Block Storage (EBS)
- Google Compute Engine Persistent Disk (GCP PD)
- Azure File
- VMware vSphere

CSI migration for these volume types is considered generally available (GA), and requires no manual intervention.

CSI automatic migration of in-tree persistent volumes (PVs) or persistent volume claims (PVCs) does not enable any new CSI driver features, such as snapshots or expansion, if the original in-tree storage plugin did not support it.

6.8.2. Storage class implications

OpenShift Container Platform 4.13, and later, uses Container Storage Interface (CSI) storage classes for new installations. Upgraded clusters receive CSI storage classes as default if none existed. Existing in-tree classes remain for backward compatibility. Switching to CSI storage classes is recommended.

For new OpenShift Container Platform 4.13, and later, installations, the default storage class is the CSI storage class. All volumes provisioned using this storage class are CSI persistent volumes (PVs).

For clusters upgraded from 4.12, and earlier, to 4.13, and later, the CSI storage class is created, and is set as the default if no default storage class was set before the upgrade. In the very unlikely case that there is a storage class with the same name, the existing storage class remains unchanged. Any existing in-tree storage classes remain, and might be necessary for certain features, such as volume expansion to work for existing in-tree PVs. While storage class referencing to the in-tree storage plugin will continue working, we recommend that you switch the default storage class to the CSI storage class.

For information about changing the default storage class, see *Changing the default storage class* under *Additional resources*.

Additional resources

- [Changing the default storage class](#)

6.9. AWS ELASTIC BLOCK STORE CSI DRIVER OPERATOR

You can provision and manage AWS Elastic Block Storage (EBS) in OpenShift Container Platform by using the AWS EBS Container Storage Interface (CSI) Driver Operator and driver, which provide dynamic volume provisioning and eliminate the need to pre-provision storage.

6.9.1. Overview of the AWS EBS CSI Driver Operator

OpenShift Container Platform is capable of provisioning persistent volumes (PVs) using the AWS Elastic Block Storage (EBS) Container Storage Interface (CSI) driver.

Familiarity with persistent storage and configuring CSI volumes is recommended when working with a CSI Operator and driver. For more information, see "Understanding persistent storage" and "Configuring CSI volumes".

To create CSI-provisioned PVs that mount to AWS EBS storage assets, OpenShift Container Platform installs the AWS EBS CSI Driver Operator (a Red Hat operator) and the AWS EBS CSI driver by default in the **openshift-cluster-csi-drivers** namespace.

AWS EBS CSI Driver Operator

The AWS EBS CSI Driver Operator provides a **StorageClass** by default that you can use to create persistent volume claims (PVCs). You can disable this default storage class if desired (see "Managing the default storage class"). You also have the option to create the AWS EBS **StorageClass** as described in "Creating the EBS storage class".

AWS EBS CSI driver

The AWS EBS CSI driver enables you to create and mount AWS EBS PVs.



NOTE

If you installed the AWS EBS CSI Operator and driver on an OpenShift Container Platform 4.5 cluster, you must uninstall the 4.5 Operator and driver before you update to OpenShift Container Platform 4.22.



IMPORTANT

OpenShift Container Platform defaults to using the CSI plugin to provision Amazon Elastic Block Store (Amazon EBS) storage.

For information about dynamically provisioning AWS EBS persistent volumes in OpenShift Container Platform, see "Dynamic provisioning".

Additional resources

- [Understanding persistent storage](#)
- [Configuring CSI volumes](#)
- [Managing the default storage class](#)
- [Creating the EBS storage class](#)
- [Dynamic provisioning](#)

6.9.2. About CSI

The Container Storage Interface (CSI) enables storage vendors to deliver plugins through a standard interface without modifying Kubernetes core code, replacing traditional embedded storage drivers.

CSI Operators give OpenShift Container Platform users storage options, such as volume snapshots, that are not possible with in-tree volume plugins.

6.9.3. User-managed encryption

The user-managed encryption feature allows you to provide keys during installation that encrypt OpenShift Container Platform node root volumes, and enables all managed storage classes to use these keys to encrypt provisioned storage volumes.

You must specify the custom key in the **platform.<cloud_type>.defaultMachinePlatform** field in the install-config YAML file.

This features supports the following storage types:

- Amazon Web Services (AWS) Elastic Block storage (EBS)



NOTE

If there is no encrypted key defined in the storage class, only set **encrypted: "true"** in the storage class. The AWS EBS CSI driver uses the AWS managed alias/aws/ebs, which is created by Amazon EBS automatically in each region by default to encrypt provisioned storage volumes. In addition, the managed storage classes all have the **encrypted: "true"** setting.

For information about installing AWS EBS with user-managed encryption, see "Optional AWS configuration parameters".

- Microsoft Azure Disk storage



NOTE

If the OS (root) disk is encrypted, and there is no encrypted key defined in the storage class, Azure Disk CSI driver uses the OS disk encryption key by default to encrypt provisioned storage volumes.

For information about installing Azure Disk with user-managed encryption, see "Preparing an Azure Disk Encryption Set".

- Google Cloud Platform (GCP) persistent disk (PD) storage
For information about installing GCP PD with user-managed encryption, see "Additional Google Cloud configuration parameters".
- IBM Cloud® Virtual Private Cloud (VPC) Block storage
For information about installing with IBM Cloud with user-managed encryption, see "User-managed encryption for IBM Cloud" and "Installing on IBM Cloud".

Additional resources

- [Optional AWS configuration parameters](#)

6.9.4. Support for European Sovereign Cloud (EUSC) region

European Sovereign Cloud (EUSC) region acts as a "digital fortress" built within a specific country's borders. Sovereign Clouds are specifically designed to meet strict legal, jurisdictional, and security requirements of a particular nation or entity.

In the context of storage, EUSC ensures that all data, including primary storage, backups, and the resulting metadata, resides physically within the specific nation's borders and remains exclusively under its legal jurisdiction.

For OpenShift Container Platform 4.22, and later, only AWS Elastic Block Storage supports EUSC. AWS Elastic File Storage (EFS) is not supported.



IMPORTANT

EUSC is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

For information about installing an OpenShift Container Platform cluster into the AWS EUSC, see "AWS EUSC region".

Additional resources

- [AWS EUSC region](#)

6.10. AWS ELASTIC FILE SERVICE CSI DRIVER OPERATOR

You can provision and manage AWS Elastic File System (EFS) storage in OpenShift Container Platform by using the AWS EFS Container Storage Interface (CSI) Driver Operator and driver, which provide dynamic volume provisioning and eliminate the need to pre-provision storage.

6.10.1. Overview

OpenShift Container Platform is capable of provisioning persistent volumes (PVs) using the Container Storage Interface (CSI) driver for AWS Elastic File Service (EFS).

Familiarity with persistent storage and configuring CSI volumes is recommended when working with a CSI Operator and driver. For more information, see "Understanding persistent storage" and "Configuring CSI volumes".

After installing the AWS EFS CSI Driver Operator, OpenShift Container Platform installs the AWS EFS CSI Operator and the AWS EFS CSI driver by default in the **openshift-cluster-csi-drivers** namespace. This allows the AWS EFS CSI Driver Operator to create CSI-provisioned PVs that mount to AWS EFS assets.

- The *AWS EFS CSI Driver Operator*, after being installed, does not create a storage class by default to use to create persistent volume claims (PVCs). However, you can manually create the AWS EFS **StorageClass**. The AWS EFS CSI Driver Operator supports dynamic volume provisioning by allowing storage volumes to be created on-demand. This eliminates the need for cluster administrators to pre-provision storage.

- The *AWS EFS CSI driver* enables you to create and mount AWS EFS PVs.

Additional resources

- [Understanding persistent storage](#)
- [Configuring CSI volumes](#)

6.10.2. About CSI

The Container Storage Interface (CSI) enables storage vendors to deliver plugins through a standard interface without modifying Kubernetes core code, replacing traditional embedded storage drivers.

CSI Operators give OpenShift Container Platform users storage options, such as volume snapshots, that are not possible with in-tree volume plugins.

6.10.3. Setting up the AWS EFS CSI Driver Operator

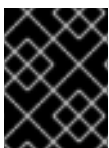
To enable AWS Elastic File System (EFS) storage in your cluster, complete the setup process by obtaining necessary credentials, installing the operator, and installing the driver.

Procedure

1. If you are using AWS EFS with AWS Secure Token Service (STS), obtain a role Amazon Resource Name (ARN) for STS. This is required for installing the AWS EFS CSI Driver Operator.
2. Install the AWS EFS CSI Driver Operator.
3. Install the AWS EFS CSI Driver.

6.10.3.1. Obtaining a role Amazon Resource Name for Security Token Service

To configure the AWS Elastic File System (EFS) Container Storage Interface (CSI) Driver Operator on clusters using Security Token Service (STS), obtain a role Amazon Resource Name (ARN) using the Cloud Credential Operator utility before installation.



IMPORTANT

Perform this procedure before you install the AWS EFS CSI Driver Operator (see *Installing the AWS EFS CSI Driver Operator* procedure).

You can obtain the ARN role in multiple ways. The following procedure shows one method that uses the same concept and CCO utility (**ccocli**) binary tool as cluster installation.



NOTE

If you are using One Zone file system, you need to create two **CredentialRequests**, one for the controller and one for the driver node. For more information, see Section *Setting up One Zone file systems with STS*.

Prerequisites

- Access to the cluster as a user with the cluster-admin role.

- AWS account credentials

Procedure

1. Extract the **ccoctl** from the OpenShift Container Platform release image, which you used to install the cluster with STS. For more information, see "Configuring the Cloud Credential Operator utility".
2. Create and save an EFS **CredentialsRequest** YAML file, such as shown in the following example, and then place it in the **credrequests** directory:

Example

```
apiVersion: cloudcredential.openshift.io/v1
kind: CredentialsRequest
metadata:
  name: openshift-aws-efs-csi-driver
  namespace: openshift-cloud-credential-operator
spec:
  providerSpec:
    apiVersion: cloudcredential.openshift.io/v1
    kind: AWSProviderSpec
    statementEntries:
      - action:
          - elasticfilesystem:*
        effect: Allow
        resource: '*'
  secretRef:
    name: aws-efs-cloud-credentials
    namespace: openshift-cluster-csi-drivers
  serviceAccountNames:
    - aws-efs-csi-driver-operator
    - aws-efs-csi-driver-controller-sa
```

3. Run the **ccoctl** tool to generate a new IAM role in AWS, and create a YAML file for it in the local file system (**<path_to_ccoctl_output_dir>/manifests/openshift-cluster-csi-drivers-aws-efs-cloud-credentials-credentials.yaml**).

```
$ ccoctl aws create-iam-roles --name=<name> --region=<aws_region> --credentials-requests-dir=<path_to_directory_with_list_of_credentials_requests>/credrequests --identity-provider-arn=arn:aws:iam::<aws_account_id>:oidc-provider/<name>-oidc.s3.<aws_region>.amazonaws.com
```

- **name=<name>** is the name used to tag any cloud resources that are created for tracking.
- **region=<aws_region>** is the AWS region where cloud resources are created.
- **dir=<path_to_directory_with_list_of_credentials_requests>/credrequests** is the directory containing the EFS CredentialsRequest file in previous step.
- **<aws_account_id>** is the AWS account ID.

Example

```
$ ccoctl aws create-iam-roles --name my-aws-efs --credentials-requests-dir credrequests
--identity-provider-arn arn:aws:iam::123456789012:oidc-provider/my-aws-efs-oidc.s3.us-
east-2.amazonaws.com
```

Example output

```
2022/03/21 06:24:44 Role arn:aws:iam::123456789012:role/my-aws-efs -openshift-
cluster-csi-drivers-aws-efs-cloud- created
2022/03/21 06:24:44 Saved credentials configuration to: /manifests/openshift-cluster-csi-
drivers-aws-efs-cloud-credentials-credentials.yaml
2022/03/21 06:24:45 Updated Role policy for Role my-aws-efs-openshift-cluster-csi-
drivers-aws-efs-cloud-
```

4. Copy the role ARN from the first line of the *Example output* in the preceding step. The role ARN is between "Role" and "created". In this example, the role ARN is "arn:aws:iam::123456789012:role/my-aws-efs -openshift-cluster-csi-drivers-aws-efs-cloud". You will need the role ARN when you install the AWS EFS CSI Driver Operator.

Next steps

Install the AWS EFS CSI Driver Operator. For information, see [Installing the AWS EFS CSI Driver Operator](#).

Additional resources

- [Installing the AWS EFS CSI Driver Operator](#)
- [Configuring the Cloud Credential Operator utility](#)
- [Installing the AWS EFS CSI Driver](#)

6.10.3.2. Installing the AWS EFS CSI Driver Operator

Install and configure the AWS EFS Container Storage Interface (CSI) Driver Operator to enable AWS EFS storage in your cluster. This Red Hat Operator is not installed by default and requires manual installation.

Prerequisites

- Access to the OpenShift Container Platform web console.

Procedure

1. Log in to the web console.
2. Install the AWS EFS CSI Operator:
 - a. Click **Ecosystem** → **Software Catalog**.
 - b. Locate the AWS EFS CSI Operator by typing **AWS EFS CSI** in the filter box.
 - c. Click the **AWS EFS CSI Driver Operator** button.



IMPORTANT

Be sure to select the **AWS EFS CSI Driver Operator** and not the **AWS EFS Operator**. The **AWS EFS Operator** is a community Operator and is not supported by Red Hat.

- a. On the **AWS EFS CSI Driver Operator** page, click **Install**.
- b. On the **Install Operator** page, ensure that:
 - If you are using AWS EFS with AWS Secure Token Service (STS), in the **role ARN** field, enter the ARN role copied from the last step of the *Obtaining a role Amazon Resource Name for Security Token Service* procedure.
 - **All namespaces on the cluster (default)** is selected.
 - **Installed Namespace** is set to **openshift-cluster-csi-drivers**.
- c. Click **Install**.
After the installation finishes, the AWS EFS CSI Operator is listed in the **Installed Operators** section of the web console.

Next steps

Install the AWS EFS CSI Driver.

6.10.3.3. Installing the AWS EFS CSI Driver

After installing the Container Storage Interface (CSI) Driver Operator (a Red Hat operator), you need to install the AWS EFS CSI driver.

Prerequisites

- Access to the OpenShift Container Platform web console.

Procedure

1. Click **Administration** → **CustomResourceDefinitions** → **ClusterCSIDriver**.
2. On the **Instances** tab, click **Create ClusterCSIDriver**.
3. Use the following YAML file:

```
apiVersion: operator.openshift.io/v1
kind: ClusterCSIDriver
metadata:
  name: efs.csi.aws.com
spec:
  managementState: Managed
```

Where **metadata.name** is the storage provisioner name.

4. Click **Create**.
5. Wait for the following Conditions to change to a "True" status:

- AWSEFSDriverNodeServiceControllerAvailable
- AWSEFSDriverControllerServiceControllerAvailable

6.10.4. About the AWS EFS storage class

To enable dynamic provisioning of persistent volumes, create a storage class that defines storage characteristics and allows users to automatically provision volumes on-demand.

The AWS Elastic File System (EFS) Container Storage Interface (CSI) Driver Operator (a Red Hat operator), after being installed, does not create a storage class by default. However, you can manually create the AWS EFS storage class.

6.10.4.1. Creating the AWS EFS storage class using the console

To enable dynamic provisioning of AWS Elastic File System (EFS) volumes using the console, create a **StorageClass** object that defines file system parameters, permissions, and access point configuration.

Procedure

1. In the OpenShift Container Platform web console, click **Storage → StorageClasses**.
2. On the **StorageClasses** page, click **Create StorageClass**.
3. On the **StorageClass** page, perform the following steps:
 - a. Enter a name to reference the storage class.
 - b. Optional: Enter the description.
 - c. Select the reclaim policy.
 - d. Select **efs.csi.aws.com** from the **Provisioner** drop-down list.
 - e. Optional: Set the configuration parameters for the selected provisioner.
4. Click **Create**.

6.10.4.2. Creating the AWS EFS storage class using the CLI

To enable dynamic provisioning of AWS Elastic File System (EFS) volumes by using the command line, create a **StorageClass** object that defines file system parameters, permissions, and access point configuration.

Procedure

- Create a **StorageClass** object using the following example YAML file:

```
kind: StorageClass
apiVersion: storage.k8s.io/v1
metadata:
  name: efs-sc
provisioner: efs.csi.aws.com
parameters:
  provisioningMode: efs-ap
```

```

fileSystemId: fs-a5324911
directoryPerms: "700"
gidRangeStart: "1000"
gidRangeEnd: "2000"
basePath: "/dynamic_provisioning"

```

- **parameters.provisioningMode:** Must be set to **efs-ap** to enable dynamic provisioning.
- **parameters.fileSystemId:** Must be the ID of the EFS volume created manually.
- **parameters.directoryPerms:** Is the default permission of the root directory of the volume. In this example, the volume is accessible only by the owner.
- **parameters.gidRangeStart** and **parameters.gidRangeEnd:** Set the range of POSIX Group IDs (GIDs) that are used to set the GID of the AWS access point. If not specified, the default range is 50000-7000000. Each provisioned volume, and thus AWS access point, is assigned a unique GID from this range.
- **parameters.basePath:** Is the directory on the EFS volume that is used to create dynamically provisioned volumes. In this case, a PV is provisioned as `"/dynamic_provisioning/<random uuid>"` on the EFS volume. Only the subdirectory is mounted to pods that use the PV.



NOTE

A cluster admin can create several **StorageClass** objects, each using a different EFS volume.

Additional resources

- [AWS EFS CSI Driver Operator](#)

6.10.5. AWS EFS CSI cross account support

To mount AWS Elastic File System (EFS) volumes across AWS accounts, configure cross-account support that allows your OpenShift Container Platform cluster in one account to access file systems in another account.

Prerequisites

- Access to an OpenShift Container Platform cluster with administrator rights
- Two valid AWS accounts
- The EFS CSI Operator has been installed. For information about installing the EFS CSI Operator, see the *Installing the AWS EFS CSI Driver Operator* section.
- Both the OpenShift Container Platform cluster and EFS file system must be located in the same AWS region.
- Ensure that the two virtual private clouds (VPCs) used in the following procedure use different network Classless Inter-Domain Routing (CIDR) ranges.
- Access to OpenShift Container Platform CLI (**oc**).
- Access to AWS CLI.

- Access to **jq** command-line JSON processor.

The following procedure explains how to set up:

- OpenShift Container Platform AWS Account A: Contains a Red Hat OpenShift Container Platform cluster v4.16, or later, deployed within a VPC
- AWS Account B: Contains a VPC (including subnets, route tables, and network connectivity). The EFS filesystem will be created in this VPC.

Procedure

1. Set up the environment:
 - a. Configure environment variables by running the following commands:

```
export CLUSTER_NAME="<CLUSTER_NAME>"
export AWS_REGION="<AWS_REGION>"
export AWS_ACCOUNT_A_ID="<ACCOUNT_A_ID>"
export AWS_ACCOUNT_B_ID="<ACCOUNT_B_ID>"
export AWS_ACCOUNT_A_VPC_CIDR="<VPC_A_CIDR>"
export AWS_ACCOUNT_B_VPC_CIDR="<VPC_B_CIDR>"
export AWS_ACCOUNT_A_VPC_ID="<VPC_A_ID>"
export AWS_ACCOUNT_B_VPC_ID="<VPC_B_ID>"
export SCRATCH_DIR="<WORKING_DIRECTORY>"
export CSI_DRIVER_NAMESPACE="openshift-cluster-csi-drivers"
export AWS_PAGER=""
```

- **<CLUSTER_NAME>**: Cluster name of choice.
- **<AWS_REGION>**: AWS region of choice.
- **<ACCOUNT_A_ID>**: AWS Account A ID.
- **<ACCOUNT_B_ID>**: AWS Account B ID.
- **<VPC_A_CIDR>**: CIDR range of VPC in Account A.
- **<VPC_B_CIDR>**: CIDR range of VPC in Account B.
- **<VPC_A_ID>**: VPC ID in Account A (cluster)
- **<VPC_B_ID>**: VPC ID in Account B (EFS cross account)
- **<WORKING_DIRECTORY>**: Any writeable directory of choice to use to store temporary files.
- **CSI_DRIVER_NAMESPACE**: If your driver is installed in a non-default namespace, change this value.
- **AWS_PAGER**: Makes AWS CLI output everything directly to stdout.

- b. Create the working directory by running the following command:

```
mkdir -p $SCRATCH_DIR
```


- c. Verify cluster connectivity by running the following command in the OpenShift Container Platform CLI:

```
$ oc whoami
```

- d. Determine the OpenShift Container Platform cluster type and set node selector:
The EFS cross account feature requires assigning AWS IAM policies to nodes running EFS CSI controller pods. However, this is not consistent for every OpenShift Container Platform type.

- If your cluster is deployed as a Hosted Control Plane (HyperShift), set the **NODE_SELECTOR** environment variable to hold the worker node label by running the following command:

```
export NODE_SELECTOR=node-role.kubernetes.io/worker
```

- For all other OpenShift Container Platform types, set the **NODE_SELECTOR** environment variable to hold the master node label by running the following command:

```
export NODE_SELECTOR=node-role.kubernetes.io/master
```

- e. Configure AWS CLI profiles as environment variables for account switching by running the following commands:

```
export AWS_ACCOUNT_A="<ACCOUNT_A_NAME>"
export AWS_ACCOUNT_B="<ACCOUNT_B_NAME>"
```

- f. Ensure that your AWS CLI is configured with JSON output format as the default for both accounts by running the following commands:

```
export AWS_DEFAULT_PROFILE=${AWS_ACCOUNT_A}
aws configure get output
export AWS_DEFAULT_PROFILE=${AWS_ACCOUNT_B}
aws configure get output
```

If the preceding commands return:

- **No value:** The default output format is already set to JSON and no changes are required.
 - **Any value:** Reconfigure your AWS CLI to use JSON format. For information about changing output formats, see *Setting the output format in the AWS CLI* in the AWS documentation.
- g. Unset **AWS_PROFILE** in your shell to prevent conflicts with **AWS_DEFAULT_PROFILE** by running the following command:

```
unset AWS_PROFILE
```

2. Configure the AWS Account B IAM roles and policies:

- a. Switch to your Account B profile by running the following command:

```
export AWS_DEFAULT_PROFILE=${AWS_ACCOUNT_B}
```

- b. Define the IAM role name for the EFS CSI Driver Operator by running the following command:

```
export ACCOUNT_B_ROLE_NAME=${CLUSTER_NAME}-cross-account-aws-efs-csi-operator
```

- c. Create the IAM trust policy file by running the following command:

```
cat <<EOF > $SCRATCH_DIR/AssumeRolePolicyInAccountB.json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::${AWS_ACCOUNT_A_ID}:root"
      },
      "Action": "sts:AssumeRole",
      "Condition": {}
    }
  ]
}
EOF
```

- d. Create the IAM role for the EFS CSI Driver Operator by running the following command:

```
ACCOUNT_B_ROLE_ARN=$(aws iam create-role \
  --role-name "${ACCOUNT_B_ROLE_NAME}" \
  --assume-role-policy-document
file://$SCRATCH_DIR/AssumeRolePolicyInAccountB.json \
  --query "Role.Arn" --output text) \
&& echo $ACCOUNT_B_ROLE_ARN
```

- e. Create the IAM policy file by running the following command:

```
cat << EOF > $SCRATCH_DIR/EfsPolicyInAccountB.json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "VisualEditor0",
      "Effect": "Allow",
      "Action": [
        "ec2:DescribeNetworkInterfaces",
        "ec2:DescribeSubnets"
      ],
      "Resource": "*"
    },
    {
      "Sid": "VisualEditor1",
      "Effect": "Allow",
      "Action": [
        "elasticfilesystem:DescribeMountTargets",
        "elasticfilesystem>DeleteAccessPoint",
        "elasticfilesystem:ClientMount",

```

```

        "elasticfilesystem:DescribeAccessPoints",
        "elasticfilesystem:ClientWrite",
        "elasticfilesystem:ClientRootAccess",
        "elasticfilesystem:DescribeFileSystems",
        "elasticfilesystem:CreateAccessPoint",
        "elasticfilesystem:TagResource"
    ],
    "Resource": "*"
}
]
}
EOF

```

- f. Create the IAM policy by running the following command:

```

ACCOUNT_B_POLICY_ARN=$(aws iam create-policy --policy-name
"${CLUSTER_NAME}-efs-csi-policy" \
    --policy-document file://$SCRATCH_DIR/EfsPolicyInAccountB.json \
    --query 'Policy.Arn' --output text) \
&& echo ${ACCOUNT_B_POLICY_ARN}

```

- g. Attach the policy to the role by running the following command:

```

aws iam attach-role-policy \
    --role-name "${ACCOUNT_B_ROLE_NAME}" \
    --policy-arn "${ACCOUNT_B_POLICY_ARN}"

```

3. Configure the AWS Account A IAM roles and policies:

- a. Switch to your Account A profile by running the following command:

```

export AWS_DEFAULT_PROFILE=${AWS_ACCOUNT_A}

```

- b. Create the IAM policy document by running the following command:

```

cat << EOF > $SCRATCH_DIR/AssumeRoleInlinePolicyPolicyInAccountA.json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "sts:AssumeRole",
      "Resource": "${ACCOUNT_B_ROLE_ARN}"
    }
  ]
}
EOF

```

- c. In AWS Account A, attach the AWS-managed policy "AmazonElasticFileSystemClientFullAccess" to the OpenShift Container Platform cluster master role by running the following command:

```

EFS_CLIENT_FULL_ACCESS_BUILTIN_POLICY_ARN=arn:aws:iam::aws:policy/AmazonElasticFileSystemClientFullAccess

```

```

declare -A ROLE_SEEN
for NODE in $(oc get nodes --selector="${NODE_SELECTOR}" -o
jsonpath='{.items[*].metadata.name}'); do
  INSTANCE_PROFILE=$(aws ec2 describe-instances \
    --filters "Name=private-dns-name,Values=${NODE}" \
    --query 'Reservations[].Instances[].IamInstanceProfile.Arn' \
    --output text | awk -F'/' '{print $NF}' | xargs)
  MASTER_ROLE_ARN=$(aws iam get-instance-profile \
    --instance-profile-name "${INSTANCE_PROFILE}" \
    --query 'InstanceProfile.Roles[0].Arn' \
    --output text | xargs)
  MASTER_ROLE_NAME=$(echo "${MASTER_ROLE_ARN}" | awk -F'/' '{print $NF}' |
xargs)
  echo "Checking role: '${MASTER_ROLE_NAME}'"
  if [[ -n "${ROLE_SEEN[${MASTER_ROLE_NAME}]:-}" ]]; then
    echo "Already processed role: '${MASTER_ROLE_NAME}', skipping."
    continue
  fi
  ROLE_SEEN[${MASTER_ROLE_NAME}]=1
  echo "Assigning policy ${EFS_CLIENT_FULL_ACCESS_BUILTIN_POLICY_ARN} to
role ${MASTER_ROLE_NAME}"
  aws iam attach-role-policy --role-name "${MASTER_ROLE_NAME}" --policy-arn
"${EFS_CLIENT_FULL_ACCESS_BUILTIN_POLICY_ARN}"
done

```

4. Attach the policy to the IAM entity to allow role assumption:

This step depends on your cluster configuration. In both of the following scenarios, the EFS CSI Driver Operator uses an entity to authenticate to AWS, and this entity must be granted permission to assume roles in Account B.

If your cluster:

- **Does not have STS enabled** The EFS CSI Driver Operator uses an IAM User entity for AWS authentication. Continue with the step "Attach policy to IAM User to allow role assumption".
- **Has STS enabled:** The EFS CSI Driver Operator uses an IAM role entity for AWS authentication. Continue with the step "Attach policy to IAM Role to allow role assumption".

5. Attach policy to IAM User to allow role assumption

- Identify the IAM User used by the EFS CSI Driver Operator by running the following command:

```

EFS_CSI_DRIVER_OPERATOR_USER=$(oc -n openshift-cloud-credential-operator get
credentialsrequest/openshift-aws-efs-csi-driver -o json | jq -r '.status.providerStatus.user')

```

- Attach the policy to the IAM user by running the following command:

```

aws iam put-user-policy \
  --user-name "${EFS_CSI_DRIVER_OPERATOR_USER}" \
  --policy-name efs-cross-account-inline-policy \
  --policy-document
file://${SCRATCH_DIR}/AssumeRoleInlinePolicyPolicyInAccountA.json

```

6. Attach the policy to the IAM role to allow role assumption:

- a. Identify the IAM role name currently used by the EFS CSI Driver Operator by running the following command:

```
EFS_CSI_DRIVER_OPERATOR_ROLE=$(oc -n ${CSI_DRIVER_NAMESPACE} get
secret/aws-efs-cloud-credentials -o jsonpath='{.data.credentials}' | base64 -d | grep
role_arn | cut -d'/' -f2) && echo ${EFS_CSI_DRIVER_OPERATOR_ROLE}
```

- b. Attach the policy to the IAM role used by the EFS CSI Driver Operator by running the following command:

```
aws iam put-role-policy \
  --role-name "${EFS_CSI_DRIVER_OPERATOR_ROLE}" \
  --policy-name efs-cross-account-inline-policy \
  --policy-document
file://$SCRATCH_DIR/AssumeRoleInlinePolicyPolicyInAccountA.json
```

7. Configure VPC peering:

- a. Initiate a peering request from Account A to Account B by running the following command:

```
export AWS_DEFAULT_PROFILE=${AWS_ACCOUNT_A}
PEER_REQUEST_ID=$(aws ec2 create-vpc-peering-connection --vpc-id
"${AWS_ACCOUNT_A_VPC_ID}" --peer-vpc-id "${AWS_ACCOUNT_B_VPC_ID}" --
peer-owner-id "${AWS_ACCOUNT_B_ID}" --query
VpcPeeringConnection.VpcPeeringConnectionId --output text)
```

- b. Accept the peering request from Account B by running the following command:

```
export AWS_DEFAULT_PROFILE=${AWS_ACCOUNT_B}
aws ec2 accept-vpc-peering-connection --vpc-peering-connection-id
"${PEER_REQUEST_ID}"
```

- c. Retrieve the route table IDs for Account A and add routes to the Account B VPC by running the following command:

```
export AWS_DEFAULT_PROFILE=${AWS_ACCOUNT_A}
for NODE in $(oc get nodes --selector=node-role.kubernetes.io/worker | tail -n +2 | awk
'{print $1}')
do
  SUBNET=$(aws ec2 describe-instances --filters "Name=private-dns-
name,Values=$NODE" --query
'Reservations[*].Instances[*].NetworkInterfaces[*].SubnetId' | jq -r '.[0][0][0]')
  echo SUBNET is ${SUBNET}
  ROUTE_TABLE_ID=$(aws ec2 describe-route-tables --filters
"Name=association.subnet-id,Values=${SUBNET}" --query
'RouteTables[*].RouteTableId' | jq -r '.[0]')
  echo Route table ID is $ROUTE_TABLE_ID
  aws ec2 create-route --route-table-id ${ROUTE_TABLE_ID} --destination-cidr-block
${AWS_ACCOUNT_B_VPC_CIDR} --vpc-peering-connection-id
${PEER_REQUEST_ID}
done
```

- d. Retrieve the route table IDs for Account B and add routes to the Account A VPC by running the following command:

```
export AWS_DEFAULT_PROFILE=${AWS_ACCOUNT_B}
for ROUTE_TABLE_ID in $(aws ec2 describe-route-tables --filters "Name=vpc-id,Values=${AWS_ACCOUNT_B_VPC_ID}" --query "RouteTables[].RouteTableId" | jq -r '.[0]')
do
    echo Route table ID is $ROUTE_TABLE_ID
    aws ec2 create-route --route-table-id ${ROUTE_TABLE_ID} --destination-cidr-block ${AWS_ACCOUNT_A_VPC_CIDR} --vpc-peering-connection-id ${PEER_REQUEST_ID}
done
```

8. Configure security groups in Account B to allow NFS traffic from Account A to EFS:

- a. Switch to your Account B profile by running the following command:

```
export AWS_DEFAULT_PROFILE=${AWS_ACCOUNT_B}
```

- b. Configure the VPC security groups for EFS access by running the following command:

```
SECURITY_GROUP_ID=$(aws ec2 describe-security-groups --filters Name=vpc-id,Values="${AWS_ACCOUNT_B_VPC_ID}" | jq -r '.SecurityGroups[].GroupId')
aws ec2 authorize-security-group-ingress \
    --group-id "${SECURITY_GROUP_ID}" \
    --protocol tcp \
    --port 2049 \
    --cidr "${AWS_ACCOUNT_A_VPC_CIDR}" | jq .
```

9. Create a region-wide EFS filesystem in Account B:

- a. Switch to your Account B profile by running the following command:

```
export AWS_DEFAULT_PROFILE=${AWS_ACCOUNT_B}
```

- b. Create a region-wide EFS file system by running the following command:

```
CROSS_ACCOUNT_FS_ID=$(aws efs create-file-system --creation-token efs-token-1 \
    --region ${AWS_REGION} \
    --encrypted | jq -r '.FileSystemId') \
&& echo $CROSS_ACCOUNT_FS_ID
```

- c. Configure region-wide mount targets for EFS by running the following command:

```
for SUBNET in $(aws ec2 describe-subnets \
    --filters "Name=vpc-id,Values=${AWS_ACCOUNT_B_VPC_ID}" \
    --region ${AWS_REGION} \
    | jq -r '.Subnets[].SubnetId'); do \
    MOUNT_TARGET=$(aws efs create-mount-target --file-system-id ${CROSS_ACCOUNT_FS_ID} \
        --subnet-id ${SUBNET} \
        --region ${AWS_REGION} \
        | jq -r '.MountTargetId'); \
    echo ${MOUNT_TARGET}; \
done
```

This creates a mount point in each subnet of your VPC.

10. Configure the EFS Operator for cross-account access:

- a. Define custom names for the secret and storage class that you will create in subsequent steps by running the following command:

```
export SECRET_NAME=my-efs-cross-account
export STORAGE_CLASS_NAME=efs-sc-cross
```

- b. Create a secret that references the role ARN in Account B by running the following command in the OpenShift Container Platform CLI:

```
oc create secret generic ${SECRET_NAME} -n ${CSI_DRIVER_NAMESPACE} --from-
literal=awsRoleArn="${ACCOUNT_B_ROLE_ARN}"
```

- c. Grant the CSI driver controller access to the newly created secret by running the following commands in the OpenShift Container Platform CLI:

```
oc -n ${CSI_DRIVER_NAMESPACE} create role access-secrets --verb=get,list,watch --
resource=secrets
oc -n ${CSI_DRIVER_NAMESPACE} create rolebinding --role=access-secrets default-
to-secrets --serviceaccount=${CSI_DRIVER_NAMESPACE}:aws-efs-csi-driver-
controller-sa
```

- d. Create a new storage class that references the EFS ID from Account B and the secret created previously by running the following command in the OpenShift Container Platform CLI:

```
cat << EOF | oc apply -f -
kind: StorageClass
apiVersion: storage.k8s.io/v1
metadata:
  name: ${STORAGE_CLASS_NAME}
provisioner: efs.csi.aws.com
parameters:
  provisioningMode: efs-ap
  filesystemId: ${CROSS_ACCOUNT_FS_ID}
  directoryPerms: "700"
  gidRangeStart: "1000"
  gidRangeEnd: "2000"
  basePath: "/dynamic_provisioning"
  csi.storage.k8s.io/provisioner-secret-name: ${SECRET_NAME}
  csi.storage.k8s.io/provisioner-secret-namespace: ${CSI_DRIVER_NAMESPACE}
EOF
```

Additional resources

- [Setting the output format in the AWS CLI](#)

6.10.6. One Zone file systems

You can use AWS Elastic File System (EFS) One Zone file systems to store data redundantly within a single Availability Zone (AZ), offering a lower-cost storage option compared to multi-AZ regional storage.

6.10.6.1. One Zone file systems overview

AWS Elastic File System (EFS) One Zone contrasts with the default EFS storage option, which stores data redundantly across multiple AZs within a region.

Clusters upgraded from OpenShift Container Platform 4.19 are compatible with the regional EFS volumes.



NOTE

Dynamic provisioning of One Zone volumes is supported only in single-zone clusters. All nodes in the cluster must be in the same AZ as the EFS volume that is used for the dynamic provisioning.

Manually provisioned One Zone volumes in regional clusters is supported, assuming that the persistent volumes (PVs) have correct **spec.nodeAffinity** that indicates the zone that the volume is in.

For Cloud Credential Operator (CCO) Mint mode or Passthrough, no extra configuration is required. However, for Security Token Service (STS), use the procedure in *Section Setting up One Zone file systems with STS*.

6.10.6.2. Setting up One Zone file systems with STS

Configure separate credential requests and role ARNs for the controller and driver nodes to use AWS Elastic File System (EFS) One Zone file systems with Security Token Service (STS) authentication.

The following procedure explains how to set up AWS One Zone file systems with Security Token Service (STS).

Prerequisites

- Access to the cluster as a user with the cluster-admin role.
- AWS account credentials

Procedure

1. Create **two CredentialsRequests** in the **credrequests** directory following the procedure under *Section Obtaining a role Amazon Resource Name for Security Token Service* :
 - For the **controller CredentialsRequest**, follow the procedure without any changes.
 - For the **driver node CredentialsRequest** use the following example file:

Example CredentialsRequest YAML file for driver node

```
apiVersion: cloudcredential.openshift.io/v1
kind: CredentialsRequest
metadata:
  annotations:
```



```

credentials.openshift.io/role-arns-vars: NODE_ROLEARN
name: openshift-aws-efs-csi-driver-node
namespace: openshift-cloud-credential-operator
spec:
  providerSpec:
    apiVersion: cloudcredential.openshift.io/v1
    kind: AWSProviderSpec
    statementEntries:
      - action:
          - elasticfilesystem:DescribeMountTargets
          - ec2:DescribeAvailabilityZones
        effect: Allow
        resource: '*'
  secretRef:
    name: node-aws-efs-cloud-credentials
    namespace: openshift-cluster-csi-drivers
  serviceAccountNames:
    - aws-efs-csi-driver-node-sa

```

Set **metadata.annotations.credentials.openshift.io/role-arns-vars** to **NODE_ROLEARN**.

Example ccoctl output

```

2025/08/26 14:05:24 Role arn:aws:iam::269733383066:role/my-arn-1-bl1l6-openshift-
cluster-csi-drivers-aws-efs-cloud-cre created
2025/08/26 14:05:24 Saved credentials configuration to: /home/my-
arn/project/go/src/github.com/openshift/myinst/aws-sts-compact-1/manifests/openshift-
cluster-csi-drivers-aws-efs-cloud-credentials-credentials.yaml
2025/08/26 14:05:24 Updated Role policy for Role my-arn-1-bl1l6-openshift-cluster-csi-
drivers-aws-efs-cloud-cre
2025/08/26 14:05:24 Role arn:aws:iam::269733383066:role/my-arn-1-bl1l6-openshift-
cluster-csi-drivers-node-aws-efs-clou created
2025/08/26 14:05:24 Saved credentials configuration to: manifests/openshift-cluster-csi-
drivers-node-aws-efs-cloud-credentials-credentials.yaml
2025/08/26 14:05:24 Updated Role policy for Role my-arn-1-bl1l6-openshift-cluster-csi-
drivers-node-aws-efs-clou

```

In this example:

- The first line shows the Controller Amazon Resource Name (ARN).
 - The fifth line shows the Driver node ARN.
2. Install the AWS EFS CSI driver using the controller ARN created earlier in this procedure.
 3. Edit the operator's subscription and add **NODE_ROLEARN** with the driver node's ARN by running a command similar to the following:

```

$ oc -n openshift-cluster-csi-drivers edit subscription aws-efs-csi-driver-operator
...
config:
  env:
    - name: ROLEARN
      value: arn:aws:iam::269733383066:role/my-arn-1-bl1l6-openshift-cluster-csi-drivers-aws-
efs-cloud-cre

```

```
- name: NODE_ROLEARN
  value: arn:aws:iam::269733383066:role/my-arn-1-bl1l6-openshift-cluster-csi-drivers-node-aws-efs-clou
...
```

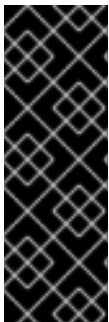
- **ROLEARN value** is the Controller ARN, which already exists.
- **NODE_ROLEARN value** is the Driver node ARN.

6.10.7. Dynamic provisioning for Amazon Elastic File Storage

To dynamically provision persistent volumes (PVs) as subdirectories of an existing Elastic File System (EFS) volume, create persistent volume claims (PVCs) referencing your EFS storage class to enable multiple independent volumes sharing the same resource.

The AWS EFS Container Storage Interface (CSI) driver supports a different form of dynamic provisioning than other CSI drivers. It provisions new PVs as subdirectories of a pre-existing EFS volume. The PVs are independent of each other. However, they all share the same EFS volume. When the volume is deleted, all PVs provisioned out of it are deleted too.

The EFS CSI driver creates an AWS Access Point for each such subdirectory. Due to AWS AccessPoint limits, you can only dynamically provision 1000 PVs from a single **StorageClass**/EFS volume.



IMPORTANT

Note that **PVC.spec.resources** is not enforced by EFS.

In the example below, you request 5 GiB of space. However, the created PV is limitless and can store any amount of data (like petabytes). A broken application, or even a rogue application, can cause significant expenses when it stores too much data on the volume.

Using monitoring of EFS volume sizes in AWS is strongly recommended.

If you have problems setting up dynamic provisioning, see *AWS EFS troubleshooting*.

Prerequisites

- You have created Amazon Elastic File Storage (Amazon EFS) volumes.
- You have created the AWS EFS storage class.

Procedure

- Create a PVC (or StatefulSet or Template) as usual, referring to the **StorageClass** created previously.

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: test
spec:
  storageClassName: efs-sc
  accessModes:
    - ReadWriteMany
resources:
```

```
requests:
  storage: 5Gi
```

Additional resources

- [AWS EFS CSI driver](#)
- [Creating the AWS EFS storage class](#)

6.10.8. Creating static PVs with Amazon Elastic File Storage

To mount an entire Amazon Elastic File Storage (EFS) volume as a single persistent volume without dynamic provisioning, create a static persistent volume (PV) that allows pods to access the full volume.

If you have problems setting up static PVs, see "AWS EFS troubleshooting".

Prerequisites

- You have created Amazon EFS volumes.

Procedure

- Create the PV using the following YAML file:

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: efs-pv
spec:
  capacity:
    storage: 5Gi
  volumeMode: Filesystem
  accessModes:
    - ReadWriteMany
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  csi:
    driver: efs.csi.aws.com
    volumeHandle: fs-ae66151a
    volumeAttributes:
      encryptInTransit: "false"
```

- **spec.capacity** does not have any meaning and is ignored by the CSI driver. It is used only when binding to a PVC. Applications can store any amount of data to the volume.
- **spec.csi.volumeHandle** must be the same ID as the EFS volume you created in AWS. If you are providing your own access point, **volumeHandle** should be **<EFS volume ID>::<access point ID>**. For example: **fs-6e633ada::fsap-081a1d293f0004630**.
- **spec.csi.volumeAttributes.encryptInTransit**: If desired, you can disable encryption in transit. Encryption is enabled by default.

6.10.9. Amazon Elastic File Storage security

When using Amazon Elastic File Storage (EFS) with access points, understand that file permissions are controlled by the access point rather than Kubernetes FSGroup settings, allowing any pod with access to read all files.

When using access points, for example, by using dynamic provisioning as described earlier, Amazon automatically replaces GIDs on files with the GID of the access point. In addition, EFS considers the user ID, group ID, and secondary group IDs of the access point when evaluating file system permissions. EFS ignores the NFS client's IDs. For more information about access points, see "Working with access points".

As a consequence, EFS volumes silently ignore FSGroup; OpenShift Container Platform is not able to replace the GIDs of files on the volume with FSGroup. Any pod that can access a mounted EFS access point can access any file on it.

Unrelated to this, encryption in transit is enabled by default. For more information, see "Encrypting data in transit".

Additional resources

- [Working with access points](#)
- [Encrypting data in transit](#)

6.10.10. AWS EFS storage CSI usage metrics

Amazon Web Services (AWS) Elastic File System (EFS) storage Container Storage Interface (CSI) usage metrics allow you to monitor how much space is used by either dynamically or statically provisioned EFS volumes.



IMPORTANT

This feature is disabled by default, because turning on metrics can lead to performance degradation.

The AWS EFS usage metrics feature collects volume metrics in the AWS EFS CSI Driver by recursively walking through the files in the volume. Because this effort can degrade performance, administrators must explicitly enable this feature.

6.10.10.1. Enabling or disabling usage metrics using the web console

To monitor Elastic File System (EFS) volume space consumption through the web console, enable usage metrics by configuring the **ClusterCSIDriver** resource with recursive walk parameters.

Prerequisites

- Access to an OpenShift Container Platform cluster with administrator rights

Procedure

1. Click **Administration** > **CustomResourceDefinitions**.
2. On the **CustomResourceDefinitions** page next to the **Name** dropdown box, type **clustercsidriver**.

3. Click **CRD ClusterCSIDriver**.
4. Click the **YAML** tab.
5. Under **spec.aws.efsVolumeMetrics.state**:
 - **Enable** metrics: Set the value to **RecursiveWalk**. RecursiveWalk` indicates that volume metrics collection in the AWS EFS CSI Driver is performed by recursively walking through the files in the volume.
 - **Disable** metrics: set the value to **Disabled**.

Example ClusterCSIDriver efs.csi.aws.com YAML file

```
spec:
  driverConfig:
    driverType: AWS
    aws:
      efsVolumeMetrics:
        state: RecursiveWalk
        recursiveWalk:
          refreshPeriodMinutes: 100
          fsRateLimit: 10
```

6. Optional: To define how the recursive walk operates, you can also set the following fields:
 - **refreshPeriodMinutes**: Specifies the refresh frequency for volume metrics in minutes. If this field is left blank, a reasonable default is chosen, which is subject to change over time. The current default is 240 minutes. The valid range is 1 to 43,200 minutes.
 - **fsRateLimit**: Defines the rate limit for processing volume metrics in goroutines per file system. If this field is left blank, a reasonable default is chosen, which is subject to change over time. The current default is 5 goroutines. The valid range is 1 to 100 goroutines.
7. Click **Save**.

6.10.10.2. Enabling or disabling usage metrics using the CLI

To monitor Elastic File System (EFS) volume space consumption through the command-line interface, enable usage metrics by configuring the **ClusterCSIDriver** resource with recursive walk parameters.

You can also disable usage metrics as needed.

Prerequisites

- Access to an OpenShift Container Platform cluster with administrator rights
- Access to OpenShift Container Platform CLI (**oc**)

Procedure

1. Edit **ClusterCSIDriver** by running the following command:

```
$ oc edit clustercsidriver efs.csi.aws.com
```

2. Under **spec.aws.efsVolumeMetrics.state**:

- **Enable** metrics: Set the value to **RecursiveWalk**. **RecursiveWalk** indicates that volume metrics collection in the AWS EFS CSI Driver is performed by recursively walking through the files in the volume.
- **Disable** metrics: set the value to **Disabled**.

Example ClusterCSIDriver efs.csi.aws.com YAML file

```
spec:
  driverConfig:
    driverType: AWS
    aws:
      efsVolumeMetrics:
        state: RecursiveWalk
        recursiveWalk:
          refreshPeriodMinutes: 100
          fsRateLimit: 10
```

3. Optional: To define how the recursive walk operates, you can also set the following fields:

- **refreshPeriodMinutes**: Specifies the refresh frequency for volume metrics in minutes. If this field is left blank, a reasonable default is chosen, which is subject to change over time. The current default is 240 minutes. The valid range is 1 to 43,200 minutes.
- **fsRateLimit**: Defines the rate limit for processing volume metrics in goroutines per file system. If this field is left blank, a reasonable default is chosen, which is subject to change over time. The current default is 5 goroutines. The valid range is 1 to 100 goroutines.

4. Save the changes to the **efs.csi.aws.com** object.**6.10.11. Amazon Elastic File Storage troubleshooting**

To diagnose and resolve AWS Elastic File System (EFS) issues, use these troubleshooting steps including gathering logs, checking operator status, and verifying network connectivity.

The following information provides guidance on how to troubleshoot issues with Amazon Elastic File Storage (Amazon EFS):

- The AWS EFS Operator and CSI driver run in namespace **openshift-cluster-csi-drivers**.
- To initiate gathering of logs of the AWS EFS Operator and CSI driver, run the following command:

```
$ oc adm must-gather
[must-gather ] OUT Using must-gather plugin-in image: quay.io/openshift-release-dev/ocp-v4.0-art-dev@sha256:125f183d13601537ff15b3239df95d47f0a604da2847b561151fedd699f5e3a5
[must-gather ] OUT namespace/openshift-must-gather-xm4wq created
[must-gather ] OUT clusterrolebinding.rbac.authorization.k8s.io/must-gather-2bd8x created
[must-gather ] OUT pod for plug-in image quay.io/openshift-release-dev/ocp-v4.0-art-dev@sha256:125f183d13601537ff15b3239df95d47f0a604da2847b561151fedd699f5e3a5 created
```

- To show AWS EFS Operator errors, view the **ClusterCSIDriver** status:

```
$ oc get clustercsidriver efs.csi.aws.com -o yaml
```

- If a volume cannot be mounted to a pod (as shown in the output of the following command):

```
$ oc describe pod
...
Type      Reason      Age   From          Message
----      -
Normal    Scheduled    2m13s default-scheduler Successfully assigned default/efs-app to ip-10-0-135-94.ec2.internal
Warning   FailedMount  13s   kubelet        MountVolume.SetUp failed for volume "pvc-d7c097e6-67ec-4fae-b968-7e7056796449" : rpc error: code = DeadlineExceeded desc = context deadline exceeded
Warning   FailedMount  10s   kubelet        Unable to attach or mount volumes: unmounted volumes=[persistent-storage], unattached volumes=[persistent-storage kube-api-access-9j477]: timed out waiting for the condition
```

The text **error: code = DeadlineExceeded desc = context deadline exceeded** is a warning message indicating that the volume is not mounted.

This error is frequently caused by AWS dropping packets between an OpenShift Container Platform node and Amazon EFS.

Check that the following are correct:

- AWS firewall and Security Groups
- Networking: port number and IP addresses

6.10.12. Uninstalling the AWS EFS CSI Driver Operator

To remove the AWS EFS Container Storage Interface (CSI) Driver Operator and free cluster resources, uninstall the operator after stopping applications and deleting persistent volumes (PVs).

All EFS PVs are inaccessible after uninstalling the AWS EFS CSI Driver Operator (a Red Hat operator).



NOTE

Before you can destroy a cluster (**openshift-install destroy cluster**), you must delete the EFS volume in AWS. An OpenShift Container Platform cluster cannot be destroyed when there is an EFS volume that uses the cluster's VPC. Amazon does not allow deletion of such a VPC.

Prerequisites

- Access to the OpenShift Container Platform web console.

Procedure

1. Log in to the web console.
2. Stop all applications that use AWS EFS PVs.

3. Delete all AWS EFS PVs:
 - a. Click **Storage** → **PersistentVolumeClaims**.
 - b. Select each PVC that is in use by the AWS EFS CSI Driver Operator, click the drop-down menu on the far right of the PVC, and then click **Delete PersistentVolumeClaims**.
4. Uninstall the AWS EFS CSI driver:

**NOTE**

Before you can uninstall the Operator, you must remove the CSI driver first.

- a. Click **Administration** → **CustomResourceDefinitions** → **ClusterCSIDriver**.
 - b. On the **Instances** tab, for **efs.csi.aws.com**, on the far left side, click the drop-down menu, and then click **Delete ClusterCSIDriver**.
 - c. When prompted, click **Delete**.
5. Uninstall the AWS EFS CSI Operator:
 - a. Click **Ecosystem** → **Installed Operators**.
 - b. On the **Installed Operators** page, scroll or type AWS EFS CSI into the **Search by name** box to find the Operator, and then click it.
 - c. On the upper, right of the **Installed Operators > Operator details** page, click **Actions** → **Uninstall Operator**.
 - d. When prompted on the **Uninstall Operator** window, click the **Uninstall** button to remove the Operator from the namespace. Any applications deployed by the Operator on the cluster need to be cleaned up manually.
After uninstalling, the AWS EFS CSI Driver Operator is no longer listed in the **Installed Operators** section of the web console.

6.10.13. Additional resources

- [Configuring CSI volumes](#)

6.11. AZURE DISK CSI DRIVER OPERATOR

You can provision and manage Azure Disk storage in OpenShift Container Platform by using the Azure Disk Container Storage Interface (CSI) Driver Operator and driver, which provide dynamic volume provisioning and eliminate the need to pre-provision storage.

6.11.1. Overview of Azure Disk CSI Driver Operator

OpenShift Container Platform is capable of provisioning persistent volumes (PVs) using the Container Storage Interface (CSI) driver for Microsoft Azure Disk Storage.

Familiarity with persistent storage and configuring CSI volumes is recommended when working with a CSI Operator and driver. For more information about these topics, see "Understanding persistent storage" and "Configuring CSI volumes".

To create CSI-provisioned PVs that mount to Azure Disk storage assets, OpenShift Container Platform installs the Azure Disk CSI Driver Operator and the Azure Disk CSI driver by default in the **openshift-cluster-csi-drivers** namespace.

Azure Disk CSI Driver Operator

The Azure Disk CSI Driver Operator provides a storage class named **managed-csi** that you can use to create persistent volume claims (PVCs). The Azure Disk CSI Driver Operator supports dynamic volume provisioning by allowing storage volumes to be created on-demand, eliminating the need for cluster administrators to pre-provision storage. You can disable this default storage class if desired (see "Managing the default storage class").

Azure Disk CSI driver

The Azure Disk CSI driver enables you to create and mount Azure Disk PVs.



NOTE

OpenShift Container Platform provides automatic migration for the Azure Disk in-tree volume plugin to its equivalent CSI driver. For more information, see "CSI automatic migration".

Additional resources

- [Understanding persistent storage](#)
- [Configuring CSI volumes](#)
- [Managing the default storage class](#)
- [CSI automatic migration](#)

6.11.2. About CSI

The Container Storage Interface (CSI) enables storage vendors to deliver plugins through a standard interface without modifying Kubernetes core code, replacing traditional embedded storage drivers.

CSI Operators give OpenShift Container Platform users storage options, such as volume snapshots, that are not possible with in-tree volume plugins.

6.11.3. Creating a storage class with storage account type

To provision persistent volumes with specific performance and redundancy characteristics, create a storage class that designates an Azure storage account type corresponding to your SKU tier.

Storage classes are used to differentiate and delineate storage levels and usages. By defining a storage class, you can obtain dynamically provisioned persistent volumes.

When creating a storage class, you can designate the storage account type. This corresponds to your Azure storage account SKU tier. Valid options are **Standard_LRS**, **Premium_LRS**, **StandardSSD_LRS**, **UltraSSD_LRS**, **Premium_ZRS**, and **StandardSSD_ZRS**. For information about finding your Azure SKU tier, see "SKU Types".

Both zone-redundant storage (ZRS) and PremiumV2_LRS have some region limitations. For information about these limitations, see "ZRS limitations" and "Premium_LRS limitations".

Prerequisites

- Access to an OpenShift Container Platform cluster with administrator rights

Procedure

1. Create a storage class designating the storage account type using a YAML file similar to the following:

```
$ oc create -f - << EOF
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: <storage-class>
provisioner: disk.csi.azure.com
parameters:
  skuName: <storage-class-account-type>
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true
EOF
```

- **metadata.name:** Specifies the storage class name.
- **parameters.skuName:** The storage account type. This corresponds to your Azure storage account SKU tier: `Standard_LRS``, **Premium_LRS**, **StandardSSD_LRS**, **UltraSSD_LRS**, **Premium_ZRS**, **StandardSSD_ZRS**, **PremiumV2_LRS**.



NOTE

For PremiumV2_LRS, specify **cachingMode: None** in **storageclass.parameters**.

2. Ensure that the storage class was created by listing the storage classes:

```
$ oc get storageclass
```

Example output

```
NAME                PROVISIONER          RECLAIMPOLICY  VOLUMEBINDINGMODE  ALLOWVOLUMEEXPANSION  AGE
azurefile-csi        file.csi.azure.com   Delete         Immediate          true                  68m
managed-csi (default) disk.csi.azure.com   Delete         WaitForFirstConsumer true                  68m
sc-prem-zrs          disk.csi.azure.com   Delete         WaitForFirstConsumer true                  4m25s
```

In this example, **sc-prem-zrs** is the new storage class with storage account type.

Additional resources

- [SKU Types](#)
- [ZRS limitations](#)

- [Premium_LRS limitations](#)

6.11.4. Performance plus for Azure Disk

You can enhance Azure disk performance by enabling performance plus to increase IOPS and throughput limits for certain Azure disk types 513 GiB, and larger.

6.11.4.1. Overview of performance plus

Performance plus increases Input/Output Operations Per Second (IOPS) and throughput limits for certain Azure disk types 513 GiB, and larger.

The following Azure disk types support performance plus:

- Azure Premium solid-state drives (SSD)
- Standard SSDs
- Standard hard disk drives (HDD)

To see what the increased limits are for IOPS and throughput, consult the columns that begin with **Expanded** in the tables in "Scalability and performance targets for VM disks".

Additional resources

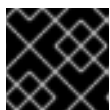
- [Scalability and performance targets for VM disks](#)

6.11.4.2. Limitations for performance plus

To successfully enable performance plus, verify that your disk configuration meets the required type, size, and provisioning criteria before attempting to use this feature.

Performance plus for Azure Disk has the following limitations:

- Can be enabled only on Standard HDD, Standard SSD, and Premium SSD managed disks that are 513 GiB or larger.



IMPORTANT

If you request a smaller value, the disk size is rounded up to 513GiB.

- Can be enabled only on new disks. For a workaround, see "Enabling performance plus by snapshot or cloning".

Additional resources

- [Enabling performance plus by snapshot or cloning](#)

6.11.4.3. Creating a storage class to use performance plus enhanced disks

To provision Azure disks with enhanced IOPS and throughput, create a storage class with performance plus enabled that automatically applies to persistent volume claims.

Prerequisites

- Access to a Microsoft Azure cluster with cluster-admin privileges.
- Access to an Azure disk with performance plus enabled.
For information about enabling performance plus on disks, see the "Microsoft Azure storage documentation".

Procedure

1. Create a storage class using the following example YAML file:

Example storage class YAML file

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: <azure-disk-performance-plus-sc>
provisioner: disk.csi.azure.com
parameters:
  skuName: Premium_LRS
  cachingMode: ReadOnly
  enablePerformancePlus: "true"
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true
```

- **metadata.name:** Specifies the name of the storage class.
 - **provisioner:** Specifies the Azure Disk Container Storage Interface (CSI) driver provisioner.
 - **parameters.skuName:** Specifies the Azure disk type SKU. In this example, **Premium_LRS** for Premium SSD Locally Redundant Storage.
 - **parameters.enablePerformancePlus:** Enables Azure Disk performance plus.
2. Create a persistent volume claim (PVC) that uses this storage class by using the following example YAML file:

Example PVC YAML file

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: <my-azure-pvc>
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: <azure-disk-performance-plus-sc>
  resources:
    requests:
      storage: 513Gi
```

- **metadata.name:** Specifies the PVC name.
- **spec.storageClassName:** References the performance plus storage class.

- **spec.resources.requests.storage**: Any disk size smaller than 513GiB is automatically rounded up.

6.11.4.4. Enabling performance plus by snapshot or cloning

To work around the limitation that performance plus applies only to new disks, snapshot or clone an existing volume to provision a new disk with performance plus enabled.

Normally, performance plus can be enabled only on new disks. For a workaround, you can use this procedure.

Prerequisites

- Access to a Microsoft Azure cluster with cluster-admin privileges.
- Access to an Azure disk with performance plus enabled.
- Have created a storage class to use performance plus enhanced Azure disks.
For more information about creating the storage class, see "Creating a storage class to use performance plus enhanced disks".

Procedure

1. Do one of the following to enable performance plus:
 - Create a snapshot of the existing volume that does not have performance plus enabled on it, and then provision a new disk from that snapshot using a storage class with **enablePerformancePlus** set to "true".
 - Clone the persistent volume claim (PVC) using a storage class with **enablePerformancePlus** set to "true" to create a new disk clone.

Additional resources

- [Creating a storage class to use performance plus enhanced disks](#)

6.11.5. User-managed encryption

The user-managed encryption feature allows you to provide keys during installation that encrypt OpenShift Container Platform node root volumes, and enables all managed storage classes to use these keys to encrypt provisioned storage volumes.

You must specify the custom key in the **platform.<cloud_type>.defaultMachinePlatform** field in the install-config YAML file.

This features supports the following storage types:

- Amazon Web Services (AWS) Elastic Block storage (EBS)

**NOTE**

If there is no encrypted key defined in the storage class, only set **encrypted: "true"** in the storage class. The AWS EBS CSI driver uses the AWS managed alias/`aws/ebs`, which is created by Amazon EBS automatically in each region by default to encrypt provisioned storage volumes. In addition, the managed storage classes all have the **encrypted: "true"** setting.

For information about installing AWS EBS with user-managed encryption, see "Optional AWS configuration parameters".

- Microsoft Azure Disk storage

**NOTE**

If the OS (root) disk is encrypted, and there is no encrypted key defined in the storage class, Azure Disk CSI driver uses the OS disk encryption key by default to encrypt provisioned storage volumes.

For information about installing Azure Disk with user-managed encryption, see "Preparing an Azure Disk Encryption Set".

- Google Cloud Platform (GCP) persistent disk (PD) storage
For information about installing GCP PD with user-managed encryption, see "Additional Google Cloud configuration parameters".
- IBM Cloud® Virtual Private Cloud (VPC) Block storage
For information about installing with IBM Cloud with user-managed encryption, see "User-managed encryption for IBM Cloud" and "Installing on IBM Cloud".

Additional resources

- [Preparing an Azure Disk Encryption Set](#)

6.11.6. Machine sets that deploy machines with ultra disks using PVCs

You can create a machine set running on Microsoft Azure that deploys machines with ultra disks. Ultra disks are high-performance storage that are intended for use with the most demanding data workloads.

Both the in-tree plugin and CSI driver support using PVCs to enable ultra disks. You can also deploy machines with ultra disks as data disks without creating a PVC.

Additional resources

- [Microsoft Azure ultra disks documentation](#)
- [Machine sets that deploy machines on ultra disks using in-tree PVCs](#)
- [Machine sets that deploy machines on ultra disks as data disks](#)

6.11.6.1. Creating machines with ultra disks by using machine sets

You can deploy machines with ultra disks on Microsoft Azure by editing your machine set YAML file.

Prerequisites

- Have an existing Microsoft Azure cluster.

Procedure

1. Copy an existing Azure **MachineSet** custom resource (CR) and edit it by running the following command:

```
$ oc edit machineset <machine_set_name>
```

where:

<machine_set_name>

Indicates the machine set that you want to provision machines with ultra disks.

2. Add the following lines in the positions indicated:

```
apiVersion: machine.openshift.io/v1beta1
kind: MachineSet
spec:
  template:
    spec:
      metadata:
        labels:
          disk: ultrassd
      providerSpec:
        value:
          ultraSSDCapability: Enabled
```

where:

spec.template.spec.metadata.labels.disk

Specifies a label to use to select a node that is created by this machine set. The example uses **disk.ultrassd** for this value.

spec.template.spec.providerSpec.value.ultraSSDCapability

Enables the use of ultra disks.

3. Create a machine set by using the updated configuration by running the following command:

```
$ oc create -f <machine_set_name>.yaml
```

4. Create a storage class that contains the following YAML definition:

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: ultra-disk-sc
parameters:
  cachingMode: None
  diskIopsReadWrite: "2000"
  diskMbpsReadWrite: "320"
  kind: managed
  skuname: UltraSSD_LRS
```

```

provisioner: disk.csi.azure.com
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer

```

where:

metadata.name

Specifies the name of the storage class. The example uses **ultra-disk-sc** for this value.

parameters.diskIopsReadWrite

Specifies the number of Input/Output Operations Per Second (IOPS) for the storage class.

parameters.diskMbpsReadWrite

Specifies the throughput in MBps for the storage class.

provisioner

For Microsoft Azure Kubernetes Service (AKS) version 1.21 or later, use **disk.csi.azure.com**.
For earlier versions of AKS, use **kubernetes.io/azure-disk**.

volumeBindingMode

Optional parameter. Specifies this parameter to wait for the creation of the pod that will use the disk.

5. Create a persistent volume claim (PVC) to reference the **ultra-disk-sc** storage class that contains the following YAML definition:

```

apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: ultra-disk
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: ultra-disk-sc
  resources:
    requests:
      storage: 4Gi

```

where:

metadata.name

Specifies the name of the PVC. The example uses **ultra-disk** for this value.

spec.storageClassName

Specifies the name of the storage class to use. The example uses **ultra-disk-sc** storage class.

spec.resources.requests.storage

Specifies the size for the storage class. The minimum value is **4Gi**.

6. Create a pod that contains the following YAML definition:

```

apiVersion: v1
kind: Pod
metadata:
  name: nginx-ultra

```



```
spec:
  nodeSelector:
    disk: ultrasssd
  containers:
  - name: nginx-ultra
    image: alpine:latest
    command:
    - "sleep"
    - "infinity"
  volumeMounts:
  - mountPath: "/mnt/azure"
    name: volume
  volumes:
  - name: volume
    persistentVolumeClaim:
      claimName: ultra-disk
```

where:

spec.nodeSelector.disk

Specifies the label of the machine set that enables the use of ultra disks. The example uses **disk.ultrasssd** for this value.

spec.volumes.persistentVolumeClaim.claimName

Specifies the name of the PVC to attach. This pod references the **ultra-disk** PVC.

Verification

1. Validate that the machines are created by running the following command:

```
$ oc get machines
```

The machines should be in the **Running** state.

2. For a machine that is running and has a node attached, validate the partition by running the following command:

```
$ oc debug node/<node_name> -- chroot /host lsblk
```

In this command, **oc debug node/<node_name>** starts a debugging shell on the node **<node_name>** and passes a command with **--**. The passed command **chroot /host** provides access to the underlying host OS binaries, and **lsblk** shows the block devices that are attached to the host OS machine.

Next steps

- To use an ultra disk from within a pod, create a workload that uses the mount point. Create a YAML file similar to the following example:

```
apiVersion: v1
kind: Pod
metadata:
  name: ssd-benchmark1
spec:
  containers:
```

```

- name: ssd-benchmark1
  image: nginx
  ports:
    - containerPort: 80
      name: "http-server"
  volumeMounts:
    - name: lun0p1
      mountPath: "/tmp"
  volumes:
    - name: lun0p1
      hostPath:
        path: /var/lib/lun0p1
        type: DirectoryOrCreate
  nodeSelector:
    disktype: ultrasd

```

6.11.6.2. Troubleshooting resources for machine sets that enable ultra disks

You can recover from issues that you might encounter when you enable ultra disks for machine sets. Review fields, such as disk settings, and ensure that the parameters are correctly configured.

6.11.6.2.1. Unable to mount a persistent volume claim backed by an ultra disk

If there is an issue mounting a persistent volume claim backed by an ultra disk, the pod becomes stuck in the **ContainerCreating** state and an alert is triggered.

For example, if the **additionalCapabilities.ultraSSDEnabled** parameter is not set on the machine that backs the node that hosts the pod, the following error message appears:

StorageAccountType UltraSSD_LRS can be used only when additionalCapabilities.ultraSSDEnabled is set.

- To resolve this issue, describe the pod by running the following command:

```
$ oc -n <stuck_pod_namespace> describe pod <stuck_pod_name>
```

6.11.7. Additional resources

- [Persistent storage using Azure Disk](#)
- [Configuring CSI volumes](#)
- [Microsoft Azure storage documentation](#)

6.12. AZURE FILE CSI DRIVER OPERATOR

You can provision and manage Azure File storage in OpenShift Container Platform by using the Azure File Container Storage Interface (CSI) Driver Operator and driver, which provide dynamic volume provisioning and eliminate the need to pre-provision storage.

6.12.1. Overview

OpenShift Container Platform is capable of provisioning persistent volumes (PVs) by using the Container Storage Interface (CSI) driver for Microsoft Azure File Storage.

Familiarity with persistent storage and configuring CSI volumes is recommended when working with a CSI Operator and driver. For more information, see "Understanding persistent storage" and "Configuring CSI volumes".

To create CSI-provisioned PVs that mount to Azure File storage assets, OpenShift Container Platform installs the Azure File CSI Driver Operator and the Azure File CSI driver by default in the **openshift-cluster-csi-drivers** namespace.

Azure File CSI Driver Operator

The Azure File CSI Driver Operator provides a storage class that is named **azurefile-csi** that you can use to create persistent volume claims (PVCs). You can disable this default storage class if desired (see "Managing the default storage").

Azure File CSI driver

The Azure File CSI driver enables you to create and mount Azure File PVs. The Azure File CSI driver supports dynamic volume provisioning by allowing storage volumes to be created on-demand, eliminating the need for cluster administrators to pre-provision storage.

Azure File CSI Driver Operator does not support the following:

- Virtual hard disks (VHD)
- Running on nodes with Federal Information Processing Standard (FIPS) mode enabled for Server Message Block (SMB) file share. However, Network File System (NFS) does support FIPS mode.

For more information about supported features, see "Supported CSI drivers and features".

Additional resources

- [Understanding persistent storage](#)
- [Configuring CSI volumes](#)
- [Managing the default storage class](#)
- [Supported CSI drivers and features](#)

6.12.2. About CSI

The Container Storage Interface (CSI) enables storage vendors to deliver plugins through a standard interface without modifying Kubernetes core code, replacing traditional embedded storage drivers.

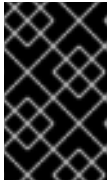
CSI Operators give OpenShift Container Platform users storage options, such as volume snapshots, that are not possible with in-tree volume plugins.

6.12.3. NFS support

OpenShift Container Platform supports the Azure File Container Storage Interface (CSI) Driver Operator with Network File System (NFS).

The following restrictions apply:

- If you create a volume smaller than 100GiB, the CSI driver rounds it up to 100GiB.
- Creating pods with Azure File NFS volumes that are scheduled to the control plane node causes the mount to be denied.
To work around this issue: If your control plane nodes are schedulable, and the pods can run on worker nodes, use **nodeSelector** or Affinity to schedule the pod in worker nodes.
- FS Group policy behavior:



IMPORTANT

Azure File CSI with NFS does not honor the `fsGroupChangePolicy` requested by pods. Azure File CSI with NFS applies a default `OnRootMismatch` FS Group policy regardless of the policy requested by the pod.

- The Azure File CSI Operator does not automatically create a storage class for NFS. You must create it manually. Use a file similar to the following:

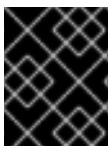
Example Azure File storage class YAML file

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: <storage-class-name>
provisioner: file.csi.azure.com
parameters:
  protocol: nfs
  skuName: Premium_LRS # available values: Premium_LRS, Premium_ZRS
mountOptions:
  - nconnect=4
```

- **metadata.name:** Specifies the storage class name.
- **provisioner:** Specifies the Azure File CSI provider.
- **parameters.protocol:** Specifies NFS as the storage backend protocol.

6.12.4. Azure File cross-subscription support

Cross-subscription support allows you to have an OpenShift Container Platform cluster in one Azure subscription and mount your Azure file share in another Azure subscription by using the Azure File Container Storage Interface (CSI) driver.



IMPORTANT

Both the OpenShift Container Platform cluster and the Azure File share (pre-provisioning or to be provisioned) should be inside the same tenant.

6.12.4.1. Dynamic provisioning across subscriptions for Azure File

Enable Azure File dynamic provisioning across Azure subscriptions by granting the cluster's Azure identity access to a storage account in a different subscription, then creating a storage class that references the target subscription.

Prerequisites

- Installed OpenShift Container Platform cluster on Azure with the service principal or managed identity as an Azure identity in one subscription (call it Subscription A)
- Access to another subscription (call it Subscription B) with the storage that is in the same tenant as the cluster
- Logged in to the Azure CLI

Procedure

1. Record the Azure identity (service principal or managed identity) by running the following applicable commands. The Azure identity is needed in a later step:

- If using the *service principal* as the Azure identity when installing the cluster:

```
$ sp_id=$(oc -n openshift-cluster-csi-drivers get secret azure-file-credentials -o
jsonpath='{.data.azure_client_id}' | base64 --decode)
```

```
$ az ad sp show --id ${sp_id} --query displayName --output tsv
```

- If using the *managed identity* as the Azure identity when installing the cluster:

```
$ mi_id=$(oc -n openshift-cluster-csi-drivers get secret azure-file-credentials -o
jsonpath='{.data.azure_client_id}' | base64 --decode)
```

```
$ az identity list --query "[?clientId=='${mi_id}'].{Name:name}" --output tsv
```

2. Grant the Azure identity (service principal or managed identity) permission to access the resource group in another Subscription B where you want to provision the Azure File share by doing one of the following:

- Run the following Azure CLI command:

```
az role assignment create \
  --assignee <object-id-or-app-id> \
  --role <role-name> \
  --scope /subscriptions/<subscription-id>/resourceGroups/<resource-
group>/providers/Microsoft.Storage/storageAccounts/<storage-account-name>
```

- **<object-id-or-app-id>**: Specifies the service principal or managed identity that you obtained from the previous step, such as **sp_id** or **mi_id**.
 - **<role-name>**: Specifies the role name. Contributor or your own role with required permissions.
 - **<subscription-id>**: Subscription B ID.
 - **<resource-group-name>**: Subscription B resource group name.
- Or

- Log in to the Azure portal and on the left menu, click **Resource groups**:

- a. Choose the resource group in Subscription B to which you want to assign a role by clicking **resource group** → **Access control (IAM)** → **Role assignments** tab to view current assignments, and then click **Add** > **Add role assignment**.
- b. On the **Role** tab, choose the contributor role to assign, and then click **Next**. You can also create and choose your own role with required permission.
- c. On the **Members** tab:
 - i. Choose an assignee by selecting the type of assignee: user, group, or service principal (or managed identity).
 - ii. Click **Select members**.
 - iii. Search for, and then select the desired service principal or managed identity recorded in the previous step.
 - iv. Click **Select** to confirm.
- d. On the **Review + assign** tab, review the settings.
- e. To finish the role assignment, click **Review + assign**.



NOTE

If you only want to use a specific storage account to provision the Azure File share, you can also obtain the Azure identity (service principal or managed identity) permission to access the storage account by using similar steps.

3. Create an Azure File storage class by using a similar configuration to the following:

Example Azure File storage class YAML file

```
allowVolumeExpansion: true
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: <sc-name>
mount options:
  - mfsymlinks
  - cache=strict
  - nosharesock
  - actimeo=30
parameters:
  subscriptionID: <xxxx-xxxx-xxxx-xxxx-xxxx>
  resourceGroup: <resource group name>
  storageAccount: <storage account>
  skuName: <skuName>
provisioner: file.csi.azure.com
reclaimPolicy: Delete
volumeBindingMode: Immediate
```

- **metadata.name:** Specifies the name of the storage class.
- **parameters.subscriptionID:** Specifies the subscription B ID.

- **parameters.resourceGroup**: Specifies the Subscription B resource group name.
 - **parameters.storageAccount**: Specifies the storage account name, if you want to specify your own.
 - **parameters.skuName**: Specifies the name of the SKU type.
4. Create a persistent volume claim (PVC) that specifies the Azure File storage class that you created in the previous step by using a similar configuration to the following:

Example PVC YAML file

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: <pvc-name>
spec:
  storageClassName: <sc-name-cross-sub>
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 5Gi
```

- **metadata.name**: Specifies the name of the PVC.
- **spec.storageClassName**: Specifies the name of the storage class that you created in the previous step.

6.12.4.2. Static provisioning across subscriptions for Azure File by creating a PV and PVC:

Provision Azure File storage across subscriptions using static provisioning by creating a secret with storage credentials, then creating a persistent volume (PV) and persistent volume claim (PVC) referencing an Azure File share in a different subscription.

Recommendation to use a storage class

In the following example of static provisioning across subscriptions, the storage class referenced in the PV and PVC is not strictly necessary, as storage classes are not required to accomplish static provisioning. However, it is advisable to use a storage class to avoid cases where a manually created PVC accidentally does not match a manually created PV, and thus potentially triggers dynamic provisioning of a new PV. Other ways to avoid this issue would be to create a storage class with **provisioner: kubernetes.io/no-provisioner** or reference a non-existing storage class, which in both cases ensures that dynamic provisioning does not occur. When using either of these strategies, if a mis-matched PV and PVC occurs, the PVC stays in a pending state, and you can correct the error.

Prerequisites

- Installed OpenShift Container Platform cluster on Azure with the service principal or managed identity as an Azure identity in one subscription (call it Subscription A)
- Access to another subscription (call it Subscription B) with the storage that is in the same tenant as the cluster
- Logged in to the Azure CLI

Procedure

1. For your Azure File share, record the resource group, storage account, storage account key, and Azure File name. These values are used for the next steps.
2. Create a secret for the persistent volume parameter **spec.csi.nodeStageSecretRef.name** by running the following command:

```
$ oc create secret generic azure-storage-account-<storageaccount-name>-secret --from-literal=azurestorageaccountname="<azure-storage-account-name>" --from-literal=azurestorageaccountkey="<azure-storage-account-key>" --type=Opaque
```

<azure-storage-account-name> and **<azure-storage-account-key>** are the Azure storage account name and key respectively that you recorded in Step 1.

3. Create a persistent volume (PV) by using a similar configuration to the following example file:

Example PV YAML file

```
apiVersion: v1
kind: PersistentVolume
metadata:
  annotations:
    pv.kubernetes.io/provisioned-by: file.csi.azure.com
  name: <pv-name>
spec:
  capacity:
    storage: 10Gi
  accessModes:
    - ReadWriteMany
  persistentVolumeReclaimPolicy: Retain
  storageClassName: <sc-name>
  mountOptions:
    - cache=strict
    - nosharesock
    - actimeo=30
    - nobrl
  csi:
    driver: file.csi.azure.com
    volumeHandle: "{resource-group-name}#{storage-account-name}#{file-share-name}"
    volumeAttributes:
      shareName: <existing-file-share-name>
    nodeStageSecretRef:
      name: <secret-name>
      namespace: <secret-namespace>
```

- **metadata.name:** Specifies the name of the PV.
- **spec.capacity.storage:** Specifies the size of the PV.
- **spec.storageClassName:** Specifies the storage class name.
- **spec.csi.volumeHandle:** Specifies the **volumeHandle** parameter. Ensure that **volumeHandle** is unique for every identical share in the cluster.

- **spec.csi.volumeAttributes.shareName:** For `<existing-file-share-name>`, use only the file share name and not the full path.
 - **spec.csi.nodeStageSecretRef.name:** Specifies the secret name created in the previous step.
 - **spec.csi.nodeStageSecretRef.namespace:** Specifies the namespace where the secret resides.
4. Create a persistent volume claim (PVC) specifying the existing Azure File share referenced in Step 1 using a similar configuration to the following:

Example PVC YAML file

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: <pvc-name>
spec:
  storageClassName: <sc-name>
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 5Gi
```

- **metadata.name:** Specifies the name of the PVC.
- **spec.storageClassName:** Specifies the name of the storage class that you specified for the PV in the previous step.

6.12.5. Static provisioning for Azure File

Use static provisioning to manually create persistent volumes (PVs) for existing Azure File shares. Create a secret with storage credentials, define a PV that references the share, and create a persistent volume claim (PVC) to consume the storage.

Prerequisites

- Access to an OpenShift Container Platform cluster with administrator rights

Procedure

1. If you have not yet created a secret for the Azure storage account, create it now:
This secret must contain the Azure Storage Account name and key with the following very specific format with two key-value pairs:
 - **azurestorageaccountname:** `<storage_account_name>`
 - **azurestorageaccountkey:** `<account_key>`
 To create a secret named **azure-secret**, run the following command:

```
oc create secret generic azure-secret -n <namespace_name> --type=Opaque --from-
literal=azurestorageaccountname="<storage_account_name>" --from-
literal=azurestorageaccountkey="<account_key>"
```

- Set **<namespace_name>** to the namespace where the PV is consumed.
- Provide your values for **<storage_account_name>** and **<account_key>**.

2. Create a PV by using the following example YAML file:

Example PV YAML file

```
apiVersion: v1
kind: PersistentVolume
metadata:
  annotations:
    pv.kubernetes.io/provisioned-by: file.csi.azure.com
  name: pv-azurefile
spec:
  capacity:
    storage: 5Gi
  accessModes:
    - ReadWriteMany
  persistentVolumeReclaimPolicy: Retain
  storageClassName: <sc-name>
  mountOptions:
    - dir_mode=0777
    - file_mode=0777
    - uid=0
    - gid=0
    - cache=strict
    - nosharesock
    - actimeo=30
    - nobrl
  csi:
    driver: file.csi.azure.com
    volumeHandle: "{resource-group-name}#{account-name}#{file-share-name}"
    volumeAttributes:
      shareName: EXISTING_FILE_SHARE_NAME
    nodeStageSecretRef:
      name: azure-secret
      namespace: <my-namespace>
```

- **spec.capacity.storage:** Specifies the Volume size.
- **spec.accessModes:** Defines the read/write and mount permissions. For more information, see "Access modes".
- **spec.persistentVolumeReclaimPolicy:** Specifies the Reclaim policy, which tells the cluster what to do with the volume after it is released. Accepted values are **Retain**, **Recycle**, or **Delete**.
- **spec.storageClassName:** Specifies the storage class name. This name is used by the PVC to bind to this specific PV. For static provisioning, a **StorageClass** object does not need to exist, but the name in the PV and PVC must match.

- **spec.mountOptions.dir_mode=0777**: Modify this permission if you want to enhance the security.
- **spec.mountOptions.cache**: Specifies the cache mode. Accepted values are **none**, **strict**, and **loose**. The default is **strict**.
- **spec.mountOptions.no_sharesock**: Use to reduce the probability of a reconnect race.
- **spec.mountOptions.actimeo**: Specifies the time (in seconds) that the CIFS client caches attributes of a file or directory before it requests attribute information from a server.
- **spec.mountOptions.nobrl**: Disables sending byte range lock requests to the server, and for applications which have challenges with POSIX locks.
- **csi.volumeHandle**: Specifies the **volumeHandle**. Ensure that **volumeHandle** is unique across the cluster. The **resource-group-name** is the Azure resource group where the storage account resides.
- **csi.volumeAttributes.shareName**: Specifies the file share name. Use only the file share name; do not use full path.
- **csi.nodeStageSecretRef.name**: Provide the name of the secret created in step 1 of this procedure. In this example, it is **azure-secret**.
- **csi.nodeStageSecretRef.namespace**: Specifies the namespace that the secret was created in. This must be the namespace where the PV is consumed.

3. Create a PVC that references the PV using the following example file:

Example PVC YAML file

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: <pvc-name>
  namespace: <my-namespace>
spec:
  volumeName: pv-azurefile
  storageClassName: <sc-name>
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 5Gi
```

- **metadata.name**: Specifies the PVC name.
- **metadata.namespace**: Specifies the namespace for the PVC.
- **spec.volumeName**: Specifies the name of the PV that you created in the previous step.
- **spec.storageClassName**: Specifies the storage class name. This name is used by the PVC to bind to this specific PV. For static provisioning, a **StorageClass** object does not need to exist, but the name in the PV and PVC must match.

- **spec.accessModes:** Specifies the access mode. Defines the requested read/write access for the PVC. Claims use the same conventions as volumes when requesting storage with specific access modes. For more information, see "Access modes".
 - **spec.resources.requests.storage:** Specifies the PVC size.
4. Ensure that the PVC is created and in **Bound** status after a while by running the following command:

```
$ oc get pvc <pvc-name>
```

Where **<pvc-name>** is the name of your PVC.

Example output

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES	STORAGECLASS
pvc-name	Bound	pv-azurefile	5Gi	ReadWriteMany	my-sc

Additional resources

- [Persistent storage using Azure File](#)
- [Access modes](#)

6.13. AZURE STACK HUB CSI DRIVER OPERATOR

You can provision and manage Azure Stack Hub Storage in OpenShift Container Platform by using the Azure Stack Hub Container Storage Interface (CSI) Driver Operator and driver, which provide dynamic volume provisioning and eliminate the need to pre-provision storage.

6.13.1. Overview of Azure Stack Hub

OpenShift Container Platform is capable of provisioning persistent volumes (PVs) using the Container Storage Interface (CSI) driver for Azure Stack Hub Storage, which allows you to run apps in an on-premise environment and deliver Azure services in your data center.

Familiarity with persistent storage and configuring CSI volumes is recommended when working with a CSI Operator and driver. For more information, see "Understanding persistent volumes" and "Configuring CSI volumes".

To create CSI-provisioned PVs that mount to Azure Stack Hub storage assets, OpenShift Container Platform installs the Azure Stack Hub CSI Driver Operator and the Azure Stack Hub CSI driver by default in the **openshift-cluster-csi-drivers** namespace.

Azure Stack Hub CSI Driver Operator

The Azure Stack CSI Driver Operator provides a storage class (**managed-csi**), with "Standard_LRS" as the default storage account type, that you can use to create persistent volume claims (PVCs). The Azure Stack Hub CSI Driver Operator supports dynamic volume provisioning by allowing storage volumes to be created on-demand, eliminating the need for cluster administrators to pre-provision storage.

Azure Stack Hub CSI driver

The Azure Stack Hub CSI driver enables you to create and mount Azure Stack Hub PVs.

6.13.2. About CSI

The Container Storage Interface (CSI) enables storage vendors to deliver plugins through a standard interface without modifying Kubernetes core code, replacing traditional embedded storage drivers.

CSI Operators give OpenShift Container Platform users storage options, such as volume snapshots, that are not possible with in-tree volume plugins.

6.13.3. Additional resources

- [Understanding persistent storage](#)
- [Configuring CSI volumes](#)

6.14. GCP PD CSI DRIVER OPERATOR

You can provision and manage Google Cloud Platform (GCP) persistent disk (PD) storage in OpenShift Container Platform by using the GCP PD Container Storage Interface (CSI) Driver Operator and driver, which provide dynamic volume provisioning and eliminate the need to pre-provision storage.

6.14.1. Overview of GCP PD CSI Driver Operator

You can provision and manage Google Cloud Platform (GCP) persistent disk (PD) storage in OpenShift Container Platform by using the GCP PD Container Storage Interface (CSI) Driver Operator and driver, which are installed by default.

Familiarity with persistent storage and configuring CSI volumes is recommended when working with a CSI Operator and driver. For more information, see "Understanding persistent storage" and "Configuring CSI volumes".

To create CSI-provisioned persistent volumes (PVs) that mount to GCP PD storage assets, OpenShift Container Platform installs the GCP PD CSI Driver Operator and the GCP PD CSI driver by default in the **openshift-cluster-csi-drivers** namespace.

GCP PD CSI Driver Operator

By default, the GCP PD CSI Driver Operator provides a storage class that you can use to create PVCs. You can disable this default storage class if desired (see "Managing the default storage class"). You also have the option to create the GCP PD storage class as described in "Persistent storage using GCE Persistent Disk".

GCP PD driver

The GCP PD driver enables you to create and mount GCP PD PVs.

GCP PD CSI driver supports the C3 instance type for bare metal and N4 machine series. The C3 instance type and N4 machine series support the hyperdisk-balanced disks and hyperdisk-balanced high-availability disks. For more information, see "C3 instance type for bare metal and N4 machine series".



NOTE

OpenShift Container Platform provides automatic migration for the GCE Persistent Disk in-tree volume plugin to its equivalent CSI driver. For more information, see "CSI automatic migration".

Additional resources

- [Understanding persistent storage](#)
- [Configuring CSI volumes](#)
- [Managing the default storage class](#)
- [Persistent storage using GCE Persistent Disk](#)
- [C3 instance type for bare metal and N4 machine series](#)
- [CSI automatic migration](#)

6.14.2. About CSI

The Container Storage Interface (CSI) enables storage vendors to deliver plugins through a standard interface without modifying Kubernetes core code, replacing traditional embedded storage drivers.

CSI Operators give OpenShift Container Platform users storage options, such as volume snapshots, that are not possible with in-tree volume plugins.

6.14.3. Reducing permissions while using the GCP PD CSI Driver Operator

By default, the Google Cloud Platform (GCP) persistent disk (PD) Container Storage Interface (CSI) Driver can impersonate any service account in the Google Cloud project. You can reduce the scope of permissions to only the required node service accounts.

To reduce permissions, grant the **iam.serviceAccountUser** role to the control plane and compute node service accounts, and then remove the **iam.serviceAccountUser** role from the project-wide service account, thus reducing the scope of the permission.



NOTE

Reducing permissions only applies to GCP clusters using Workload Identity Federation (WIF).

Procedure

1. Grant scoped **iam.serviceAccountUser** role for node service accounts by running the following Bash commands:

```
gcloud iam service-accounts add-iam-policy-binding "${MASTER_NODE_SA}" --
project="${GOOGLE_PROJECT_ID}" --
member="serviceAccount:${SERVICE_ACCOUNT_EMAIL}" --
role="roles/iam.serviceAccountUser" --condition=None
gcloud iam service-accounts add-iam-policy-binding "${WORKER_NODE_SA}" --
project="${GOOGLE_PROJECT_ID}" --
member="serviceAccount:${SERVICE_ACCOUNT_EMAIL}" --
role="roles/iam.serviceAccountUser" --condition=None
```

- **GOOGLE_PROJECT_ID**: The unique ID of your Google Cloud project.
- **SERVICE_ACCOUNT_EMAIL**: The email address of the "Member" (the person or service account) who is being granted the new permissions. To find the service account, on WIF

clusters, there is a default service account on GCP for the CSI driver based on the cluster name, for example: **`${CLUSTER_NAME}-openshift-gcp-pd-csi-*`**.

- **MASTER_NODE_SA**: The email address of the service account used by your cluster's master node.
 - **WORKER_NODE_SA**: The email address of the service account used by your cluster's worker nodes.
2. Remove project-level **iam.serviceAccountUser** role from the binding created by the installation program by running the following Bash commands:

```
gcloud projects remove-iam-policy-binding "${GOOGLE_PROJECT_ID}" --
member="serviceAccount:${SERVICE_ACCOUNT_EMAIL}" --
role="roles/iam.serviceAccountUser" --condition=None
```

- **SERVICE_ACCOUNT_EMAIL**: The email address of the account losing the permission. For example, **`my-app-sa@my-project.iam.gserviceaccount.com`**. To find the service account, on WIF clusters, there is a default service account on GCP for the CSI driver based on the cluster name, for example: **`${CLUSTER_NAME}-openshift-gcp-pd-csi-*`**.
- **GOOGLE_PROJECT_ID**: The unique ID of the Google Cloud project where this is occurring. For example, **`prod-data-789`**.

6.14.4. GCP PD CSI driver storage class parameters

To configure persistent volume provisioning behavior for Google Cloud Platform (GCP) persistent disk (PD), use storage class parameters that control disk type, replication, and encryption settings.

The GCP PD Container Storage Interface (CSI) driver uses the CSI **external-provisioner** sidecar as a controller. This is a separate helper container that is deployed with the CSI driver. The sidecar manages persistent volumes (PVs) by triggering the **CreateVolume** operation.

The GCP PD CSI driver uses the **csi.storage.k8s.io/fstype** parameter key to support dynamic provisioning. The following table describes all the GCP PD CSI storage class parameters that are supported by OpenShift Container Platform.

Table 6.5. CreateVolume Parameters

Parameter	Values	Default	Description
type	pd-ssd , pd-standard , pd-balanced , or hyperdisk-balanced	pd-standard	Allows you to choose between standard PVs or solid-state-drive PVs. The driver does not validate the value, thus all the possible values are accepted. For hyperdisk-balanced, be sure to check the limitations under "C3 and N4 instance type limitations".

Parameter	Values	Default	Description
replication-type	none or regional-pd	none	Allows you to choose between zonal or regional PVs.
disk-encryption-kms-key	Fully qualified resource identifier for the key to use to encrypt new disks.	Empty string	Uses customer-managed encryption keys (CMEK) to encrypt new disks.

Additional resources

- [C3 and N4 instance type limitations](#)

6.14.5. C3 instance type for bare metal and N4 machines series

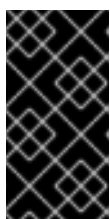
You can use hyperdisk-balanced storage on Google Cloud Platform (GCP) C3 bare metal and N4 machine series instances to achieve high performance.

6.14.5.1. C3 and N4 instance type limitations

Before deploying hyperdisk-balanced disks on C3 bare metal or N4 machine series instances, review the volume size, cloning, resizing, and storage class requirements to ensure successful configuration.

The GCP PD CSI driver support for the C3 instance type for bare metal and N4 machine series have the following limitations:

- You must set the volume size to at least 4Gi when you create hyperdisk-balanced disks. OpenShift Container Platform does not round up to the minimum size, so you must specify the correct size yourself.
- Cloning volumes is not supported when using storage pools.
- For cloning or resizing, hyperdisk-balanced disks original volume size must be 6Gi or greater.
- The default storage class is standard-csi.



IMPORTANT

You need to manually create a storage class.

For information about creating the storage class, see Step 2 in "Setting up hyperdisk-balanced disks".

- Clusters with mixed virtual machines (VMs) that use different storage types, for example, N2 and N4, are not supported. This is due to hyperdisks-balanced disks not being usable on most legacy VMs. Similarly, regular persistent disks are not usable on N4/C3 VMs.
- A GCP cluster with c3-standard-2, c3-standard-4, n4-standard-2, and n4-standard-4 nodes can erroneously exceed the maximum attachable disk number, which should be 16. For more information, see "OCPBUGS-39258".

- For more limitations, see Google Cloud documentation "Limitations for Hyperdisk".

Additional resources

- [Setting up hyperdisk-balanced disk](#)
- [OCPBUGS-39258](#)
- [Limitations for Hyperdisk](#)

6.14.5.2. Hyperdisk-balanced high availability disks overview

You can improve application resilience against zone failures by using Hyperdisk Balanced High Availability volumes that synchronously replicate data across two zones in the same region.

Hyperdisk Balanced High Availability volumes are useful for:

- Protecting your applications from a zonal outage by synchronously replicating data across two zones in the same region
- When you require write access to the same volume in multiple zones



NOTE

Volume Attributes Classes (VAC) does not work on Hyperdisk Balanced High Availability disks.

To set up Hyperdisk Balanced High Availability disks, see "Setting up hyperdisk-balanced disks".

Additional resources

- [Setting up hyperdisk-balanced disk](#)

6.14.5.3. Storage pools for hyperdisk-balanced disks overview

You can simplify storage management and reduce costs by using hyperdisk storage pools to aggregate capacity, throughput, and IOPS into a single pool instead of managing individual disks.

Hyperdisk storage pools can be used with Compute Engine for large-scale storage. A hyperdisk storage pool is a purchased collection of capacity, throughput, and IOPS, which you can then provision for your applications as needed. You can use hyperdisk storage pools to create and manage disks in pools and use the disks across multiple workloads. By managing disks in aggregate, you can save costs while achieving expected capacity and performance growth. By using only the storage that you need in hyperdisk storage pools, you reduce the complexity of forecasting capacity and reduce management by going from managing hundreds of disks to managing a single storage pool.

6.14.5.4. Setting up hyperdisk-balanced disks

To provision high-performance hyperdisk-balanced storage volumes, configure a storage class, create persistent volume claims, and deploy applications that use the hyperdisk storage.

Prerequisites

- Access to the cluster with administrative privileges

Procedure

1. Create a GCP cluster with attached disks provisioned with hyperdisk-balanced disks.
2. Create a storage class specifying the hyperdisk-balanced disks during installation:
 - a. Follow the procedure in "Installing a cluster on GCP with customizations".
For your install-config.yaml file, use the following example file:

Example install-config YAML file

```
apiVersion: v1
metadata:
  name: ci-op-9976b7t2-8aa6b

sshKey: |
  XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
baseDomain: XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
platform:
  gcp:
    projectID: XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
    region: us-central1
controlPlane:
  architecture: amd64
  name: master
  platform:
    gcp:
      type: n4-standard-4
    osDisk:
      diskType: hyperdisk-balanced
      diskSizeGB: 200
  replicas: 3
compute:
  - architecture: amd64
    name: worker
    replicas: 3
    platform:
      gcp:
        type: n4-standard-4
      osDisk:
        diskType: hyperdisk-balanced
```

- **controlPlane.platform.gcp.type** and **compute.platform.gcp.type**: Specifies the node type as n4-standard-4.
- **controlPlane.platform.gcp.osDisk.diskType** and **compute.platform.osDisk.diskType**: Specifies the node has the root disk backed by hyperdisk-balanced disk type. All nodes in the cluster should use the same disk type, either hyperdisks-balanced or pd-*



NOTE

All nodes in the cluster must support hyperdisk-balanced volumes. Clusters with mixed nodes are not supported, for example N2 and N3 using hyperdisk-balanced disks.

b. After step 3 in "Incorporating the Cloud Credential Operator utility manifests", copy the following manifests into the manifests directory created by the installation program:

- `cluster_csi_driver.yaml` - specifies opting out of the default storage class creation
- `storageclass.yaml` - creates a hyperdisk-specific storage class

Example cluster CSI driver YAML file

```
apiVersion: operator.openshift.io/v1
kind: "ClusterCSIDriver"
metadata:
  name: "pd.csi.storage.gke.io"
spec:
  logLevel: Normal
  managementState: Managed
  operatorLogLevel: Normal
  storageClassState: Unmanaged
```

spec.storageClassState specifies disabling creation of the default OpenShift Container Platform storage classes.

Example storage class YAML file

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: hyperdisk-sc
  annotations:
    storageclass.kubernetes.io/is-default-class: "true"
provisioner: pd.csi.storage.gke.io
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true
reclaimPolicy: Delete
parameters:
  type: hyperdisk-balanced
  replication-type: none
  provisioned-throughput-on-create: "140Mi"
  provisioned-iops-on-create: "3000"
  storage-pools: projects/my-project/zones/us-east4-c/storagePools/pool-us-east4-c
allowedTopologies:
- matchLabelExpressions:
  - key: topology.kubernetes.io/zone
    values:
    - us-east4-c
...
```

- **metadata.name:** Specifies the name for your storage class. In this example, it is **hyperdisk-sc**.
- **provisioner:** **pd.csi.storage.gke.io** specifies GCP CSI provisioner.
- **parameters.type:** Specifies using hyperdisk-balanced disks. To specify high availability hyperdisk-balanced disk, set the value to **hyperdisk-balanced-high-availability**.

- **parameters.provisioned-throughput-on-create:** Specifies the throughput value in MiBps using the "Mi" qualifier. For example, if your required throughput is 250 MiBps, specify "250Mi". If you do not specify a value, the capacity is based upon the disk type default.
 - **parameters.provisioned-iops-on-create:** Specifies the IOPS value without any qualifiers. For example, if you require 7,000 IOPS, specify "7000". If you do not specify a value, the capacity is based upon the disk type default.
 - **parameters.storage-pools:** If using storage pools, specifies a list of specific storage pools that you want to use in the format:
projects/PROJECT_ID/zones/ZONE/storagePools/STORAGE_POOL_NAME.
 - **parameters.allowedTopologies:** If using storage pools, set **allowedTopologies** to restrict the topology of provisioned volumes to where the storage pool exists. In this example, **us-east4-c**.
3. Create a persistent volume claim (PVC) that uses the hyperdisk-specific storage class using the following example YAML file:

Example PVC YAML file

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-pvc
spec:
  storageClassName: hyperdisk-sc
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 2048Gi
```

- **spec.storageClassName:** The PVC references the storage pool-specific storage class. In this example, **hyperdisk-sc**.
 - **spec.resources.requests.storage:** Target storage capacity of the hyperdisk-balanced volume. In this example, **2048Gi**.
4. Create a deployment that uses the PVC that you just created. Using a deployment helps ensure that your application has access to the persistent storage even after the pod restarts and rescheduling:
- Ensure a node pool with the specified machine series is up and running before creating the deployment. Otherwise, the pod fails to schedule.
 - Use the following example YAML file to create the deployment:

Example deployment YAML file

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: postgres
```

```
spec:
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      nodeSelector:
        cloud.google.com/machine-family: n4
      containers:
        - name: postgres
          image: postgres:14-alpine
          args: [ "sleep", "3600" ]
          volumeMounts:
            - name: sdk-volume
              mountPath: /usr/share/data/
      volumes:
        - name: sdk-volume
          persistentVolumeClaim:
            claimName: my-pvc
```

- **spec.template.spec.nodeSelector:** Specifies the machine family. In this example, it is **n4**.
- **spec.template.spec.volumes.persistentVolumeClaim.claimName:** Specifies the name of the PVC created in the preceding step. In this example, it is **my-pvc**.

- c. Confirm that the deployment was successfully created by running the following command:

```
$ oc get deployment
```

Example output

```
NAME      READY  UP-TO-DATE  AVAILABLE  AGE
postgres  0/1    1           0          42s
```

It might take a few minutes for hyperdisk instances to complete provisioning and display a READY status.

- d. Confirm that PVC **my-pvc** has been successfully bound to a persistent volume (PV) by running the following command:

```
$ oc get pvc my-pvc
```

Example output

```
NAME      STATUS  VOLUME                                     CAPACITY  ACCESS MODES
STORAGECLASS  VOLUMEATTRIBUTESCLASS  AGE
my-pvc      Bound   pvc-1ff52479-4c81-4481-aa1d-b21c8f8860c6  2Ti       RWO
hyperdisk-sc  <unset>                2m24s
```

- e. Confirm the expected configuration of your hyperdisk-balanced disk:

```
$ gcloud compute disks list
```

Example output

NAME	LOCATION	LOCATION_SCOPE	SIZE_GB	TYPE
STATUS				
instance-20240914-173145-boot	us-central1-a	zone	150	pd-standard
READY				
instance-20240914-173145-data-workspace	us-central1-a	zone	100	pd-
balanced				
READY				
c4a-rhel-vm	us-central1-a	zone	50	hyperdisk-balanced
READY				

Where **c4a-rhel.vm** is a hyperdisk-balanced disk.

- f. If using storage pools, check that the volume is provisioned as specified in your storage class and PVC by running the following command:

```
$ gcloud compute storage-pools list-disks pool-us-east4-c --zone=us-east4-c
```

Example output

NAME	STATUS	PROVISIONED_IOPS
PROVISIONED_THROUGHPUT	SIZE_GB	
pvc-1ff52479-4c81-4481-aa1d-b21c8f8860c6	READY	3000
2048		140

Additional resources

- [Create a Hyperdisk Storage Pool](#)
- [Installing a cluster on GCP with customizations](#)

6.14.6. Creating a custom-encrypted persistent volume

To enhance data security beyond default encryption, create persistent volumes with customer-managed encryption keys (CMEK) that use Google Cloud Key Management Service for encryption control.

When you create a **PersistentVolumeClaim** object, OpenShift Container Platform provisions a new persistent volume (PV) and creates a **PersistentVolume** object. You can add a custom encryption key in Google Cloud Platform (GCP) to protect a PV in your cluster by encrypting the newly created PV.

For encryption, the newly attached PV that you create uses customer-managed encryption keys (CMEK) on a cluster by using a new or existing Google Cloud Key Management Service (KMS) key.

Prerequisites

- You are logged in to a running OpenShift Container Platform cluster.
- You have created a Cloud KMS key ring and key version.

For more information about CMEK and Cloud KMS resources, see Google Cloud documentation "Using customer-managed encryption keys (CMEK)".

Procedure

1. Create a storage class with the Cloud KMS key. The following example enables dynamic provisioning of encrypted volumes:

Example

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: csi-gce-pd-cmek
provisioner: pd.csi.storage.gke.io
volumeBindingMode: "WaitForFirstConsumer"
allowVolumeExpansion: true
parameters:
  type: pd-standard
  disk-encryption-kms-key: projects/<key-project-id>/locations/<location>/keyRings/<key-
ring>/cryptoKeys/<key>
```

The **parameters.disk-encryption-kms-key** field must be the resource identifier for the key that will be used to encrypt new disks. Values are case-sensitive. For more information about providing key ID values, see Google Cloud documentation "Retrieving a resource's ID" and "Getting a Cloud KMS resource ID".



NOTE

You cannot add the **disk-encryption-kms-key** parameter to an existing storage class. However, you can delete the storage class and re-create it with the same name and a different set of parameters. If you do this, the provisioner of the existing class must be **pd.csi.storage.gke.io**.

2. Deploy the storage class on your OpenShift Container Platform cluster by using the **oc** command:

```
$ oc describe storageclass csi-gce-pd-cmek
```

Example output

```
Name:          csi-gce-pd-cmek
IsDefaultClass: No
Annotations:    None
Provisioner:    pd.csi.storage.gke.io
Parameters:    disk-encryption-kms-key=projects/key-project-
id/locations/location/keyRings/ring-name/cryptoKeys/key-name,type=pd-standard
AllowVolumeExpansion: true
MountOptions:   none
ReclaimPolicy:  Delete
VolumeBindingMode: WaitForFirstConsumer
Events:         none
```

3. Create a file named **pvc.yaml** that matches the name of your storage class object that you created in the previous step:

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: podpvc
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: csi-gce-pd-cmek
resources:
  requests:
    storage: 6Gi
```



NOTE

If you marked the new storage class as default, you can omit the **storageClassName** field.

4. Apply the PVC on your cluster:

```
$ oc apply -f pvc.yaml
```

5. Get the status of your PVC and verify that it is created and bound to a newly provisioned PV:

```
$ oc get pvc
```

Example output

NAME	STATUS	VOLUME	CAPACITY	ACCESS MODES
STORAGECLASS	AGE			
podpvc	Bound	pvc-e36abf50-84f3-11e8-8538-42010a800002	10Gi	RWO
gce-pd-cmek	9s			csi-



NOTE

If your storage class has the **volumeBindingMode** field set to **WaitForFirstConsumer**, you must create a pod to use the PVC before you can verify it.

Your CMEK-protected PV is now ready to use with your OpenShift Container Platform cluster.

Additional resources

- [Using customer-managed encryption keys \(CMEK\)](#)
- [Retrieving a resource's ID](#)
- [Getting a Cloud KMS resource ID](#)

6.14.7. User-managed encryption

The user-managed encryption feature allows you to provide keys during installation that encrypt OpenShift Container Platform node root volumes, and enables all managed storage classes to use these keys to encrypt provisioned storage volumes.

You must specify the custom key in the **platform.<cloud_type>.defaultMachinePlatform** field in the install-config YAML file.

This features supports the following storage types:

- Amazon Web Services (AWS) Elastic Block storage (EBS)



NOTE

If there is no encrypted key defined in the storage class, only set **encrypted: "true"** in the storage class. The AWS EBS CSI driver uses the AWS managed alias/aws/ebs, which is created by Amazon EBS automatically in each region by default to encrypt provisioned storage volumes. In addition, the managed storage classes all have the **encrypted: "true"** setting.

For information about installing AWS EBS with user-managed encryption, see "Optional AWS configuration parameters".

- Microsoft Azure Disk storage



NOTE

If the OS (root) disk is encrypted, and there is no encrypted key defined in the storage class, Azure Disk CSI driver uses the OS disk encryption key by default to encrypt provisioned storage volumes.

For information about installing Azure Disk with user-managed encryption, see "Preparing an Azure Disk Encryption Set".

- Google Cloud Platform (GCP) persistent disk (PD) storage
For information about installing GCP PD with user-managed encryption, see "Additional Google Cloud configuration parameters".
- IBM Cloud® Virtual Private Cloud (VPC) Block storage
For information about installing with IBM Cloud with user-managed encryption, see "User-managed encryption for IBM Cloud" and "Installing on IBM Cloud".

Additional resources

- [Additional Google Cloud configuration parameters](#)

6.14.8. Volume snapshot class **csi-gce-pd-vsc-images**

By default, you cannot restore more than six volumes per snapshot per hour. So in Kubevirt environments, you normally cannot create more than six VMs per hour from a "golden image" (templates saved as snapshots).

For Google Cloud Platform (GCP) persistent disk (PD) storage CSI, there is a non-default **VolumeSnapshotClass**, named **csi-gce-pd-vsc-images**, that uses the **snapshot-type: images** parameter. When using KubeVirt, it allows you overcome the six VMs per hour restriction, so that you can

create VMs from "golden images".



NOTE

Snapshots using the images snapshot class are strictly limited to ReadWriteOnce (RWO) sources, but you can restore them to ReadWriteMany (RWX) hyperdisk-balanced disks.

For more information, see, "Volume snapshots CRD: VolumeSnapshotClass".

Additional resources

- [Volume snapshots CRD: VolumeSnapshotClass](#)

6.14.9. Additional resources

- [Persistent storage using GCE Persistent Disk](#)

6.15. GOOGLE CLOUD FILESTORE CSI DRIVER OPERATOR

The Google Cloud Platform (GCP) Filestore Container Storage Interface (CSI) Driver Operator provisions and manages GCP Filestore Storage in OpenShift Container Platform with dynamic volume provisioning, eliminating the need to pre-provision storage.

6.15.1. Overview

OpenShift Container Platform is capable of provisioning persistent volumes (PVs) using the Container Storage Interface (CSI) driver for Google Compute Platform (GCP) Filestore Storage.

Familiarity with persistent storage and configuring CSI volumes is recommended when working with a CSI Operator and driver. For more information, see "Understanding persistent storage" and "Configuring CSI volumes".

To create CSI-provisioned PVs that mount to Google Cloud Filestore Storage assets, you install the Google Cloud Filestore CSI Driver Operator and the Google Cloud Filestore CSI driver in the **openshift-cluster-csi-drivers** namespace.

- The *Google Cloud Filestore CSI Driver Operator* does not provide a storage class by default, but you can create one if needed (for more information, see "Creating a storage class for GCP Filestore storage"). The Google Cloud Filestore CSI Driver Operator supports dynamic volume provisioning by allowing storage volumes to be created on-demand, eliminating the need for cluster administrators to pre-provision storage.
- The *Google Cloud Filestore CSI driver* enables you to create and mount Google Cloud Filestore PVs.

OpenShift Container Platform Google Cloud Filestore supports Workload Identity. This allows users to access Google Cloud resources using federated identities instead of a service account key. GCP Workload Identity must be enabled globally during installation, and then configured for the Google Cloud Filestore CSI Driver Operator.

Additional resources

- [Understanding persistent storage](#)

- [Configuring CSI volumes](#)
- [Creating a storage class for GCP Filestore storage](#)

6.15.2. About CSI

The Container Storage Interface (CSI) enables storage vendors to deliver plugins through a standard interface without modifying Kubernetes core code, replacing traditional embedded storage drivers.

CSI Operators give OpenShift Container Platform users storage options, such as volume snapshots, that are not possible with in-tree volume plugins.

6.15.3. Installing the Google Cloud Filestore CSI Driver Operator

Since the Google Compute Platform (Google Cloud) Filestore Container Storage Interface (CSI) Driver Operator is not installed in OpenShift Container Platform by default, you must install the Google Cloud Filestore CSI Driver Operator in your cluster.

6.15.3.1. Preparing to install the Google Cloud Filestore CSI Driver Operator with Workload Identity

If you are planning to use GCP Workload Identity with Google Compute Platform Filestore, you must obtain certain parameters that you will use during the installation of the Google Cloud Filestore Container Storage Interface (CSI) Driver Operator.

Prerequisites

- Access to the cluster as a user with the cluster-admin role.

Procedure

1. Obtain the project number:
 - a. Obtain the project ID by running the following command:

```
$ export PROJECT_ID=$(oc get infrastructure/cluster -o
jsonpath='{.status.platformStatus.gcp.projectID}')
```

- b. Obtain the project number, using the project ID, by running the following command:

```
$ gcloud projects describe $PROJECT_ID --format="value(projectNumber)"
```

2. Find the identity pool ID and the provider ID:
During cluster installation, the names of these resources are provided to the Cloud Credential Operator utility (**ccctl**) with the **--name parameter**. See "Creating Google Cloud resources with the Cloud Credential Operator utility".
3. Create Workload Identity resources for the Google Cloud Filestore Operator:
 - a. Create a **CredentialsRequest** file using the following example file:

Example Credentials Request YAML file

```
apiVersion: cloudcredential.openshift.io/v1
```

```

kind: CredentialsRequest
metadata:
  name: openshift-gcp-filestore-csi-driver-operator
  namespace: openshift-cloud-credential-operator
  annotations:
    include.release.openshift.io/self-managed-high-availability: "true"
    include.release.openshift.io/single-node-developer: "true"
spec:
  serviceAccountNames:
    - gcp-filestore-csi-driver-operator
    - gcp-filestore-csi-driver-controller-sa
  secretRef:
    name: gcp-filestore-cloud-credentials
    namespace: openshift-cluster-csi-drivers
  providerSpec:
    apiVersion: cloudcredential.openshift.io/v1
kind: GCPProviderSpec
  predefinedRoles:
    - roles/file.editor
    - roles/resourcemanager.tagUser
  skipServiceCheck: true

```

- b. Use the **CredentialsRequest** file to create a Google Cloud service account by running the following command:

```

$ ./ccctl gcp create-service-accounts --name=<filestore-service-account> \
  --workload-identity-pool=<workload-identity-pool> \
  --workload-identity-provider=<workload-identity-provider> \
  --project=<project-id> \
  --credentials-requests-dir=/tmp/credreq

```

- **<filestore-service-account>** is a user-chosen name.
- **<workload-identity-pool>** comes from Step 2 above.
- **<workload-identity-provider>** comes from Step 2 above.
- **<project-id>** comes from Step 1.a above.
- **credentials-requests-dir** is the name of directory where the **CredentialsRequest** file resides.

Example output

```

2025/02/10 17:47:39 Credentials loaded from gcloud CLI defaults
2025/02/10 17:47:42 IAM service account filestore-service-account-openshift-gcp-
filestore-csi-driver-operator created
2025/02/10 17:47:44 Unable to add predefined roles to IAM service account, retrying...
2025/02/10 17:47:59 Updated policy bindings for IAM service account filestore-service-
account-openshift-gcp-filestore-csi-driver-operator
2025/02/10 17:47:59 Saved credentials configuration to: /tmp/install-dir/
openshift-cluster-csi-drivers-gcp-filestore-cloud-credentials-credentials.yaml

```

Where **/tmp/install-dir/** is the current directory.

- c. Find the service account email of the newly created service account by running the following command:

```
$ cat /tmp/install-dir/manifests/openshift-cluster-csi-drivers-gcp-filestore-cloud-credentials-credentials.yaml | yq '.data["service_account.json"]' | base64 -d | jq '.service_account_impersonation_url'
```

Example output

```
https://iamcredentials.googleapis.com/v1/projects/-/serviceAccounts/filestore-se-openshift-g-ch8cm@openshift-gce-devel.iam.gserviceaccount.com:generateAccessToken
```

In this example output, the service account email is **filestore-se-openshift-g-ch8cm@openshift-gce-devel.iam.gserviceaccount.com**.

Results

You now have the following parameters that you need to install the Google Cloud Filestore CSI Driver Operator:

- Project number - from Step 1.b
- Pool ID - from Step 2
- Provider ID - from Step 2
- Service account email - from Step 3.c

Additional resources

- [Creating Google Cloud resources with the Cloud Credential Operator utility](#)

6.15.3.2. Installing the Google Cloud Filestore CSI Driver Operator

Since the Google Compute Platform (Google Cloud) Filestore Container Storage Interface (CSI) Driver Operator is not installed in OpenShift Container Platform by default, you must install the Google Cloud Filestore CSI Driver Operator in your cluster.

Prerequisites

- Access to the OpenShift Container Platform web console.
- If using GCP Workload Identity, certain GCP Workload Identity parameters are needed. See the preceding Section *Preparing to install the Google Cloud Filestore CSI Driver Operator with Workload Identity*.

Procedure

1. Log in to the web console.
2. Enable the Filestore API in the GCE project by running the following command:

```
$ gcloud services enable file.googleapis.com --project <my_gce_project>
```

Replace **<my_gce_project>** with your Google Cloud project.

You can also do this using Google Cloud web console.

3. Install the Google Cloud Filestore CSI Operator:

- a. Click **Ecosystem → Software Catalog**.
- b. Locate the Google Cloud Filestore CSI Operator by typing **Google Cloud Filestore** in the filter box.
- c. Click the **Google Cloud Filestore CSI Driver Operator** button.
- d. On the **Google Cloud Filestore CSI Driver Operator** page, click **Install**.
- e. On the **Install Operator** page, ensure that:
 - **All namespaces on the cluster (default)** is selected.
 - **Installed Namespace** is set to **openshift-cluster-csi-drivers**.
If using GCP Workload Identity, enter values for the following fields obtained from the procedure in Section *Preparing to install the Google Cloud Filestore CSI Driver Operator with Workload Identity*.
 - **Google Cloud Project Number**
 - **Google Cloud Pool ID**
 - **Google Cloud Provider ID**
 - **Google Cloud Service Account Email**
- f. Click **Install**.
After the installation finishes, the Google Cloud Filestore CSI Operator is listed in the **Installed Operators** section of the web console.

4. Install the Google Cloud Filestore CSI Driver:

- a. Click **administration → CustomResourceDefinitions → ClusterCSIDriver**.
- b. On the **Instances** tab, click **Create ClusterCSIDriver**.
Use the following YAML file:

```
apiVersion: operator.openshift.io/v1
kind: ClusterCSIDriver
metadata:
  name: filestore.csi.storage.gke.io
spec:
  managementState: Managed
```

- c. Click **Create**.
- d. Wait for the following Conditions to change to a "true" status:
 - **GCPFilestoreDriverCredentialsRequestControllerAvailable**
 - **GCPFilestoreDriverNodeServiceControllerAvailable**

- `GCPFilestoreDriverControllerServiceControllerAvailable`

Additional resources

- [Enabling an API in your Google Cloud](#)
- [Enabling an API using the Google Cloud web console](#)

6.15.4. Creating a storage class for GCP Filestore Storage

To enable dynamic provisioning of GCP Filestore volumes, create a storage class that specifies VPC network configuration and connection mode settings.

Ensure that the Operator is installed before creating a storage class for dynamic provisioning of Google Compute Platform (GCP) Filestore volumes.

Prerequisites

- You are logged in to the running OpenShift Container Platform cluster.

Procedure

1. Create a storage class using the following example YAML file:

Example YAML file

```
kind: StorageClass
apiVersion: storage.k8s.io/v1
metadata:
  name: filestore-csi
provisioner: filestore.csi.storage.gke.io
parameters:
  connect-mode: DIRECT_PEERING
  network: network-name
allowVolumeExpansion: true
volumeBindingMode: WaitForFirstConsumer
```

- **parameters.connect-mode:** For a shared VPC, use the **connect-mode** parameter set to **PRIVATE_SERVICE_ACCESS**. For a non-shared VPC, the value is **DIRECT_PEERING**, which is the default setting.
 - **parameters.network:** Specify the name of the GCP virtual private cloud (VPC) network where Filestore instances should be created in.
2. Specify the name of the VPC network where Filestore instances should be created in. It is recommended to specify the VPC network that the Filestore instances should be created in. If no VPC network is specified, the Container Storage Interface (CSI) driver tries to create the instances in the default VPC network of the project.

On IPI installations, the VPC network name is typically the cluster name with the suffix "-network". However, on UPI installations, the VPC network name can be any value chosen by the user.

For a shared VPC (**connect-mode** = **PRIVATE_SERVICE_ACCESS**), the network needs to be the full VPC name. For example: **projects/shared-vpc-name/global/networks/gcp-filestore-network**.

You can find out the VPC network name by inspecting the **MachineSets** objects with the following command:

```
$ oc -n openshift-machine-api get machinesets -o yaml | grep "network:"
- network: gcp-filestore-network
(...)
```

In this example, the VPC network name in this cluster is "gcp-filestore-network".

6.15.5. NFS export options

To restrict access to Google Cloud Filestore volumes beyond default project-wide permissions, configure NFS export options that limit access by IP range and user/group IDs.

By default, a Filestore instance grants root level read/write access to all clients that share the same Google Cloud project and virtual private cloud (VPC) network. Network File System (NFS) export options can limit this access to certain IP ranges and specific user/group IDs for the Filestore instance. When creating a storage class, you can set these options using the **nfs-export-options-on-create** parameter.

Prerequisites

- Access to the cluster as a user with the cluster-admin role.
- The Google Cloud Filestore CSI Driver Operator and Google Cloud Filestore CSI driver installed.

Procedure

1. Create a storage class using a file similar to the following sample YAML file:



NOTE

For more information about creating a storage class, see Section *Creating a storage class for GCP Filestore Operator*.

Example storage class YAML file with NFS export options

```
kind: StorageClass
apiVersion: storage.k8s.io/v1
metadata:
  name: SC-name
provisioner: filestore.csi.storage.gke.io
parameters:
  connect-mode: DIRECT_PEERING
  network: project-network
  nfs-export-options-on-create: '[
  {
    "accessMode": "READ_WRITE",
    "squashMode": "NO_ROOT_SQUASH",
```



```

    "anonUid": 65534
    "anonGid": 65534
    "ipRanges": [
      "10.0.0.0/16"
    ]
  }
}
allowVolumeExpansion: true

```

For **parameters.nfs-export-options-on-create** arguments:

- **AccessMode:** Either **READ_ONLY**, which allows only read requests on the exported directory; or **READ_WRITE**, which allows both read and write requests. The default is **READ_WRITE**.
- **SquashMode:** Either **NO_ROOT_SQUASH**, which allows root access on the exported directory; or **ROOT_SQUASH**, which does not allow root access. The default is **NO_ROOT_SQUASH**.
- **AnonUid:** An integer representing the anonymous user ID with a default value of 65534. **AnonUid** can only be set with **squashMode** set to **ROOT_SQUASH**; Otherwise, an error occurs.
- **AnonGid:** An integer representing the anonymous group ID with a default value of 65534. **AnonGid** can only be set with
- **ipRanges:** List of either an IPv4 addresses in the format {octet1}.{octet2}.{octet3}.{octet4}, or CIDR ranges in the format {octet1}.{octet2}.{octet3}.{octet4}/{mask size}, which can mount the file share. Overlapping IP ranges are not allowed, both within and across **NfsExportOptions**, otherwise, an error is returned. The limit is 64 IP ranges or addresses for each **FileShareConfig** among all NFS export options.

6.15.6. Destroying clusters and GCP Filestore

To prevent orphaned resources and potential costs, verify that all Google Cloud Filestore resources are deleted after cluster destruction, as automated cleanup might not remove all resources.

Typically, if you destroy a cluster, the OpenShift Container Platform installer deletes all of the cloud resources that belong to that cluster. However, due to the special nature of the Google Compute Platform (GCP) Filestore resources, the automated cleanup process might not remove all of them in some rare cases.

Therefore, Red Hat recommends that you verify that all cluster-owned Filestore resources are deleted by the uninstall process.

Procedure

1. Access your Google Cloud account using the GUI or CLI.
2. Search for any resources with the **kubernetes-io-cluster-\${CLUSTER_ID}=owned** label. Since the cluster ID is unique to the deleted cluster, there should not be any remaining resources with that cluster ID.
3. In the unlikely case there are some remaining resources, delete them.

6.15.7. Additional resources

- [Configuring CSI volumes](#)
- [CCO-based workflow for OLM-managed Operators with Google Cloud Workload Identity](#)

6.16. IBM CLOUD(R) VPC BLOCK CSI DRIVER OPERATOR

You can provision and manage IBM Cloud® Virtual Private Cloud (VPC) Block Storage in OpenShift Container Platform using the IBM Cloud® VPC Block Container Storage Interface (CSI) Driver Operator and driver, which provide dynamic volume provisioning.

6.16.1. Overview of IBM Cloud(R) VPC Block CSI Driver Operator

You can dynamically provision block storage volumes on IBM Cloud® Virtual Private Cloud (VPC) infrastructure by using the Container Storage Interface (CSI) driver, which supports multiple IOPS tiers to match your workload performance requirements.

Familiarity with persistent storage and configuring CSI volumes is recommended when working with a CSI Operator and driver. For more information, see "Understanding persistent storage" and "Configuring CSI volumes".

To create CSI-provisioned PVs that mount to IBM Cloud® VPC Block storage assets, OpenShift Container Platform installs the IBM Cloud® VPC Block CSI Driver Operator and the IBM Cloud® VPC Block CSI driver by default in the **openshift-cluster-csi-drivers** namespace.

IBM Cloud® VPC Block CSI Driver Operator

The IBM Cloud® VPC Block CSI Driver Operator provides three storage classes named **ibmc-vpc-block-10iops-tier** (default), **ibmc-vpc-block-5iops-tier**, and **ibmc-vpc-block-custom** for different tiers that you can use to create persistent volume claims (PVCs). The IBM Cloud® VPC Block CSI Driver Operator supports dynamic volume provisioning by allowing storage volumes to be created on-demand, eliminating the need for cluster administrators to pre-provision storage. You can disable this default storage class if necessary (see "Managing the default storage class").

IBM Cloud® VPC Block CSI driver

The IBM Cloud® VPC Block CSI driver enables you to create and mount IBM Cloud® VPC Block PVs.

Additional resources

- [Understanding persistent storage](#)
- [Configuring CSI volumes](#)
- [Managing the default storage class](#)

6.16.2. About CSI

The Container Storage Interface (CSI) enables storage vendors to deliver plugins through a standard interface without modifying Kubernetes core code, replacing traditional embedded storage drivers.

CSI Operators give OpenShift Container Platform users storage options, such as volume snapshots, that are not possible with in-tree volume plugins.

6.16.3. User-managed encryption

The user-managed encryption feature allows you to provide keys during installation that encrypt OpenShift Container Platform node root volumes, and enables all managed storage classes to use these keys to encrypt provisioned storage volumes.

You must specify the custom key in the **platform.<cloud_type>.defaultMachinePlatform** field in the install-config YAML file.

This features supports the following storage types:

- Amazon Web Services (AWS) Elastic Block storage (EBS)



NOTE

If there is no encrypted key defined in the storage class, only set **encrypted: "true"** in the storage class. The AWS EBS CSI driver uses the AWS managed alias/aws/ebs, which is created by Amazon EBS automatically in each region by default to encrypt provisioned storage volumes. In addition, the managed storage classes all have the **encrypted: "true"** setting.

For information about installing AWS EBS with user-managed encryption, see "Optional AWS configuration parameters".

- Microsoft Azure Disk storage



NOTE

If the OS (root) disk is encrypted, and there is no encrypted key defined in the storage class, Azure Disk CSI driver uses the OS disk encryption key by default to encrypt provisioned storage volumes.

For information about installing Azure Disk with user-managed encryption, see "Preparing an Azure Disk Encryption Set".

- Google Cloud Platform (GCP) persistent disk (PD) storage
For information about installing GCP PD with user-managed encryption, see "Additional Google Cloud configuration parameters".
- IBM Cloud® Virtual Private Cloud (VPC) Block storage
For information about installing with IBM Cloud with user-managed encryption, see "User-managed encryption for IBM Cloud" and "Installing on IBM Cloud".

Additional resources

- [User-managed encryption for IBM Cloud](#)
- [Preparing to install on IBM Cloud](#)

6.17. IBM POWER VIRTUAL SERVER BLOCK CSI DRIVER OPERATOR

You can provision and manage IBM Power® Virtual Server Block storage in OpenShift Container Platform by using the Container Storage Interface (CSI) Driver Operator and driver, which provide dynamic volume provisioning.

6.17.1. Introduction to IBM Power(R) Virtual Server Block CSI Driver Operator

The IBM Power® Virtual Server Block CSI Driver is installed through the IBM Power® Virtual Server Block CSI Driver Operator and the operator is based on **library-go**.

The OpenShift Container Platform **library-go** framework is a collection of functions that allows users to build OpenShift operators easily. Most of the functionality of a CSI Driver Operator is already available there. The IBM Power® Virtual Server Block CSI Driver Operator is installed by the Cluster Storage Operator. The Cluster Storage Operator installs the IBM Power® Virtual Server Block CSI Driver Operator if the platform type is Power Virtual Servers.

6.17.2. IBM Power(R) Virtual Server Block CSI Driver Operator overview

OpenShift Container Platform can provision persistent volumes (PVs) by using the Container Storage Interface (CSI) driver for IBM Power® Virtual Server Block Storage.

Familiarity with persistent storage and configuring CSI volumes is helpful when working with a CSI Operator and driver. For more information, see "Understanding persistent storage" and "Configuring CSI volumes".

To create CSI-provisioned PVs that mount to IBM Power® Virtual Server Block storage assets, OpenShift Container Platform installs the IBM Power® Virtual Server Block CSI Driver Operator and the IBM Power® Virtual Server Block CSI driver by default in the **openshift-cluster-csi-drivers** namespace.

- The *IBM Power® Virtual Server Block CSI Driver Operator* provides two storage classes named **ibm-powervs-tier1** (default), and **ibm-powervs-tier3** for different tiers that you can use to create persistent volume claims (PVCs). The IBM Power® Virtual Server Block CSI Driver Operator supports dynamic volume provisioning by allowing storage volumes to be created on-demand, eliminating the need for cluster administrators to pre-provision storage.
- With the *IBM Power® Virtual Server Block CSI driver* you can create and mount IBM Power® Virtual Server Block PVs.

Additional resources

- [Understanding persistent storage](#)
- [Configuring CSI volumes](#)

6.17.3. About CSI

The Container Storage Interface (CSI) enables storage vendors to deliver plugins through a standard interface without modifying Kubernetes core code, replacing traditional embedded storage drivers.

CSI Operators give OpenShift Container Platform users storage options, such as volume snapshots, that are not possible with in-tree volume plugins.

6.18. OPENSTACK CINDER CSI DRIVER OPERATOR

You can provision and manage OpenStack Cinder storage in OpenShift Container Platform using the OpenStack Cinder Container Storage Interface (CSI) Driver Operator and driver, which provide dynamic volume provisioning.

6.18.1. Overview

OpenShift Container Platform is capable of provisioning persistent volumes (PVs) using the Container Storage Interface (CSI) driver for OpenStack Cinder.

Familiarity with persistent storage and configuring CSI volumes is recommended when working with a CSI Operator and driver. For more information, see "Understanding persistent storage" and "Configuring CSI volumes."

To create CSI-provisioned PVs that mount to OpenStack Cinder storage assets, OpenShift Container Platform installs the OpenStack Cinder CSI Driver Operator and the OpenStack Cinder CSI driver in the **openshift-cluster-csi-drivers** namespace.

- The *OpenStack Cinder CSI Driver Operator* provides a CSI storage class that you can use to create PVCs. You can disable this default storage class if needed (see "Managing the default storage class").
- The *OpenStack Cinder CSI driver* enables you to create and mount OpenStack Cinder PVs.



NOTE

OpenShift Container Platform provides automatic migration for the Cinder in-tree volume plugin to its equivalent CSI driver. For more information, see "CSI automatic migration".



IMPORTANT

OpenShift Container Platform defaults to using the CSI plugin to provision Cinder storage.

Additional resources

- [Understanding persistent storage](#)
- [Configuring CSI volumes](#)
- [Managing the default storage class](#)
- [CSI automatic migration](#)

6.18.2. About CSI

The Container Storage Interface (CSI) enables storage vendors to deliver plugins through a standard interface without modifying Kubernetes core code, replacing traditional embedded storage drivers.

CSI Operators give OpenShift Container Platform users storage options, such as volume snapshots, that are not possible with in-tree volume plugins.

6.18.3. Making OpenStack Cinder CSI the default storage class

To use the OpenStack Cinder Container Storage Interface (CSI) driver for dynamic provisioning instead of the in-tree driver, change the default storage class from **standard** to **standard-csi** by updating storage class annotations.

The OpenStack Cinder CSI driver uses the **cinder.csi.openstack.org** parameter key to support dynamic provisioning.

To enable OpenStack Cinder CSI provisioning in OpenShift Container Platform, it is recommended that you overwrite the default in-tree storage class with **standard-csi**. Alternatively, you can create the persistent volume claim (PVC) and specify the storage class as "standard-csi".

In OpenShift Container Platform, the default storage class references the in-tree Cinder driver. However, with CSI automatic migration enabled, volumes created using the default storage class actually use the CSI driver.

Use the following steps to apply the **standard-csi** storage class by overwriting the default in-tree storage class.

Procedure

1. List the storage class by running the following command:

```
$ oc get storageclass
```

Example output

NAME	PROVISIONER	RECLAIMPOLICY	VOLUMEBINDINGMODE
standard(default)	cinder.csi.openstack.org	Delete	WaitForFirstConsumer
standard-csi	kubernetes.io/cinder	Delete	WaitForFirstConsumer

2. Change the value of the annotation **storageclass.kubernetes.io/is-default-class** to **false** for the default storage class, as shown in the following example:

```
$ oc patch storageclass standard -p '{"metadata": {"annotations": {"storageclass.kubernetes.io/is-default-class": "false"}}}'
```

3. Make another storage class the default by adding or modifying the annotation as **storageclass.kubernetes.io/is-default-class=true**.

```
$ oc patch storageclass standard-csi -p '{"metadata": {"annotations": {"storageclass.kubernetes.io/is-default-class": "true"}}}'
```

4. Verify that the PVC is now referencing the CSI storage class by default:

```
$ oc get storageclass
```

Example output

NAME	PROVISIONER	RECLAIMPOLICY	VOLUMEBINDINGMODE
standard	kubernetes.io/cinder	Delete	WaitForFirstConsumer
standard-csi(default)	cinder.csi.openstack.org	Delete	WaitForFirstConsumer

5. Optional: You can define a new PVC without having to specify the storage class:

```

apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: cinder-claim
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi

```

A PVC that does not specify a specific storage class is automatically provisioned by using the default storage class.

6. Optional: After the new file has been configured, create it in your cluster:

```
$ oc create -f cinder-claim.yaml
```

6.19. OPENSTACK MANILA CSI DRIVER OPERATOR

You can provision and manage OpenStack Manila storage in OpenShift Container Platform using the OpenStack Manila Container Storage Interface (CSI) Driver Operator and driver, which provide dynamic volume provisioning.

6.19.1. Overview of Manila CSI Driver Operator

OpenShift Container Platform is capable of provisioning persistent volumes (PVs) using the Container Storage Interface (CSI) driver for the OpenStack Manila shared file system service.

Familiarity with persistent storage] and configuring CSI volumes is recommended when working with CSI Operator and driver. For more information, see "Understanding persistent storage" and "Configuring CSI volumes".

To create CSI-provisioned PVs that mount to Manila storage assets, OpenShift Container Platform installs the Manila CSI Driver Operator and the Manila CSI driver by default on any OpenStack cluster that has the Manila service enabled.

Manila CSI Driver Operator

The Manila CSI Driver Operator creates the required storage class that is needed to create persistent volumes claims (PVCs) for all available Manila share types. The Operator is installed in the **openshift-cluster-csi-drivers** namespace.

Manila CSI driver

The Manila CSI driver enables you to create and mount Manila PVs. The driver is installed in the **openshift-manila-csi-driver** namespace.

Additional resources

- [Understanding persistent storage](#)
- [Configuring CSI volumes](#)

6.19.2. About CSI

The Container Storage Interface (CSI) enables storage vendors to deliver plugins through a standard interface without modifying Kubernetes core code, replacing traditional embedded storage drivers.

CSI Operators give OpenShift Container Platform users storage options, such as volume snapshots, that are not possible with in-tree volume plugins.

6.19.3. Manila CSI Driver Operator limitations

Before deploying the Manila CSI Driver Operator, understand the protocol, snapshot, and FSGroup limitations to ensure compatibility with your OpenStack storage configuration.

The following limitations apply to the Manila Container Storage Interface (CSI) Driver Operator:

Only NFS is supported

OpenStack Manila supports many network-attached storage protocols, such as NFS, CIFS, and CEPHFS, and these can be selectively enabled in the OpenStack cloud. The Manila CSI Driver Operator in OpenShift Container Platform only supports using the NFS protocol. If NFS is not available and enabled in the underlying OpenStack cloud, you cannot use the Manila CSI Driver Operator to provision storage for OpenShift Container Platform.

Snapshots are not supported if the back end is CephFS-NFS

To take snapshots of persistent volumes (PVs) and revert volumes to snapshots, you must ensure that the Manila share type that you are using supports these features. A Red Hat OpenStack administrator must enable support for snapshots (**share type extra-spec snapshot_support**) and for creating shares from snapshots (**share type extra-spec create_share_from_snapshot_support**) in the share type associated with the storage class you intend to use.

FSGroups are not supported

Since Manila CSI provides shared file systems for access by multiple readers and multiple writers, it does not support the use of FSGroups. This is true even for persistent volumes created with the ReadWriteOnce access mode. It is therefore important not to specify the **fsType** attribute in any storage class that you manually create for use with Manila CSI Driver.



IMPORTANT

In Red Hat OpenStack Platform 16.x and 17.x, the Shared File Systems service (Manila) with CephFS through NFS fully supports serving shares to OpenShift Container Platform through the Manila CSI. However, this solution is not intended for massive scale. Be sure to review important recommendations in "CephFS NFS Manila-CSI Workload Recommendations for Red Hat OpenStack Platform".

Additional resources

- [CephFS NFS Manila-CSI Workload Recommendations for Red Hat OpenStack Platform](#)

6.19.4. Manila CSI volumes dynamic provisioning

OpenShift Container Platform installs a storage class for each available Manila share type. To dynamically provision shared storage volumes that support concurrent access from multiple pods, you must create persistent volume claims (PVCs) by using Manila Container Storage Interface CSI storage classes.

The YAML files that are created are completely decoupled from Manila and from its Container Storage Interface (CSI) plugin. As an application developer, you can dynamically provision ReadWriteMany

(RWX) storage and deploy pods with applications that safely consume the storage using YAML manifests.

You can use the same pod and persistent volume claim (PVC) definitions on-premise that you use with OpenShift Container Platform on AWS, Google Cloud, Azure, and other platforms, with the exception of the storage class reference in the PVC definition.



IMPORTANT

By default, the access rule that is assigned to a volume is **0.0.0.0/0**, which allows access from all IPv4 clients. To limit client access, create custom storage classes that use specific client IP addresses or subnets. For more information, see "Customizing Manila share access rules".



NOTE

Manila service is optional. If the service is not enabled in Red Hat OpenStack Platform (RHOSP), the Manila CSI driver is not installed and the storage classes for Manila are not created.

6.19.4.1. Dynamically provisioning Manila CSI volumes by using the web console

To dynamically provision shared storage volumes using the OpenShift Container Platform web console, create a persistent volume claim (PVC) by selecting a Manila CSI storage class.

Prerequisites

- RHOSP is deployed with appropriate Manila share infrastructure so that it can be used to dynamically provision and mount volumes in OpenShift Container Platform.

Procedure

1. In the OpenShift Container Platform web console, click **Storage → Persistent Volume Claims**
2. In the persistent volume claims overview, click **Create Persistent Volume Claim**
3. Define the required options on the resulting page.
 - a. Select the appropriate storage class.
 - b. Enter a unique name for the storage claim.
 - c. Select the access mode to specify read and write access for the PVC you are creating.



IMPORTANT

Use RWX if you want the PV that fulfills this PVC to be mounted to multiple pods on multiple nodes in the cluster.

4. Define the size of the storage claim.
5. Click **Create** to create the PVC and generate a PV.

Results

You can now use the new PVC to configure a pod.

6.19.4.2. Dynamically provisioning Manila CSI volumes by using the CLI

To dynamically provision shared storage volumes using the CLI, create and apply a persistent volume claim (PVC) YAML file that references a Manila CSI storage class.

Prerequisites

- RHOSP is deployed with appropriate Manila share infrastructure so that it can be used to dynamically provision and mount volumes in OpenShift Container Platform.

Procedure

1. Create and save a file with the **PersistentVolumeClaim** object described by the following YAML:

pvc-manila.yaml

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-manila
spec:
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 10Gi
  storageClassName: csi-manila-gold
```

- **spec.accessModes:** Use RWX if you want the PV that fulfills this PVC to be mounted to multiple pods on multiple nodes in the cluster.
 - **spec.storageClassName:** Specifies the name of the storage class that provisions the storage back end. Manila storage classes are provisioned by the Operator and have the **csi-manila-** prefix.
2. Create the object you saved in the previous step by running the following command:

```
$ oc create -f pvc-manila.yaml
```

A new PVC is created.

3. To verify that the volume was created and is ready, run the following command:

```
$ oc get pvc pvc-manila
```

The **pvc-manila** shows that it is **Bound**.

Results

You can now use the new PVC to configure a pod.

6.19.5. Customizing Manila share access rules

To improve storage security by controlling which clients can mount volumes, create custom Manila storage classes that limit access to specific IP addresses or subnets.

By default, OpenShift Container Platform creates Manila storage classes that provide access to all IPv4 clients. To limit client access, you can define custom storage classes that use specific client IP addresses or subnets by using the **nfs-ShareClient** parameter.



IMPORTANT

When using custom storage classes with restricted access rules, ensure that:

- The specified IP addresses or subnets include all OpenShift Container Platform nodes that need to access the storage.
- The Manila service in RHOSP supports the share type specified in the storage class.
- Network connectivity exists between the allowed clients and the Manila share servers.

Prerequisites

- Red Hat OpenStack Platform (RHOSP) is deployed with appropriate Manila share infrastructure.
- Access to a cluster with administrator privileges.

Procedure

1. Create a YAML file for your custom storage class based on the following example:

Example custom storage class file

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: csi-manila-gold-restricted
provisioner: manila.csi.openstack.org
parameters:
  type: gold
  nfs-ShareClient: "10.0.0.0/24,192.168.1.100"
  csi.storage.k8s.io/provisioner-secret-name: manila-csi-secret
  csi.storage.k8s.io/provisioner-secret-namespace: openshift-manila-csi-driver
  csi.storage.k8s.io/controller-expand-secret-name: manila-csi-secret
  csi.storage.k8s.io/controller-expand-secret-namespace: openshift-manila-csi-driver
  csi.storage.k8s.io/node-stage-secret-name: manila-csi-secret
  csi.storage.k8s.io/node-stage-secret-namespace: openshift-manila-csi-driver
  csi.storage.k8s.io/node-publish-secret-name: manila-csi-secret
  csi.storage.k8s.io/node-publish-secret-namespace: openshift-manila-csi-driver
allowVolumeExpansion: true
```

- **metadata.name:** Specifies a descriptive name for your custom storage class.

- **parameters.type**: Specifies the Manila share type. This type must match an existing share type in your RHOSP environment.
- **parameters.nfs.ShareClient**: Comma-separated list of IP addresses or CIDR subnets allowed to access the NFS shares. The **nfs-ShareClient** parameter accepts various formats:
 - Single IP address: **192.168.1.100**
 - CIDR subnet: **10.0.0.0/24**
 - Multiple entries: **10.0.0.0/24,192.168.1.100,172.16.0.0/16**

Ensure that the specified IP addresses or subnets include the OpenShift Container Platform cluster nodes to allow proper mounting of the persistent volumes.

In this example, access is restricted to the **10.0.0.0/24** subnet, and the specific IP address is **192.168.1.100**.

2. Apply the storage class from the file by running the following command:

```
$ oc apply -f custom-manila-storageclass.yaml
```

3. Verify that the storage class was created by running the following command:

```
$ oc get storageclass csi-manila-gold-restricted
```

Example output

NAME	PROVISIONER	RECLAIMPOLICY	VOLUMEBINDINGMODE
csi-manila-gold-restricted	manila.csi.openstack.org	Delete	Immediate
43m			true

4. Create a persistent volume claim (PVC) that uses the custom storage class based on the following example:

Example PVC file

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-manila-restricted
spec:
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 10Gi
  storageClassName: csi-manila-gold-restricted
```

spec.storageClassName is the name of your custom storage class that has restricted access. In this example, the name is **csi-manila-gold-restricted**.

5. Apply the PVC from the file by running the following command:

```
$ oc apply -f pvc-manila-restricted.yaml
```

6.20. SECRETS STORE CONTAINER STORAGE INTERFACE DRIVER OPERATOR

To improve secret security and integrate with enterprise secret management systems, you can use the Secrets Store CSI Driver Operator to mount secrets from external stores without persisting them on the cluster after pod termination.

6.20.1. Overview of Secrets Store CSI Driver Operator

To store and manage your secrets securely, configure the Secrets Store CSI Driver Operator to mount secrets from an external secret management system, such as Azure Key Vault, by using a provider plugin. Applications can then use the secret, but the secret does not persist on the system after pod termination.

Secret objects are stored with Base64 encoding. etcd provides encryption at rest for these secrets, but when secrets are retrieved, they are decrypted and presented to the user. If role-based access control is not configured properly on your cluster, anyone with API or etcd access can retrieve or modify a secret. Additionally, anyone who is authorized to create a pod in a namespace can use that access to read any secret in that namespace.

The Secrets Store CSI Driver Operator, **secrets-store.csi.k8s.io**, enables OpenShift Container Platform to mount multiple secrets, keys, and certificates stored in enterprise-grade external secrets stores into pods as a volume. The Secrets Store CSI Driver Operator communicates with the provider using gRPC to fetch the mount contents from the specified external secrets store. After the volume is attached, the data in it is mounted into the container's file system. Secrets store volumes are mounted in-line.

For more information about CSI inline volumes, see "CSI inline ephemeral volumes".

Familiarity with persistent storage and configuring CSI volumes is recommended when working with a CSI driver. For more information, see "Understanding persistent storage" and "Configuring CSI volumes".

Additional resources

- [CSI inline ephemeral volumes](#)
- [Understanding persistent storage](#)
- [Configuring CSI volumes](#)

6.20.1.1. Secrets store providers

You can store sensitive information needed by your applications in an external secret management system and use the Secrets Store CSI Driver Operator to mount the secret content as a pod volume. Using an external secret store protects information that you do not want developers to have and can be more secure than **secret** objects.

The Secrets Store CSI Driver Operator has been tested with the following secrets store providers:

- AWS Secrets Manager
- AWS Systems Manager Parameter Store

- Azure Key Vault
- Google Secret Manager
- HashiCorp Vault

**IMPORTANT**

IBM Z supports only HashiCorp Vault.

**NOTE**

Red Hat does not test all factors associated with third-party secrets store provider functionality. For more information about third-party support, see the "Red Hat third-party support policy".

6.20.2. About CSI

The Container Storage Interface (CSI) enables storage vendors to deliver plugins through a standard interface without modifying Kubernetes core code, replacing traditional embedded storage drivers.

CSI Operators give OpenShift Container Platform users storage options, such as volume snapshots, that are not possible with in-tree volume plugins.

6.20.3. Support for disconnected environments

You can use certain secrets store providers in disconnected OpenShift Container Platform clusters by configuring VPC endpoints or equivalent connectivity to enable communication between the driver and external secret management systems.

The following secrets store providers support using the Secrets Store CSI driver in disconnected clusters:

- AWS Secrets Manager
- Azure Key Vault
- Google Secret Manager
- HashiCorp Vault

To enable communication between Secrets Store CSI driver and the secrets store provider, configure Virtual Private Cloud (VPC) endpoints or equivalent connectivity to the corresponding secrets store provider, the OpenID Connect (OIDC) issuer, and the Secure Token Service (STS). The exact configuration depends on the secrets store provider, the authentication method, and the type of disconnected cluster.

**NOTE**

For more information about disconnected environments, see "About disconnected environments".

Additional resources

- [About disconnected environments](#)

6.20.4. Support for network policies

The Secrets Store CSI Driver Operator uses pre-defined **NetworkPolicies** to control ingress and egress traffic for enhanced security of the Operator and driver components.

The following table summarizes the default ingress and egress rules:

Component	Ingress ports	Egress ports	Description
Secrets Store CSI Driver Operator	8443	6443	Accesses metrics and communicates with the API server
Secrets Store CSI driver	8095	6443	Accesses metrics and communicates with the API server

6.20.5. Installing the Secrets Store CSI driver

To enable OpenShift Container Platform to mount secrets from external secret management systems, install the Secrets Store CSI Driver Operator and create a **ClusterCSIDriver** instance.

Prerequisites

- Access to the OpenShift Container Platform web console.
- Administrator access to the cluster.

Procedure

1. Install the Secrets Store CSI Driver Operator:
 - a. Log in to the web console.
 - b. Click **Ecosystem** → **Software Catalog**.
 - c. Locate the Secrets Store CSI Driver Operator by typing "Secrets Store CSI" in the filter box.
 - d. Click the **Secrets Store CSI Driver Operator** button.
 - e. On the **Secrets Store CSI Driver Operator** page, click **Install**.
 - f. On the **Install Operator** page, ensure that:
 - **All namespaces on the cluster (default)** is selected.
 - **Installed Namespace** is set to **openshift-cluster-csi-drivers**.
 - g. Click **Install**.
After the installation finishes, the Secrets Store CSI Driver Operator is listed in the **Installed Operators** section of the web console.
2. Create the **ClusterCSIDriver** instance for the driver (**secrets-store.csi.k8s.io**):

- a. Click **Administration** → **CustomResourceDefinitions** → **ClusterCSIDriver**.
- b. On the **Instances** tab, click **Create ClusterCSIDriver**.
Use the following YAML file:

```
apiVersion: operator.openshift.io/v1
kind: ClusterCSIDriver
metadata:
  name: secrets-store.csi.k8s.io
spec:
  managementState: Managed
```

- c. Click **Create**.

6.20.6. Installing the Secrets Store CSI driver by using the CLI

The Secrets Store CSI driver is typically installed in the namespace **openshift-cluster-csi-drivers**. This namespace is present in the cluster as part of the installation of the Cluster Storage Operator.

Procedure

1. Create an **OperatorGroup** object by running the following command:

```
$ oc apply -f - <<EOF
apiVersion: operators.coreos.com/v1
kind: OperatorGroup
metadata:
  name: openshift-cluster-csi-drivers
  namespace: openshift-cluster-csi-drivers
spec: {}
EOF
```

2. Create a **Subscription** object by running the following command:

```
$ oc apply -f - <<EOF
apiVersion: operators.coreos.com/v1alpha1
kind: Subscription
metadata:
  name: secrets-store-csi-driver-operator
  namespace: openshift-cluster-csi-drivers
spec:
  channel: stable
  installPlanApproval: Automatic
  name: secrets-store-csi-driver-operator
  source: redhat-operators
  sourceNamespace: openshift-marketplace
EOF
```

3. Wait for the Operator to be up and running. You can check the Operator status by running the following command:

```
$ oc get csv -n openshift-cluster-csi-drivers
```


Example output

NAME	DISPLAY	VERSION	RELEASE	REPLACES	PHASE
secrets-store-csi-driver-operator.v4.22.0-202607151755					Secrets Store CSI Driver Operator
4.22.0-202607151755				Succeeded	

4. Create the Cluster CSI driver by running the following command:

```
$ oc apply -f - <<EOF
apiVersion: operator.openshift.io/v1
kind: ClusterCSIDriver
metadata:
  name: secrets-store.csi.k8s.io
spec:
  managementState: Managed
EOF
```

Verification

- Verify that the Operator is running:

```
$ oc get operator -n openshift-cluster-csi-drivers
```

Example output

NAME	AGE
secrets-store-csi-driver-operator.openshift-cluster-csi-drivers	7m55s

- Verify that the pods are running:

```
$ oc get pods -n openshift-cluster-csi-drivers
```

Example output

NAME	READY	STATUS	RESTARTS	AGE
....				
....				
secrets-store-csi-driver-node-bhp2d	3/3	Running	0	45s
secrets-store-csi-driver-operator-74696fc7bc-qjrl8	1/1	Running	0	7m40s

Next steps

- [Mounting secrets from an external secrets store to a CSI volume](#)

6.20.7. Uninstalling the Secrets Store CSI Driver Operator

To remove the Secrets Store CSI Driver Operator and free cluster resources, uninstall the Operator after stopping applications and removing the CSI driver.

Prerequisites

- Access to the OpenShift Container Platform web console.

- Administrator access to the cluster.

Procedure

1. Stop all application pods that use the **secrets-store.csi.k8s.io** provider.
2. Remove any third-party provider plug-in for your chosen secret store.
3. Remove the Container Storage Interface (CSI) driver and associated manifests:
 - a. Click **Administration** → **CustomResourceDefinitions** → **ClusterCSIDriver**.
 - b. On the **Instances** tab, for **secrets-store.csi.k8s.io**, on the far left side, click the drop-down menu, and then click **Delete ClusterCSIDriver**.
 - c. When prompted, click **Delete**.
4. Verify that the CSI driver pods are no longer running.
5. Uninstall the Secrets Store CSI Driver Operator:



NOTE

Before you can uninstall the Operator, you must remove the CSI driver first.

- a. Click **Ecosystem** → **Installed Operators**.
- b. On the **Installed Operators** page, scroll or type "Secrets Store CSI" into the **Search by name** box to find the Operator, and then click it.
- c. On the upper, right of the **Installed Operators > Operator details** page, click **Actions** → **Uninstall Operator**.
- d. When prompted on the **Uninstall Operator** window, click the **Uninstall** button to remove the Operator from the namespace. Any applications deployed by the Operator on the cluster need to be cleaned up manually.
After uninstalling, the Secrets Store CSI Driver Operator is no longer listed in the **Installed Operators** section of the web console.

6.20.8. Additional resources

- [Configuring CSI volumes](#)

6.21. CIFS/SMB CSI DRIVER OPERATOR

You can provision and manage Common Internet File System (CIFS)/Server Message Block (SMB) network shares in OpenShift Container Platform by using the CIFS/SMB Container Storage Interface (CSI) Driver Operator, which supports dynamic volume provisioning.

6.21.1. Overview of the CIFS/SMB Driver Operator

You can provision persistent volumes (PVs) that mount to network file shares by using the CIFS/SMB CSI Driver Operator, which supports dynamic volume provisioning for on-demand storage.

Familiarity with persistent storage and configuring CSI volumes is recommended when working with a CSI Operator and driver. For more information, see "Understanding persistent volumes" and "Configuring CSI volumes".

After installing the CIFS/SMB CSI Driver Operator, OpenShift Container Platform installs corresponding pods for the Operator and driver in the **openshift-cluster-csi-drivers** namespace by default. This allows the CIFS/SMB CSI Driver to create CSI-provisioned persistent volumes (PVs) that mount to CIFS/SMB shares.

CIFS/SMB CSI Driver Operator

After the CIFS/SMB CSI Driver Operator is installed, it does not create a storage class by default to use to create persistent volume claims (PVCs). However, you can manually create the CIFS/SMB **StorageClass** for dynamic provisioning (see "Dynamic provisioning"). The CIFS/SMB CSI Driver Operator supports dynamic volume provisioning by allowing storage volumes to be created on-demand. This eliminates the need for cluster administrators to pre-provision storage.

CIFS/SMB CSI driver

The CIFS/SMB CSI driver enables you to create and mount CIFS/SMB PVs.

Additional resources

- [Understanding persistent storage](#)
- [Configuring CSI volumes](#)
- [Dynamic provisioning](#)

6.21.2. About CSI

The Container Storage Interface (CSI) enables storage vendors to deliver plugins through a standard interface without modifying Kubernetes core code, replacing traditional embedded storage drivers.

CSI Operators give OpenShift Container Platform users storage options, such as volume snapshots, that are not possible with in-tree volume plugins.

6.21.3. Limitations of the CIFS/SMB Driver Operator

Before you deploy Common Internet File System (CIFS)/Server Message Block (SMB) storage, review the CSI driver limitations, including no support for FIPS mode, HTTP proxy, DFS, or Kerberos authentication.

The following limitations apply to the CIFS/SMB CSI Driver Operator:

- FIPS mode is not supported:
When Federal Information Processing Standards (FIPS) mode is enabled, the use of md4 and md5 are disabled, which prevents users from using ntlm, ntlmv2, or ntlmssp authentication. Also, signing cannot be used because it uses md5. Any CIFS mount that uses these methods fails when FIPS mode is enabled.
- Using HTTP proxy configuration to connect to outside of the cluster SMB servers is not supported by the CSI driver.
Since CIFS/SMB is a LAN protocol, and though it can be routed to subnets, it is not designed to be extended over the WAN, and does not support HTTP proxy settings.
- The CIFS/SMB CSI Driver Operator does *not* support Windows Distributed File System (DFS).

- Kerberos authentication is not supported.
- SMB CSI was tested with Samba v4.21.2 and Windows Server 2019 and Windows Server 2022.

6.21.4. Installing the CIFS/SMB CSI Driver Operator

Install and configure the CIFS/SMB Container Storage Interface (CSI) Driver Operator to enable CIFS/SMB storage in your cluster. This Red Hat Operator is not installed by default and requires manual installation.

Prerequisites

- Access to the OpenShift Container Platform web console.

Procedure

1. Log in to the web console.
2. Install the CIFS/SMB CSI Operator:
 - a. Click **Ecosystem** → **Software Catalog**.
 - b. Locate the CIFS/SMB CSI Operator by typing **CIFS/SMB CSI** in the filter box.
 - c. Click the **CIFS/SMB CSI Driver Operator** button.
 - d. On the **CIFS/SMB CSI Driver Operator** page, click **Install**.
 - e. On the **Install Operator** page, ensure that:
 - **All namespaces on the cluster (default)** is selected.
 - **Installed Namespace** is set to **openshift-cluster-csi-drivers**.
 - f. Click **Install**.After the installation finishes, the CIFS/SMB CSI Operator is listed in the **Installed Operators** section of the web console.

Next steps

Install the CIFS/SMB CSI Driver.

6.21.5. Installing the CIFS/SMB CSI Driver

To use network file shares as persistent volumes (PVs) in your cluster, create a **ClusterCSIDriver** resource after installing the CIFS/SMB CSI Driver Operator.

Prerequisites

- Access to the OpenShift Container Platform web console.
- CIFS/SMB CSI Driver Operator installed.

Procedure

1. Click **Administration** → **CustomResourceDefinitions** → **ClusterCSIDriver**.

2. On the **Instances** tab, click **Create ClusterCSIDriver**.

3. Use the following YAML file:

```
apiVersion: operator.openshift.io/v1
kind: ClusterCSIDriver
metadata:
  name: smb.csi.k8s.io
spec:
  managementState: Managed
```

4. Click **Create**.

5. Wait for the following Conditions to change to a "True" status:

- **SambaDriverControllerServiceControllerAvailable**
- **SambaDriverNodeServiceControllerAvailable**

6.21.6. Dynamic provisioning for CIFS/SMB

To automatically provision Common Internet File System (CIFS)/Server Message Block (SMB) volumes without pre-creating shares, create a Secret for server credentials, a storage class for the SMB source, and persistent volume claims (PVCs).

When dynamically provisioning volumes, a subdirectory is created with the persistent volume (PV) name under **source** defined in the storage class.

Prerequisites

- CIFS/SMB CSI Driver Operator and driver installed.
- You are logged in to the running OpenShift Container Platform cluster.
- You have installed the SMB server and know the following information about the server:
 - Hostname
 - Share name
 - Username and password

Procedure

1. Create a Secret for access to the Samba server running the following command with the following example YAML file:

```
$ oc create -f <file_name>.yaml
```

Example Secret YAML file

```
apiVersion: v1
kind: Secret
metadata:
  name: smbcreds
```

```
namespace: samba-server
stringData:
  username: <username>
  password: <password>
```

- **metadata.name:** Specifies the name of the Secret for the Samba server.
 - **metadata.namespace:** Specifies the namespace for the Secret for the Samba server.
 - **stringData.username:** Specifies the username for the Secret for the Samba server.
 - **stringData.password:** Specifies the password for the Secret for the Samba server.
2. Create a storage class by running the following command with the following example YAML file:

```
$ oc create -f <sc_file_name>.yaml
```

Where **<sc_file_name>.yaml** is the name of the storage class YAML file.

Example storage class YAML file

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: <sc_name>
provisioner: smb.csi.k8s.io
parameters:
  source: //<hostname>/<shares>
  csi.storage.k8s.io/provisioner-secret-name: smbcreds
  csi.storage.k8s.io/provisioner-secret-namespace: samba-server
  csi.storage.k8s.io/node-stage-secret-name: smbcreds
  csi.storage.k8s.io/node-stage-secret-namespace: samba-server
reclaimPolicy: Delete
volumeBindingMode: Immediate
mountOptions:
  - dir_mode=0777
  - file_mode=0777
  - uid=1001
  - gid=1001
```

- **metadata.name:** Specifies the name of the storage class.
 - **parameters.source:** The Samba server must be installed somewhere that is reachable from the cluster with **<hostname>** being the hostname for the Samba server and **<shares>** the path the server is configured to have among the exported shares.
 - **parameters.csi.storage.k8s.io/provisioner-secret-name:** Specifies the name of the Secret for the Samba server that was set in the previous step. If the **csi.storage.k8s.io/provisioner-secret** is provided, a subdirectory is created with the PV name under **source**.
 - **parameters.csi.storage.k8s.io/provisioner-secret-namespace:** Specifies the namespace for the Secret for the Samba server that was set in the previous step.
3. Create a PVC:

- a. Create a PVC by running the following command with the following example YAML file:

```
$ oc create -f <pv_file_name>.yaml
```

Where **<pv_file_name>.yaml** is the name of the PVC YAML file.

Example PVC YAML file

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: <pvc_name>
spec:
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: <storage_amount>
  storageClassName: <sc_name>
```

- **metadata.name:** Specifies the name of the PVC.
- **spec.resources.requests.storage:** Specifies the storage request amount.
- **spec.storage.ClassName:** Specifies the name of the CIFS/SMB storage class that you created in the previous step.

- b. Ensure that the PVC was created and is in the "Bound" status by running the following command:

```
$ oc describe pvc <pvc_name>
```

Where **<pvc_name>** is the name of the PVC that you created in the preceding step.

Example output

```
Name:      pvc-test
Namespace: default
StorageClass: samba
Status:    Bound
...
```

PVC is in Bound status.

6.21.7. Static provisioning for CIFS/SMB

You can use static provisioning to create a persistent volume (PV) and persistent volume claim (PVC) to consume existing Server Message Block protocol (SMB) shares:

Prerequisites

- Access to the OpenShift Container Platform web console.
- CIFS/SMB CSI Driver Operator and driver installed.

- You have installed the SMB server and know the following information about the server:
 - Hostname
 - Share name
 - Username and password

Procedure

1. Create a Secret for access to the Samba server running the following command with the following example YAML file:

```
$ oc create -f <file_name>.yaml
```

Example Secret YAML file

```
apiVersion: v1
kind: Secret
metadata:
  name: smbcreds
  namespace: samba-server
stringData:
  username: <username>
  password: <password>
```

- **metadata.name:** Specifies the name of the Secret for the Samba server.
 - **metadata.namespace:** Specifies the namespace for the Secret for the Samba server.
 - **stringData.username:** Specifies the username for the Secret for the Samba server.
 - **stringData.password:** Specifies the password for the Secret for the Samba server.
2. Create a PV by running the following command with the following example YAML file:

```
$ oc create -f <pv_file_name>.yaml
```

Where **<pv_file_name>.yaml** is the name of the PV YAML file.

Example PV YAML file

```
apiVersion: v1
kind: PersistentVolume
metadata:
  annotations:
    pv.kubernetes.io/provisioned-by: smb.csi.k8s.io
  name: <pv_name>
spec:
  capacity:
    storage: 100Gi
  accessModes:
    - ReadWriteMany
  persistentVolumeReclaimPolicy: Retain
  storageClassName: ""
```



```

mountOptions:
  - dir_mode=0777
  - file_mode=0777
csi:
  driver: smb.csi.k8s.io
  volumeHandle: smb-server.default.svc.cluster.local/share##
  volumeAttributes:
    source: //<hostname>/<shares>
  nodeStageSecretRef:
    name: <secret_name_shares>
    namespace: <namespace>

```

- **metadata.name:** Specifies the name of the PV.
- **spec.csi.volumeHandle** format: **{smb-server-address}#{sub-dir-name}#{share-name}**. Ensure that this value is unique for every share in the cluster.
- **spec.csi.volumeAttributes.source:** The Samba server must be installed somewhere that is reachable from the cluster with **<hostname>** being the hostname for the Samba server and **<shares>** the path the server is configured to have among the exported shares.
- **spec.csi.nodeStageSecretRef.name:** Specifies the name of the Secret for the shares.
- **spec.csi.nodeStageSecretRef.namespace:** Specifies the applicable namespace.

3. Create a PVC:

- Create a PVC by running the following command with the following example YAML file:

```
$ oc create -f <pv_file_name>.yaml
```

Where **<pv_file_name>.yaml** is the name of the PVC YAML file.

Example PVC YAML file

```

kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: <pvc_name>
spec:
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: <storage_amount>
  storageClassName: ""
  volumeName: <pv_name>

```

- **metadata.name:** Specifies the name of the PVC.
- **spec.resources.requests.storage:** Specifies the storage request amount.
- **spec.volumeName:** Specifies the name of the PV from the first step.

- b. Ensure that the PVC was created and is in the **Bound** status by running the following command:

```
$ oc describe pvc <pvc_name>
```

Where **<pvc_name>** is the name of the PVC that you created in the preceding step.

Example output

```
Name:      pvc-test
Namespace: default
StorageClass:
Status:    Bound
...
```

PVC is in **Bound** status.

4. Create a deployment on Linux by running the following command with the following example YAML file:



NOTE

The following deployment is not mandatory for using the PV and PVC created in the previous steps. It is example of how they can be used.

```
$ oc create -f <deployment_file_name>.yaml
```

Where **<deployment_file_name>.yaml** is the name of the deployment YAML file.

Example deployment YAML file

```
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app: nginx
  name: <deployment_name>
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
      name: <deployment_name>
    spec:
      nodeSelector:
        "kubernetes.io/os": linux
      containers:
        - name: <deployment_name>
          image: quay.io/centos/centos:stream8
          command:
```

```

- "/bin/bash"
- "-c"
- set -euo pipefail; while true; do echo $(date) >> <mount_path>/outfile; sleep 1; done

1
volumeMounts:
- name: <vol_mount_name>
  mountPath: <mount_path>
  readOnly: false
volumes:
- name: <vol_mount_name>
  persistentVolumeClaim:
    claimName: <pvc_name>
strategy:
  rollingUpdate:
    maxSurge: 0
    maxUnavailable: 1
  type: RollingUpdate

```

- **metadata.name** and **spec.template.name**: Specifies the name of the deployment.
- **spec.template.spec.containers.command.set** and **spec.template.spec.containers.volumeMounts.mountpath**: Specifies the volume mount path.
- **spec.template.spec.containers.volumeMounts.mountpath** and **spec.template.spec.volumes.name**: Specifies the name of the volume mount.
- **spec.template.spec.volumes.persistentVolumeClaim.claimName**: Specifies the name of the PVC created in the preceding step.

5. Check the setup by running the **df -h** command in the container:

```
$ oc exec -it <pod_name> -- df -h
```

Where **<pod_name>** is the name of the pod.

Example output

```

Filesystem      Size  Used Avail Use% Mounted on
...
/dev/sda1       97G   21G   77G   22% /etc/hosts
//20.43.191.64/share 97G   21G   77G   22% /mnt/smb
...

```

In this example, there is a **/mnt/smb** directory mounted as a Common Internet File System (CIFS) filesystem.

6.22. VMWARE VSPHERE CSI DRIVER OPERATOR

You can provision and manage vSphere storage in OpenShift Container Platform by using the vSphere Container Storage Interface (CSI) Driver Operator and driver, which provide dynamic volume provisioning and eliminate the need to pre-provision storage.

6.22.1. Overview of vSphere CSI Driver Operator

OpenShift Container Platform can provision persistent volumes (PVs) using the Container Storage Interface (CSI) VMware vSphere driver for Virtual Machine Disk (VMDK) volumes.

Familiarity with persistent storage and configuring CSI volumes is recommended when working with a CSI Operator and driver. For more information, see "Understanding persistent volumes" and "Configuring CSI volumes".

To create CSI-provisioned persistent volumes (PVs) that mount to vSphere storage assets, OpenShift Container Platform installs the vSphere CSI Driver Operator and the vSphere CSI driver by default in the **openshift-cluster-csi-drivers** namespace.

vSphere CSI Driver Operator

The Operator provides a storage class, called **thin-csi**, that you can use to create persistent volume claims (PVCs). The vSphere CSI Driver Operator supports dynamic volume provisioning by allowing storage volumes to be created on-demand, eliminating the need for cluster administrators to pre-provision storage. You can disable this default storage class if desired (see "Managing the default storage class").

vSphere CSI driver

The driver enables you to create and mount vSphere PVs. In OpenShift Container Platform 4.20, and later, the driver version is 3.6.0 The vSphere CSI driver supports all of the file systems supported by the underlying Red Hat Enterprise Linux CoreOS (RHCOS) release, including XFS and Ext4. For more information about supported file systems, see "Overview of available file systems".



NOTE

For new installations, OpenShift Container Platform 4.13 and later provides automatic migration for the vSphere in-tree volume plugin to its equivalent CSI driver. Updating to OpenShift Container Platform 4.15 and later also provides automatic migration. For more information about updating and migration, see "CSI automatic migration".

CSI automatic migration should be seamless. Migration does not change how you use all existing API objects, such as persistent volumes, persistent volume claims, and storage classes.

Additional resources

- [Understanding persistent storage](#)
- [Configuring CSI volumes](#)
- [Managing the default storage class](#)
- [Overview of available file systems](#)
- [CSI automatic migration](#)

6.22.2. About CSI

The Container Storage Interface (CSI) enables storage vendors to deliver plugins through a standard interface without modifying Kubernetes core code, replacing traditional embedded storage drivers.

CSI Operators give OpenShift Container Platform users storage options, such as volume snapshots, that are not possible with in-tree volume plugins.

6.22.3. vSphere CSI limitations

Before deploying vSphere storage volumes, verify that your configuration meets the static provisioning and snapshot restoration requirements to avoid compatibility issues.

The following limitations apply to the vSphere Container Storage Interface (CSI) Driver Operator:

- The vSphere CSI Driver supports dynamic and static provisioning. However, when using static provisioning in the PV specifications, do not use the key **storage.kubernetes.io/csiProvisionerIdentity** in **csi.volumeAttributes** because this key indicates dynamically provisioned PVs.
- OpenShift Container Platform does not support restoring volume snapshots in a topology domain that does not have access to the datastore where the snapshot resides. You must manually schedule pods that use a persistent volume claim (PVC) that restore a snapshot to a region and zone with the snapshot. Using a shared datastore across all regions and zones meets this requirement.

6.22.4. vSphere storage policy

The vSphere CSI Driver Operator storage class uses vSphere's storage policy. OpenShift Container Platform automatically creates a storage policy that targets datastore configured in cloud configuration.

```
kind: StorageClass
apiVersion: storage.k8s.io/v1
metadata:
  name: thin-csi
provisioner: csi.vsphere.vmware.com
parameters:
  StoragePolicyName: "$openshift-storage-policy-xxxx"
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: false
reclaimPolicy: Delete
```

6.22.5. ReadWriteMany vSphere volume support

You can provision ReadWriteMany (RWX) volumes that allow multiple pods to access the same storage simultaneously when vSAN file service is configured in your vSphere environment.

If vSAN file service is not configured, then ReadWriteOnce (RWO) is the only access mode available. If you do not have vSAN file service configured, and you request RWX, the volume fails to get created and an error is logged.

For more information about configuring the vSAN file service in your environment, see "vSAN File Service".

You can request RWX volumes by making the following persistent volume claim (PVC):

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: myclaim
spec:
  resources:
    requests:
```

```

storage: 1Gi
accessModes:
- ReadWriteMany
storageClassName: thin-csi

```

Requesting a PVC of the RWX volume type should result in provisioning of persistent volumes (PVs) backed by the vSAN file service.

Additional resources

- [vSAN File Service](#)

6.22.6. VMware vSphere CSI Driver Operator requirements

To successfully install and operate the vSphere CSI Driver Operator, verify that your environment meets the minimum VMware vSphere, vCenter, and virtual machine version requirements.

To install the vSphere Container Storage Interface (CSI) Driver Operator, the following requirements must be met:

- VMware vSphere version 8.0 Update 1 or later; or VMware vSphere Foundation (VVF) 9; or VMware Cloud Foundation (VCF) 5 or later
- vCenter version 8.0 Update 1 or later; or VVF 9; or VCF 5 or later
- Virtual machines of hardware version 15 or later
- No third-party vSphere CSI driver already installed in the cluster

If a third-party vSphere CSI driver is present in the cluster, OpenShift Container Platform does not overwrite it. The presence of a third-party vSphere CSI driver prevents OpenShift Container Platform from updating to OpenShift Container Platform 4.13 or later.



NOTE

The VMware vSphere CSI Driver Operator is supported only on clusters deployed with **platform: vsphere** in the installation manifest.

You can create a custom role for the Container Storage Interface (CSI) driver, the vSphere CSI Driver Operator, and the vSphere Problem Detector Operator. The custom role can include privilege sets that assign a minimum set of permissions to each vSphere object. This means that the CSI driver, the vSphere CSI Driver Operator, and the vSphere Problem Detector Operator can establish a basic interaction with these objects.



IMPORTANT

Installing an OpenShift Container Platform cluster in a vCenter is tested against a full list of privileges as described in the "Required vCenter account privileges" section. By adhering to the full list of privileges, you can reduce the possibility of unexpected and unsupported behaviors that might occur when creating a custom role with a set of restricted privileges.

To remove a third-party CSI driver, see "Removing a third-party vSphere CSI Driver".

Additional resources

Additional resources

- [Removing a third-party vSphere CSI Driver Operator](#)

6.22.7. Removing a third-party vSphere CSI Driver Operator

To enable cluster upgrades to OpenShift Container Platform 4.13 or later, remove any third-party vSphere CSI drivers and allow the Red Hat-supported built-in driver to manage storage.

OpenShift Container Platform 4.10, and later, includes a built-in version of the vSphere Container Storage Interface (CSI) Operator Driver that is supported by Red Hat. If you have installed a vSphere CSI driver provided by the community or another vendor, updates to the next major version of OpenShift Container Platform, such as 4.13, or later, might be disabled for your cluster.

OpenShift Container Platform 4.12, and later, clusters are still fully supported, and updates to z-stream releases of 4.12, such as 4.12.z, are not blocked, but you must correct this state by removing the third-party vSphere CSI Driver before updates to next major version of OpenShift Container Platform can occur. Removing the third-party vSphere CSI driver does not require deletion of associated persistent volume (PV) objects, and no data loss should occur.

**NOTE**

These instructions may not be complete, so consult the vendor or community provider uninstall guide to ensure removal of the driver and components.

Procedure

1. Delete the third-party vSphere CSI Driver (VMware vSphere Container Storage Plugin) Deployment and Daemonset objects.
2. Delete the configmap and secret objects that were installed previously with the third-party vSphere CSI Driver.
3. Delete the third-party vSphere CSI driver **CSIDriver** object by running the following command:

```
$ oc delete CSIDriver csi.vsphere.vmware.com
```

Example output

```
csidriver.storage.k8s.io "csi.vsphere.vmware.com" deleted
```

Result

After you have removed the third-party vSphere CSI Driver from the OpenShift Container Platform cluster, installation of Red Hat's vSphere CSI Driver Operator automatically resumes, and any conditions that could block upgrades to OpenShift Container Platform 4.11, or later, are automatically removed. If you had existing vSphere CSI PV objects, their lifecycle is now managed by Red Hat's vSphere CSI Driver Operator.

6.22.8. vSphere persistent disks encryption

You can encrypt virtual machines (VMs) and dynamically provisioned persistent volumes (PVs) on OpenShift Container Platform running on top of vSphere.

**NOTE**

OpenShift Container Platform does not support RWX-encrypted PVs. You cannot request RWX PVs out of a storage class that uses an encrypted storage policy.

You must encrypt VMs before you can encrypt PVs, which you can do during or after installation.

For information about encrypting VMs, see:

- ["Requirements for encrypting virtual machines"](#)
- ["During installation: Step 7 of Installing RHCOS and starting the OpenShift Container Platform bootstrap process"](#)
- ["Enabling encryption on a vSphere cluster"](#)

After encrypting VMs, you can configure a storage class that supports dynamic encryption volume provisioning using the vSphere Container Storage Interface (CSI) driver. This can be accomplished in one of two ways using:

- **Datastore URL:** This approach is not very flexible, and forces you to use a single datastore. It also does not support topology-aware provisioning.
- **Tag-based placement:** Encrypts the provisioned volumes and uses tag-based placement to target specific datastores.

Additional resources

- [Requirements for encrypting virtual machines](#)
- [During installation: Step 7 of Installing RHCOS and starting the OpenShift Container Platform bootstrap process](#)
- [Enabling encryption on a vSphere cluster](#)

6.22.8.1. Using datastore URL

To encrypt persistent volumes by targeting a specific datastore, create a storage class that references an encryption-enabled storage policy and datastore URL.

Procedure

1. Find out the name of the default storage policy in your datastore that supports encryption. This is same policy that was used for encrypting your VMs.
2. Create a storage class that uses this storage policy:

```
kind: StorageClass
apiVersion: storage.k8s.io/v1
metadata:
  name: encryption
provisioner: csi.vsphere.vmware.com
parameters:
  storagePolicyName: <storage-policy-name>
  datastoreurl: "ds:///vmfs/volumes/vsan:522e875627d-b090c96b526bb79c/"
```


parameters.storagePolicyName is the name of the default storage policy in your datastore that supports encryption.

6.22.8.2. Using tag-based placement

To encrypt persistent volumes with flexible datastore selection, use tag-based placement that targets multiple datastores through vCenter tags and supports topology-aware provisioning.

Procedure

1. In vCenter create a category for tagging datastores that will be made available to this storage class. Also, ensure that **StoragePod(Datastore clusters)**, **Datastore**, and **Folder** are selected as Associable Entities for the created category.
2. In vCenter, create a tag that uses the category created earlier.
3. Assign the previously created tag to each datastore that will be made available to the storage class. Make sure that datastores are shared with hosts participating in the OpenShift Container Platform cluster.
4. In vCenter, from the main menu, click **Policies and Profiles**.
5. On the **Policies and Profiles** page, in the navigation pane, click **VM Storage Policies**.
6. Click **CREATE**.
7. Type a name for the storage policy.
8. Select **Enable host based rules** and **Enable tag based placement rules**.
9. In the **Next** tab:
 - a. Select **Encryption** and **Default Encryption Properties**.
 - b. Select the tag category created earlier, and select tag selected. Verify that the policy is selecting matching datastores.
10. Create the storage policy.
11. Create a storage class that uses the storage policy:

```
kind: StorageClass
apiVersion: storage.k8s.io/v1
metadata:
  name: csi-encrypted
provisioner: csi.vsphere.vmware.com
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer
parameters:
  storagePolicyName: <storage-policy-name>
```

parameters.storagePolicyName is the name of the storage policy that you created for encryption.

6.22.9. Multiple vCenter support for vSphere CSI

To achieve high availability across vSphere infrastructure without shared storage, configure up to three vCenter clusters during OpenShift Container Platform installation.

OpenShift Container Platform v4.17, and later, supports this capability.



NOTE

Multiple vCenters can only be configured **during** installation. Multiple vCenters **cannot** be configured after installation.

The maximum number of supported vCenter clusters is three.

6.22.9.1. Configuring multiple vCenters during installation

To enable multi-vCenter support for high availability without shared storage, specify multiple vSphere clusters in the installation configuration before deploying your cluster.

Procedure

- Specify multiple vSphere clusters during installation. For information, see "Installation configuration parameters for vSphere".

Additional resources

- [Installation configuration parameters for vSphere](#)

6.22.10. vSphere CSI topology overview

You can improve cluster resilience and avoid single points of failure by deploying OpenShift Container Platform across multiple vSphere zones and regions using topology-aware storage provisioning.

This is accomplished by defining zone and region categories in vCenter, and then assigning these categories to different failure domains, such as a compute cluster, by creating tags for these zone and region categories. After you have created the appropriate categories, and assigned tags to vCenter objects, you can create additional machinesets that create virtual machines (VMs) that are responsible for scheduling pods in those failure domains.

The following example defines two failure domains with one region and two zones:

Table 6.6. vSphere storage topology with one region and two zones

Compute cluster	Failure domain	Description
Compute cluster: ocp1, Data center: Atlanta	openshift-region: us-east-1 (tag), openshift-zone: us-east-1a (tag)	This defines a failure domain in region us-east-1 with zone us-east-1a.
Computer cluster: ocp2, Data center: Atlanta	openshift-region: us-east-1 (tag), openshift-zone: us-east-1b (tag)	This defines a different failure domain within the same region called us-east-1b.

6.22.10.1. vSphere CSI topology requirements

The following guidelines are recommended for vSphere CSI topology:

- You are strongly recommended to add topology tags to data centers and compute clusters, and **not** to hosts.
vsphere-problem-detector provides alerts if the **openshift-region** or **openshift-zone** tags are not defined at the data center or compute cluster level, and each topology tag (**openshift-region** or **openshift-zone**) should occur only once in the hierarchy.



NOTE

Ignoring this recommendation only results in a log warning from the CSI driver and duplicate tags lower in the hierarchy, such as hosts, are ignored; VMware considers this an invalid configuration, and therefore to prevent problems you should not use it.

- Volume provisioning requests in topology-aware environments attempt to create volumes in datastores accessible to all hosts under a given topology segment. This includes hosts that do not have Kubernetes node VMs running on them. For example, if the vSphere Container Storage Plug-in driver receives a request to provision a volume in **zone-a**, applied on the data center **dc-1**, all hosts under **dc-1** must have access to the datastore selected for volume provisioning. The hosts include those that are directly under **dc-1**, and those that are a part of clusters inside **dc-1**.
- For additional recommendations, you should read the VMware "Guidelines and Best Practices for Deployment with Topology".

Additional resources

- [Guidelines and Best Practices for Deployment with Topology](#)

6.22.10.2. Creating vSphere storage topology during installation

To enable automatic topology-aware storage provisioning across vSphere failure domains, configure regions and zones during cluster installation.

Procedure

- Specify the topology during installation. See "Configuring regions and zones for a VMware vCenter".
No additional action is necessary and the default storage class that is created by OpenShift Container Platform is topology aware and should allow provisioning of volumes in different failure domains.

Additional resources

- [Configuring regions and zones for a VMware vCenter](#)

6.22.10.3. Creating vSphere storage topology postinstallation

To enable topology-aware storage provisioning after cluster installation, configure vCenter tags, create failure domains, and define storage classes that target datastores in specific zones and regions.

Procedure

1. In the VMware vCenter vSphere client GUI, define appropriate zone and region categories and tags.

While vSphere allows you to create categories with any arbitrary name, OpenShift Container Platform strongly recommends use of **openshift-region** and **openshift-zone** names for defining topology categories.

For more information about vSphere categories and tags, see the "VMware vCenter documentation".

2. In OpenShift Container Platform, create failure domains. For more information, see "Specifying multiple regions and zones for your cluster on vSphere".
3. Create a tag to assign to datastores across failure domains:
When an OpenShift Container Platform spans more than one failure domain, the datastore might not be shared across those failure domains, which is where topology-aware provisioning of persistent volumes (PVs) is useful.

- a. In vCenter, create a category for tagging the datastores. For example, **openshift-zonal-datastore-cat**. You can use any other category name, provided the category uniquely is used for tagging datastores participating in OpenShift Container Platform cluster. Also, ensure that **StoragePod**, **Datastore**, and **Folder** are selected as Associable Entities for the created category.
- b. In vCenter, create a tag that uses the previously created category. This example uses the tag name **openshift-zonal-datastore**.
- c. Assign the previously created tag (in this example **openshift-zonal-datastore**) to each datastore in a failure domain that would be considered for dynamic provisioning.



NOTE

You can use any names you like for datastore categories and tags. The names used in this example are provided as recommendations. Ensure that the tags and categories that you define uniquely identify only datastores that are shared with all hosts in the OpenShift Container Platform cluster.

4. As needed, create a storage policy that targets the tag-based datastores in each failure domain:
 - a. In vCenter, from the main menu, click **Policies and Profiles**.
 - b. On the **Policies and Profiles** page, in the navigation pane, click **VM Storage Policies**.
 - c. Click **CREATE**.
 - d. Type a name for the storage policy.
 - e. For the rules, choose Tag Placement rules and select the tag and category that targets the desired datastores (in this example, the **openshift-zonal-datastore** tag).
The datastores are listed in the storage compatibility table.
5. Create a new storage class that uses the new zoned storage policy:
 - a. Click **Storage > StorageClasses**.
 - b. On the **StorageClasses** page, click **Create StorageClass**.

- c. Type a name for the new storage class in **Name**.
- d. Under **Provisioner**, select **csi.vsphere.vmware.com**.
- e. Under **Additional parameters**, for the StoragePolicyName parameter, set **Value** to the name of the new zoned storage policy that you created earlier.
- f. Click **Create**.

Example output

```
kind: StorageClass
apiVersion: storage.k8s.io/v1
metadata:
  name: zoned-sc
provisioner: csi.vsphere.vmware.com
parameters:
  StoragePolicyName: zoned-storage-policy
reclaimPolicy: Delete
allowVolumeExpansion: true
volumeBindingMode: WaitForFirstConsumer
```

- **metadata.name:** New topology aware storage class name.
- **parameters.StoragePolicyName:** Specify zoned storage policy.



NOTE

You can also create the storage class by editing the preceding YAML file and running the command **oc create -f \$FILE**.

Additional resources

- [Specifying multiple regions and zones for your cluster on vSphere](#)
- [VMware vSphere tag documentation](#)

6.22.10.4. Creating vSphere storage topology without an infra topology

To configure topology-aware storage provisioning without using the **infrastructure** object, define vCenter categories and tags, then configure the **ClusterCSIDriver** object to recognize topology zones and regions.



NOTE

OpenShift Container Platform recommends using the **infrastructure** object for specifying failure domains in a topology aware setup. Specifying failure domains in the **infrastructure** object and specify topology-categories in the **ClusterCSIDriver** object at the same time is an unsupported operation.

Procedure

1. In the VMware vCenter vSphere client GUI, define appropriate zone and region categories and tags.

While vSphere allows you to create categories with any arbitrary name, OpenShift Container Platform strongly recommends use of **openshift-region** and **openshift-zone** names for defining topology.

For more information about vSphere categories and tags, see the "VMware vCenter documentation".

2. To allow the container storage interface (CSI) driver to detect this topology, edit the **clusterCSIDriver** object YAML file **driverConfig** section:

- Specify the **openshift-zone** and **openshift-region** categories that you created earlier.
- Set **driverType** to **vSphere**.

```
~ $ oc edit clustercsidriver csi.vsphere.vmware.com -o yaml
```

Example output

```
apiVersion: operator.openshift.io/v1
kind: ClusterCSIDriver
metadata:
  name: csi.vsphere.vmware.com
spec:
  logLevel: Normal
  managementState: Managed
  observedConfig: null
  operatorLogLevel: Normal
  unsupportedConfigOverrides: null
  driverConfig:
    driverType: vSphere
    vSphere:
      topologyCategories:
        - openshift-zone
        - openshift-region
```

- **spec.driverConfig.driverType**: Ensure that **driverType** is set to **vSphere**.
- **spec.driverConfig.driverType.vSphere.topologyCategories**: **openshift-zone** and **openshift-region** categories created earlier in vCenter.

3. Verify that **CSINode** object has topology keys by running the following commands:

```
~ $ oc get csinode
```

Example output

```
NAME DRIVERS AGE
co8-4s88d-infra-2m5vd 1 27m
co8-4s88d-master-0 1 70m
co8-4s88d-master-1 1 70m
co8-4s88d-master-2 1 70m
co8-4s88d-worker-j2hmg 1 47m
co8-4s88d-worker-mbb46 1 47m
co8-4s88d-worker-zlk7d 1 47m
```

```
~ $ oc get csinode co8-4s88d-worker-j2hmg -o yaml
```

Example output

```
...
spec:
  drivers:
  - allocatable:
      count: 59
      name: csi-vsphere.vmware.com
      nodeID: co8-4s88d-worker-j2hmg
      topologyKeys:
      - topology.csi.vmware.com/openshift-zone
      - topology.csi.vmware.com/openshift-region
```

spec.topologyKeys lists the topology keys from vSphere **openshift-zone** and **openshift-region** categories.



NOTE

CSINode objects might take some time to receive updated topology information. After the driver is updated, **CSINode** objects should have topology keys in them.

4. Create a tag to assign to datastores across failure domains:
When an OpenShift Container Platform spans more than one failure domain, the datastore might not be shared across those failure domains, which is where topology-aware provisioning of persistent volumes (PVs) is useful.
 - a. In vCenter, create a category for tagging the datastores. For example, **openshift-zonal-datastore-cat**. You can use any other category name, provided the category uniquely is used for tagging datastores participating in OpenShift Container Platform cluster. Also, ensure that **StoragePod**, **Datastore**, and **Folder** are selected as Associable Entities for the created category.
 - b. In vCenter, create a tag that uses the previously created category. This example uses the tag name **openshift-zonal-datastore**.
 - c. Assign the previously created tag (in this example **openshift-zonal-datastore**) to each datastore in a failure domain that would be considered for dynamic provisioning.



NOTE

You can use any names you like for categories and tags. The names used in this example are provided as recommendations. Ensure that the tags and categories that you define uniquely identify only datastores that are shared with all hosts in the OpenShift Container Platform cluster.

5. Create a storage policy that targets the tag-based datastores in each failure domain:
 - a. In vCenter, from the main menu, click **Policies and Profiles**.
 - b. On the **Policies and Profiles** page, in the navigation pane, click **VM Storage Policies**.

- c. Click **CREATE**.
 - d. Type a name for the storage policy.
 - e. For the rules, choose Tag Placement rules and select the tag and category that targets the desired datastores (in this example, the **openshift-zonal-datastore** tag).
The datastores are listed in the storage compatibility table.
6. Create a new storage class that uses the new zoned storage policy:
- a. Click **Storage > StorageClasses**.
 - b. On the **StorageClasses** page, click **Create StorageClass**.
 - c. Type a name for the new storage class in **Name**.
 - d. Under **Provisioner**, select **csi.vsphere.vmware.com**.
 - e. Under **Additional parameters**, for the StoragePolicyName parameter, set **Value** to the name of the new zoned storage policy that you created earlier.
 - f. Click **Create**.

Example output

```
kind: StorageClass
apiVersion: storage.k8s.io/v1
metadata:
  name: zoned-sc
provisioner: csi.vsphere.vmware.com
parameters:
  StoragePolicyName: zoned-storage-policy
reclaimPolicy: Delete
allowVolumeExpansion: true
volumeBindingMode: WaitForFirstConsumer
```

- **metadata.name:** New topology aware storage class name.
- **parameters.StoragePolicyName:** Specify zoned storage policy.



NOTE

You can also create the storage class by editing the preceding YAML file and running the command **oc create -f \$FILE**.

Additional resources

- [VMware vSphere tag documentation](#)

6.22.10.5. vSphere topology results

To verify that topology-aware storage provisioning is working correctly, check that persistent volumes include zone and region node affinity labels that match the pod scheduling requirements.

Creating persistent volume claims (PVCs) and PVs from the topology aware storage class are truly zonal, and should use the datastore in their respective zone depending on how pods are scheduled:

Procedure

- Check that persistent volumes include zone and region labels as expected by running the following command:

```
$ oc get pv <pv_name> -o yaml
```

Example output

```
...
nodeAffinity:
  required:
    nodeSelectorTerms:
      - matchExpressions:
          - key: topology.csi.vmware.com/openshift-zone
            operator: In
            values:
              - <openshift_zone>
          - key: topology.csi.vmware.com/openshift-region
            operator: In
            values:
              - <openshift_region>
      ...
    persistentVolumeclaimPolicy: Delete
    storageClassName: <zoned_storage_class_name>
    volumeMode: Filesystem
  ...
```

- **nodeAffinity.required.nodeSelectorTerms.matchExpressions.key:** PV has zoned keys.
- **storageClassName:** PV is using the zoned storage class.

6.22.11. Changing the maximum number of snapshots for vSphere

Configure the maximum number of snapshots per volume globally or for specific datastore types to balance storage capacity and performance in your vSphere environment.

The default maximum number of snapshots per volume in vSphere Container Storage Interface (CSI) is 3. You can change the maximum number up to 32 per volume. However, be aware that increasing the snapshot maximum involves a performance trade off, so for better performance use only 2 to 3 snapshots per volume.

For more VMware snapshot performance recommendations, see "Best practices for using VMware snapshots in the vSphere environment".

Prerequisites

- Access to the cluster with administrator rights.

Procedure

1. Check the current secret by the running the following command:

```
$ oc -n openshift-cluster-csi-drivers get secret/vsphere-csi-config-secret -o
jsonpath='{.data.cloud\.conf}' | base64 -d
```

Example output

```
# Labels with topology values are added dynamically via operator
[Global]
cluster-id = vsphere-01-cwv8p

# Populate VCenters (multi) after here
[VirtualCenter "vcenter.openshift.com"]
insecure-flag      = true
datacenters        = DEVQEdatacenter
password           = "xxxxxxx"
user               = "xxxxxxx@devcluster.openshift.com"
migration-datastore-url = ds:///vmfs/volumes/vsan:52c842f232751e0d-3253aadeac21ca82/
```

In this example, the global maximum number of snapshots is not configured, so the default value of 3 is applied.

2. Change the snapshot limit by running the following command:

- Set **global** snapshot limit:

```
$ oc patch clustercsidriver/csi.vsphere.vmware.com --type=merge -p '{"spec":
{"driverConfig":{"vSphere":{"globalMaxSnapshotsPerBlockVolume": 10}}}'

clustercsidriver.operator.openshift.io/csi.vsphere.vmware.com patched
```

In this example, the global limit is being changed to 10 (**globalMaxSnapshotsPerBlockVolume** set to 10).

- Set **Virtual Volume** snapshot limit:
This parameter sets the limit on the Virtual Volumes datastore only. The Virtual Volume maximum snapshot limit overrides the global constraint if set, but defaults to the global limit if it is not set.

```
$ oc patch clustercsidriver/csi.vsphere.vmware.com --type=merge -p '{"spec":
{"driverConfig":{"vSphere":{"granularMaxSnapshotsPerBlockVolumeInVVOL": 5}}}'
clustercsidriver.operator.openshift.io/csi.vsphere.vmware.com patched
```

In this example, the Virtual Volume limit is being changed to 5 (**granularMaxSnapshotsPerBlockVolumeInVVOL** set to 5).

- Set **vSAN** snapshot limit:
This parameter sets the limit on the vSAN datastore only. The vSAN maximum snapshot limit overrides the global constraint if set, but defaults to the global limit if it is not set. You can set a maximum value of 32 under vSAN ESA setup.

```
$ oc patch clustercsidriver/csi.vsphere.vmware.com --type=merge -p '{"spec":
{"driverConfig":{"vSphere":{"granularMaxSnapshotsPerBlockVolumeInVSAN": 7}}}'
clustercsidriver.operator.openshift.io/csi.vsphere.vmware.com patched
```

In this example, the vSAN limit is being changed to 7 (**granularMaxSnapshotsPerBlockVolumeInVSAN** set to 7).

Verification

- Verify that any changes you made are reflected in the config map by running the following command:

```
$ oc -n openshift-cluster-csi-drivers get secret/vsphere-csi-config-secret -o
jsonpath='{.data.cloud\conf}' | base64 -d
```

Example output

```
# Labels with topology values are added dynamically via operator
[Global]
cluster-id = vsphere-01-cwv8p

# Populate VCenters (multi) after here
[VirtualCenter "vcenter.openshift.com"]
insecure-flag      = true
datacenters        = DEVQEdatacenter
password           = "xxxxxxx"
user               = "xxxxxxx@devcluster.openshift.com"
migration-datastore-url = ds:///vmfs/volumes/vsan:52c842f232751e0d-3253aadeac21ca82/

[Snapshot]
global-max-snapshots-per-block-volume = 10
```

The parameter **global-max-snapshots-per-block-volume** is now set to 10.

Additional resources

- [Best practices for using VMware snapshots in the vSphere environment](#)

6.22.12. Migrating CNS volumes between datastores for vSphere

To optimize storage performance or free up capacity, you can migrate vSphere Cloud Native Storage (CNS) volumes between datastores without data loss.

If you are running out of space in your current datastore, or want to move to a more performant datastore, you can migrate VMware CNS volumes between datastores. This applies to both attached and detached volumes.

6.22.12.1. Limitations

- Requires VMware vSphere 8.0.2 or later, or VMware vSphere Foundation (VVF) 9, or VMware Cloud Foundation (VCF) 9
- Only one volume can be migrated at a time.
- RWX volumes are not supported.
- CNS volume should only be migrated to a datastore that is shared with all hosts that make up the OpenShift Container Platform cluster.

- Migrating volumes between different datastore in different datacenters is not supported.
- VMware HCX is not supported.

6.22.12.2. Additional limitations

For additional limitations, see "For vSphere 8", "For VCF 9", "For vSphere v8.0, more general information", and "For VCF 9, more general information".

Additional resources

- [For vSphere 8](#)
- [For VCF 9](#)
- [For vSphere v8.0, more general information](#)
- [For VCF 9, more general information](#)

6.22.13. Disabling and enabling storage on vSphere

To control storage integration with your vSphere environment during Day 2 operations, you can disable and re-enable the VMware vSphere Container Storage Interface (CSI) Driver as needed.

6.22.13.1. Consequences of disabling and enabling storage on vSphere

Before disabling or re-enabling vSphere storage, understand the impact on persistent volumes, pods, storage classes, and the Container Storage Interface (CSI) driver components.

The consequences of disabling and enabling storage on vSphere are described in the following table.

Table 6.7. Consequences of disabling/enabling storage on vSphere

Disabling	Enabling
● vSphere CSI Driver Operator un-installs the CSI driver. ● Storage container orchestration (CO) should be healthy. ● vSphere-problem-detector continues running, but does not emit alerts or events, and checks less frequently (once per 24 hours). ● All existing persistent volumes (PVs), persistent volume claims (PVCs), and vSphere storage policies are unchanged: ○ vSphere PVs cannot be used in new pods. ○ vSphere PVs stay mounted and attached forever to existing nodes for existing pods. These pods remain in terminating state indefinitely after deletion. ● Storage classes are removed	* vSphere CSI Driver Operator re-installs the CSI driver. * If necessary, the vSphere CSI Driver Operator creates the vSphere storage policy.

6.22.13.2. Disabling and enabling storage on vSphere

To control vSphere storage integration with your cluster, disable or enable the CSI driver by changing the management state in the **ClusterCSIDriver** resource.



IMPORTANT

Before running this procedure, carefully review the preceding "Consequences of disabling and enabling storage on vSphere" table and potential impacts to your environment.

Procedure

1. Click **Administration > CustomResourceDefinitions**.
2. On the **CustomResourceDefinitions** page next to the **Name** dropdown box, type "clustersidriver".
3. Click **CRD ClusterCSIDriver**.
4. Click the **Instances** tab.
5. Click **csi.vsphere.vmware.com**.
6. Click the **YAML** tab.
7. For **spec.managementState**, change the value to **Removed** or **Managed**:
 - **Removed**: storage is disabled

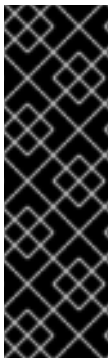
- **Managed:** storage is enabled
8. Click **Save**.
 9. If you are disabling storage, confirm that the driver has been removed:
 - a. Click **Workloads > Pods**.
 - b. On the **Pods** page, in the **Name** filter box type "vmware-vsphere-csi-driver".
The only item that should appear is the operator. For example: "vmware-vsphere-csi-driver-operator-559b97ffc5-w99fm"

6.22.14. Adding bare-metal nodes

OpenShift Container Platform has the ability to add bare-metal nodes to a cluster on vSphere as a Technology Preview feature.

However, if you add bare-metal nodes, you must remove the vSphere CSI Driver, otherwise the cluster is marked as degraded. For information about how to remove the driver and the consequences of doing this, see "Disabling and enabling storage on vSphere".

For information about how to add bare-metal nodes, see "Adding bare-metal compute machines to a vSphere cluster".



IMPORTANT

Adding bare-metal nodes is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

Additional resources

- [Disabling and enabling storage on vSphere](#)
- [Adding bare-metal compute machines to a vSphere cluster](#)

6.22.15. Increasing maximum volumes per node for vSphere

To support more storage volumes on individual nodes, you can increase the maximum volumes per node on homogeneous vSphere 8 or later environments.

For vSphere version 8 or later, or VMware vSphere Foundation (VVF) 9, or VMware Cloud Foundation (VCF) 9, you can increase the allowable number of volumes per node to a maximum of 255. Otherwise, the default value remains at 59.

**IMPORTANT**

You must have an homogeneous vSphere 8 environment that only contains ESXi 8 hypervisors, or an homogeneous VVF or VCF 9 environment that only contains ESXi 9 hypervisors. Heterogeneous environments that contain a mix of versions of ESXi are not allowed. In such heterogenous environment, if you set a value greater than 59, the cluster degrades.

Limitations

- You must be running VMware vSphere version 8 or later, or VVF 9, or VCF 9.
- You can potentially exceed the limit of 2048 virtual disks per host if you increase the maximum number of volumes per node on enough nodes. This can occur because there is no Distributed Resource scheduler (DRS) validation for vSphere to ensure you do not exceed this limit.

**IMPORTANT**

Increasing volumes per node is a Technology Preview feature only. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

6.22.15.1. Increasing the maximum allowable volumes per node for vSphere

To accommodate workloads requiring more than 59 persistent volumes per node, increase the maximum allowable volumes by configuring the **ClusterCSIDriver** resource.

Prerequisites

- Access to the OpenShift Container Platform web console.
- Access to the cluster as a user with the cluster-admin role.
- Access to VMware vSphere vCenter.
- In vCenter, ensure that the parameter **pvscsiCtrlr256DiskSupportEnabled** is set to 'True'.

**IMPORTANT**

Changing the **pvscsiCtrlr256DiskSupportEnabled** parameter is not fully supported by VMware. Also, the parameter is a cluster-wide option.

Procedure

1. Click **Administration > CustomResourceDefinitions**.
2. On the **CustomResourceDefinitions** page next to the **Name** dropdown box, type "clustercsidriver".

3. Click **CRD ClusterCSIDriver**.
4. Click the **Instances** tab.
5. Click **csi.vsphere.vmware.com**.
6. Click the **YAML** tab.
7. Set the parameter **spec.driverConfig.driverType** to **vSphere**.
8. Add the parameter **spec.driverConfig.vSphere.maxAllowedBlockVolumesPerNode** to the YAML file, and provide a value for the desired maximum number of volumes per node as in the following sample YAML file:

Example YAML file for adding the parameter maxAllowedBlockVolumesPerNode

```
...
spec:
  driverConfig:
    driverType: vSphere
    vSphere:
      maxAllowedBlockVolumesPerNode: 59
...
```

For **maxAllowedBlockVolumesPerNode**, enter the desired value here for the maximum number of volumes per node. The default is 59. The minimum value is 1 and the maximum value is 255.

9. Click **Save**.

6.22.16. Additional resources

- [Best practices for using VMware snapshots in the vSphere environment](#)
- [VMware vCenter documentation](#)

CHAPTER 7. GENERIC EPHEMERAL VOLUMES

Generic ephemeral volumes provide per-pod temporary storage backed by any storage driver that supports dynamic provisioning, unlike **emptyDir** volumes which are limited to local node storage. This flexibility lets you use network storage backends, control storage classes and volume characteristics, and leverage delayed volume binding for optimal pod scheduling.

7.1. OVERVIEW OF GENERIC EPHEMERAL VOLUMES

Generic ephemeral volumes support network-attached storage, size limits, initial data population, and operations like cloning and snapshotting for temporary storage, with some driver-specific limitations.

Generic ephemeral volumes have the following features:

- Storage can be local or network-attached.
- Volumes can have a fixed size that pods cannot exceed.
- Volumes might have some initial data, depending on the driver and parameters.
- Typical operations on volumes are supported, assuming that the driver supports them, including snapshotting, cloning, resizing, and storage capacity tracking.



NOTE

Generic ephemeral volumes do not support offline snapshotting and resizing.

Due to this limitation, the following Container Storage Interface (CSI) drivers do not support the following features for generic ephemeral volumes:

- Azure Disk CSI driver does not support resize.
- Cinder CSI driver does not support snapshot.

7.2. LIFECYCLE AND PERSISTENT VOLUME CLAIMS

Generic ephemeral volumes follow pod lifecycle through automatically managed persistent volume claims created at pod startup and deleted at termination. Choose volume binding mode and reclaim policy based on this lifecycle behavior.

Generic ephemeral volumes are specified inline in the pod spec and follow the pod's lifecycle. They are created and deleted along with the pod.

The parameters for a volume claim are allowed inside a volume source of a pod. Labels, annotations, and the whole set of fields for PVCs are supported. When such a pod is created, the ephemeral volume controller then creates an actual PVC object (from the template shown in the *Creating generic ephemeral volumes* procedure) in the same namespace as the pod, and ensures that the PVC is deleted when the pod is deleted.

This triggers volume binding and provisioning in one of two ways:

- Either immediately, if the storage class uses immediate volume binding. With immediate binding, the scheduler is forced to select a node that has access to the volume after it is available.

- When the pod is tentatively scheduled onto a node (**WaitForFirstConsumer** volume binding mode).
This volume binding option is recommended for generic ephemeral volumes because then the scheduler can choose a suitable node for the pod.

In terms of resource ownership, a pod that has generic ephemeral storage is the owner of the PVCs that provide that ephemeral storage. When the pod is deleted, the Kubernetes garbage collector deletes the PVC, which then usually triggers deletion of the volume because the default reclaim policy of storage classes is to delete volumes. You can create quasi-ephemeral local storage by using a storage class with a reclaim policy of retain. The storage outlives the pod, and in this case, you must ensure that volume clean-up happens separately. While these PVCs exist, they can be used like any other PVC. In particular, they can be referenced as data sources in volume cloning or snapshotting. The PVC object also holds the current status of the volume.

Additional resources

- [Creating generic ephemeral volumes](#)

7.3. SECURITY

Generic ephemeral volumes allow users who can create pods to indirectly create persistent volume claims (PVCs), even without direct PVC creation permissions. You can restrict this behavior if it conflicts with your security model.

To restrict this behavior, use an admission webhook that rejects objects such as pods that have a generic ephemeral volume.

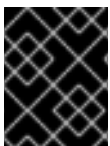
The normal namespace quota for PVCs still applies, so even if users are allowed to use this new mechanism, they cannot use it to circumvent other policies.

7.4. PERSISTENT VOLUME CLAIM NAMING

Automatically created persistent volume claims (PVCs) are named using pod name and volume name with a hyphen separator, potentially causing conflicts with other pods or manual PVCs.

For example, **pod-a** with volume **scratch** and **pod** with volume **a-scratch** both end up with the same PVC name, **pod-a-scratch**.

Such conflicts are detected, and a PVC is only used for an ephemeral volume if it was created for the pod. This check is based on the ownership relationship. An existing PVC is not overwritten or modified, but this does not resolve the conflict. Without the right PVC, a pod cannot start.



IMPORTANT

Be careful when naming pods and volumes inside the same namespace so that naming conflicts do not occur.

7.5. CREATING GENERIC EPHEMERAL VOLUMES

To create ephemeral volumes that are automatically provisioned and deleted with pod lifecycle, define a **volumeClaimTemplate** in your pod spec specifying storage class, size, and access modes.

Procedure

1. Create the **pod** object definition and save it to a file.
2. Include the generic ephemeral volume information in the file.

my-example-pod-with-generic-vols.yaml

```
kind: Pod
apiVersion: v1
metadata:
  name: my-app
spec:
  containers:
    - name: my-frontend
      image: busybox:1.28
      volumeMounts:
        - mountPath: "/mnt/storage"
          name: data
      command: [ "sleep", "1000000" ]
  volumes:
    - name: data
      ephemeral:
        volumeClaimTemplate:
          metadata:
            labels:
              type: my-app-ephvol
          spec:
            accessModes: [ "ReadWriteOnce" ]
            storageClassName: "gp2-csi"
            resources:
              requests:
                storage: 1Gi
```

Where **spec.volumes.name** is the name of the generic ephemeral volume.

CHAPTER 8. EXPANDING PERSISTENT VOLUMES

Expand persistent volumes to increase storage capacity as your application data grows. You can resize volumes without recreating volumes or disrupting running workloads.

8.1. ENABLING VOLUME EXPANSION SUPPORT

To enable volume expansion, the **StorageClass** object must have the **allowVolumeExpansion** field set to **true**. This prerequisite configuration allows persistent volume claims (PVCs) to be expanded after creation as your storage needs grow.

Procedure

- Edit the **StorageClass** object and add the **allowVolumeExpansion** attribute by running the following command:

```
$ oc edit storageclass <storage_class_name>
```

Enter the name of storage class in **<storage_class_name>**.

The following example shows adding this line at the bottom of the storage class configuration.

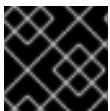
Example storage class YAML file with **allowVolumeExpansion** field set to **true**

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
...
parameters:
  type: gp2
reclaimPolicy: Delete
allowVolumeExpansion: true
```

- **parameters.allowVolumeExpansion:** Setting this field to **true** allows persistent volume claims (PVCs) to be expanded after creation.

8.2. EXPANDING CSI VOLUMES

You can use the Container Storage Interface (CSI) to expand storage volumes after they have already been created.



IMPORTANT

Shrinking persistent volumes (PVs) is *not* supported.

Prerequisites

- The underlying CSI driver supports resize.
For information about which CSI drivers support resizing, see under the *Additional resources* section "CSI drivers supported by OpenShift Container Platform".
- Dynamic provisioning is used.
- The controlling **StorageClass** object has **allowVolumeExpansion** set to **true**.

For more information, see section *Enabling volume expansion support*.

Procedure

- For the persistent volume claim (PVC), set **.spec.resources.requests.storage** to the desired new size.

Verification

- To confirm that the resize is finished, look at the **status.conditions** field of the PVC. OpenShift Container Platform adds the **Resizing** condition to the PVC during expansion, which is removed after expansion completes.

8.3. EXPANDING FLEXVOLUME WITH A SUPPORTED DRIVER

To expand your FlexVolume storage capacity and meet growing data needs, update the storage request in your persistent volume claim (PVC). This increases capacity for existing volumes without recreating them.

Similar to other volume types, FlexVolume volumes can also be expanded when in use by a pod.



IMPORTANT

Because OpenShift Container Platform does not support installation of FlexVolume plugins on control plane nodes, it does not support control plane expansion of FlexVolume.

Prerequisites

- The underlying volume driver supports resize.
For information about which CSI drivers support resizing, see under the *Additional resources* section "CSI drivers supported by OpenShift Container Platform".
- The driver is set with the **RequiresFSResize** capability to **true**. The FlexVolume can then be expanded after restarting the pod.
- Dynamic provisioning is used.
- The controlling **StorageClass** object has **allowVolumeExpansion** set to **true**.
For more information, see section *Enabling volume expansion support*.

Procedure

- To use resizing in the FlexVolume plugin, you must implement the **ExpandableVolumePlugin** interface using these methods:
 - **RequiresFSResize**
If **true**, updates the capacity directly. If **false**, calls the **ExpandFS** method to finish the filesystem resize.
 - **ExpandFS**
If **true**, calls **ExpandFS** to resize filesystem after physical volume expansion is done. The volume driver can also perform physical volume resize together with filesystem resize.

8.4. EXPANDING LOCAL VOLUMES

To expand your Local Storage Operator (LSO) storage capacity and meet growing data needs, update the storage request in your persistent volume (PV) and persistent volume claim (PVC). This increases capacity for existing volumes without recreating them.

Procedure

1. Expand the underlying devices. Ensure that appropriate capacity is available on these devices.
2. Update the corresponding PV objects to match the new device sizes by editing the **.spec.capacity** field of the PV.
3. For the storage class that is used for binding the PVC to PV, set the **allowVolumeExpansion** field to **true**.
4. For the PVC, set **.spec.resources.requests.storage** to match the new size.

Result

Kubelet should automatically expand the underlying file system on the volume, if necessary, and update the status field of the PVC to reflect the new size.

8.5. EXPANDING PERSISTENT VOLUME CLAIMS (PVCS) WITH A FILE SYSTEM

To expand your storage capacity and meet growing data needs, you can resize existing volumes without recreating them.

Expanding persistent volume claims (PVCs) based on volume types that need file system resizing, such as Google Cloud Platform (GCP) persistent disk (PD), AWS Elastic Block Storage (EBS), and Cinder, is a two-step process. First, expand the volume objects in the cloud provider. Second, expand the file system on the node.

Expanding the file system on the node only happens when a new pod is started with the volume.

Prerequisites

- The controlling storage class has the **allowVolumeExpansion** field set to **true**.
For more information, see section *Enabling volume expansion support*.

Procedure

- Edit the PVC and request a new size by editing **spec.resources.requests**. For example, the following expands the **ebs** PVC to 8 Gi:

Example PVC YAML file

```
kind: PersistentVolumeClaim
apiVersion: v1
metadata:
  name: ebs
spec:
  storageClass: "storageClassWithFlagSet"
```

```

accessModes:
  - ReadWriteOnce
resources:
  requests:
    storage: 8Gi

```

Where updating **spec.resources.requests** to a larger amount expands the PVC.

Verification

After the cloud provider object has finished resizing, the PVC is set to **FileSystemResizePending**.

- Check the condition by running the following command:

```
$ oc describe pvc <pvc_name>
```

Next steps

When the cloud provider object has finished resizing, the **PersistentVolume** object reflects the newly requested size in **PersistentVolume.Spec.Capacity**. You can now create or recreate a new pod from the PVC to finish the file system resizing. After the pod is running, the newly requested size is available and the **FileSystemResizePending** condition is removed from the PVC.

8.6. RECOVERING FROM FAILURE WHEN EXPANDING VOLUMES

If a resize request fails or remains in a pending state, you can try again by entering a different resize value in **.spec.resources.requests.storage** for the persistent volume claim (PVC). The new value must be larger than the original volume size.

If entering another smaller resize value in **.spec.resources.requests.storage** for the PVC does not work, use the following procedure to recover.

Procedure

1. Mark the persistent volume (PV) that is bound to the PVC with the **Retain** reclaim policy. Change the **persistentVolumeReclaimPolicy** field to **Retain**.
2. Delete the PVC.
3. Manually edit the PV and delete the **claimRef** entry from the PV specification to ensure that the newly created PVC can bind to the PV marked **Retain**. This marks the PV as **Available**.
4. Recreate the PVC in a smaller size, or a size that can be allocated by the underlying storage provider.
5. Set the **volumeName** field of the PVC to the name of the PV. This binds the PVC to the provisioned PV only.
6. Restore the reclaim policy on the PV.

8.7. VIEWING THE STATUS OF VOLUME RESIZE

The volume resize status shows the progress of persistent volume claim (PVC) expansion operations. By checking this status you can confirm that volume expansions are progressing correctly and troubleshoot stuck or failed resizes.

You can view the status of volume resizing with the **pvc.Status.AllocatedResourceStatus** field. If a user changes the size of their PVCs, the **pvc.Status.AllocatedResourceStatus** field allows resource quota to be tracked accurately.

The possible values for **pvc.Status.AllocatedResourceStatus** are:

- **ControllerResizeInProgress**: Controller resize attempt is in progress.
- **ControllerResizeFailed**: Controller resize attempt failed.
- **NodeResizeInProgress**: Node resize attempt is in progress.
- **NodeResizeFailed**: Node resize attempt failed.

For a typical block volume, the field transitions between **ControllerResizeInProgress**, **NodeResizePending**, **NodeResizeInProgress**, and then nil/empty when the volume expansion finishes.

8.8. ADDITIONAL RESOURCES

- [CSI drivers supported by OpenShift Container Platform](#)

CHAPTER 9. DYNAMIC PROVISIONING

In dynamic provisioning, instead of manually creating a pool of persistent volumes (PVs), an administrator creates a storage class. Using the storage class, OpenShift Container Platform automatically triggers the storage backend to create a brand-new volume of the exact size and type requested, creates the PV object, and then the PV binds to the persistent volume claim (PVC).

9.1. ABOUT DYNAMIC PROVISIONING

The **StorageClass** resource object describes and classifies storage that can be requested, and provides a means for passing parameters for dynamically provisioned storage on-demand.

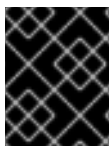
StorageClass objects can also serve as a management mechanism for controlling different levels of storage and access to the storage. Cluster Administrators (**cluster-admin**) or Storage Administrators (**storage-admin**) define and create the **StorageClass** objects that users can request without needing any detailed knowledge about the underlying storage volume sources.

The OpenShift Container Platform persistent volume framework enables this functionality and allows administrators to provision a cluster with persistent storage. The framework also gives users a way to request those resources without having any knowledge of the underlying infrastructure.

Many storage types are available for use as persistent volumes in OpenShift Container Platform. While all of them can be statically provisioned by an administrator, some types of storage are created dynamically using the built-in provider and plugin APIs.

9.2. AVAILABLE DYNAMIC PROVISIONING PLUGINS

Provisioner plugins automatically create storage resources on-demand by connecting to your cloud provider's API. This lets you dynamically provision persistent volumes (PVs) without manual intervention, adapting to your cluster's storage needs as they arise.



IMPORTANT

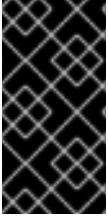
Any chosen provisioner plugin also requires configuration for the relevant cloud, host, or third-party provider as in the relevant documentation.

Storage type	Provisioner plugin name	Notes
Red Hat OpenStack Platform (RHOSP) Cinder	kubernetes.io/cinder	
RHOSP Manila Container Storage Interface (CSI)	manila.csi.openstack.org	After being installed, the OpenStack Manila CSI Driver Operator and ManilaDriver automatically create the required storage classes for all available Manila share types needed for dynamic provisioning.

Storage type	Provisioner plugin name	Notes
Amazon Elastic Block Store (Amazon EBS)	ebs.csi.aws.com	For dynamic provisioning when using multiple clusters in different zones, tag each node with Key=kubernetes.io/cluster/<cluster_name>,Value=<cluster_id> where <cluster_name> and <cluster_id> are unique per cluster.
Azure Disk	kubernetes.io/azure-disk	
Azure File	kubernetes.io/azure-file	The persistent-volume-binder service account requires permissions to create and get secrets to store the Azure storage account and keys.
GCE Persistent Disk (gcePD)	kubernetes.io/gce-pd	In multi-zone configurations, it is advisable to run one OpenShift Container Platform cluster per GCE project to avoid persistent volumes (PVs) from being created in zones where no node exists in the current cluster.
IBM Power® Virtual Server Block	powervs.csi.ibm.com	After installation, the IBM Power® Virtual Server Block CSI Driver Operator and IBM Power® Virtual Server Block CSI Driver automatically create the required storage classes for dynamic provisioning.
VMware vSphere	kubernetes.io/vsphere-volume	

9.3. DEFINING A STORAGE CLASS

StorageClass objects apply cluster-wide and are available to all namespaces. Only users with cluster-admin or storage-admin privileges can create or modify them. This centralized control ensures consistent storage policies across your cluster while requiring application teams to coordinate with administrators for custom storage configurations.



IMPORTANT

The Cluster Storage Operator might install a default storage class depending on the platform in use. This storage class is owned and controlled by the Operator. It cannot be deleted or modified beyond defining annotations and labels. If different behavior is required, you must define a custom storage class.

The following sections describe the basic definition for a **StorageClass** object and specific examples for each of the supported plugin types.

9.3.1. Basic StorageClass object definition

A **StorageClass** object defines the metadata, provisioner type, and plugin-specific parameters that determine how persistent volumes (PVs) are dynamically created in your cluster. Each storage provisioner type requires different parameters, and annotations control cluster-wide defaults, making this structure the foundation for all dynamic storage provisioning.

Example StorageClass definition

```
kind: StorageClass
apiVersion: storage.k8s.io/v1
metadata:
  name: <storage-class-name>
  annotations:
    storageclass.kubernetes.io/is-default-class: 'true'
  ...
provisioner: kubernetes.io/aws-ebs
parameters:
  type: gp3
...
```

- **kind**: API object type.
- **apiversion**: The current apiVersion.
- **metadata.name**: The name of the storage class.
- Optional: **metadata.annotations**: Annotations for the storage class.
- **provisioner**: The type of provisioner associated with this storage class.
- Optional: **parameters**: The parameters required for the specific provisioner. This is different for each plugin.

9.3.2. RHOSP Cinder StorageObject object definition

This RHOSP Cinder storage class example demonstrates how to configure volume types for performance optimization, control availability zone placement for high availability, and specify filesystem types for your persistent volumes.

Example Cinder storage class YAML file

```
kind: StorageClass
apiVersion: storage.k8s.io/v1
```

```

metadata:
  name: <storage-class-name>
provisioner: kubernetes.io/cinder
parameters:
  type: fast
  availability: nova
  fsType: ext4

```

- **metadata.name:** Name of the storage class. The persistent volume claim uses this storage class for provisioning the associated persistent volumes.
- **parameter.type:** Volume type created in Cinder. Default is empty.
- **parameters.availability:** Availability Zone. If not specified, volumes are generally round-robinbed across all active zones where the OpenShift Container Platform cluster has a node.
- **parameters.fsType:** File system that is created on dynamically provisioned volumes. This value is copied to the **fsType** field of dynamically provisioned persistent volumes, and the file system is created when the volume is mounted for the first time. The default value is **ext4**. The following is a RHOSP Cinder example storage class object definition.

9.3.3. RHOSP Manila Container Storage Interface (CSI) object definition

The OpenStack Manila CSI Driver Operator automatically creates storage classes for all available Manila share types immediately after installation, eliminating manual configuration. This automation ensures you can start provisioning persistent volumes right away without needing to understand Manila share type details or write custom **StorageClass** definitions.

9.3.4. AWS Elastic Block Store (EBS) StorageObject object definition

This AWS EBS storage class example shows how to configure volume type, IOPS performance, encryption settings, and filesystem type for dynamically provisioned persistent volumes.

Example AWS EBS storage class YAML file

```

kind: StorageClass
apiVersion: storage.k8s.io/v1
metadata:
  name: <storage-class-name>
provisioner: ebs.csi.aws.com
parameters:
  type: io1
  iopsPerGB: "10"
  encrypted: "true"
  kmsKeyId: keyvalue
  fsType: ext4

```

- **metadata.name:** Name of the storage class. The persistent volume claim uses this storage class for provisioning the associated persistent volumes.
- **parameters.type:** Select from **io1**, **gp3**, **sc1**, **st1**. The default is **gp3**. For valid Amazon Resource Name (ARN) values, see the AWS documentation, *Identify AWS resources with Amazon Resource Names (ARNs)*.

- Optional: **parameters.iopsPerGB**. Only for **io1** volumes. I/O operations per second per GiB. The AWS volume plugin multiplies this with the size of the requested volume to compute IOPS of the volume. The maximum value is 20,000 IOPS, which is the maximum supported by AWS.

For more information, see the AWS documentation, *Identify AWS resources with Amazon Resource Names (ARNs)*.

- Optional: **parameters.encrypted**. Indicates whether to encrypt the EBS volume. Valid values are **true** or **false**.
- Optional: **parameters.kmsKeyId**. The full ARN of the key to use when encrypting the volume. If none is supplied, but **encrypted** is set to **true**, then AWS generates a key. For valid ARN values, see the AWS documentation, *Identify AWS resources with Amazon Resource Names (ARNs)*.
- Optional: **parameters.fsType**. File system that is created on dynamically provisioned volumes. This value is copied to the **fsType** field of dynamically provisioned persistent volumes, and the file system is created when the volume is mounted for the first time. The default value is **ext4**.

Additional resources

- [AWS documentation](#)

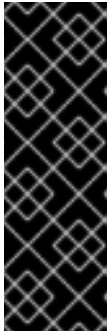
9.3.5. Azure Disk StorageClass object definition

This Azure Disk storage class example demonstrates how to configure managed disks with delayed volume binding for optimal zone placement, volume expansion, and performance tiers. Key parameters ensure compatibility with OpenShift nodes, which require managed disks rather than shared or dedicated storage accounts.

Example Azure Disk storage class YAML file

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: <storage-class-name>
provisioner: kubernetes.io/azure-disk
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true
parameters:
  kind: Managed
  storageaccounttype: Premium_LRS
reclaimPolicy: Delete
```

- **metadata.name**: Name of the storage class. The persistent volume claim uses this storage class for provisioning the associated persistent volumes.
- **volumeBindingMode**: Using **WaitForFirstConsumer** is strongly recommended. This provisions the volume while allowing enough storage to schedule the pod on a free worker node from an available zone.
- **parameters.kind**: Possible values are **Shared** (default), **Managed**, and **Dedicated**.



IMPORTANT

Red Hat only supports the use of **Managed**.

With **Shared** and **Dedicated**, Azure creates unmanaged disks, while OpenShift Container Platform creates a managed disk for machine operating system (root) disks. But because Azure Disk does not allow the use of both managed and unmanaged disks on a node, unmanaged disks created with **Shared** or **Dedicated** cannot be attached to OpenShift Container Platform nodes.

- **parameters.storageaccounttype**: Azure storage account SKU tier. Default is empty. Note that Premium VMs can attach both **Standard_LRS** and **Premium_LRS** disks. Standard VMs can only attach **Standard_LRS** disks. Managed VMs can only attach managed disks. Unmanaged VMs can only attach unmanaged disks.
 - **Shared**: Azure creates all unmanaged disks in a few shared storage accounts in the same resource group as the cluster.
 - **Managed**: Azure creates new managed disks.
 - **Dedicated**, and a **storageAccount** is not specified: Azure creates a new dedicated storage account for the new unmanaged disk in the same resource group as the cluster.
 - **Dedicated**, and a **storageAccount** is specified: Azure uses the specified storage account for the new unmanaged disk in the same resource group as the cluster. For this to work, the specified storage account must be in the same region, and Azure Cloud Provider must have write access to the storage account.

9.3.6. Azure File object definition

To enable Azure File storage classes to dynamically provision file shares, grant the persistent volume binder permissions to create and manage secrets containing Azure storage credentials. This allows the provisioner to securely store and access the Azure storage account name and key required for file share creation.

Procedure

1. Define a **ClusterRole** object that allows access to create and view secrets as in the following example file:

Cluster role example YAML file

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: <persistent_volume_binder_role>
rules:
- apiGroups: [""]
  resources: ['secrets']
  verbs: ['get','create']
```

- **Metadata.name**: The name of the cluster role to view and create secrets.
2. Add the cluster role to the service account by running the following command:

–

```
$ oc adm policy add-cluster-role-to-user <persistent-volume-binder-role>
system:serviceaccount:kube-system:persistent-volume-binder
```

Where **<persistent-volume-binder-role>** is the name of the cluster role you provided in the preceding step.

3. Create the Azure File **StorageClass** object as in the following example file:

Example Azure File storage class YAML file

```
kind: StorageClass
apiVersion: storage.k8s.io/v1
metadata:
  name: <azure-file>
provisioner: kubernetes.io/azure-file
parameters:
  location: eastus
  skuName: Standard_LRS
  storageAccount: <storage-account>
reclaimPolicy: Delete
volumeBindingMode: Immediate
```

- **metadata.name:** Name of the storage class. The persistent volume claim uses this storage class for provisioning the associated persistent volumes.
- **parameters.location:** Location of the Azure storage account, such as **eastus**. The default is empty, meaning that a new Azure storage account is created in the OpenShift Container Platform cluster's location.
- **parameters.skuName:** SKU tier of the Azure storage account, such as **Standard_LRS**. The default is empty, meaning that a new Azure storage account is created with the **Standard_LRS** SKU.
- **parameters.storageAccount:** Name of the Azure storage account. If a storage account is provided, then **skuName** and **location** are ignored. If no storage account is provided, the storage class searches for storage accounts associated with the resource group for accounts that match the defined **skuName** and **location**.

9.3.6.1. Considerations when using Azure File

Azure File storage has inherent file system limitations, including lack of support for symlinks, hard links, and sparse files by default, plus ownership mismatches between mounted directories and container processes. Understanding these constraints and using mount options such as `uid`, `gid`, and `mfsymlinks` helps you configure Azure File storage classes that work correctly with your containerized applications.

The following features are not supported:

- Symlinks
- Hard links
- Extended attributes
- Sparse files

- Named pipes

Additionally, the owner user identifier (UID) of the Azure File mounted directory is different from the process UID of the container. You can specify the **uid** mount option in the **StorageClass** object to define a specific user identifier to use for the mounted directory.

The following **StorageClass** object demonstrates modifying the **uid** and group identifier (**gid**), along with enabling symlinks for the mounted directory.

Example Azure File storage class YAML file with modified **uid** and **gid**

```
kind: StorageClass
apiVersion: storage.k8s.io/v1
metadata:
  name: azure-file
mountOptions:
  - uid=1500
  - gid=1500
  - mfsymlinks
provisioner: kubernetes.io/azure-file
parameters:
  location: eastus
  skuName: Standard_LRS
reclaimPolicy: Delete
volumeBindingMode: Immediate
```

- **mountOptions.uid**: Specifies the user identifier to use for the mounted directory.
- **mountOptions.gid**: Specifies the group identifier to use for the mounted directory.
- **mountOptions.mfsymlinks**: Enables symlinks.

9.3.7. GCE PersistentDisk (gcePD) object definition

This Google Compute Engine Persistent Disk (GCE PD) storage class example shows how to configure disk performance tiers (SSD, standard, or hyperdisk-balanced), enable volume expansion, and use delayed binding to optimize zone placement for your workloads.

Example GCE PD storage class YAML file

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: <storage-class-name>
provisioner: kubernetes.io/gce-pd
parameters:
  type: pd-ssd
  replication-type: none
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true
reclaimPolicy: Delete
```

- **metadata.name**: The name of the storage class. The persistent volume claim uses this storage class for provisioning the associated persistent volumes.

- **parameters.type**: Select **pd-ssd**, **pd-standard**, or **hyperdisk-balanced**. The default is **pd-ssd**.

9.3.8. VMware vSphere object definition

This VMware vSphere storage class example demonstrates the basic structure and Container Storage Interface (CSI) provisioner configuration required to enable dynamic storage provisioning on vSphere infrastructure. This minimal definition provides the foundation for vSphere storage integration, which you can extend with storage policies, datastore preferences, and other vSphere-specific parameters.

Example vSphere storage class YAML file

```
kind: StorageClass
apiVersion: storage.k8s.io/v1
metadata:
  name: <storage-class-name> 1
provisioner: csi.vsphere.vmware.com 2
```

- **metadata.name**: Name of the storage class. The persistent volume claim uses this storage class for provisioning the associated persistent volumes.
- **provisioner**: The name of the provisioner for the storage class. For vSphere, this is **csi.vsphere.vmware.com**.

9.4. SETTING THE DEFAULT STORAGE CLASS

A default storage class automatically provisions persistent volumes when you create persistent volume claims (PVCs) without specifying a storage class. This simplifies storage management by removing the need for users to select a storage class for each claim. To designate a storage class as the cluster-wide default, add an annotation to the storage class metadata.

Prerequisites

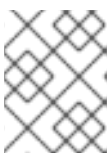
- Logged in to a running OpenShift Container Platform cluster with administrator privileges.

Procedure

1. For your required storage class, set the **metadata.annotations.storageclass.kubernetes.io/is-default-class** field to **true** as in the following example:

Example storage class YAML file

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  annotations:
    storageclass.kubernetes.io/is-default-class: "true"
...
```



NOTE

The beta annotation **storageclass.beta.kubernetes.io/is-default-class** is still working; however, it will be removed in a future release.

- Optional: Create a storage class description in the **metadata.annotations.kubernetes.io/description** field as in the following example:

Example storage class YAML file

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  annotations:
    kubernetes.io/description: My Storage Class Description
...
```

9.5. CHANGING THE DEFAULT STORAGE CLASS

Change the default storage class to ensure new persistent volume claims (PVCs) automatically use your preferred storage backend. This helps you optimize costs, align with infrastructure changes, or ensure consistent storage types across new deployments without requiring users to specify a storage class for each claim.

In this example, you have two defined storage classes, **gp3** and **standard**, and you want to change the default storage class from **gp3** to **standard**.

Prerequisites

- Access to the cluster with cluster-admin privileges.

Procedure

- List the storage classes by running the following command:

```
$ oc get storageclass
```

Example output

NAME	TYPE
gp3 (default)	ebs.csi.aws.com
standard	ebs.csi.aws.com

The text **(default)** indicates the default storage class. In this example **gp3** is the current default storage class.

- Make the required storage class the default.
For the required storage class, set the **storageclass.kubernetes.io/is-default-class** annotation to **true** by running the following command:

```
$ oc patch storageclass standard -p '{"metadata": {"annotations": {"storageclass.kubernetes.io/is-default-class": "true"}}}'
```



NOTE

You can have many default storage classes for a short time. However, you must ensure that only one default storage class exists eventually.

With many default storage classes present, any persistent volume claim (PVC) requesting the default storage class (**pvc.spec.storageClassName=nil**) gets the most recently created default storage class, regardless of the default status of that storage class. The administrator receives an alert in the alerts dashboard that there are many default storage classes, **MultipleDefaultStorageClasses**.

3. Remove the default storage class setting from the old default storage class.
For the old default storage class, change the value of the **storageclass.kubernetes.io/is-default-class** annotation to **false** by running the following command:

```
$ oc patch storageclass gp3 -p '{"metadata": {"annotations": {"storageclass.kubernetes.io/is-default-class": "false"}}}'
```

4. Verify the changes by running the following command:

```
$ oc get storageclass
```

Example output

NAME	TYPE
gp3	ebs.csi.aws.com
standard (default)	ebs.csi.aws.com

The **standard** storage class is now the default.

CHAPTER 10. DETACH VOLUMES AFTER NON-GRACEFUL NODE SHUTDOWN

Automatic volume detachment after non-graceful node shutdowns prevents volumes from remaining attached to failed nodes, enabling faster workload recovery by allowing pods to reschedule and reattach volumes on healthy nodes without manual intervention.

10.1. OVERVIEW

Non-graceful node shutdowns from hardware failures or system crashes leave volumes attached to failed nodes, blocking pod rescheduling. Applying an out-of-service taint triggers automatic volume detachment from failed nodes, enabling workload recovery without manual volume management.

A graceful node shutdown occurs when the kubelet's node shutdown manager detects the upcoming node shutdown action. Non-graceful shutdowns occur when the kubelet does not detect a node shutdown action, which can occur because of system or hardware failures. Also, the kubelet might not detect a node shutdown action when the shutdown command does not trigger the Inhibitor Locks mechanism used by the kubelet on Linux, or because of a user error, for example, if the `shutdownGracePeriod` and `shutdownGracePeriodCriticalPods` details are not configured correctly for that node.

10.2. ADDING AN OUT-OF-SERVICE TAINT MANUALLY FOR AUTOMATIC VOLUME DETACHMENT

After non-graceful shutdowns, to trigger automatic volume detachment and enable pod rescheduling, apply an out-of-service taint to the node. This recovers workloads faster than manually detaching volumes from failed nodes.

Prerequisites

- Access to the cluster with cluster-admin privileges.

Procedure

1. After a node is detected as unhealthy, shut down the worker node.
2. Ensure that the node is shutdown by running the following command and checking the status:

```
$ oc get node <node_name>
```

- Use the **<node_name>** to specify the node that shut down non-gracefully.



IMPORTANT

If the node is not completely shut down, do not proceed with tainting the node. If the node is still up and the taint is applied, filesystem corruption can occur.

3. Taint the corresponding node object by running the following command:



IMPORTANT

Tainting a node this way deletes all pods on that node. This also causes any pods that are backed by statefulsets to be evicted, and replacement pods to be created on a different node.

```
$ oc adm taint node <node_name> node.kubernetes.io/out-of-
service=nodeshutdown:NoExecute
```

- Use the **<node_name>** to specify the node that shut down non-gracefully. After the taint is applied, the volumes detach from the shutdown node allowing their disks to be attached to a different node.

The resulting YAML file resembles the following example file:

Example node YAML file with out-of-service taint applied

```
spec:
  taints:
  - effect: NoExecute
    key: node.kubernetes.io/out-of-service
    value: nodeshutdown
```

4. Restart the node.
5. Remove the taint from the corresponding node object by running the following command:

```
$ oc adm taint node <node_name> node.kubernetes.io/out-of-
service=nodeshutdown:NoExecute-
```

- Use the **<node_name>** to specify the node that shut down non-gracefully